

Android Framework 精编内核解析

Android Framework 精编内核解析.....	1
第一章 深入解析 Binder.....	7
第一节 Binder 系列—开篇.....	7
一、概述.....	7
二、 Binder.....	7
2.1 IPC 原理.....	8
2.2 Binder 原理.....	8
2.3 C/S 模式.....	10
三、 提纲.....	10
四. 源码目录.....	11
第二节 Binder Driver 初探.....	13
一、Binder 驱动概述.....	13
1.1 概述.....	13
1.2 系统调用.....	13
二、 Binder 核心方法.....	14
2.1 binder_init.....	14
2.2 binder_open.....	16
2.3 binder_mmap.....	17
2.4 binder_ioctl.....	26
2.4 Command 使用场景.....	35
2.5 小节.....	35
三、附录.....	35
3.0 结构体列表.....	35
3.1 binder_proc.....	37
3.2 binder_thread.....	40
3.3 binder_node.....	41
3.4 binder_ref.....	43
3.5 binder_ref_death.....	44
3.6 binder_write_read.....	44
3.7 binder_transaction_data.....	45
3.8 flat_binder_object.....	47
3.9 binder_buffer.....	48
3.10 binder_transaction.....	49
3.11 binder_work.....	50
3.12 binder_state.....	51
第三节 Binder Driver 再探.....	51
一、Binder 通信简述.....	51
二、Binder 通信协议.....	52
2.1 通信模型.....	52
2.2 binder_thread_write.....	54

2.3 binder_thread_read.....	59
三. 场景总结.....	74
四、Binder 内存机制.....	77
第四节 Binder 启动 ServiceManager.....	79
一. 概述.....	79
1.1 流程图.....	80
二. 启动过程.....	80
2.1 main.....	81
2.2 binder_open.....	82
2.3 binder_become_context_manager.....	84
2.4 binder_loop.....	88
2.5 binder_parse.....	93
2.6 svcmgr_handler.....	97
三. 核心工作.....	100
3.1 do_find_service.....	100
3.2 do_add_service.....	103
3.3 binder_link_to_death.....	107
3.4 binder_send_reply.....	115
四. 总结.....	117
第五节 获取 ServiceManager.....	117
一. 概述.....	118
1.1 流程图.....	118
1.2 defaultServiceManager.....	119
二. 获取 ProcessState 对象.....	120
三. 获取 BpBinder 对象.....	123
四. 获取 BpServiceManager.....	126
4.1 interface_cast.....	126
4.2 IServiceManager::asInterface.....	127
4.3 DECLARE_META_INTERFACE.....	127
4.4 IMPLEMENT_META_INTERFACE.....	128
4.5 BpServiceManager 实例化.....	130
五. 总结.....	131
5.1 模板函数.....	131
第六节 注册服务(addService).....	133
一.概述.....	133
二. ProcessState.....	136
三. 服务注册.....	140
3.1 instantiate.....	140
3.2 BpSM.addService.....	140
3.3 BpBinder::transact.....	143
3.4 IPC::transact.....	145
3.5 IPC.writeTransactionData.....	147
3.6 IPC.waitForResponse.....	148
3.7 IPC.talkWithDriver.....	149

四. Binder Driver.....	151
4.1 binder_ioctl_write_read.....	151
4.2 binder_thread_write.....	152
4.3 binder_transaction.....	153
五. ServiceManager.....	162
5.1 binder_parse.....	162
5.2 svcmgr_handler.....	163
5.3 do_add_service.....	164
5.4 binder_send_reply.....	166
六. 总结.....	167
第七节 获取服务(getService).....	169
一、 获取服务.....	169
二. 获取 Media 服务.....	170
2.1 getMediaPlayerService.....	170
2.2 BpSM.getService.....	171
2.3 BpSM.checkService.....	172
2.4 BpBinder::transact.....	173
2.5 IPC::transact.....	175
2.6 IPC.writeTransactionData.....	176
2.7 IPC.waitForResponse.....	178
2.8 IPC.talkWithDriver.....	179
2.9 readStrongBinder.....	186
二. 总结.....	189
第八节 framework 层分析.....	189
一、 概述.....	190
1.1 Binder 架构.....	190
1.2 Binder 类图.....	191
1.3 Binder 类分层.....	192
二、 初始化.....	193
2.1 startReg.....	193
2.2 register_android_os_Binder.....	194
2.3 注册 Binder.....	195
2.4 注册 BinderInternal.....	197
2.5 注册 BinderProxy.....	198
三、 注册服务.....	200
3.1 SM.addService.....	200
3.2 getIServiceManager.....	200
3.3 SMN.asInterface.....	203
3.4 SMP.addService.....	205
3.5 writeStrongBinder(Java).....	205
3.6 writeStrongBinder(C++).....	208
3.7 BinderProxy.transact.....	210
3.8 android_os_BinderProxy_transact.....	211
3.9 小结.....	212

四、获取服务.....	212
4.1 SM.getService.....	212
4.2 SMP.getService.....	213
4.3 BinderProxy.transact.....	214
4.4 android_os_BinderProxy.transact.....	214
4.5 BpBinder.transact.....	215
4.6 IPC.transact.....	216
4.7 IPC.waitForResponse.....	217
4.8 readStrongBinder.....	219
4.9 小结.....	222
五、实例.....	223
5.1 Binder.....	224
5.2 BinderProxy.....	225
第九节如何使用 Binder.....	225
一、Native 层 Binder.....	225
1.1 服务端.....	225
1.2 客户端.....	226
1.3 创建 MyService.....	226
1.4 原理图.....	229
1.5 运行.....	230
二、Framework 层 Binder.....	232
2.1 Server 端.....	232
2.2 Client 端.....	235
2.3 原理图.....	237
2.4 运行.....	238
第十节如何使用 AIDL.....	239
一、AIDL.....	239
1.1 Server 端.....	239
1.2 Client 端.....	241
1.3 AIDL 文件.....	245
1.4 Parcel 数据.....	246
1.5 运行.....	248
二、原理分析.....	250
2.1 源码.....	250
第十一节 Binder 总结.....	255
1. Binder 概述.....	255
2. Binder 架构.....	256
3. Binder 进程与线程.....	256
4. Binder 传输过程.....	257
5. Binder 路由.....	258
第十二节 Binder 面试题全解析.....	260
第二章深入解析 Handler 消息机制.....	264
第一节 宏观理论分析与 Message 源码分析.....	264
预热.....	264

设计 Handler 的初衷.....	264
Java 多线程通信.....	265
Android 多线程的特殊性.....	266
Android 多线程通信.....	266
小结.....	267
Handler 提供的一些方法.....	267
发送消息类方法.....	267
处理消息类方法.....	268
切换线程类方法.....	268
使用 Handler.....	269
小结.....	272
Handler 实现原理 - 理论分析.....	272
线程中接收消息端的特殊性.....	273
Looper - 线程独立.....	274
MessageQueue - 多线程同时发消息.....	276
小结.....	277
Message 源码分析.....	277
Message 中的数据属性与方法.....	278
创建与回收 Message 对象的基本方法.....	279
扩展方法.....	290
小结.....	293
第二节 MessageQueue 的源码分析.....	294
消息队列存储原理.....	294
使用 JNI 实现的 native 方法.....	295
创建与销毁.....	295
消息入队管理 enqueueMessage()方法.....	298
同步消息拦截器.....	301
队列空闲处理器 IdleHandler.....	303
消息出队管理 next()方法.....	304
总结.....	309
第三节 Looper 的源码分析.....	309
创建与退出 Looper.....	309
运行 Looper 处理消息.....	312
总结.....	314
第四节 handler 的源码分析.....	314
初始化.....	314
发送消息.....	317
接收消息.....	320
内存泄漏的可能.....	321
第五节 Handler 机制实现原理总结.....	323
Message 缓存池.....	323
真正的阻塞发生在 MessageQueue.....	324
为什么推荐使用 Handler 实现线程间通信.....	324
第六节 Handler 面试题全解析.....	324

第三章 Dalvik VM 进程系统.....	331
第一节 Android 系统启动-综述.....	331
一. 概述.....	331
二. init.....	332
三. Zygote.....	335
四. system_server.....	341
五. app.....	349
六. 启动日志分析.....	351
七. 总结.....	354
第二节 Init 篇.....	355
一、概述.....	355
二、信号处理.....	361
三、rc 文件语法.....	366
四、启动服务.....	368
五、属性服务.....	372
六、总结.....	379
第三节 zygote 篇.....	380
一. 概述.....	380
二、Zygote 启动过程.....	381
2.1 App_main.main.....	381
2.2 start.....	384
2.3 startVm.....	387
2.4 startReg.....	389
三. 进入 Java 层.....	391
3.1 ZygoteInit.main.....	391
3.2 registerZygoteSocket.....	393
3.3 preload.....	394
3.4 startSystemServer.....	395
3.5 runSelectLoop.....	397
3.6 runOnce.....	399
四、总结.....	401
第四节 SystemServer 上篇.....	403
启动流程.....	403
1. startSystemServer.....	403
2 forkSystemServer.....	405
3. nativeForkSystemServer.....	406
4. ForkAndSpecializeCommon.....	407
5. handleSystemServerProcess.....	409
6. performSystemServerDexOpt.....	410
7. zygoteInit.....	411
8. commonInit.....	412
9. nativeZygoteInit.....	413
10. applicationInit.....	414
11. invokeStaticMain.....	415

12. MethodAndArgsCaller.....	416
第五节 SystemServer 下篇.....	417
一. SystemServer 启动.....	417
1.1 SystemServer.main.....	418
1.2 SystemServer.run.....	418
1.3 createSystemContext.....	421
1.4 startBootstrapServices.....	422
1.5 startCoreServices.....	424
1.6 startOtherServices.....	424
二、服务启动阶段.....	426
三、服务类别.....	432
第四章深入解析 WMS.....	439
1.WMS 概述.....	439
2.WMS 的诞生.....	440
第五章 PackageMS 启动.....	448
1. PackageMS 相关框架类.....	448
2.PackageMS 启动过程.....	448

第一章 深入解析 Binder

第一节 Binder 系列—开篇

一、概述

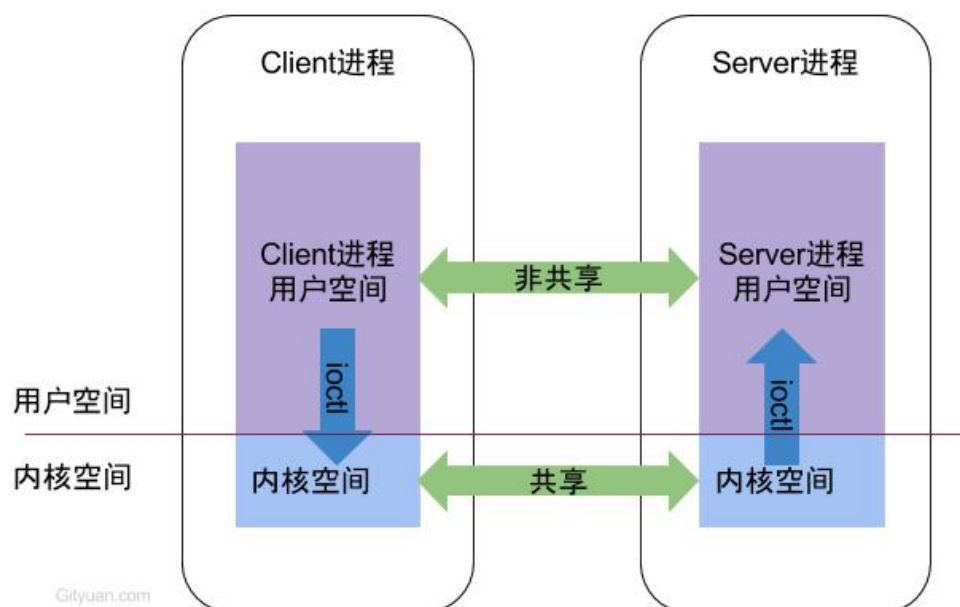
Android 系统中，每个应用程序是由 Android 的 **Activity**，**Service**，**Broadcast**，**ContentProvider** 这四剑客的中一个或多个组合而成，这四剑客所涉及的多进程间的通信底层都是依赖于 Binder IPC 机制。例如当进程 A 中的 Activity 要向进程 B 中的 Service 通信，这便需要依赖于 Binder IPC。不仅如此，整个 Android 系统架构中，大量采用了 Binder 机制作为 IPC（进程间通信）方案，当然也存在部分其他的 IPC 方式，比如 Zygote 通信便是采用 socket。

Binder 作为 Android 系统提供的一种 IPC 机制，无论从事系统开发还是应用开发，都应该有所了解，这是 Android 系统最重要的组成，也是最难理解的一块知识点，错综复杂。要深入了解 Binder 机制，最好的方法便是阅读源码，借用 Linux 鼻祖 Linus Torvalds 曾说过的一句话：**Read The Fucking Source Code**。

二、Binder

2.1 IPC 原理

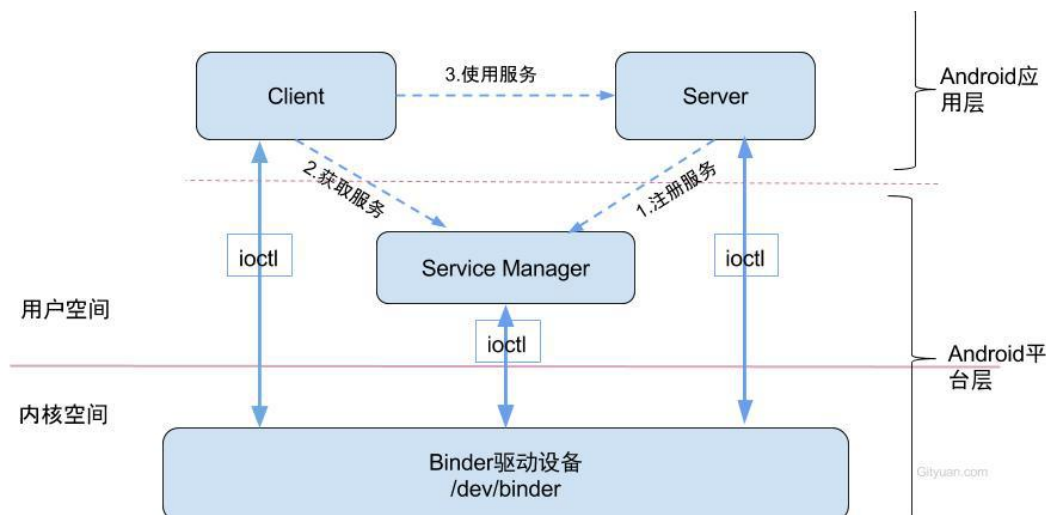
从进程角度来看 IPC 机制



每个 Android 的进程，只能运行在自己进程所拥有的虚拟地址空间。对应一个 4GB 的虚拟地址空间，其中 3GB 是用户空间，1GB 是内核空间，当然内核空间的大小是可以通过参数配置调整的。对于用户空间，不同进程之间彼此是不能共享的，而内核空间却是可共享的。Client 进程向 Server 进程通信，恰恰是利用进程间可共享的内核内存空间来完成底层通信工作的，Client 端与 Server 端进程往往采用 ioctl 等方法跟内核空间的驱动进行交互。

2.2 Binder 原理

Binder 通信采用 C/S 架构，从组件视角来说，包含 Client、Server、ServiceManager 以及 binder 驱动，其中 ServiceManager 用于管理系统中的各种服务。架构图如下所示：



可以看出无论是注册服务和获取服务的过程都需要 **ServiceManager**，需要注意的是此处的 **Service Manager** 是指 **Native 层**的 **ServiceManager (C++)**，并非指 **framework 层**的 **ServiceManager(Java)**。**ServiceManager** 是整个 **Binder** 通信机制的大管家，是 **Android** 进程间通信机制 **Binder** 的守护进程，要掌握 **Binder** 机制，首先需要了解系统是如何首次启动 **Service Manager**。当 **Service Manager** 启动之后，**Client** 端和 **Server** 端通信时都需要先获取 **Service Manager** 接口，才能开始通信服务。

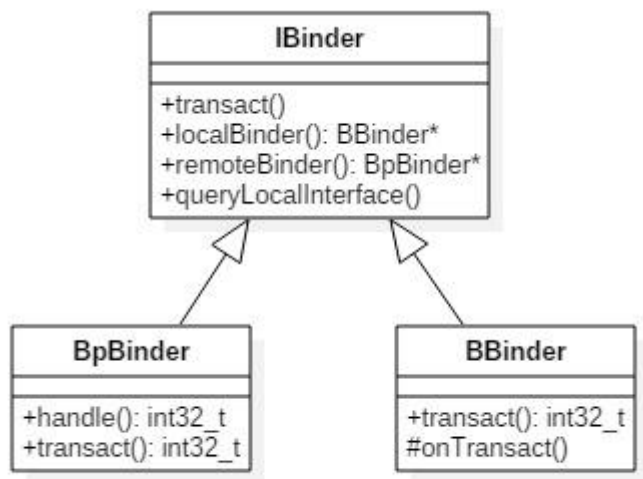
图中 **Client/Server/ServiceManager** 之间的相互通信都是基于 **Binder** 机制。既然基于 **Binder** 机制通信，那么同样也是 **C/S** 架构，则图中的 3 大步骤都有相应的 **Client** 端与 **Server** 端。

1. **注册服务 (addService)**： **Server** 进程要先注册 **Service** 到 **ServiceManager**。该过程：**Server** 是客户端，**ServiceManager** 是服务端。
2. **获取服务 (getService)**： **Client** 进程使用某个 **Service** 前，须先向 **ServiceManager** 中获取相应的 **Service**。该过程：**Client** 是客户端，**ServiceManager** 是服务端。
3. **使用服务**：**Client** 根据得到的 **Service** 信息建立与 **Service** 所在的 **Server** 进程通信的通路，然后就可以直接与 **Service** 交互。该过程：**client** 是客户端，**server** 是服务端。

图中的 **Client,Server,Service Manager** 之间交互都是虚线表示，是由于它们彼此之间不是直接交互的，而是都通过与 **Binder 驱动**进行交互的，从而实现 **IPC** 通信方式。其中 **Binder 驱动**位于内核空间，**Client,Server,Service Manager** 位于用户空间。**Binder 驱动**和 **Service Manager**可以看做是 **Android** 平台的基础架构，而 **Client** 和 **Server** 是 **Android** 的应用层，开发人员只需自定义实现 **client**、**Server** 端，借助 **Android** 的基本平台架构便可以直接进行 **IPC** 通信。

2.3 C/S 模式

BpBinder(客户端)和 BBinder(服务端)都是 Android 中 Binder 通信相关的代表，它们都从 IBinder 类中派生而来，关系图如下：



- client 端：BpBinder.transact()来发送事务请求；
- server 端：BBinder.onTransact()会接收到相应事务。

三、 提纲

在后续的 Binder 源码分析过程中所涉及的源码，会有部分的精简，主要是去掉所有 log 输出语句，已减少代码篇幅过于长。通过前面的介绍，下面罗列一下关于 Binder 系列文章的提纲：

- Binder 系列 1—Binder Driver 初探
- Binder 系列 2—Binder Driver 再探
- Binder 系列 3—启动 Service Manager
- Binder 系列 4—获取 Service Manager
- Binder 系列 5—注册服务(addService)
- Binder 系列 6—获取服务(getService)
- Binder 系列 7—framework 层分析
- Binder 系列 8—如何使用 Binder
- Binder 系列 9—如何使用 AIDL
- Binder 系列 10—总结

文章是从底层驱动往上层写的，这并不适合大家的理解，建议读者还是从上层往底层看。下面说说这个系列文章之间的彼此联系，也是对你阅读顺序的一个建议，更好的建议，大家可以上微博跟@Gityuan，或许邮件跟我进行交流与反馈：

首先阅读 [Binder 系列 5—注册服务\(addService\)](#)和 [Binder 系列 6—获取服务\(getService\)](#)，这两个过程都需要于 `ServiceManager` 打交道，那么这两个过程在开始之前都需要 [Binder 系列 4—获取 Service Manager](#)，既然要获取 `Service Manager`，那么就需要先 [Binder 系列 3—启动 Service Manager](#)。在看 `Binder` 服务的注册和获取这两个过程中，不断追溯下去，最终调用到底层 `Binder` 底层驱动，这时需要了解 [Binder 系列 1—Binder Driver 初探](#)和 [Binder 系列 2—Binder Driver 再探](#)。

看完 `Binder` 系列 1~系列 6，那么对 `Binder` 的整个流程会有一个比较清晰的认知，这还只是停留在 `Native` 层(C/C++)。接下来，可以看看上层 [Binder 系列 7—framework 层分析](#)的 `Binder` 架构情况，`Java` 层 `Binder` 架构的核心逻辑都是交由 `Native` 架构来完成，更多的是对 `Binder` 的一个封装过程，只有真正理解了 `Native` 层 `Binder` 架构，才能算掌握的 `Binder`。

前面的这些都是讲述 `Binder` 整个流程以及原理，再接下来你可能想要自己写一套 `Binder` 的 C/S 架构服务。如果你是系统工程师可能会比较关心 `Native` 层和 `framework` 层分别该如何实现自己的自定义的 `Binder` 通信服务，见 [Binder 系列 8—如何使用 Binder](#)；如果你是应用开发工程师则应该更关心 `App` 是如何使用 `Binder` 的，那么可以查看文章 [Binder 系列 9—如何使用 AIDL](#)。

最后是对 `Binder` 的一个简单总结 [Binder 系列 10—总结](#)。

四. 源码目录

从上之下，整个 `Binder` 架构所涉及的总共有以下 5 个目录：

```
/framework/base/core/java/           (Java)/framework/base/core/jni/
(JNI)
/framework/native/libs/binder          (Native)/framework/native/cmds/servicem
anager/ (Native)
/kernel/drivers/staging/android        (Driver)
```

4.1 Java framework

```
/framework/base/core/java/android/os/
```

- IInterface.java
- IBinder.java
- Parcel.java
- IServiceManager.java
- ServiceManager.java
- ServiceManagerNative.java
- Binder.java

```
/framework/base/core/jni/
```

- android_os_Parcel.cpp
- AndroidRuntime.cpp
- android_util_Binder.cpp (核心类)

4.2 Native framework

```
/framework/native/libs/binder
```

- IServiceManager.cpp
- BpBinder.cpp
- Binder.cpp
- IPCThreadState.cpp (核心类)
- ProcessState.cpp (核心类)

```
/framework/native/include/binder/
```

- IServiceManager.h
- IInterface.h

```
/framework/native/cmds/servicemanager/
```

- service_manager.c
- binder.c

4.3 Kernel

```
/kernel/drivers/staging/android/
```

```
- binder.c
```

```
- uapi/binder.h
```

第二节 Binder Driver 初探

基于 Android 6.0 的源码剖析，在讲解 Binder 原理之前，先从 kernel 的角度来讲解 Binder Driver。

```
kernel/drivers/ (不同 Linux 分支路径略有不同)
```

```
- staging/android/binder.c
```

```
- android/binder.c
```

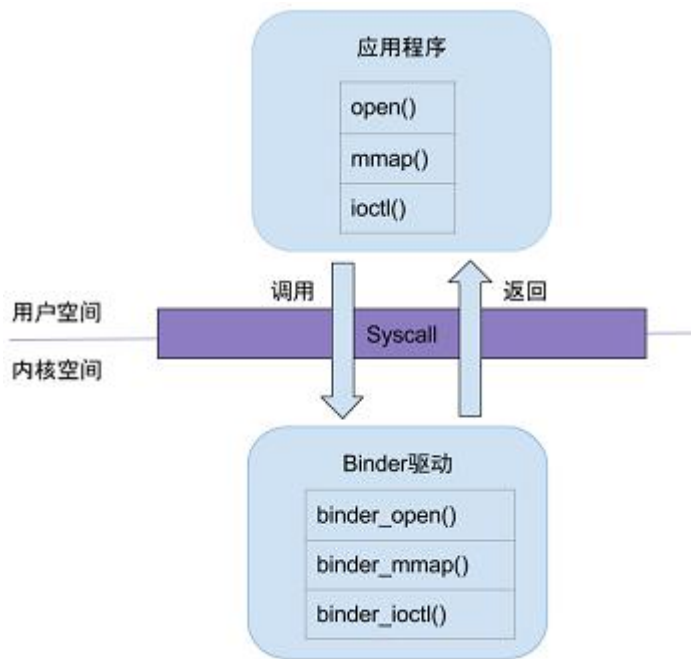
一、Binder 驱动概述

1.1 概述

Binder 驱动是 Android 专用的，但底层的驱动架构与 Linux 驱动一样。binder 驱动在以 misc 设备进行注册，作为虚拟字符设备，没有直接操作硬件，只是对设备内存的处理。主要是驱动设备的初始化(binder_init)，打开 (binder_open)，映射(binder_mmap)，数据操作(binder_ioctl)。

1.2 系统调用

用户态的程序调用 Kernel 层驱动是需要陷入内核态，进行系统调用(syscall)，比如打开 Binder 驱动方法的调用链为：open->__open()->binder_open()。open()为用户空间的方法，__open()便是系统调用中相应的处理方法，通过查找，对应调用到内核 binder 驱动的 binder_open()方法，至于其他的从用户态陷入内核态的流程也基本一致。



简单说，当用户空间调用 `open()` 方法，最终会调用 `binder` 驱动的 `binder_open()` 方法；`mmap()/ioctl()` 方法也是同理，在 `Binder` 系列的后续文章从用户态进入内核态，都依赖于系统调用过程。

二、 Binder 核心方法

2.1 binder_init

主要工作是为了注册 `misc` 设备

```
static int __init binder_init(void)
{
    int ret;

    //创建名为 binder 的工作队列

    binder_deferred_workqueue = create_singlethread_workqueue("binder");

    ...

    binder_debugfs_dir_entry_root = debugfs_create_dir("binder", NULL);
```

```
if (binder_debugfs_dir_entry_root)

    binder_debugfs_dir_entry_proc = debugfs_create_dir("proc",
                                                    binder_debugfs_dir_entry_root);

// 注册 misc 设备【见小节 2.1.1】
ret = misc_register(&binder_miscdev);

if (binder_debugfs_dir_entry_root) {
    ... //在 debugfs 文件系统中创建一系列的文件
}

return ret;
}
```

`debugfs_create_dir` 是指在 `debugfs` 文件系统中创建一个目录，返回值是指向 `dentry` 的指针。当 `kernel` 中禁用 `debugfs` 的话，返回值是 `-%ENODEV`。默认是禁用的。如果需要打开，在目录 `/kernel/arch/arm64/configs/` 下找到目标 `defconfig` 文件中添加一行 `CONFIG_DEBUG_FS=y`，再重新编译版本，即可打开 `debug_fs`。

2.1.1 misc_register

注册 `misc` 设备，`miscdevice` 结构体，便是前面注册 `misc` 设备时传递进去的参数

```
static struct miscdevice binder_miscdev = {

    .minor = MISC_DYNAMIC_MINOR, //次设备号 动态分配

    .name = "binder", //设备名

    .fops = &binder_fops //设备的文件操作结构，这是 file_operations 结构
};
```

`file_operations` 结构体,指定相应文件操作的方法

```
static const struct file_operations binder_fops = {
```

```
.owner = THIS_MODULE,  
  
.poll = binder_poll,  
  
.unlocked_ioctl = binder_ioctl,  
  
.compat_ioctl = binder_ioctl,  
  
.mmap = binder_mmap,  
  
.open = binder_open,  
  
.flush = binder_flush,  
  
.release = binder_release,  
  
};
```

2.2 binder_open

打开 binder 驱动设备

```
static int binder_open(struct inode *nodp, struct file *filp)  
{  
    struct binder_proc *proc; // binder 进程 【见附录 3.1】  
  
    proc = kzalloc(sizeof(*proc), GFP_KERNEL); // 为 binder_proc 结构体在分配 kernel 内存空间  
  
    if (proc == NULL)  
        return -ENOMEM;  
  
    get_task_struct(current);  
  
    proc->tsk = current; //将当前线程的 task 保存到 binder 进程的 tsk  
  
    INIT_LIST_HEAD(&proc->todo); //初始化 todo 列表  
  
    init_waitqueue_head(&proc->wait); //初始化 wait 队列  
  
    proc->default_priority = task_nice(current); //将当前进程的 nice 值转换为进程优先级  
  
    binder_lock(__func__); //同步锁，因为 binder 支持多线程访问  
  
    binder_stats_created(BINDER_STAT_PROC); //BINDER_PROC 对象创建数加 1
```



```
hlist_add_head(&proc->proc_node, &binder_procs); //将 proc_node 节点添加到 binder_procs 为表头的队列

proc->pid = current->group_leader->pid;

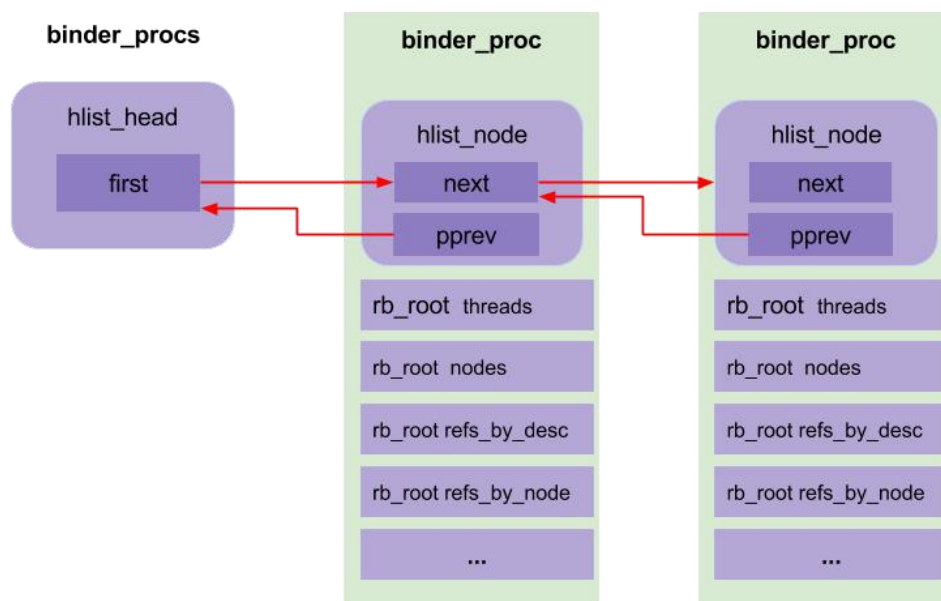
INIT_LIST_HEAD(&proc->delivered_death); //初始化已分发的死亡通知列表

filp->private_data = proc; //file 文件指针的 private_data 变量指向 binder_proc 数据

binder_unlock(__func__); //释放同步锁

return 0;
}
```

创建 `binder_proc` 对象，并把当前进程等信息保存到 `binder_proc` 对象，该对象管理 IPC 所需的各种信息并拥有其他结构体的根结构体；再把 `binder_proc` 对象保存到文件指针 `filp`，以及把 `binder_proc` 加入到全局链表 `binder_procs`。



Binder 驱动中通过 `static HLIST_HEAD(binder_procs);`，创建了全局的哈希链表 `binder_procs`，用于保存所有的 `binder_proc` 队列，每次新创建的 `binder_proc` 对象都会加入 `binder_procs` 链表中。

2.3 binder_mmap

binder_mmap(文件描述符, 用户虚拟内存空间)

主要功能: 首先在内核虚拟地址空间, 申请一块与用户虚拟内存相同大小的内存; 然后再申请 1 个 page 大小的物理内存, 再将同一块物理内存分别映射到内核虚拟地址空间和用户虚拟内存空间, 从而实现了用户空间的 Buffer 和内核空间的 Buffer 同步操作的功能。

```
static int binder_mmap(struct file *filp, struct vm_area_struct *vma)
{
    int ret;

    struct vm_struct *area; //内核虚拟空间

    struct binder_proc *proc = filp->private_data;

    const char *failure_string;

    struct binder_buffer *buffer; //【见附录 3.9】

    if (proc->tsk != current)
        return -EINVAL;

    if ((vma->vm_end - vma->vm_start) > SZ_4M)
        vma->vm_end = vma->vm_start + SZ_4M; //保证映射内存大小不超过 4M

    mutex_lock(&binder_mmap_lock); //同步锁

    //采用 IOREMAP 方式, 分配一个连续的内核虚拟空间, 与进程虚拟空间大小一致
    area = get_vm_area(vma->vm_end - vma->vm_start, VM_IOREMAP);

    if (area == NULL) {
        ret = -ENOMEM;

        failure_string = "get_vm_area";

        goto err_get_vm_area_failed;
    }

    proc->buffer = area->addr; //指向内核虚拟空间的地址

    //地址偏移量 = 用户虚拟地址空间 - 内核虚拟地址空间

    proc->user_buffer_offset = vma->vm_start - (uintptr_t)proc->buffer;
```

```
mutex_unlock(&binder_mmap_lock); //释放锁

...

//分配物理页的指针数组，数组大小为 vma 的等效 page 个数；
proc->pages = kzalloc(sizeof(proc->pages[0]) * ((vma->vm_end - vma->vm_start)
/ PAGE_SIZE), GFP_KERNEL);

if (proc->pages == NULL) {
    ret = -ENOMEM;
    failure_string = "alloc page array";
    goto err_alloc_pages_failed;
}

proc->buffer_size = vma->vm_end - vma->vm_start;

vma->vm_ops = &binder_vm_ops;
vma->vm_private_data = proc;

//分配物理页面，同时映射到内核空间和进程空间，先分配 1 个物理页 【见小节 2.3.1】
if (binder_update_page_range(proc, 1, proc->buffer, proc->buffer + PAGE_SIZE,
vma)) {
    ret = -ENOMEM;
    failure_string = "alloc small buf";
    goto err_alloc_small_buf_failed;
}

buffer = proc->buffer; //binder_buffer 对象 指向 proc 的 buffer 地址
INIT_LIST_HEAD(&proc->buffers); //创建进程的 buffers 链表头
list_add(&buffer->entry, &proc->buffers); //将 binder_buffer 地址 加入到所属进
程的 buffers 队列
buffer->free = 1;

//将空闲 buffer 放入 proc->free_buffers 中
binder_insert_free_buffer(proc, buffer);

//异步可用空间大小为 buffer 总大小的一半。
```

```
proc->free_async_space = proc->buffer_size / 2;

barrier();

proc->files = get_files_struct(current);

proc->vma = vma;

proc->vma_vm_mm = vma->vm_mm;

return 0;

...// 错误 flags 跳转处, free 释放内存之类的操作

return ret;
}
```

binder_mmap 通过加锁, 保证一次只有一个进程分配内存, 保证多进程间的并发访问。其中 `user_buffer_offset` 是虚拟进程地址与虚拟内核地址的差值(该值为负数)。也就是说同一物理地址, 当内核地址为 `kernel_addr`, 则进程地址为 `proc_addr = kernel_addr + user_buffer_offset`。

2.3.1 binder_update_page_range

```
static int binder_update_page_range(struct binder_proc *proc, int allocate,
    void *start, void *end,
    struct vm_area_struct *vma)
{
    void *page_addr;
    unsigned long user_page_addr;
    struct page **page;
    struct mm_struct *mm; // 内存结构体

    if (vma)
        mm = NULL; // binder_mmap 过程 vma 不为空, 其他情况都为空
    else
        mm = get_task_mm(proc->tsk); // 获取 mm 结构体
}
```

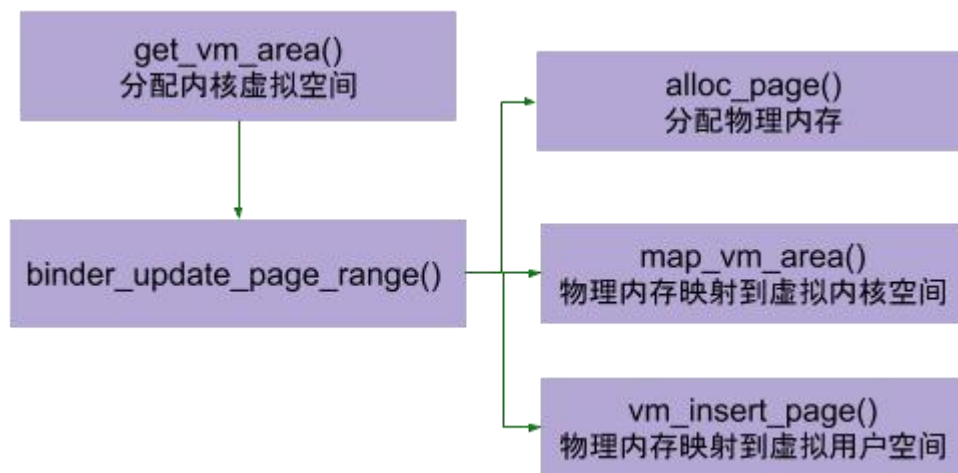
```
if (mm) {  
    down_write(&mm->mmap_sem); //获取 mm_struct 的写信号量  
    vma = proc->vma;  
}  
  
//此处 allocate 为 1, 代表分配过程。如果为 0 则代表释过程  
if (allocate == 0)  
    goto free_range;  
  
for (page_addr = start; page_addr < end; page_addr += PAGE_SIZE) {  
    int ret;  
    page = &proc->pages[(page_addr - proc->buffer) / PAGE_SIZE];  
    //分配一个 page 的物理内存  
    *page = alloc_page(GFP_KERNEL | __GFP_HIGHMEM | __GFP_ZERO);  
    //物理空间映射到虚拟内核空间  
    ret = map_kernel_range_noflush((unsigned long)page_addr,  
        PAGE_SIZE, PAGE_KERNEL, page);  
    flush_cache_vmap((unsigned long)page_addr, (unsigned long)page_addr + PAGE_SIZE);  
    user_page_addr = (uintptr_t)page_addr + proc->user_buffer_offset;  
    //物理空间映射到虚拟进程空间  
    ret = vm_insert_page(vma, user_page_addr, page[0]);  
}  
  
if (mm) {  
    up_write(&mm->mmap_sem); //释放内存的写信号量  
    mmpu(mm); //减少 mm->mm_users 计数  
}
```

```
return 0;

free_range:
... //释放内存的流程

return -ENOMEM;
}
```

主要工作如下：



`binder_update_page_range` 主要完成工作：分配物理空间，将物理空间映射到内核空间，将物理空间映射到进程空间。另外，不同参数下该方法也可以释放物理页面。

`binder_update_page_range` 的调用时机：

- `binder_mmap`: 用于分配内存，分配大小为 1page, `vma` 不为空；
- `binder_alloc_buf`: 用于分配内存，`vma` 为空；
- `binder_free_buf`: 用于释放内存，`vma` 为空；
- `binder_delete_free_buffer`: 同样用于释放内存，`vma` 为空。

关于 `mm_struct` 结构体，定义在 `mm_types.h` 文件：

```
struct mm_struct {
```

```
struct vm_area_struct *mmap; //VMA 列表

struct rb_root mm_rb;

pgd_t * pgd;

atomic_t mm_users; //使用该内存的进程个数

atomic_t mm_count; //结构体 mm_struct 的引用个数

struct rw_semaphore mmap_sem; //读写信号量，用于同步

unsigned long flags;

...

};
```

下面，再说一说 binder_alloc_buf 过程

2.3.2 binder_alloc_buf

通过 binder_alloc_buf()方法来分配 binder_buffer 结构体，只有在 binder_transaction 过程才需要分配 buffer.

```
static struct binder_buffer *binder_alloc_buf(struct binder_proc *proc,
                                              size_t data_size, size_t offsets_size, int is_async)
{
    struct rb_node *n = proc->free_buffers.rb_node;

    struct binder_buffer *buffer;

    size_t buffer_size;

    struct rb_node *best_fit = NULL;

    void *has_page_addr;

    void *end_page_addr;

    size_t size;

    if (proc->vma == NULL) {

        return NULL; //虚拟地址空间为空，直接返回

    }
```

```
size = ALIGN(data_size, sizeof(void *)) + ALIGN(offsets_size, sizeof(void
*));

if (size < data_size || size < offsets_size) {

    return NULL; //非法的 size

}

if (is_async && proc->free_async_space < size + sizeof(struct binder_buffer))
{

    return NULL; // 剩余可用的异步空间，小于所需的大小

}

while (n) { //从 binder_buffer 的红黑树中查找大小相等的 buffer 块

    buffer = rb_entry(n, struct binder_buffer, rb_node);

    buffer_size = binder_buffer_size(proc, buffer);

    if (size < buffer_size) {

        best_fit = n;

        n = n->rb_left;

    } else if (size > buffer_size)

        n = n->rb_right;

    else {

        best_fit = n;

        break;

    }

}

if (best_fit == NULL) {

    return NULL; //内存分配失败，地址空间为空

}

if (n == NULL) {

    buffer = rb_entry(best_fit, struct binder_buffer, rb_node);

    buffer_size = binder_buffer_size(proc, buffer);

}
```



```
    has_page_addr = (void *)(((uintptr_t)buffer->data + buffer_size) & PAGE_MASK);

    if (n == NULL) {

        if (size + sizeof(struct binder_buffer) + 4 >= buffer_size)

            buffer_size = size;

        else

            buffer_size = size + sizeof(struct binder_buffer);

    }

    end_page_addr = (void *)PAGE_ALIGN((uintptr_t)buffer->data + buffer_size);

    if (end_page_addr > has_page_addr)

        end_page_addr = has_page_addr;

    if (binder_update_page_range(proc, 1,

        (void *)PAGE_ALIGN((uintptr_t)buffer->data), end_page_addr, NULL))

        return NULL;

    rb_erase(best_fit, &proc->free_buffers);

    buffer->free = 0;

    binder_insert_allocated_buffer(proc, buffer);

    if (buffer_size != size) {

        struct binder_buffer *new_buffer = (void *)buffer->data + size;

        list_add(&new_buffer->entry, &buffer->entry);

        new_buffer->free = 1;

        binder_insert_free_buffer(proc, new_buffer);

    }

    buffer->data_size = data_size;

    buffer->offsets_size = offsets_size;

    buffer->async_transaction = is_async;

    if (is_async) {

        proc->free_async_space -= size + sizeof(struct binder_buffer);

    }

}
```

```
return buffer;  
}
```

这里介绍的 `binder_alloc_buf` 是内存分配函数。除此之外，还有内存释放相关方法：

- `binder_free_buf`
- `binder_delete_free_buffer`
- `binder_transaction_buffer_release`

这里涉及强弱引用相关函数的操作：

强/弱引用操作函数	功能
<code>binder_inc_ref(ref, 0, NULL)</code>	<code>binder_ref->weak++</code>
<code>binder_inc_ref(ref, 1, NULL)</code>	<code>binder_ref->strong++</code> ，或 <code>binder_node->internal_strong_refs++</code>
<code>binder_dec_ref(&ref, 0)</code>	<code>binder_ref->weak -</code>
<code>binder_dec_ref(&ref, 1)</code>	<code>binder_ref->strong -</code> ，或 <code>binder_node->internal_strong_refs -</code>
<code>binder_dec_node(node, 0, 0)</code>	<code>binder_node->pending_weak_ref = 0</code> ，且 <code>binder_node->local_weak_ref -</code>
<code>binder_dec_node(node, 1, 0)</code>	<code>binder_node->pending_strong_ref = 0</code> ，且 <code>binder_node->local_strong_ref -</code>

2.4 binder_ioctl

`binder_ioctl()`函数负责在两个进程间收发 IPC 数据和 IPC reply 数据。

ioctl(文件描述符, ioctl 命令, 数据类型)

(1) 文件描述符, 是通过 `open()` 方法打开 **Binder Driver** 后返回值;

(2) `ioctl` 命令和数据类型是一体的, 不同的命令对应不同的数据类型

ioctl 命令	数据类型	操作
BINDER_WRITE_READ	struct binder_write_read	收发 Binder IPC 数据
BINDER_SET_MAX_THREADS	__u32	设置 Binder 线程最大个数
BINDER_SET_CONTEXT_MGR	__s32	设置 Service Manager 节点
BINDER_THREAD_EXIT	__s32	释放 Binder 线程
BINDER_VERSION	struct binder_version	获取 Binder 版本信息
BINDER_SET_IDLE_TIMEOUT	__s64	没有使用
BINDER_SET_IDLE_PRIORITY	__s32	没有使用

这些命令中 `BINDER_WRITE_READ` 命令使用率最为频繁, 也是 `ioctl` 最为核心的命令。

源码:

```
static long binder_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    int ret;
```

```
    struct binder_proc *proc = filp->private_data;

    struct binder_thread *thread; // binder 线程

    unsigned int size = _IOC_SIZE(cmd);

    void __user *ubuf = (void __user *)arg;

    //进入休眠状态, 直到中断唤醒

    ret = wait_event_interruptible(binder_user_error_wait, binder_stop_on_user_e
rror < 2);

    if (ret)

        goto err_unlocked;

    binder_lock(__func__);

    //获取 binder_thread【见 2.4.1】

    thread = binder_get_thread(proc);

    if (thread == NULL) {

        ret = -ENOMEM;

        goto err;

    }

    switch (cmd) {

    case BINDER_WRITE_READ: //进行 binder 的读写操作

        ret = binder_ioctl_write_read(filp, cmd, arg, thread); //【见 2.4.2】

        if (ret)

            goto err;

        break;

    case BINDER_SET_MAX_THREADS: //设置 binder 最大支持的线程数

        if (copy_from_user(&proc->max_threads, ubuf, sizeof(proc->max_threads)))
        {

            ret = -EINVAL;

            goto err;

        }

        break;
```

```
case BINDER_SET_CONTEXT_MGR: //成为 binder 的上下文管理者，也就是 ServiceManager
成为守护进程

    ret = binder_ioctl_set_ctx_mgr(filp);

    if (ret)

        goto err;

    break;

case BINDER_THREAD_EXIT: //当 binder 线程退出，释放 binder 线程

    binder_free_thread(proc, thread);

    thread = NULL;

    break;

case BINDER_VERSION: { //获取 binder 的版本号

    struct binder_version _user *ver = ubuf;

    if (size != sizeof(struct binder_version)) {

        ret = -EINVAL;

        goto err;

    }

    if (put_user(BINDER_CURRENT_PROTOCOL_VERSION,

        &ver->protocol_version)) {

        ret = -EINVAL;

        goto err;

    }

    break;

}

default:

    ret = -EINVAL;

    goto err;

}

ret = 0;

err:
```

```
if (thread)
    thread->looper &= ~BINDER_LOOPER_STATE_NEED_RETURN;
    binder_unlock(__func__);
    wait_event_interruptible(binder_user_error_wait, binder_stop_on_user_error <
2);

err_unlocked:
    trace_binder_ioctl_done(ret);
    return ret;
}
```

执行 ioctl 过程，便需要加上 binder lock.

2.4.1 binder_get_thread

从 binder_proc 中查找 binder_thread,如果当前线程已经加入到 proc 的线程队列则直接返回,如果不存在则创建 binder_thread,并将当前线程添加到当前的 proc

```
static struct binder_thread *binder_get_thread(struct binder_proc *proc)
{
    struct binder_thread *thread = NULL;
    struct rb_node *parent = NULL;
    struct rb_node **p = &proc->threads.rb_node;

    while (*p) { //根据当前进程的 pid, 从 binder_proc 中查找相应的 binder_thread
        parent = *p;
        thread = rb_entry(parent, struct binder_thread, rb_node);
        if (current->pid < thread->pid)
            p = &(*p)->rb_left;
        else if (current->pid > thread->pid)
            p = &(*p)->rb_right;
        else
```

```
        break;
    }

    if (*p == NULL) {
        thread = kzalloc(sizeof(*thread), GFP_KERNEL); //新建 binder_thread 结构体
        if (thread == NULL)
            return NULL;

        binder_stats_created(BINDER_STAT_THREAD);
        thread->proc = proc;
        thread->pid = current->pid; //保存当前进程(线程)的 pid
        init_waitqueue_head(&thread->wait);
        INIT_LIST_HEAD(&thread->todo);
        rb_link_node(&thread->rb_node, parent, p);
        rb_insert_color(&thread->rb_node, &proc->threads);
        thread->looper |= BINDER_LOOPER_STATE_NEED_RETURN;
        thread->return_error = BR_OK;
        thread->return_error2 = BR_OK;
    }

    return thread;
}
```

2.4.2 binder_ioctl_write_read

对于 `ioctl()` 方法中, 传递进来的命令是 `cmd = BINDER_WRITE_READ` 时执行该方法, `arg` 是一个 `binder_write_read` 结构体

```
static int binder_ioctl_write_read(struct file *filp,
                                   unsigned int cmd, unsigned long arg,
                                   struct binder_thread *thread)
{
    int ret = 0;
```

```
    struct binder_proc *proc = filp->private_data;

    unsigned int size = _IOC_SIZE(cmd);

    void __user *ubuf = (void __user *)arg;

    struct binder_write_read bwr;

    if (size != sizeof(struct binder_write_read)) {
        ret = -EINVAL;
        goto out;
    }

    if (copy_from_user(&bwr, ubuf, sizeof(bwr))) { //把用户空间数据 ubuf 拷贝到 bwr
        ret = -EFAULT;
        goto out;
    }

    if (bwr.write_size > 0) {
        //当写缓存中有数据, 则执行 binder 写操作
        ret = binder_thread_write(proc, thread,
                                   bwr.write_buffer, bwr.write_size, &bwr.write_consumed);
        trace_binder_write_done(ret);
        if (ret < 0) { //当写失败, 再将 bwr 数据写回用户空间, 并返回
            bwr.read_consumed = 0;
            if (copy_to_user(ubuf, &bwr, sizeof(bwr)))
                ret = -EFAULT;
            goto out;
        }
    }

    if (bwr.read_size > 0) {
        //当读缓存中有数据, 则执行 binder 读操作
        ret = binder_thread_read(proc, thread,
                                   bwr.read_buffer, bwr.read_size, &bwr.read_consumed,
```



```
        filp->f_flags & O_NONBLOCK);

    trace_binder_read_done(ret);

    if (!list_empty(&proc->todo))

        wake_up_interruptible(&proc->wait); //唤醒等待状态的线程

    if (ret < 0) { //当读失败，再将 bwr 数据写回用户空间，并返回

        if (copy_to_user(ubuf, &bwr, sizeof(bwr)))

            ret = -EFAULT;

        goto out;

    }

}

}

if (copy_to_user(ubuf, &bwr, sizeof(bwr))) { //将内核数据 bwr 拷贝到用户空间 ubu
f
    ret = -EFAULT;

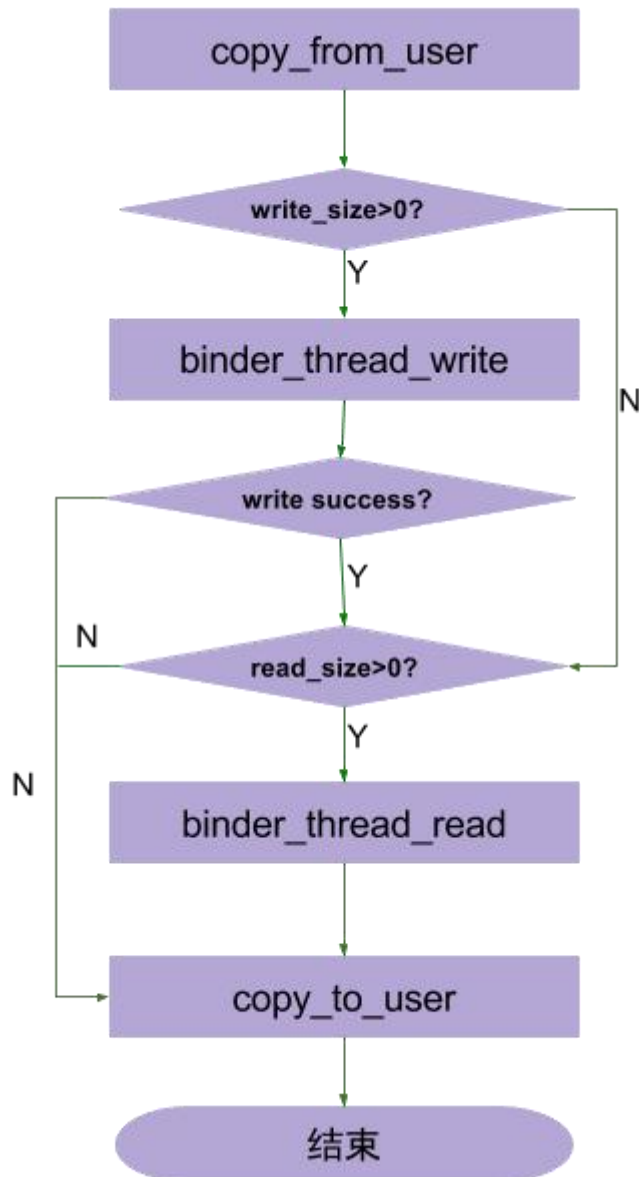
    goto out;

}out:

return ret;

}
```

对于 `binder_ioctl_write_read` 的流程图，如下：



流程：

- 首先，把用户空间数据 ubuf 拷贝到内核空间 bwr；
- 当 bwr 写缓存有数据，则执行 binder_thread_write；当写失败则将 bwr 数据写回用户空间并退出；
- 当 bwr 读缓存有数据，则执行 binder_thread_read；当读失败则再将 bwr 数据写回用户空间并退出；
- 最后，把内核数据 bwr 拷贝到用户空间 ubuf。

这里涉及两个核心方法 `binder_thread_write()` 和 `binder_thread_read()` 方法，在 Binder 系列的后续文章 [Binder Driver](#) 再探中详细介绍。

2.4 Command 使用场景

ioctl 命令常见命令的使用场景，其中 BINDER_WRITE_READ 最为频繁

- BINDER_WRITE_READ
 - Binder 读写交互场景，IPC.talkWithDriver
- BINDER_SET_CONTEXT_MGR
 - servicemanager 进程成为上下文管理者，binder_become_context_manager()
- BINDER_SET_MAX_THREADS
 - 初始化 ProcessState 对象，open_driver()
 - 主动调整参数，ProcessState.setThreadPoolMaxThreadCount()
- BINDER_VERSION
 - 初始化 ProcessState 对象，open_driver()

2.5 小节

- binder_init: 初始化字符设备;
- binder_open: 打开驱动设备，过程需要持有 binder_main_lock 同步锁;
- binder_mmap: 申请内存空间，该过程需要持有 binder_mmap_lock 同步锁;
- binder_ioctl: 执行相应的 ioctl 操作，该过程需要持有 binder_main_lock 同步锁;
 - 当处于 binder_thread_read 过程，read_buffer 无数据则释放同步锁，并处于 wait_event_freezable 过程，等有数据到来则唤醒并尝试持有同步锁。

三、附录

下面列举 Binder 驱动相关的一些重要结构体

3.0 结构体列表

序号	结构体	名称	解释
----	-----	----	----

序号	结构体	名称	解释
1	binder_proc	binder 进程	每个进程调用 open() 打开 binder 驱动都会创建该结构体，用于管理 IPC 所需的各种信息
2	binder_thread	binder 线程	对应于上层的 binder 线程
3	binder_node	binder 实体	对应于 BBinder 对象，记录 BBinder 的进程、指针、引用计数等
4	binder_ref	binder 引用	对应于 BpBinder 对象，记录 BpBinder 的引用计数、死亡通知、BBinder 指针等
5	binder_ref_death	binder 死亡引用	记录 binder 死亡的引用信息
6	binder_write_read	binder 读写	记录 buffer 中读和写的数据信息
7	binder_transaction_data	binder 事务数据	记录传输数据内容，比如发送方 pid/uid，RPC 数据
8	flat_binder_object	binder 扁平对象	Binder 对象在两个进程间传递的扁平结构
9	binder_buffer	binder 内存	调用 mmap() 创建用于 Binder 传输数据的缓存区
10	binder_transaction	binder 事务	记录传输事务的发送方和接收方线程、进程等
11	binder_work	binder 工作	记录 binder 工作类型
12	binder_state	binder 状态	

6~9 用于数据传输相关，其中 `binder_write_read`，`binder_transaction_data` 进程空间和内核空间是通用的。

BWR 核心数据图表

`binder_write_read` 是整个 Binder IPC 过程，最为核心的数据结构之一。

3.1 binder_proc

`binder_proc` 结构体：用于管理 IPC 所需的各种信息，拥有其他结构体的结构体。

类型	成员变量	解释
<code>struct hlist_node</code>	<code>proc_node</code>	进程节点
<code>struct rb_root</code>	<code>threads</code>	<code>binder_thread</code> 红黑树的根节点
<code>struct rb_root</code>	<code>nodes</code>	<code>binder_node</code> 红黑树的根节点
<code>struct rb_root</code>	<code>refs_by_desc</code>	<code>binder_ref</code> 红黑树的根节点 (以 <code>handle</code> 为 key)
<code>struct rb_root</code>	<code>refs_by_node</code>	<code>binder_ref</code> 红黑树的根节点 (以 <code>ptr</code> 为 key)
<code>int</code>	<code>pid</code>	相应进程 id
<code>struct vm_area_struct *</code>	<code>vma</code>	指向进程虚拟地址空间的指针

类型	成员变量	解释
struct mm_struct *	vma_vm_mm	相应进程的内存结构体
struct task_struct *	tsk	相应进程的 task 结构体
struct files_struct *	files	相应进程的文件结构体
struct hlist_node	deferred_work_node	
int	deferred_work	
void *	buffer	内核空间的起始地址
ptrdiff_t	user_buffer_offset	内核空间与用户空间的地址偏移量
struct list_head	buffers	所有的 buffer
struct rb_root	free_buffers	空闲的 buffer
struct rb_root	allocated_buffers	已分配的 buffer
size_t	free_async_space	异步的可用空闲空间大小
struct page **	pages	指向物理内存页指针的指针
size_t	buffer_size	映射的内核空间大小
uint32_t	buffer_free	可用内存总大小
struct list_head	todo	进程将要做的事
wait_queue_head_t	wait	等待队列

类型	成员变量	解释
struct binder_stats	stats	binder 统计信息
struct list_head	delivered_death	已分发的死亡通知
int	max_threads	最大线程数
int	requested_threads	请求的线程数
int	requested_threads_started	已启动的请求线程数
int	ready_threads	准备就绪的线程个数
long	default_priority	默认优先级
struct dentry *	debugfs_entry	

- **free_buffers**: 记录所有空闲的 **buffer**，记录以 **buffer_size** 为 key 的 **binder_buffer** 的红黑树结构
- **allocated_buffers**: 记录所有已分配的 **buffer**，记录以 **buffer_size** 为 key 的 **binder_buffer** 的红黑树结构
- **buffers**: 所有 **buffer**（包含空闲的和已分配的 **buffer**）的按地址由从低到高都连入到 **buffers** 链表中
- **ready_threads**: 准备就绪的线程个数，往往是指进入 **binder_thread_read()**，处于休眠等待状态的线程个数；**ready_threads** 线程个数越多，代表系统越空闲。
- **requested_threads_started**: 是指系统已经启动的线程个数，在方法 **binder_thread_write()** 中，执行一次 **BC_REGISTER_LOOPER**，则 **requested_threads_started++**，**requested_threads--**；上限为 **max_threads.BC_REGISTER_LOOPER** 次数与 **requested_threads_started** 个数应该相等；
- **requested_threads**: 请求的线程个数，在方法 **binder_thread_read()** 中，当同时满足 **requested_threads_started** 小于最大线程数，没有 **ready_threads** 线程，且 **requested_threads=0**，则执行 **requested_threads++**。可见 **requested_threads** 取值要么为 0，要么为 1。

3.2 binder_thread

binder_thread 结构体代表当前 binder 操作所在的线程

类型	成员变量	解释
struct binder_proc *	proc	线程所属的进程
struct rb_node	rb_node	
int	pid	线程 pid
int	looper	looper 的状态
struct binder_transaction *	transaction_stack	线程正在处理的事务
struct list_head	todo	将要处理的链表
uint32_t	return_error	write 失败后, 返回的错误码
uint32_t	return_error2	write 失败后, 返回的错误码 2
wait_queue_head_t	wait	等待队列的队头
struct binder_stats	stats	binder 线程的统计信息

looper 的状态如下:

```
enum {  
    BINDER_LOOPER_STATE_REGISTERED = 0x01, // 创建注册线程 BC_REGISTER_LOOPER  
    BINDER_LOOPER_STATE_ENTERED    = 0x02, // 创建主线程 BC_ENTER_LOOPER  
    BINDER_LOOPER_STATE_EXITED     = 0x04, // 已退出  
    BINDER_LOOPER_STATE_INVALID    = 0x08, // 非法  
    BINDER_LOOPER_STATE_WAITING    = 0x10, // 等待中  
    BINDER_LOOPER_STATE_NEED_RETURN = 0x20, // 需要返回  
};
```


binder_thread_write()过程:

- 收到 BC_REGISTER_LOOPER, 则线程状态为 BINDER_LOOPER_STATE_REGISTERED;
- 收到 BC_ENTER_LOOPER, 则线程状态为 BINDER_LOOPER_STATE_ENTERED;
- 收到 BC_EXIT_LOOPER, 则线程状态为 BINDER_LOOPER_STATE_EXITED;

其他 3 个状态的时机:

- BINDER_LOOPER_STATE_WAITING:
 - 当停留在 binder_thread_read()的 wait_event_xxx 过程, 则设置该状态;
- BINDER_LOOPER_STATE_NEED_RETURN:
 - binder_get_thread()过程, 根据 binder_proc 查询不到当前线程所对应的 binder_thread, 会新建 binder_thread 对象;
 - binder_deferred_flush()过程;
- BINDER_LOOPER_STATE_INVALID:
 - 当 binder_thread 创建过程状态不正确时会设置.

3.3 binder_node

binder_node 代表一个 binder 实体

类型	成员变量	解释
int	debug_id	节点创建时分配, 具有全局唯一性, 用于调试使用
struct binder_work	work	
struct rb_node	rb_node	binder 节点正常使用, union

类型	成员变量	解释
struct hlist_node	dead_node	binder 节点已销毁, union
struct binder_proc *	proc	binder 所在的进程
struct hlist_head	refs	所有指向该节点的 binder 引用队列
int	internal_strong_refs	
int	local_weak_refs	
int	local_strong_refs	
binder_uintptr_t	ptr	指向用户空间 binder_node 的指针
binder_uintptr_t	cookie	附件数据
unsigned	has_strong_ref	占位 1bit
unsigned	pending_strong_ref	占位 1bit
unsigned	has_weak_ref	占位 1bit
unsigned	pending_weak_ref	占位 1bit
unsigned	has_async_transaction	占位 1bit
unsigned	accept_fds	占位 1bit
unsigned	min_priority	占位 8bit, 最小优先级
struct list_head	async_todo	异步 todo 队列

binder_node 有一个联合类型：

```
union {  
    struct rb_node rb_node;  
    struct hlist_node dead_node;  
};
```

当 Binder 对象已销毁，但还存在该 Binder 节点引用，则采用 dead_node，并加入到全局列表 binder_dead_nodes；否则使用 rb_node 节点。

另外：

- binder_node.ptr 对应于 flat_binder_object.binder；
- binder_node.cookie 对应于 flat_binder_object.cookie。

3.4 binder_ref

类型	成员变量	解释
int	debug_id	用于调试使用
struct rb_node	rb_node_desc	以 desc 为索引的红黑树
struct rb_node	rb_node_node	以 node 为索引的红黑树
struct hlist_node	node_entry	
struct binder_proc *	proc	binder 进程
struct binder_node *	node	binder 节点
uint32_t	desc	handle

类型	成员变量	解释
int	strong	强引用次数
int	weak	弱引用次数
struct binder_ref_death *	death	当应用注册死亡通知时，此域不为空

binder 引用的查询方式如下：

- node + proc => ref (transaction)
- desc + proc => ref (transaction, inc/dec ref)
- node => refs + procs (proc exit)

3.5 binder_ref_death

```
struct binder_ref_death {  
    struct binder_work work;  
    binder_uintptr_t cookie;  
};
```

cookie 只是死亡通知的 BpBinder 代理对象的指针

3.6 binder_write_read

用户空间程序和 Binder 驱动程序交互基本都是通过 BINDER_WRITE_READ 命令，来进行数据的读写操作。

类型	成员变量	解释
binder_size_t	write_size	write_buffer 的总字节数

类型	成员变量	解释
binder_size_t	write_consumed	write_buffer 已消费的字节数
binder_uintptr_t	write_buffer	写缓冲数据的指针
binder_size_t	read_size	read_buffer 的总字节数
binder_size_t	read_consumed	read_buffer 已消费的字节数
binder_uintptr_t	read_buffer	读缓存数据的指针

- **write_buffer** 变量：用于发送 IPC(或 IPC reply)数据，即传递经由 Binder Driver 的数据时使用。
- **read_buffer** 变量：用于接收来自 Binder Driver 的数据，即 Binder Driver 在接收 IPC(或 IPC reply)数据后，保存到 read_buffer，再传递到用户空间；

write_buffer 和 read_buffer 都是包含 Binder 协议命令和 binder_transaction_data 结构体。

- copy_from_user()将用户空间 IPC 数据拷贝到内核态 binder_write_read 结构体；
- copy_to_user()将用内核态 binder_write_read 结构体数据拷贝到用户空间；

3.7 binder_transaction_data

当 BINDER_WRITE_READ 命令的目标是本地 Binder node 时，target 使用 ptr，否则使用 handle。只有当这是 Binder node 时，cookie 才有意义，表示附加数据，由进程自己解释。

```
struct binder_transaction_data {  
    union {  
        __u32 handle; //binder_ref (即 handle)  
    };  
};
```

```
binder_uintptr_t ptr;    //Binder_node 的内存地址

} target; //RPC 目标

binder_uintptr_t cookie; //BBinder 指针

__u32 code; //RPC 代码，代表 Client 与 Server 双方约定的命令码

__u32 flags; //标志位，比如 TF_ONE_WAY 代表异步，即不等待 Server 端回复

pid_t sender_pid; //发送端进程的 pid

uid_t sender_euid; //发送端进程的 uid

binder_size_t data_size; //data 数据的总大小

binder_size_t offsets_size; //IPC 对象的大小

union {
    struct {
        binder_uintptr_t buffer; //数据区起始地址
        binder_uintptr_t offsets; //数据区 IPC 对象偏移量
    } ptr;
    __u8 buf[8];
} data; //RPC 数据

};
```

- **target**: 对于 BpBinder 则使用 handle，对于 BBinder 则使用 ptr，故使用 union 数据类型来表示；
- **code**: 比如注册服务过程 code 为 ADD_SERVICE_TRANSACTION，又比如获取服务 code 为 CHECK_SERVICE_TRANSACTION
- **data**: 代表整个数据区，其中 data.ptr 指向的是传递给 Binder 驱动的数据区的起始地址，data.offsets 指的是数据区中 IPC 数据地址的偏移量。
- **cookie**: 记录着 BBinder 指针。
- **data_size**: 代表本次传输的 parcel 数据的大小；
- **offsets_size**: 代表传递的 IPC 对象的大小；根据这个可以推测出传递了多少个 binder 对象。
 - 对于 64 位 IPC，一个 IPC 对象大小等于 8；
 - 对于 32 位 IPC，一个 IPC 对象大小等于 4；

3.8 flat_binder_object

flat_binder_object 结构体代表 Binder 对象在两个进程间传递的扁平结构。

类型	成员变量	解释
__u32	type	类型
__u32	flags	记录优先级、文件描述符许可
binder_uintptr_t	binder	(union) 当传递的是 binder_node 时使用, 指向 binder_node 在应用程序的地址
__u32	handle	(union) 当传递的是 binder_ref 时使用, 存放 Binder 在进程中的引用号
binder_uintptr_t	cookie	只对 binder_node 有效, 存放 binder_node 的额外数据

此处的类型 type 的可能取值来自于 enum, 成员如下:

成员变量	解释
BINDER_TYPE_BINDER	binder_node 的强引用
BINDER_TYPE_WEAK_BINDER	binder_node 的弱引用
BINDER_TYPE_HANDLE	binder_ref 强引用
BINDER_TYPE_WEAK_HANDLE	binder_ref 弱引用
BINDER_TYPE_FD	binder 文件描述符

说明:

- 当 `type` 等于 `BINDER_TYPE_BINDER` 或 `BINDER_TYPE_WEAK_BINDER` 类型时, 代表 `Server` 进程向 `ServiceManager` 进程注册服务, 则创建 `binder_node` 对象;
- 当 `type` 等于 `BINDER_TYPE_HANDLE` 或 `BINDER_TYPE_WEAK_HANDLE` 类型时, 代表 `Client` 进程向 `Server` 进程请求代理, 则创建 `binder_ref` 对象;
- 当 `type` 等于 `BINDER_TYPE_FD` 类型时, 代表进程向另一个进程发送文件描述符, 只打开文件, 则无需创建任何对象。

3.9 binder_buffer

每一次 `Binder` 传输数据时, 都会先从 `Binder` 内存缓存区中分配一个 `binder_buffer` 来存储传输数据。

类型	成员变量	解释
<code>struct list_head</code>	<code>entry</code>	buffer 实体的地址
<code>struct rb_node</code>	<code>rb_node</code>	buffer 实体的地址
<code>unsigned</code>	<code>free</code>	标记是否是空闲 buffer, 占位 1bit
<code>unsigned</code>	<code>allow_user_free</code>	是否允许用户释放, 占位 1bit
<code>unsigned</code>	<code>async_transaction</code>	占位 1bit
<code>unsigned</code>	<code>debug_id</code>	占位 29bit
<code>struct binder_transaction *</code>	<code>transaction</code>	该缓存区的需要处理的事务
<code>struct binder_node *</code>	<code>target_node</code>	该缓存区所需处理的 Binder 实体

类型	成员变量	解释
size_t	data_size	数据大小
size_t	offsets_size	数据偏移量
uint8_t	data[0]	数据地址

每一个 binder_buffer 分为空闲和已分配的，通过 free 标记来区分。空闲和已分配的 binder_buffer 通过各自的成员变量 rb_node 分别连入 binder_proc 的 free_buffers(红黑树)和 allocated_buffers(红黑树)。

3.10 binder_transaction

类型	成员变量	解释
int	debug_id	用于调试
struct binder_work	work	binder 工作类型
struct binder_thread *	from	发送端线程
struct binder_transaction *	from_parent	上一个事务
struct binder_proc *	to_proc	接收端进程
struct binder_thread *	to_thread	接收端线程
struct binder_transaction *	to_parent	下一个事务
unsigned	need_reply	是否需要回复
struct binder_buffer *	buffer	数据 buffer

类型	成员变量	解释
unsigned int	code	通信方法，比如 startService
unsigned int	flags	标志，比如是否 oneway
long	priority	优先级
long	saved_priority	保存的优先级
kuid_t	sender_euid	发送端 uid

执行 binder_transaction()过程创建的结构体

- debug_id: 是一个全局静态变量，每当创建一个 binder_transaction 或 binder_node 或 binder_ref 对象，则++debug_id
- from 与 to_thread 是一对，分别是发送端线程和接收端线程；
- from_parent 与 to_parent 是一对，分别是上一个和下一个 binder_transaction，组成一个链表。
 - 执行 binder_transaction() 方法过程，当非 oneway 的 BC_TRANSACTION 时，则设置当前事务 t->from_parent 等于当前线程的 transaction_stack；
 - 执行 binder_thread_read() 方法过程，当非 oneway 的 BR_TRANSACTION 时，则设置当前事务 t->to_parent 等于当前线程的 transaction_stack；

3.11 binder_work

```
struct binder_work {  
    struct list_head entry;  
    enum {  
        BINDER_WORK_TRANSACTION = 1,  
        BINDER_WORK_TRANSACTION_COMPLETE,  
        BINDER_WORK_NODE,  
    }  
};
```

```
BINDER_WORK_DEAD_BINDER,  
  
BINDER_WORK_DEAD_BINDER_AND_CLEAR,  
  
BINDER_WORK_CLEAR_DEATH_NOTIFICATION,  
  
} type;  
  
};
```

binder_work.type 设置时机:

- binder_transaction()
- binder_thread_write()
- binder_new_node()

3.12 binder_state

类型	成员变量	解释
int	fd	文件描述符
void *	mapped	映射到进程空间的起始地址
size_t	mapsize	内存空间的映射大小

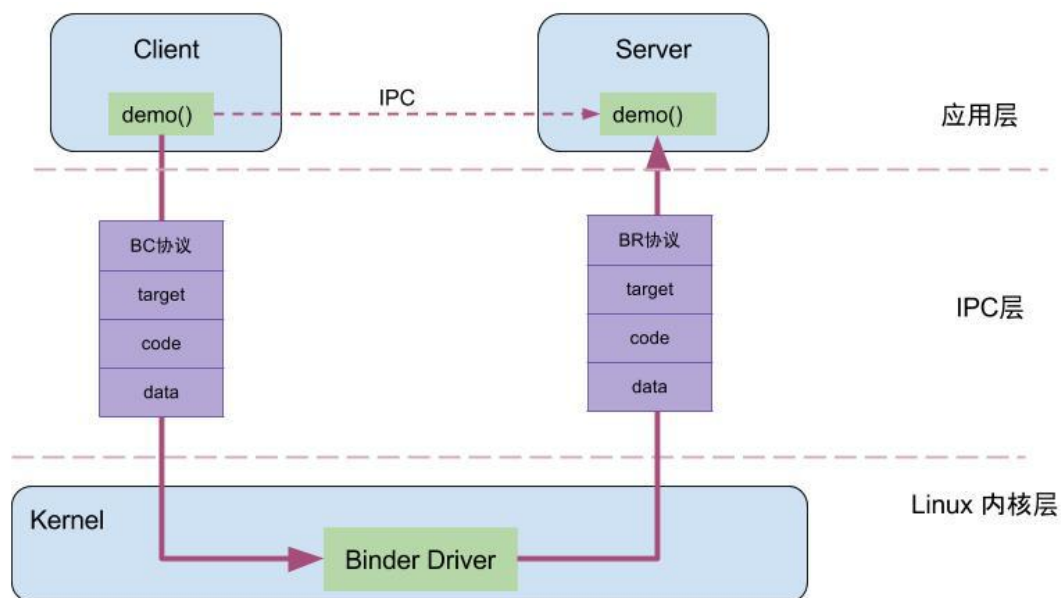
第三节 Binder Driver 再探

基于 Android 6.0 的源码剖析，在讲解 Binder 原理之前，先从 kernel 的角度来讲解 Binder Driver。

一、Binder 通信简述

上一篇文章 [Binder Driver 初探](#) 介绍了 Binder 驱动的 `init`、`open`、`mmap`、`ioctl` 这 4 个核心方法，并说明与 Binder 相关的常见结构体。

Client 进程通过 RPC(Remote Procedure Call Protocol)与 Server 通信，可以简单地划分为三层，驱动层、IPC 层、业务层。`demo()`便是 Client 端和 Server 共同协商好的统一方法；`handle`、RPC 数据、代码、协议这 4 项组成了 IPC 层的数据，通过 IPC 层进行数据传输；而真正在 Client 和 Server 两端建立通信的基础设施便是 Binder Driver。

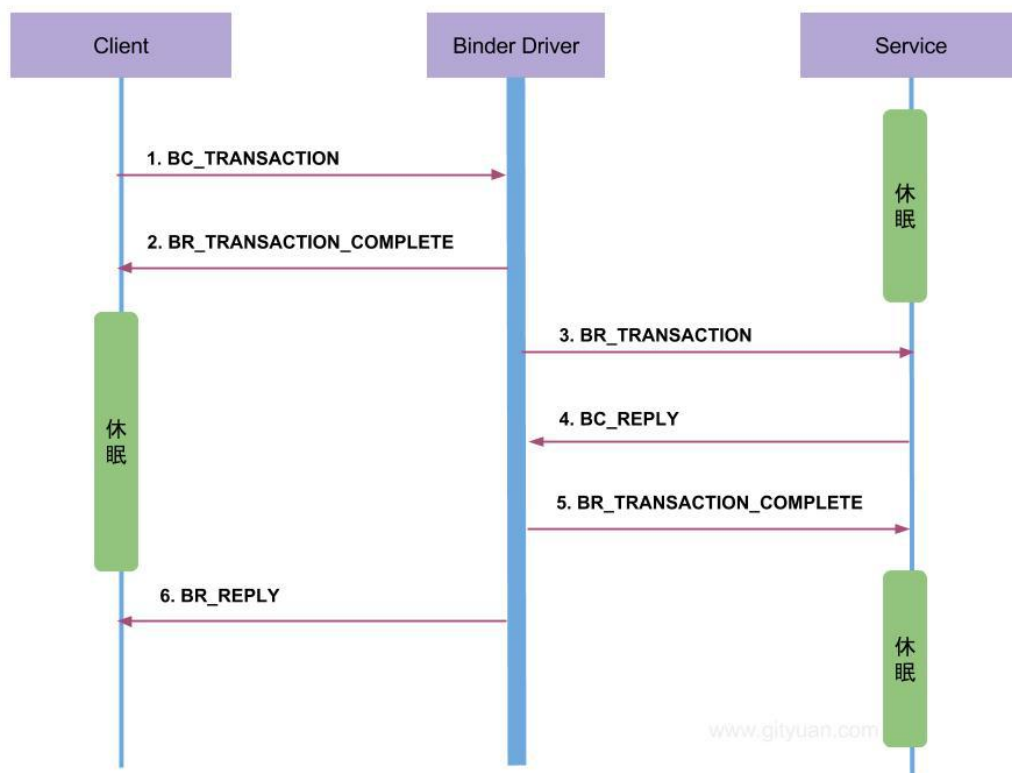


例如，当名为 `BatteryStatsService` 的 Client 向 `ServiceManager` 注册服务的过程中，IPC 层的数据组成为：`Handle=0`，RPC 代码为 `ADD_SERVICE_TRANSACTION`，RPC 数据为 `BatteryStatsService`，Binder 协议为 `BC_TRANSACTION`。

二、Binder 通信协议

2.1 通信模型

先列举一次完整的 Binder 通信过程：



Binder 协议包含在 IPC 数据中，分为两类：

1. **BINDER_COMMAND_PROTOCOL**：binder 请求码，以"BC_"开头，简称 BC 码，用于从 IPC 层传递到 Binder Driver 层；
2. **BINDER_RETURN_PROTOCOL**：binder 响应码，以"BR_"开头，简称 BR 码，用于从 Binder Driver 层传递到 IPC 层；

Binder IPC 通信至少是两个进程的交互：

- client 进程执行 `binder_thread_write`，根据 BC_XXX 命令，生成相应的 `binder_work`；
- server 进程执行 `binder_thread_read`，根据 `binder_work.type` 类型，生成 BR_XXX，发送到用户空间处理。

2.1.1 通信过程

其中 `binder_work.type` 共有 6 种类型：

```
BINDER_WORK_TRANSACTION //最常见类型
BINDER_WORK_TRANSACTION_COMPLETE
BINDER_WORK_NODE
BINDER_WORK_DEAD_BINDER
BINDER_WORK_DEAD_BINDER_AND_CLEAR
BINDER_WORK_CLEAR_DEATH_NOTIFICATION
```

2.2 binder_thread_write

请求处理过程是通过 `binder_thread_write()` 方法，该方法用于处理 Binder 协议中的请求码。当 `binder_buffer` 存在数据，`binder` 线程的写操作循环执行。

```
binder_thread_write(){
    while (ptr < end && thread->return_error == BR_OK) {
        get_user(cmd, (uint32_t __user *)ptr); //获取 IPC 数据中的 Binder 协议(BC 码)
        switch (cmd) {
            case BC_INCREFS: ...
            case BC_ACQUIRE: ...
            case BC_RELEASE: ...
            case BC_DECREFS: ...
            case BC_INCREFS_DONE: ...
            case BC_ACQUIRE_DONE: ...
            case BC_FREE_BUFFER: ... break;
            case BC_TRANSACTION:
            case BC_REPLY: {
                struct binder_transaction_data tr;
                copy_from_user(&tr, ptr, sizeof(tr)); //拷贝用户空间 tr 到内核
                // 【见小节 2.2.1】
```

```
binder_transaction(proc, thread, &tr, cmd == BC_REPLY);  
  
break;  
  
case BC_REGISTER_LOOPER: ...  
case BC_ENTER_LOOPER: ...  
case BC_EXIT_LOOPER: ...  
case BC_REQUEST_DEATH_NOTIFICATION: ...  
case BC_CLEAR_DEATH_NOTIFICATION: ...  
case BC_DEAD_BINDER_DONE: ...  
}  
}  
}  
}
```

对于请求码为 `BC_TRANSACTION` 或 `BC_REPLY` 时,会执行 `binder_transaction()`方法,这是最为频繁的操作。 对于其他命令则不同。

2.2.1 binder_transaction

```
static void binder_transaction(struct binder_proc *proc,  
                               struct binder_thread *thread,  
                               struct binder_transaction_data *tr, int reply){  
    //根据各种判定, 获取以下信息:  
    struct binder_thread *target_thread; //目标线程  
    struct binder_proc *target_proc;     //目标进程  
    struct binder_node *target_node;     //目标 binder 节点  
    struct list_head *target_list;       //目标 TODO 队列  
    wait_queue_head_t *target_wait;     //目标等待队列  
    ...  
  
    //分配两个结构体内存
```

```
struct binder_transaction *t = kzalloc(sizeof(*t), GFP_KERNEL);

struct binder_work *tcomplete = kzalloc(sizeof(*tcomplete), GFP_KERNEL);

//从 target_proc 分配一块 buffer【见小节 3.2】

t->buffer = binder_alloc_buf(target_proc, tr->data_size,

for (; offp < off_end; offp++) {

    switch (fp->type) {

        case BINDER_TYPE_BINDER: ...

        case BINDER_TYPE_WEAK_BINDER: ...

        case BINDER_TYPE_HANDLE: ...

        case BINDER_TYPE_WEAK_HANDLE: ...

        case BINDER_TYPE_FD: ...

    }

}

//向目标进程的 target_list 添加 BINDER_WORK_TRANSACTION 事务

t->work.type = BINDER_WORK_TRANSACTION;

list_add_tail(&t->work.entry, target_list);

//向当前线程的 todo 队列添加 BINDER_WORK_TRANSACTION_COMPLETE 事务

tcomplete->type = BINDER_WORK_TRANSACTION_COMPLETE;

list_add_tail(&tcomplete->entry, &thread->todo);

if (target_wait)

    wake_up_interruptible(target_wait);

return;

}
```

路由过程: handle -> ref -> target_node -> target_proc

- reply 的过程会找到 target_thread;
- 非 reply 则一般找到 target_proc;
- 对特殊的嵌套 binder call 会根据 transaction_stack 来决定是插入事务到目标线程还是目标进程。

2.2.2 BC_PROTOCOL

binder 请求码，是用 `enum binder_driver_command_protocol` 来定义的，是用于应用程序向 binder 驱动设备发送请求消息，应用程序包含 Client 端和 Server 端，以 BC_ 开头，总 17 条；(-代表目前不支持的请求码)

请求码	参数类型	作用
BC_TRANSACTION	binder_transaction_data	Client 向 Binder 驱动发送请求数据
BC_REPLY	binder_transaction_data	Server 向 Binder 驱动发送请求数据
BC_FREE_BUFFER	binder_uintptr_t (指针)	释放内存
BC_INCREFS	__u32(descriptor)	binder_ref 弱引用加 1 操作
BC_DECREFS	__u32(descriptor)	binder_ref 弱引用减 1 操作
BC_ACQUIRE	__u32(descriptor)	binder_ref 强引用加 1 操作
BC_RELEASE	__u32(descriptor)	binder_ref 强引用减 1 操作
BC_ACQUIRE_DONE	binder_ptr_cookie	binder_node 强引用减 1 操作
BC_INCREFS_DONE	binder_ptr_cookie	binder_node 弱引用减 1 操作
BC_REGISTER_LOOPER	无参数	创建新的 looper 线程
BC_ENTER_LOOPER	无参数	应用线程进入 looper

请求码	参数类型	作用
BC_EXIT_LOOPER	无参数	应用线程退出 looper
BC_REQUEST_DEATH_NOTIFICATION	binder_handle_cookie	注册死亡通知
BC_CLEAR_DEATH_NOTIFICATION	binder_handle_cookie	取消注册的死亡通知
BC_DEAD_BINDER_DONE	binder_uintptr_t (指针)	已完成 binder 的死亡通知
BC_ACQUIRE_RESULT	-	-
BC_ATTEMPT_ACQUIRE	-	-

1. BC_FREE_BUFFER:

- 通过 `mmap()` 映射内存，其中 `ServiceManager` 映射的空间大小为 128K，其他 `Binder` 应用进程映射的内存大小为 1M-8K。
- `Binder` 驱动基于这块映射的内存采用最佳匹配算法来动态分配和释放，通过 `binder_buffer` 结构体中的 `free` 字段来表示相应的 `buffer` 是空闲还是已分配状态。对于已分配的 `buffers` 加入到 `binder_proc` 中的 `allocated_buffers` 红黑树；对于空闲的 `buffers` 加入到 `binder_proc` 中的 `free_buffers` 红黑树。
- 当应用程序需要内存时，根据所需内存大小从 `free_buffers` 中找到最合适的内存，并放入 `allocated_buffers` 树；当应用程序处理完后必须尽快使用 `BC_FREE_BUFFER` 命令来释放该 `buffer`，从而添加回到 `free_buffers` 树。

2. `BC_INCREFS`、`BC_ACQUIRE`、`BC_RELEASE`、`BC_DECREFS` 等请求码的作用是对 `binder` 的强/弱引用的计数操作，用于实现强/弱指针的功能。

3. 参数类型主要有以下几类：

- `binder_transaction_data`（结构体）
- `binder_ptr_cookie`（由 `binder` 指针和 `cookie` 组成）
- `binder_uintptr_t`（指针）
- `__u32`（无符号整型）

4. `Binder` 线程创建与退出：

- BC_ENTER_LOOPER: binder 主线程(由应用层发起)的创建会向驱动发送该消息; joinThreadPool()过程创建 binder 主线程;
- BC_REGISTER_LOOPER: Binder 用于驱动层决策而创建新的 binder 线程; joinThreadPool()过程,创建非 binder 主线程;
- BC_EXIT_LOOPER: 退出 Binder 线程, 对于 binder 主线程是不能退出;joinThreadPool()的过程出现 timeout,并且非 binder 主线程,则会退出该 binder 线程;

2.3 binder_thread_read

响应处理过程是通过 `binder_thread_read()` 方法, 该方法根据不同的 `binder_work->type` 以及不同状态, 生成相应的响应码。

```
binder_thread_read () {
    wait_for_proc_work = thread->transaction_stack == NULL &&
        list_empty(&thread->todo);

    //根据 wait_for_proc_work 来决定 wait 在当前线程还是进程的等待队列
    if (wait_for_proc_work) {
        ret = wait_event_freezable_exclusive(proc->wait, binder_has_proc_work(proc, thread));
        ...
    } else {
        ret = wait_event_freezable(thread->wait, binder_has_thread_work(thread));
        ...
    }
}

while (1) {
    //当&thread->todo 和&proc->todo 都为空时, goto 到 retry 标志处, 否则往下执行:
    struct binder_transaction_data tr;
    struct binder_transaction *t = NULL;
    switch (w->type) {
        case BINDER_WORK_TRANSACTION: ...
```

```
        case BINDER_WORK_TRANSACTION_COMPLETE: ...
        case BINDER_WORK_NODE: ...
        case BINDER_WORK_DEAD_BINDER: ...
        case BINDER_WORK_DEAD_BINDER_AND_CLEAR: ...
        case BINDER_WORK_CLEAR_DEATH_NOTIFICATION: ...
    }
    ...
}

done:
    *consumed = ptr - buffer;

    //当满足请求线程加已准备线程数等于 0，已启动线程数小于最大线程数(15)，
    //且 looper 状态为已注册或已进入时创建新的线程。

    if (proc->requested_threads + proc->ready_threads == 0 &&
        proc->requested_threads_started < proc->max_threads &&
        (thread->looper & (BINDER_LOOPER_STATE_REGISTERED |
            BINDER_LOOPER_STATE_ENTERED))) {
        proc->requested_threads++;

        // 生成 BR_SPAWN_LOOPER 命令，用于创建新的线程
        put_user(BR_SPAWN_LOOPER, (uint32_t __user *)buffer);
    }

    return 0;
}
```

说明：

当 transaction 堆栈为空，且线程 todo 链表为空，且 non_block=false 时，意味着没有任何事务需要处理的，会进入等待客户端请求的状态。当有事务需要处理时便会进入循环处理过程，并生成相应的响应码。在 Binder 驱动层，只有在进入 binder_thread_read()方法时，同时满足以下条件，才会生成 BR_SPAWN_LOOPER 命令，当用户态进程收到该命令则会创建新线程：

1. binder_proc 的 requested_threads 线程数为 0;

2. binder_proc 的 ready_threads 线程数为 0;
3. binder_proc 的 requested_threads_started 个数小于 15(即最大线程个数);
4. binder_thread 的 loop 状态为 BINDER_LOOPER_STATE_REGISTERED 或 BINDER_LOOPER_STATE_ENTERED。

那么在哪里处理响应码呢？通过前面的 Binder 通信协议图，可以知道处理响应码的过程是在用户态处理，即后续文章会讲到的用户空间 IPCThreadState 类中的 IPCThreadState::waitForResponse()和 IPCThreadState::executeCommand()两个方法共同处理 Binder 协议中的 18 个响应码。

2.3.1 BR_PROTOCOL

binder 响应码，是用 enum binder_driver_return_protocol 来定义的，是 binder 设备向应用程序回复的消息，应用程序包含 Client 端和 Server 端，以 BR_开头，总 18 条；

响应码	参数类型	作用
-----	------	----

响应码	参数类型	作用
BR_ERROR	__s32	操作发生错误
BR_OK	无参数	操作完成

响应码	参数类型	作用
BR_NOOP	无参数	不做任何事
BR_SPAWN_LOOPER	无参数	创建新的 Looper 线程

响应码	参数类型	作用
BR_TRANSACTION	binder_transaction_data	Binder 驱动向 Server 端发送请求数据
BR_REPLY	binder_transaction_data	Binder 驱动向 Client 端发送回复数据

响应码	参数类型	作用
BR_TRANSACTION_COMPLETE	无参数	对请求发送的成功反馈
BR_DEAD_REPLY	无参数	回复失败，往往是线程或节点为空

响应码	参数类型	作用
BR_FAILED_REPLY	无参数	回复失败，往往是 transaction 出错导致

响应码	参数类型	作用
BR_INCREFS	binder_ptr_cookie	binder_ref 弱引用加 1 操作（Server 端）

响应码	参数类型	作用
BR_DECREFS	binder_ptr_cookie	binder_ref 弱引用减 1 操作（Server 端）

响应码	参数类型	作用
BR_ACQUIRE	binder_ptr_cookie	binder_ref 强引用加 1 操作（Server 端）

响应码	参数类型	作用
BR_RELEASE	binder_ptr_cookie	binder_ref 强引用减 1 操作（Server 端）

响应码	参数类型	作用
BR_DEAD_BINDER	binder_uintptr_t(指针)	Binder 驱动向 client 端 发送死亡通知

响应码	参数类型	作用
BR_CLEAR_DEATH_NOTIFICATION_DONE	binder_uintptr_t(指针)	BC_CLEAR_DEATH_NOTIFICATION 命令对应的响应码
BR_ACQUIRE_RESULT	—	—

响应码	参数类型	作用
BR_ATTEMPT_ACQUIRE	-	-
BR_FINISHED	-	-

BR_SPAWN_LOOPER: binder 驱动已经检测到进程中没有线程等待即将到来的事务。那么当一个进程接收到这条命令时，该进程必须创建一条新的服务线程并注册该线程，在接下来的响应过程会看到何时生成该响应码。

BR_TRANSACTION_COMPLETE: 当 Client 端向 Binder 驱动发送 BC_TRANSACTION 命令后，Client 会收到 BR_TRANSACTION_COMPLETE 命令，告知 Client 端请求命令发送成功；对于 Server 向 Binder 驱动发送 BC_REPLY 命令后，Server 端会收到 BR_TRANSACTION_COMPLETE 命令，告知 Server 端请求回应命令发送成功。

BR_DEAD_REPLY: 当应用层向 Binder 驱动发送 Binder 调用时，若 Binder 应用层的另一个端已经死亡，则驱动回应 BR_DEAD_BINDER 命令。

BR_FAILED_REPLY: 当应用层向 Binder 驱动发送 Binder 调用时，若 transaction 出错，比如调用的函数号不存在，则驱动回应 BR_FAILED_REPLY。

三. 场景总结

3.1 BC 协议使用场景

BC 协议	调用方法
BC_TRANSACTION	IPC.transact()
BC_REPLY	IPC.sendReply()
BC_FREE_BUFFER	IPC.freeBuffer()
BC_REQUEST_DEATH_NOTIFICATION	IPC.requestDeathNotification()
BC_CLEAR_DEATH_NOTIFICATION	IPC.clearDeathNotification()
BC_DEAD_BINDER_DONE	IPC.execute()

binder_thread_write()根据不同的 BC 协议而执行不同的流程。其中 BC_TRANSACTION 和 BC_REPLY 协议，会进入 binder_transaction()过程。

2.4.2 binder_work 类型

- BINDER_WORK_TRANSACTION
 - binder_transaction()
 - binder_release_work()
- BINDER_WORK_TRANSACTION_COMPLETE
 - binder_transaction()
 - binder_release_work()
- BINDER_WORK_NODE
 - binder_new_node()
- BINDER_WORK_DEAD_BINDER
 - binder_thread_write() , 收到 BC_REQUEST_DEATH_NOTIFICATION
- BINDER_WORK_DEAD_BINDER_AND_CLEAR
 - binder_thread_write() , 收到 BC_CLEAR_DEATH_NOTIFICATION
- BINDER_WORK_CLEAR_DEATH_NOTIFICATION
 - binder_thread_write() , 收到 BC_CLEAR_DEATH_NOTIFICATION
 - binder_thread_write(), 收到 BC_DEAD_BINDER_DONE

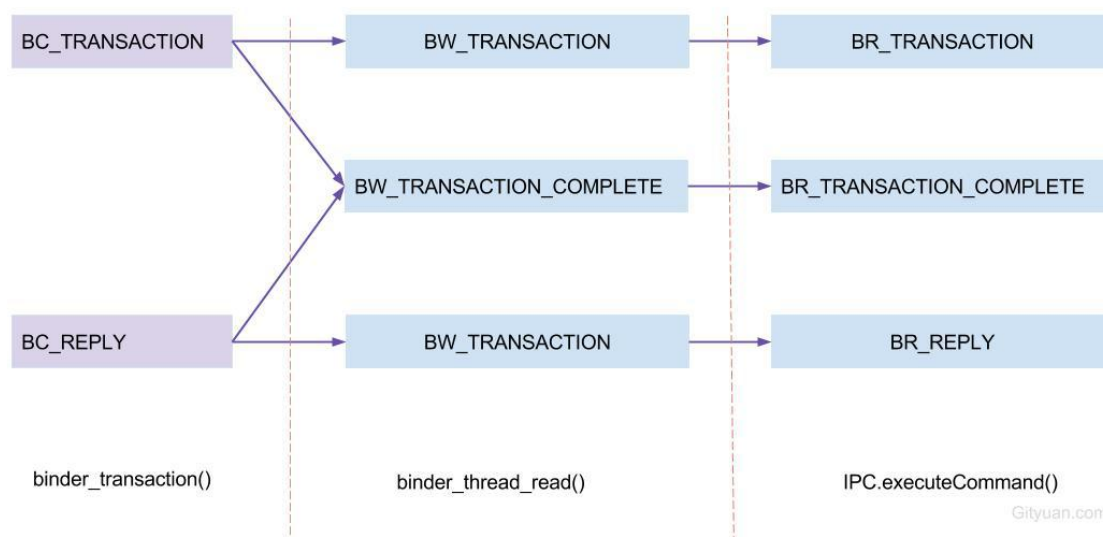
3.2 BR 协议使用场景

BC 协议	触发时机
BR_TRANSACTION	收到 BINDER_WORK_TRANSACTION
BR_REPLY	收到 BINDER_WORK_TRANSACTION
BR_TRANSACTION_COMPLETE	收到 BINDER_WORK_TRANSACTION_COMPLETE

BC 协议	触发时机
BR_DEAD_BINDER	收到 BINDER_WORK_DEAD_BINDER 或 BINDER_WORK_DEAD_BINDER_AND_CLEAR
BR_CLEAR_DEATH_NOTIFICATION_DONE	收到 BINDER_WORK_CLEAR_DEATH_NOTIFICATION

BR_DEAD_REPLY, BR_FAILED_REPLY, BR_ERROR 这些都是失败或错误相关的应答协议

3.3 协议转换图



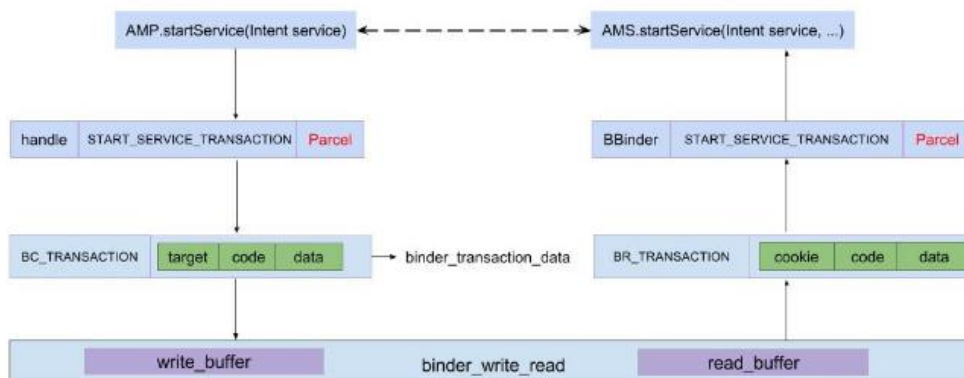
图解: (以 BC_TRANSACTION 为例)

- 发起端进程: binder_transaction()过程将 BC_TRANSACTION 转换为 BW_TRANSACTION;
- 接收端进程: binder_thread_read()过程, 将 BW_TRANSACTION 转换为 BR_TRANSACTION;
- 接收端进程: IPC.execute()过程, 处理 BR_TRANSACTION 命令。

注: BINDER_WORK_xxx -> BW_xxx

更多录播视频+架构学习资料免费领取 QQ 3607409029

3.4 数据转换图



图(左)说明:

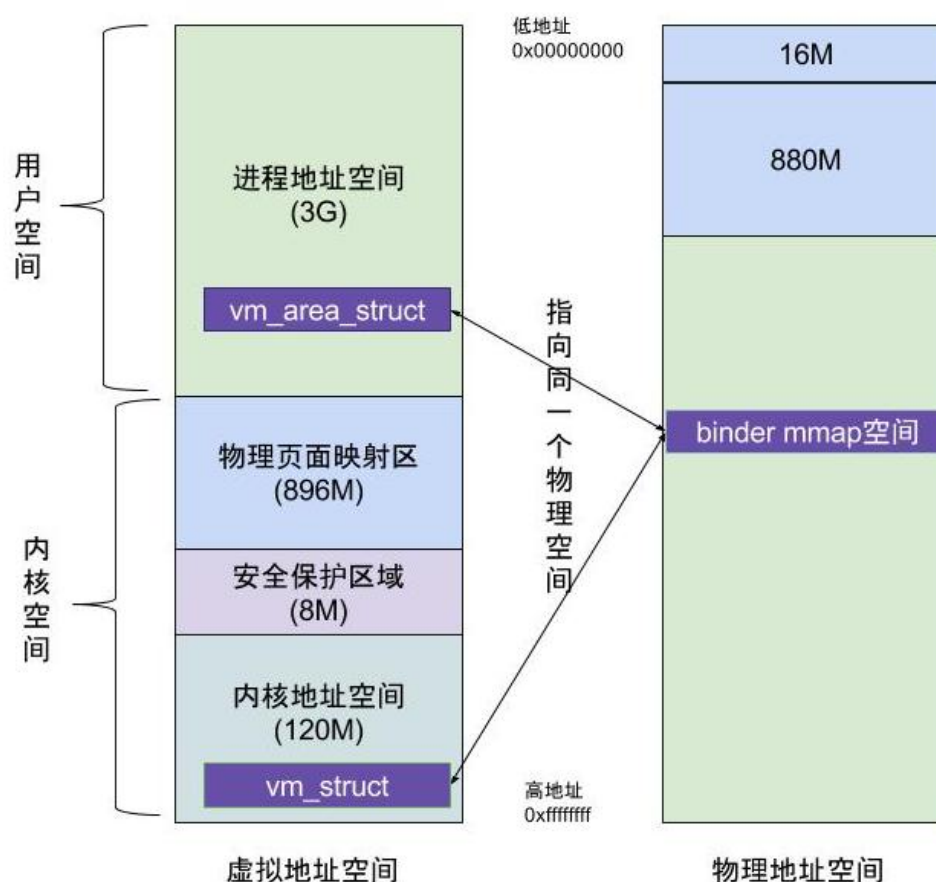
- `AMP.startService`: 将数据封装到 `Parcel` 类型;
- `IPC.writeTransactionData`: 将数据封装到 `binder_transaction_data` 结构体;
- `IPC.talkWithDriver`: 将数据进一步封装到 `binder_write_read` 结构体;
 - 再通过 `ioctl()` 写入命令 `BINDER_WRITE_READ` 和 `binder_write_read` 结构体到驱动层
- `binder_transaction`: 将发起端数据拷贝到接收端进程的 `buffer` 结构体;

图(右)说明:

- `binder_thread_read`: 根据 `binder_transaction` 结构体和 `binder_buffer` 结构体数据生成新的 `binder_transaction_data` 结构体, 写入 `bwr` 的 `write_buffer`, 传递到用户空间。
- `IPC.executeCommand`: 解析 `binder_transaction_data` 数据, 找到目标 `BBinder` 并调用其 `transact()` 方法;
- `AMN.onTransact`: 解析 `Parcel` 数据, 然后调用目标服务的目标方法;
- `AMS.startService`: 层层封装和拆分后, 执行真正的业务逻辑。

四、Binder 内存机制

在上一篇文章从代码角度阐释了 `binder_mmap()`，这也是 Binder 进程间通信效率高的核心机制所在，如下图：

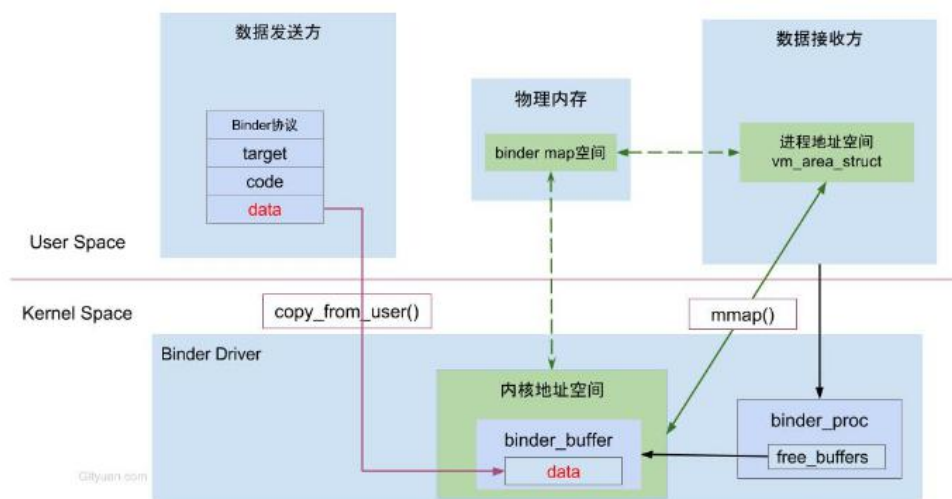


虚拟进程地址空间(`vm_area_struct`)和虚拟内核地址空间(`vm_struct`)都映射到同一块物理内存空间。当 Client 端与 Server 端发送数据时，Client（作为数据发送端）先从自己的进程空间把 IPC 通信数据 `copy_from_user` 拷贝到内核空间，而 Server 端（作为数据接收端）与内核共享数据，不再需要拷贝数据，而是通过内存地址空间的偏移量，即可获悉内存地址，整个过程只发生一次内存拷贝。一般地做法，需要 Client 端进程空间拷贝到内核空间，再由内核空间拷贝到 Server 进程空间，会发生两次拷贝。

对于进程和内核虚拟地址映射到同一个物理内存的操作是发生在数据接收端，而数据发送端还是需要将用户态的数据复制到内核态。到此，可能有读者会好奇，为何不直接让发送端和接收端直接映射到同一个物理空间，那样就连一次复制的操作都不需要了，0 次复制操作那就与 Linux 标准内核的共享内存的 IPC 机制没有区别了，对于共享内存虽然效率高，但是对于多进程的同步问题比较复杂，而管道/消息队列等 IPC 需要复制 2 两次，效率较低。这里就不先展开讨论 Linux

现有的各种 IPC 机制跟 Binder 的详细对比，总之 Android 选择 Binder 的基于速度和安全性的考虑。

下面这图是从 Binder 在进程间数据通信的流程图，从图中更能明了 Binder 的内存转移关系。



第四节 Binder 启动 ServiceManager

基于 Android 6.0 的源码剖析， 本文详细地讲解了 ServiceManager 启动流程

```
framework/native/cmds/servicemanager/
```

```
- service_manager.c
```

```
- binder.c
```

```
■
```

```
kernel/drivers/ (不同 Linux 分支路径略有不同)
```

```
- staging/android/binder.c
```

```
- android/binder.c
```

一. 概述

ServiceManager 是 Binder IPC 通信过程中的守护进程，本身也是一个 Binder 服务，但并没有采用 libbinder 中的多线程模型来与 Binder 驱动通信，而是自行

编写了 `binder.c` 直接和 `Binder` 驱动来通信，并且只有一个循环 `binder_loop` 来进行读取和处理事务，这样的好处是简单而高效。

`ServiceManager` 本身工作相对简单，其功能：查询和注册服务。对于 `Binder IPC` 通信过程中，其实更多的情形是 `BpBinder` 和 `BBinder` 之间的通信，比如 `ActivityManagerProxy` 和 `ActivityManagerService` 之间的通信等。

1.1 流程图

启动过程主要以下几个阶段：

1. 打开 `binder` 驱动：`binder_open`;
2. 注册成为 `binder` 服务的大管家：`binder_become_context_manager`;
3. 进入无限循环，处理 `client` 端发来的请求：`binder_loop`;

二. 启动过程

`ServiceManager` 是由 `init` 进程通过解析 `init.rc` 文件而创建的，其所对应的可执行程序 `/system/bin/servicemanager`，所对应的源文件是 `service_manager.c`，进程名为 `/system/bin/servicemanager`。

```
service servicemanager /system/bin/servicemanager
```

```
class core
```

```
user system
```

```
group system
```

```
critical
```

```
onrestart restart healthd
```

```
onrestart restart zygote
```

```
onrestart restart media
```

```
onrestart restart surfaceflinger
```

```
onrestart restart drm
```


启动 Service Manager 的入口函数是 service_manager.c 中的 main()方法,代码如下:

2.1 main

[-> service_manager.c]

```
int main(int argc, char **argv) {  
  
    struct binder_state *bs;  
  
    //打开 binder 驱动, 申请 128k 字节大小的内存空间 【见小节 2.2】  
    bs = binder_open(128*1024);  
  
    ...  
  
    //成为上下文管理者 【见小节 2.3】  
    if (binder_become_context_manager(bs)) {  
        return -1;  
    }  
  
    selinux_enabled = is_selinux_enabled(); //selinux 权限是否使能  
    sehandle = selinux_android_service_context_handle();  
    selinux_status_open(true);  
  
    if (selinux_enabled > 0) {  
        if (sehandle == NULL) {  
            abort(); //无法获取 sehandle  
        }  
  
        if (getcon(&service_manager_context) != 0) {  
            abort(); //无法获取 service_manager 上下文  
        }  
    }  
  
    ...  
}
```

```
//进入无限循环，处理 client 端发来的请求 【见小节 2.4】  
  
binder_loop(bs, svcmgr_handler);  
  
return 0;  
  
}
```

2.2 binder_open

[-> servicemanager/binder.c]

```
struct binder_state *binder_open(size_t mapsize)  
{  
    struct binder_state *bs; 【见小节 2.2.1】  
    struct binder_version vers;  
  
    bs = malloc(sizeof(*bs));  
    if (!bs) {  
        errno = ENOMEM;  
        return NULL;  
    }  
  
    //通过系统调用陷入内核，打开 Binder 设备驱动  
    bs->fd = open("/dev/binder", O_RDWR);  
    if (bs->fd < 0) {  
        goto fail_open; // 无法打开 binder 设备  
    }  
  
    //通过系统调用，ioctl 获取 binder 版本信息  
    if ((ioctl(bs->fd, BINDER_VERSION, &vers) == -1) ||  
        (vers.protocol_version != BINDER_CURRENT_PROTOCOL_VERSION)) {
```

```
goto fail_open; //内核空间与用户空间的 binder 不是同一版本
}

bs->mapsize = mapsize;

//通过系统调用，mmap 内存映射，mmap 必须是 page 的整数倍
bs->mapped = mmap(NULL, mapsize, PROT_READ, MAP_PRIVATE, bs->fd, 0);

if (bs->mapped == MAP_FAILED) {

goto fail_map; // binder 设备内存无法映射
}

return bs;

fail_map:
close(bs->fd);
fail_open:
free(bs);
return NULL;
}
```

打开 binder 驱动相关操作:

先调用 `open()` 打开 binder 设备，`open()` 方法经过系统调用，进入 Binder 驱动，然后调用方法 `binder_open()`，该方法会在 Binder 驱动层创建一个 `binder_proc` 对象，再将 `binder_proc` 对象赋值给 `fd->private_data`，同时放入全局链表 `binder_procs`。再通过 `ioctl()` 检验当前 binder 版本与 Binder 驱动层的版本是否一致。

调用 `mmap()` 进行内存映射，同理 `mmap()` 方法经过系统调用，对应于 Binder 驱动层的 `binder_mmap()` 方法，该方法会在 Binder 驱动层创建 `Binder_buffer` 对象，并放入当前 `binder_proc` 的 `proc->buffers` 链表。

2.2.1 binder_state

[-> servicemanager/binder.c]

```
struct binder_state
{
    int fd; // dev/binder 的文件描述符
    void *mapped; //指向 mmap 的内存地址
    size_t mapsize; //分配的内存大小，默认为 128KB
};
```

2.3 binder_become_context_manager

[-> servicemanager/binder.c]

```
int binder_become_context_manager(struct binder_state *bs){
    //通过 ioctl, 传递 BINDER_SET_CONTEXT_MGR 指令【见小节 2.3.1】
    return ioctl(bs->fd, BINDER_SET_CONTEXT_MGR, 0);
}
```

成为上下文的管理者，整个系统中只有一个这样的管理者。通过 `ioctl()` 方法经过系统调用，对应于 Binder 驱动层的 `binder_ioctl()` 方法。

2.3.1 binder_ioctl

[-> kernel/drivers/android/binder.c]

```
static long binder_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) {
    binder_lock(__func__);
    switch (cmd) {
        case BINDER_SET_CONTEXT_MGR:
            ret = binder_ioctl_set_ctx_mgr(filp); //【见小节 2.3.2】
    }
```

```
        break;
    }
    case ...:
    }
    binder_unlock(__func__);
}
```

根据参数 `BINDER_SET_CONTEXT_MGR`，最终调用 `binder_ioctl_set_ctx_mgr()` 方法，这个过程会持有 `binder_main_lock`。

2.3.2 binder_ioctl_set_ctx_mgr

[-> kernel/drivers/android/binder.c]

```
static int binder_ioctl_set_ctx_mgr(struct file *filp)
{
    int ret = 0;
    struct binder_proc *proc = filp->private_data;
    kuid_t curr_euid = current_euid();

    //保证只创建一次 mgr_node 对象
    if (binder_context_mgr_node != NULL) {
        ret = -EBUSY;
        goto out;
    }

    if (uid_valid(binder_context_mgr_uid)) {
        ...
    } else {
        //设置当前线程 euid 作为 Service Manager 的 uid
        binder_context_mgr_uid = curr_euid;
    }
}
```

```
}

//创建 ServiceManager 实体【见小节 2.3.3】
binder_context_mgr_node = binder_new_node(proc, 0, 0);
...
binder_context_mgr_node->local_weak_refs++;
binder_context_mgr_node->local_strong_refs++;
binder_context_mgr_node->has_strong_ref = 1;
binder_context_mgr_node->has_weak_ref = 1;out:
return ret;
}
```

进入 binder 驱动，在 Binder 驱动中定义的静态变量

```
// service manager 所对应的 binder_node;static struct binder_node *binder_context_mgr_node;
// 运行 service manager 的线程uidstatic kuid_t binder_context_mgr_uid = INVALID_UID;
```

创建了全局的 binder_node 对象 `binder_context_mgr_node`，并将 `binder_context_mgr_node` 的强弱引用各加 1。

2.3.3 binder_new_node

[-> kernel/drivers/android/binder.c]

```
static struct binder_node *binder_new_node(struct binder_proc *proc,
binder_uintptr_t ptr,
binder_uintptr_t cookie)
{
struct rb_node **p = &proc->nodes.rb_node;
struct rb_node *parent = NULL;
```

```
    struct binder_node *node;

    //首次进来为空
    while (*p) {
        parent = *p;
        node = rb_entry(parent, struct binder_node, rb_node);

        if (ptr < node->ptr)
            p = &(*p)->rb_left;
        else if (ptr > node->ptr)
            p = &(*p)->rb_right;
        else
            return NULL;
    }

    //给新创建的 binder_node 分配内核空间
    node = kzalloc(sizeof(*node), GFP_KERNEL);
    if (node == NULL)
        return NULL;

    binder_stats_created(BINDER_STAT_NODE);
    // 将新创建的 node 对象添加到 proc 红黑树;
    rb_link_node(&node->rb_node, parent, p);
    rb_insert_color(&node->rb_node, &proc->nodes);
    node->debug_id = ++binder_last_id;
    node->proc = proc;
    node->ptr = ptr;
    node->cookie = cookie;
    node->work.type = BINDER_WORK_NODE; //设置 binder_work 的 type
    INIT_LIST_HEAD(&node->work.entry);
    INIT_LIST_HEAD(&node->async_todo);

    return node;
```

}

在 Binder 驱动层创建 `binder_node` 结构体对象，并将当前 `binder_proc` 加入到 `binder_node` 的 `node->proc`。并创建 `binder_node` 的 `async_todo` 和 `binder_work` 两个队列。

2.4 binder_loop

[-> servicemanager/binder.c]

```
void binder_loop(struct binder_state *bs, binder_handler func) {  
  
    int res;  
  
    struct binder_write_read bwr;  
  
    uint32_t readbuf[32];  
  
    bwr.write_size = 0;  
    bwr.write_consumed = 0;  
    bwr.write_buffer = 0;  
  
    readbuf[0] = BC_ENTER_LOOPER;  
  
    //将 BC_ENTER_LOOPER 命令发送给 binder 驱动，让 Service Manager 进入循环 【见小节 2.4.1】  
  
    binder_write(bs, readbuf, sizeof(uint32_t));  
  
    for (;;) {  
  
        bwr.read_size = sizeof(readbuf);  
        bwr.read_consumed = 0;  
        bwr.read_buffer = (uintptr_t) readbuf;  
  
        res = ioctl(bs->fd, BINDER_WRITE_READ, &bwr); //进入循环，不断地 binder 读写过程  
  
        if (res < 0) {
```



```
        break;
    }

    // 解析 binder 信息 【见小节 2.5】
    res = binder_parse(bs, 0, (uintptr_t) readbuf, bwr.read_consumed, func);
    if (res == 0) {
        break;
    }
    if (res < 0) {
        break;
    }
}
}
```

进入循环读写操作，由 `main()` 方法传递过来的参数 `func` 指向 `svcmgr_handler`。

`binder_write` 通过 `ioctl()` 将 `BC_ENTER_LOOPER` 命令发送给 `binder` 驱动，此时 `bwr` 只有 `write_buffer` 有数据，进入 `binder_thread_write()` 方法。接下来进入 `for` 循环，执行 `ioctl()`，此时 `bwr` 只有 `read_buffer` 有数据，那么进入 `binder_thread_read()` 方法。

2.4.1 binder_write

[-> servicemanager/binder.c]

```
int binder_write(struct binder_state *bs, void *data, size_t len) {
    struct binder_write_read bwr;

    int res;

    bwr.write_size = len;
    bwr.write_consumed = 0;
    bwr.write_buffer = (uintptr_t) data; //此处 data 为 BC_ENTER_LOOPER
```

```
bwr.read_size = 0;

bwr.read_consumed = 0;

bwr.read_buffer = 0;

//【见小节 2.4.2】

res = ioctl(bs->fd, BINDER_WRITE_READ, &bwr);

return res;

}
```

根据传递进来的参数，初始化 `bwr`，其中 `write_size` 大小为 4，`write_buffer` 指向缓冲区的起始地址，其内容为 `BC_ENTER_LOOPER` 请求协议号。通过 `ioctl` 将 `bwr` 数据发送给 `binder` 驱动，则调用其 `binder_ioctl` 方法，如下：

2.4.2 binder_ioctl

[-> kernel/drivers/android/binder.c]

```
static long binder_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    int ret;

    struct binder_proc *proc = filp->private_data;

    struct binder_thread *thread;

    ret = wait_event_interruptible(binder_user_error_wait, binder_stop_on_user_error < 2);

    ...

    binder_lock(__func__);

    thread = binder_get_thread(proc); //获取 binder_thread

    switch (cmd) {

        case BINDER_WRITE_READ: //进行 binder 的读写操作

            ret = binder_ioctl_write_read(filp, cmd, arg, thread); //【见小节 2.4.3】

            if (ret)
```

```
        goto err;

        break;

        case ...
    }

    ret = 0;

err:
    if (thread)
        thread->looper &= ~BINDER_LOOPER_STATE_NEED_RETURN;

    binder_unlock(__func__);

    ...

    return ret;
}
```

2.4.3 binder_ioctl_write_read

[-> kernel/drivers/android/binder.c]

```
static int binder_ioctl_write_read(struct file *filp,
                                   unsigned int cmd, unsigned long arg,
                                   struct binder_thread *thread)
{
    int ret = 0;

    struct binder_proc *proc = filp->private_data;

    void __user *ubuf = (void __user *)arg;

    struct binder_write_read bwr;

    if (copy_from_user(&bwr, ubuf, sizeof(bwr))) { //把用户空间数据 ubuf 拷贝到 bwr

        ret = -EFAULT;

        goto out;
    }
```

```
}

    if (bwr.write_size > 0) { //此时写缓存有数据【见小节 2.4.4】
        ret = binder_thread_write(proc, thread,
                                   bwr.write_buffer, bwr.write_size, &bwr.write_consumed);
        ...
    }

    if (bwr.read_size > 0) { //此时读缓存无数据
        ...
    }

    if (copy_to_user(ubuf, &bwr, sizeof(bwr))) { //将内核数据 bwr 拷贝到用户空间 ubuf
        ret = -EFAULT;
        goto out;
    }out:
    return ret;
}
```

此处将用户空间的 binder_write_read 结构体 拷贝到内核空间.

2.4.4 binder_thread_write

[-> kernel/drivers/android/binder.c]

```
static int binder_thread_write(struct binder_proc *proc, struct binder_thread *
thread, binder_uintptr_t binder_buffer, size_t size, binder_size_t *consumed) {
    uint32_t cmd;

    void __user *buffer = (void __user *) (uintptr_t) binder_buffer;

    void __user *ptr = buffer + *consumed;
```

```
void __user *end = buffer + size;

while (ptr < end && thread->return_error == BR_OK) {
    get_user(cmd, (uint32_t __user *)ptr); //获取命令
    switch (cmd) {
        case BC_ENTER_LOOPER:
            //设置该线程的 looper 状态
            thread->looper |= BINDER_LOOPER_STATE_ENTERED;
            break;
        case ...;
    }
}
```

从 bwr.write_buffer 拿出 cmd 数据,此处为 BC_ENTER_LOOPER. 可见上层本次调用 binder_write()方法, 主要是完成设置当前线程的 looper 状态为 BINDER_LOOPER_STATE_ENTERED。

2.5 binder_parse

[-> servicemanager/binder.c]

```
int binder_parse(struct binder_state *bs, struct binder_io *bio,
                 uintptr_t ptr, size_t size, binder_handler func)
{
    int r = 1;
    uintptr_t end = ptr + (uintptr_t) size;

    while (ptr < end) {
        uint32_t cmd = *(uint32_t *) ptr;
        ptr += sizeof(uint32_t);
        switch(cmd) {
```

```
case BR_NOOP: //无操作，退出循环

    break;

case BR_TRANSACTION_COMPLETE:

    break;

case BR_INCREFS:

case BR_ACQUIRE:

case BR_RELEASE:

case BR_DECREFS:

    ptr += sizeof(struct binder_ptr_cookie);

    break;

case BR_TRANSACTION: {

    struct binder_transaction_data *txn = (struct binder_transaction_data
    *) ptr;

    ...

    binder_dump_txn(txn);

    if (func) {

        unsigned rdata[256/4];

        struct binder_io msg;

        struct binder_io reply;

        int res;

        // 【见小节 2.5.1】

        bio_init(&reply, rdata, sizeof(rdata), 4);

        bio_init_from_txn(&msg, txn); //从 txn 解析出 binder_io 信息

        // 【见小节 2.6】

        res = func(bs, txn, &msg, &reply);

        // 【见小节 3.4】

        binder_send_reply(bs, &reply, txn->data.ptr.buffer, res);

    }

    ptr += sizeof(*txn);

    break;

}
```

```
    }  
  
    case BR_REPLY: {  
        struct binder_transaction_data *txn = (struct binder_transaction_data *) ptr;  
        ...  
        binder_dump_txn(txn);  
        if (bio) {  
            bio_init_from_txn(bio, txn);  
            bio = 0;  
        }  
        ptr += sizeof(*txn);  
        r = 0;  
        break;  
    }  
  
    case BR_DEAD_BINDER: {  
        struct binder_death *death = (struct binder_death *) (uintptr_t) *(binder_uintptr_t *) ptr;  
        ptr += sizeof(binder_uintptr_t);  
        // binder 死亡消息【见小节 3.3】  
        death->func(bs, death->ptr);  
        break;  
    }  
  
    case BR_FAILED_REPLY:  
        r = -1;  
        break;  
  
    case BR_DEAD_REPLY:  
        r = -1;  
        break;  
  
    default:  
        return -1;  
    }  
}
```

```
    }  
  
    return r;  
  
}
```

解析 binder 信息，此处参数 ptr 指向 BC_ENTER_LOOPER，func 指向 svcmgr_handler。故有请求到来，则调用 svcmgr_handler。

2.5.1 bio_init

[-> servicemanager/binder.c]

```
void bio_init(struct binder_io *bio, void *data,  
              size_t maxdata, size_t maxoffs)  
{  
    size_t n = maxoffs * sizeof(size_t);  
    if (n > maxdata) {  
        ...  
    }  
  
    bio->data = bio->data0 = (char *) data + n;  
    bio->offs = bio->offs0 = data;  
    bio->data_avail = maxdata - n;  
    bio->offs_avail = maxoffs;  
    bio->flags = 0;  
}
```

其中

```
struct binder_io  
{  
    char *data;          /* pointer to read/write from */  
    ...  
}
```



```
binder_size_t *offs; /* array of offsets */

size_t data_avail; /* bytes available in data buffer */

size_t offs_avail; /* entries available in offsets array */


char *data0; //data buffer 起点位置

binder_size_t *offs0; //buffer 偏移量的起点位置

uint32_t flags;

uint32_t unused;

};
```

2.5.2 bio_init_from_txn

[-> servicemanager/binder.c]

```
void bio_init_from_txn(struct binder_io *bio, struct binder_transaction_data *txn)
{
    bio->data = bio->data0 = (char *) (intptr_t) txn->data.ptr.buffer;

    bio->offs = bio->offs0 = (binder_size_t *) (intptr_t) txn->data.ptr.offsets;

    bio->data_avail = txn->data_size;

    bio->offs_avail = txn->offsets_size / sizeof(size_t);

    bio->flags = BIO_F_SHARED;
}
```

将 readbuf 的数据赋给 bio 对象的 data

2.6 svcmgr_handler

[-> service_manager.c]

```
int svcmgr_handler(struct binder_state *bs,
```

```
        struct binder_transaction_data *txn,
        struct binder_io *msg,
        struct binder_io *reply)
{
    struct svcinfo *si; //【见小节 2.6.1】
    uint16_t *s;
    size_t len;
    uint32_t handle;
    uint32_t strict_policy;
    int allow_isolated;
    ...

    strict_policy = bio_get_uint32(msg);
    s = bio_get_string16(msg, &len);
    ...

    switch(txn->code) {
        case SVC_MGR_GET_SERVICE:
        case SVC_MGR_CHECK_SERVICE:
            s = bio_get_string16(msg, &len); //服务名
            //根据名称查找相应服务 【见小节 3.1】
            handle = do_find_service(bs, s, len, txn->sender_euid, txn->sender_pid);
            //【见小节 3.1.2】
            bio_put_ref(reply, handle);
            return 0;

        case SVC_MGR_ADD_SERVICE:
            s = bio_get_string16(msg, &len); //服务名
            handle = bio_get_ref(msg); //handle【见小节 3.2.3】
            allow_isolated = bio_get_uint32(msg) ? 1 : 0;
```

```
//注册指定服务 【见小节 3.2】

if (do_add_service(bs, s, len, handle, txn->sender_euid,
    allow_isolated, txn->sender_pid))
    return -1;

break;

case SVC_MGR_LIST_SERVICES: {
    uint32_t n = bio_get_uint32(msg);

    if (!svc_can_list(txn->sender_pid)) {
        return -1;
    }

    si = svclist;
    while ((n-- > 0) && si)
        si = si->next;

    if (si) {
        bio_put_string16(reply, si->name);
        return 0;
    }

    return -1;
}

default:
    return -1;
}

bio_put_uint32(reply, 0);

return 0;
}
```

该方法的功能：查询服务，注册服务，以及列举所有服务

2.6.1 svcinfo

```
struct svcinfo
{
    struct svcinfo *next;

    uint32_t handle; //服务的 handle 值

    struct binder_death death;

    int allow_isolated;

    size_t len; //名字长度

    uint16_t name[0]; //服务名
};
```

每一个服务用 `svcinfo` 结构体来表示，该 `handle` 值是在注册服务的过程中，由服务所在进程那一端所确定的。

三. 核心工作

`servicemanager` 的核心工作就是注册服务和查询服务。

3.1 do_find_service

[-> service_manager.c]

```
uint32_t do_find_service(struct binder_state *bs, const uint16_t *s, size_t len,
    uid_t uid, pid_t spid)
{
    //查询相应的服务 【见小节 3.1.1】

    struct svcinfo *si = find_svc(s, len);

    if (!si || !si->handle) {
        return 0;
    }
}
```

```
}

    if (!si->allow_isolated) {
        uid_t appid = uid % AID_USER;

        //检查该服务是否允许孤立于进程而单独存在
        if (appid >= AID_ISOLATED_START && appid <= AID_ISOLATED_END) {
            return 0;
        }
    }

    //服务是否满足查询条件
    if (!svc_can_find(s, len, spid)) {
        return 0;
    }

    return si->handle;
}
```

查询到目标服务，并返回该服务所对应的 handle

3.1.1 find_svc

```
struct svcinfo *find_svc(const uint16_t *s16, size_t len)
{
    struct svcinfo *si;

    for (si = svclist; si; si = si->next) {
        //当名字完全一致，则返回查询到的结果
        if ((len == si->len) &&
            !memcmp(s16, si->name, len * sizeof(uint16_t))) {
            return si;
        }
    }
}
```

```
    }  
    return NULL;  
}
```

从 `svclist` 服务列表中，根据服务名遍历查找是否已经注册。当服务已存在 `svclist`，则返回相应的服务名，否则返回 `NULL`。

当找到服务的 `handle`，则调用 `bio_put_ref(reply, handle)`，将 `handle` 封装到 `reply`。

3.1.2 bio_put_ref

```
void bio_put_ref(struct binder_io *bio, uint32_t handle){  
    struct flat_binder_object *obj;  
  
    if (handle)  
        obj = bio_alloc_obj(bio); //[见小节 3.1.3]  
    else  
        obj = bio_alloc(bio, sizeof(*obj));  
  
    if (!obj)  
        return;  
  
    obj->flags = 0x7f | FLAT_BINDER_FLAG_ACCEPTS_FDS;  
    obj->type = BINDER_TYPE_HANDLE; //返回的是 HANDLE 类型  
    obj->handle = handle;  
    obj->cookie = 0;  
}
```

3.1.3 bio_alloc_obj

```
static struct flat_binder_object *bio_alloc_obj(struct binder_io *bio)
```

```
{  
  
    struct flat_binder_object *obj;  
  
    obj = bio_alloc(bio, sizeof(*obj)); // [见小节 3.1.4]  
  
    if (obj && bio->offs_avail) {  
        bio->offs_avail--;  
        *bio->offs++ = ((char*) obj) - ((char*) bio->data0);  
        return obj;  
    }  
  
    bio->flags |= BIO_F_OVERFLOW;  
    return NULL;  
}
```

3.1.4 bio_alloc

```
static void *bio_alloc(struct binder_io *bio, size_t size)  
{  
    size = (size + 3) & (~3);  
    if (size > bio->data_avail) {  
        bio->flags |= BIO_F_OVERFLOW;  
        return NULL;  
    } else {  
        void *ptr = bio->data;  
        bio->data += size;  
        bio->data_avail -= size;  
        return ptr;  
    }  
}
```

3.2 do_add_service

[-> service_manager.c]

```
int do_add_service(struct binder_state *bs,
                  const uint16_t *s, size_t len,
                  uint32_t handle, uid_t uid, int allow_isolated,
                  pid_t spid)
{
    struct svcinfo *si;

    if (!handle || (len == 0) || (len > 127))
        return -1;

    //权限检查【见小节 3.2.1】
    if (!svc_can_register(s, len, spid)) {
        return -1;
    }

    //服务检索【见小节 3.1.1】
    si = find_svc(s, len);
    if (si) {
        if (si->handle) {
            svcinfo_death(bs, si); //服务已注册时，释放相应的服务【见小节 3.2.2】
        }
        si->handle = handle;
    } else {
        si = malloc(sizeof(*si) + (len + 1) * sizeof(uint16_t));
        if (!si) { //内存不足，无法分配足够内存
            return -1;
        }
        si->handle = handle;
```



```
    si->len = len;

    memcpy(si->name, s, (len + 1) * sizeof(uint16_t)); //内存拷贝服务信息

    si->name[len] = '\0';

    si->death.func = (void*) svcinfo_death;

    si->death.ptr = si;

    si->allow_isolated = allow_isolated;

    si->next = svclist; // svclist 保存所有已注册的服务

    svclist = si;

}

//以 BC_ACQUIRE 命令, handle 为目标的信息, 通过 ioctl 发送给 binder 驱动

binder_acquire(bs, handle);

//以 BC_REQUEST_DEATH_NOTIFICATION 命令的信息, 通过 ioctl 发送给 binder 驱动, 主要用于清理内存等收尾工作。[见小节 3.3]

binder_link_to_death(bs, handle, &si->death);

return 0;

}
```

注册服务的分以下 3 部分工作:

- **svc_can_register**: 检查权限, 检查 **selinux** 权限是否满足;
- **find_svc**: 服务检索, 根据服务名来查询匹配的服务;
- **svcinfo_death**: 释放服务, 当查询到已存在同名的服务, 则先清理该服务信息, 再将当前的服务加入到服务列表 **svclist**;

3.2.1 svc_can_register

[-> service_manager.c]

```
static int svc_can_register(const uint16_t *name, size_t name_len, pid_t spid) {

    const char *perm = "add";
```

```
//检查 selinux 权限是否满足  
  
return check_mac_perms_from_lookup(spid, perm, str8(name, name_len)) ? 1 :  
0;  
  
}
```

3.2.2 svcinfo_death

[-> service_manager.c]

```
void svcinfo_death(struct binder_state *bs, void *ptr){  
  
    struct svcinfo *si = (struct svcinfo* ) ptr;  
  
    if (si->handle) {  
        binder_release(bs, si->handle);  
        si->handle = 0;  
    }  
  
}
```

3.2.3 bio_get_ref

[-> servicemanager/binder.c]

```
uint32_t bio_get_ref(struct binder_io *bio)  
{  
  
    struct flat_binder_object *obj;  
  
    obj = _bio_get_obj(bio);  
  
    if (!obj)  
        return 0;  
  
}
```

```
if (obj->type == BINDER_TYPE_HANDLE)

    return obj->handle;

return 0;

}
```

3.3 binder_link_to_death

[-> servicemanager/binder.c]

```
void binder_link_to_death(struct binder_state *bs, uint32_t target, struct binder_death *death) {

    struct {

        uint32_t cmd;

        struct binder_handle_cookie payload;

    } __attribute__((packed)) data;

    data.cmd = BC_REQUEST_DEATH_NOTIFICATION;

    data.payload.handle = target;

    data.payload.cookie = (uintptr_t) death;

    binder_write(bs, &data, sizeof(data)); //[见小节 3.3.1]

}
```

binder_write 经过跟小节 2.4.1 一样的方式, 进入 Binder driver 后, 直接调用后进入 binder_thread_write, 处理 BC_REQUEST_DEATH_NOTIFICATION 命令

3.3.1 binder_ioctl_write_read

[-> kernel/drivers/android/binder.c]

```
static int binder_ioctl_write_read(struct file *filp,
```

```
        unsigned int cmd, unsigned long arg,
        struct binder_thread *thread)
{
    int ret = 0;

    struct binder_proc *proc = filp->private_data;
    void __user *ubuf = (void __user *)arg;
    struct binder_write_read bwr;

    if (copy_from_user(&bwr, ubuf, sizeof(bwr))) { //把用户空间数据 ubuf 拷贝到 bwr
        ret = -EFAULT;
        goto out;
    }

    if (bwr.write_size > 0) { //此时写缓存有数据【见小节 3.3.2】
        ret = binder_thread_write(proc, thread,
            bwr.write_buffer, bwr.write_size, &bwr.write_consumed);
        if (ret < 0) {
            bwr.read_consumed = 0;
            if (copy_to_user(ubuf, &bwr, sizeof(bwr)))
                ret = -EFAULT;
            goto out;
        }
    }

    if (bwr.read_size > 0) { //此时读缓存有数据【见小节 3.3.3】
        ret = binder_thread_read(proc, thread, bwr.read_buffer,
            bwr.read_size,
            &bwr.read_consumed,
            filp->f_flags & O_NONBLOCK);
        if (!list_empty(&proc->todo)) //进程 todo 队列不为空,则唤醒该队列中的线程
            wake_up_interruptible(&proc->wait);
    }
}
```

```
    if (ret < 0) {
        if (copy_to_user(ubuf, &bwr, sizeof(bwr)))
            ret = -EFAULT;
        goto out;
    }
}

if (copy_to_user(ubuf, &bwr, sizeof(bwr))) { //将内核数据 bwr 拷贝到用户空间 ubu
f
    ret = -EFAULT;
    goto out;
}out:
return ret;
}
```

3.3.2 binder_thread_write

[-> kernel/drivers/android/binder.c]

```
static int binder_thread_write(struct binder_proc *proc,
    struct binder_thread *thread,
    binder_uintptr_t binder_buffer, size_t size,
    binder_size_t *consumed){
    uint32_t cmd;
    struct binder_context *context = proc->context;
    void __user *buffer = (void __user *) (uintptr_t) binder_buffer;
    void __user *ptr = buffer + *consumed; //ptr 指向小节 3.2.3 中 bwr 中 write_buffer
    的 data.
    void __user *end = buffer + size;
    while (ptr < end && thread->return_error == BR_OK) {
        get_user(cmd, (uint32_t __user *) ptr); //获取 BC_REQUEST_DEATH_NOTIFICATION
```

```
ptr += sizeof(uint32_t);

switch (cmd) {

    case BC_REQUEST_DEATH_NOTIFICATION:{ //注册死亡通知

        uint32_t target;

        void __user *cookie;

        struct binder_ref *ref;

        struct binder_ref_death *death;

        get_user(target, (uint32_t __user *)ptr); //获取 target

        ptr += sizeof(uint32_t);

        get_user(cookie, (void __user * __user *)ptr); //获取 death

        ptr += sizeof(void *);

        ref = binder_get_ref(proc, target); //拿到目标服务的 binder_ref

        if (cmd == BC_REQUEST_DEATH_NOTIFICATION) {

            if (ref->death) {

                break; //已设置死亡通知

            }

            death = kzalloc(sizeof(*death), GFP_KERNEL);

            INIT_LIST_HEAD(&death->work.entry);

            death->cookie = cookie;

            ref->death = death;

            if (ref->node->proc == NULL) { //当目标 binder 服务所在进程已死,则发送死亡通知

                ref->death->work.type = BINDER_WORK_DEAD_BINDER;

                //当前线程为 binder 线程,则直接添加到当前线程的 todo 队列。接下来,进入[小节 3.2.6]

                if (thread->looper & (BINDER_LOOPER_STATE_REGISTERED | BINDER_LOOPER_STATE_ENTERED)) {
```

```
list_add_tail(&ref->death->work.entry, &thread->todo);  
} else {  
list_add_tail(&ref->death->work.entry, &proc->todo);  
wake_up_interruptible(&proc->wait);  
}  
}  
} else {  
...  
}  
} break;  
case ...:  
}  
*consumed = ptr - buffer;  
}  
}
```

此方法中的 `proc`, `thread` 都是指当前 `servicemanager` 进程的信息. 此时 `TODO` 队列有数据,则进入 `binder_thread_read`.

那么哪些场景会向队列增加 `BINDER_WORK_DEAD_BINDER` 事务呢? 那就是当 `binder` 所在进程死亡后,会调用 `binder_release` 方法, 然后调用 `binder_node_release`.这个过程便会发出死亡通知的回调.

3.3.3 binder_thread_read

```
static int binder_thread_read(struct binder_proc *proc,  
struct binder_thread *thread,  
binder_uintptr_t binder_buffer, size_t size,  
binder_size_t *consumed, int non_block)  
...  
//只有当前线程 todo 队列为空, 并且 transaction_stack 也为空, 才会开始处于当前进程的  
事务
```

```
    if (wait_for_proc_work) {  
        ...  
        ret = wait_event_freezable_exclusive(proc->wait, binder_has_proc_work(proc, thread));  
    } else {  
        ...  
        ret = wait_event_freezable(thread->wait, binder_has_thread_work(thread));  
    }  
    binder_lock(__func__); //加锁  
  
    if (wait_for_proc_work)  
        proc->ready_threads--; //空闲的 binder 线程减 1  
    thread->looper &= ~BINDER_LOOPER_STATE_WAITING;  
  
    while (1) {  
        uint32_t cmd;  
        struct binder_transaction_data tr;  
        struct binder_work *w;  
        struct binder_transaction *t = NULL;  
  
        //从 todo 队列拿出前面放入的 binder_work, 此时 type 为 BINDER_WORK_DEAD_BINDER  
        if (!list_empty(&thread->todo)) {  
            w = list_first_entry(&thread->todo, struct binder_work,  
                                entry);  
        } else if (!list_empty(&proc->todo) && wait_for_proc_work) {  
            w = list_first_entry(&proc->todo, struct binder_work,  
                                entry);  
        }  
  
        switch (w->type) {
```



```
case BINDER_WORK_DEAD_BINDER: {
    struct binder_ref_death *death;

    uint32_t cmd;

    death = container_of(w, struct binder_ref_death, work);

    if (w->type == BINDER_WORK_CLEAR_DEATH_NOTIFICATION)
        ...
    else
        cmd = BR_DEAD_BINDER; //进入此分支

    put_user(cmd, (uint32_t __user *)ptr); //拷贝到用户空间[见小节 3.3.4]

    ptr += sizeof(uint32_t);

    //此处的 cookie 是前面传递的 svcinfo_death

    put_user(death->cookie, (binder_uintptr_t __user *)ptr);

    ptr += sizeof(binder_uintptr_t);

    if (w->type == BINDER_WORK_CLEAR_DEATH_NOTIFICATION) {
        ...
    } else
        list_move(&w->entry, &proc->delivered_death);

    if (cmd == BR_DEAD_BINDER)
        goto done;

    } break;
}

}

...

return 0;
}
```

将命令 BR_DEAD_BINDER 写到用户空间，此处的 cookie 是前面传递的 svcinfo_death。当 binder_loop 下一次 执行 binder_parse 的过程便会处理该消息。

3.3.4 binder_parse

[-> servicemanager/binder.c]

```
int binder_parse(struct binder_state *bs, struct binder_io *bio, uintptr_t ptr,
size_t size, binder_handler func) {
    int r = 1;
    uintptr_t end = ptr + (uintptr_t) size;

    while (ptr < end) {
        uint32_t cmd = *(uint32_t *) ptr;
        ptr += sizeof(uint32_t);
        switch(cmd) {
            case BR_DEAD_BINDER: {
                struct binder_death *death = (struct binder_death *) (uintptr_t) *
                (binder_uintptr_t *) ptr;
                ptr += sizeof(binder_uintptr_t);
                // binder 死亡消息【见小节 3.3.5】
                death->func(bs, death->ptr);
                break;
            }
            ...
        }
    }
    return r;
}
```

由小节 3.2 的 `si->death.func = (void*) svcinfo_death`; 可知此处 `death->func` 便是执行 `svcinfo_death()` 方法.

3.3.5 svcinfo_death

[-> service_manager.c]

```
void svcinfo_death(struct binder_state *bs, void *ptr){  
  
    struct svcinfo *si = (struct svcinfo* ) ptr;  
  
    if (si->handle) {  
        binder_release(bs, si->handle);  
        si->handle = 0;  
    }  
}
```

3.3.6 binder_release

[-> service_manager.c]

```
void binder_release(struct binder_state *bs, uint32_t target) {  
  
    uint32_t cmd[2];  
  
    cmd[0] = BC_RELEASE;  
    cmd[1] = target;  
  
    binder_write(bs, cmd, sizeof(cmd));  
}
```

向 Binder Driver 写入 `BC_RELEASE` 命令, 最终进入 Binder Driver 后执行 `binder_dec_ref(ref, 1)` 来减少 binder node 的引用.

3.4 binder_send_reply

[-> servicemanager/binder.c]

```
void binder_send_reply(struct binder_state *bs, struct binder_io *reply, binder
_uintptr_t buffer_to_free, int status) {
    struct {
        uint32_t cmd_free;
        binder_uintptr_t buffer;
        uint32_t cmd_reply;
        struct binder_transaction_data txn;
    } __attribute__((packed)) data;

    data.cmd_free = BC_FREE_BUFFER; //free buffer 命令
    data.buffer = buffer_to_free;
    data.cmd_reply = BC_REPLY; // reply 命令
    data.txn.target.ptr = 0;
    data.txn.cookie = 0;
    data.txn.code = 0;
    if (status) {
        data.txn.flags = TF_STATUS_CODE;
        data.txn.data_size = sizeof(int);
        data.txn.offsets_size = 0;
        data.txn.data.ptr.buffer = (uintptr_t)&status;
        data.txn.data.ptr.offsets = 0;
    } else {
        data.txn.flags = 0;
        data.txn.data_size = reply->data - reply->data0;
        data.txn.offsets_size = ((char*) reply->offs) - ((char*) reply->offs0);
        data.txn.data.ptr.buffer = (uintptr_t)reply->data0;
        data.txn.data.ptr.offsets = (uintptr_t)reply->offs0;
    }

    //向 Binder 驱动通信
```

```
binder_write(bs, &data, sizeof(data));
```

```
}
```

当小节 2.5 执行 `binder_parse` 方法，先调用 `svcmgr_handler()`，再然后执行 `binder_send_reply` 过程。该方法会调用 [小节 2.4.1] `binder_write` 进入 `binder` 驱动后，将 `BC_FREE_BUFFER` 和 `BC_REPLY` 命令协议发送给 `Binder` 驱动，向 `client` 端发送 `reply`。其中 `data` 的数据区中保存的是 `TYPE` 为 `HANDLE`。

四. 总结

`ServiceManger` 集中管理系统内的所有服务，通过权限控制进程是否有权注册服务,通过字符串名称来查找对应的 `Service`；由于 `ServiceManger` 进程建立跟所有向其注册服务的死亡通知，那么当服务所在进程死亡后，会只需告知 `ServiceManager`。每个 `Client` 通过查询 `ServiceManager` 可获取 `Server` 进程的情况，降低所有 `Client` 进程直接检测会导致负载过重。

`ServiceManager` 启动流程：

1. 打开 `binder` 驱动，并调用 `mmap()` 方法分配 128k 的内存映射空间：`binder_open()`;
2. 通知 `binder` 驱动使其成为守护进程：`binder_become_context_manager()`;
3. 验证 `selinux` 权限，判断进程是否有权注册或查看指定服务；
4. 进入循环状态，等待 `Client` 端的请求：`binder_loop()`。
5. 注册服务的过程，根据服务名称，但同一个服务已注册，重新注册前会先移除之前的注册信息；
6. 死亡通知：当 `binder` 所在进程死亡后,会调用 `binder_release` 方法,然后调用 `binder_node_release`.这个过程便会发出死亡通知的回调。

`ServiceManager` 最核心的两个功能为查询和注册服务：

- 注册服务：记录服务名和 `handle` 信息，保存到 `svclist` 列表；
- 查询服务：根据服务名查询相应的 `handle` 信息。

第五节获取 `ServiceManager`

基于 `Android 6.0` 的源码剖析， 本文详细地讲解 `defaultServiceManager` 流程

```
framework/native/libs/binder/
```

```
- ProcessState.cpp
```

```
- BpBinder.cpp
```

```
- Binder.cpp
```

```
- IServiceManager.cpp
```

```
framework/native/include/binder/
```

```
- IServiceManager.h
```

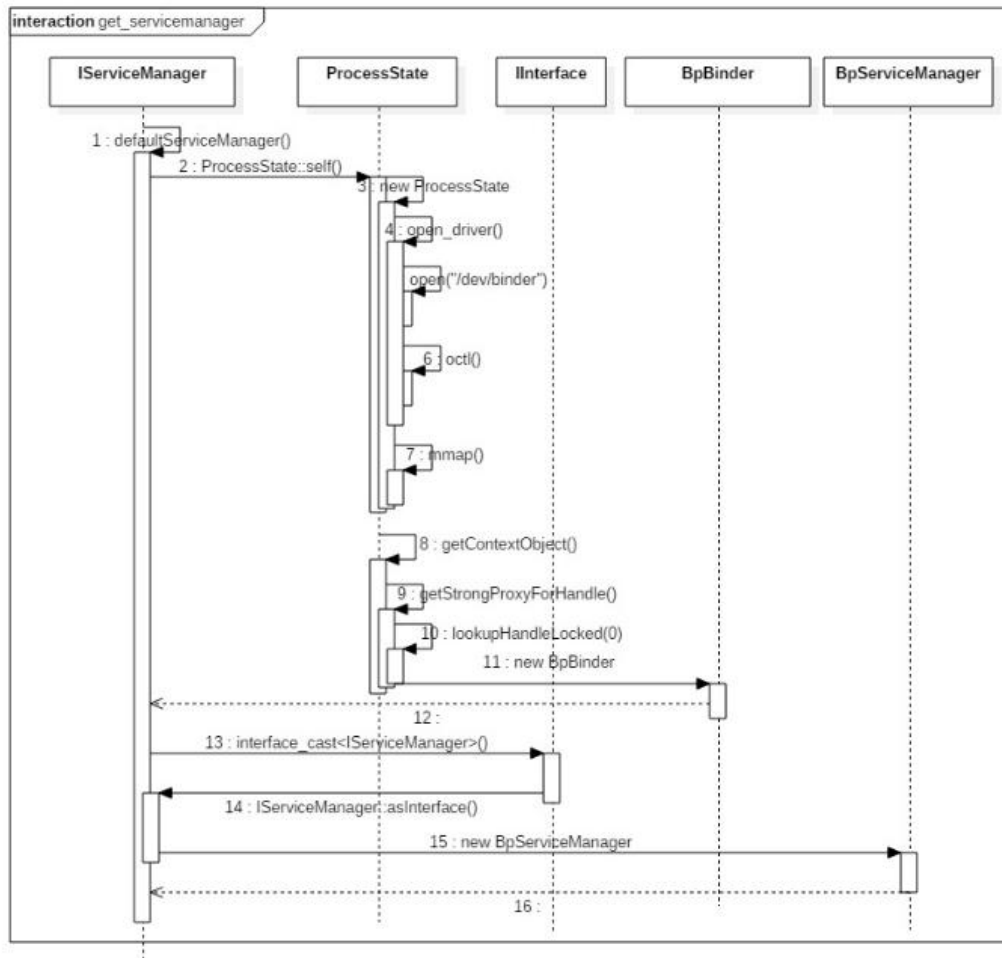
```
- IInterface.h
```

一. 概述

获取 Service Manager 是通过 `defaultServiceManager()` 方法来完成，当进程注册服务(`addService`)或 获取服务(`getService`)的过程之前，都需要先调用 `defaultServiceManager()` 方法来获取 `gDefaultServiceManager` 对象。对于 `gDefaultServiceManager` 对象，如果存在则直接返回；如果不存在则创建该对象，创建过程包括调用 `open()` 打开 binder 驱动设备，利用 `mmap()` 映射内核的地址空间。

1.1 流程图

[点击查看大图](#)



1.2 defaultServiceManager

[-> IServiceManager.cpp]

```
sp<IServiceManager> defaultServiceManager()
{
    if (gDefaultServiceManager != NULL) return gDefaultServiceManager;

    {
        AutoMutex _l(gDefaultServiceManagerLock); //加锁

        while (gDefaultServiceManager == NULL) {
            // 【见下文小节二,三,四】

            gDefaultServiceManager = interface_cast<IServiceManager>(<

```

```
ProcessState::self()->getContextObject(NULL));  
  
if (gDefaultServiceManager == NULL)  
  
    sleep(1);  
  
}  
  
}  
  
return gDefaultServiceManager;  
  
}
```

获取 ServiceManager 对象采用单例模式，当 gDefaultServiceManager 存在，则直接返回，否则创建一个新对象。发现与一般的单例模式不太一样，里面多了一层 while 循环，这是 google 在 2013 年 1 月 Todd Poynor 提交的修改。当尝试创建或获取 ServiceManager 时，ServiceManager 可能尚未准备就绪，这时通过 sleep 1 秒后，循环尝试获取直到成功。gDefaultServiceManager 的创建过程,可分解为以下 3 个步骤：

- **ProcessState::self()**：用于获取 ProcessState 对象(也是单例模式)，每个进程有且只有一个 ProcessState 对象，存在则直接返回，不存在则创建，详情见【小节二】；
- **getContextObject()**：用于获取 BpBinder 对象，对于 handle=0 的 BpBinder 对象，存在则直接返回，不存在才创建，详情见【小节三】；
- **interface_cast<IServiceManager>()**：用于获取 BpServiceManager 对象，详情见【小节四】；

二. 获取 ProcessState 对象

2.1 ProcessState::self

[-> ProcessState.cpp]

```
sp<ProcessState> ProcessState::self()  
{  
  
    Mutex::Autolock _l(gProcessMutex);  
  
    if (gProcess != NULL) {  
  
        return gProcess;  
  
    }
```



```
}  
  
//实例化 ProcessState 【见小节 2.2】  
gProcess = new ProcessState;  
return gProcess;  
}
```

获得 ProcessState 对象: 这也是**单例模式**, 从而保证每一个进程只有一个 ProcessState 对象。其中 gProcess 和 gProcessMutex 是保存在 Static.cpp 类的全局变量。

2.2 初始化 ProcessState

[-> ProcessState.cpp]

```
ProcessState::ProcessState()  
  
: mDriverFD(open_driver()) // 打开 Binder 驱动【见小节 2.3】  
  , mVMStart(MAP_FAILED)  
  , mThreadCountLock(PTHREAD_MUTEX_INITIALIZER)  
  , mThreadCountDecrement(PTHREAD_COND_INITIALIZER)  
  , mExecutingThreadsCount(0)  
  , mMaxThreads(DEFAULT_MAX_BINDER_THREADS)  
  , mManagesContexts(false)  
  , mBinderContextCheckFunc(NULL)  
  , mBinderContextUserData(NULL)  
  , mThreadPoolStarted(false)  
  , mThreadPoolSeq(1)  
{  
    if (mDriverFD >= 0) {  
        //采用内存映射函数 mmap, 给 binder 分配一块虚拟地址空间,用来接收事务
```

```
mVMStart = mmap(0, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE | MAP_NORESERV  
E, mDriverFD, 0);  
  
if (mVMStart == MAP_FAILED) {  
  
    close(mDriverFD); //没有足够空间分配给/dev/binder,则关闭驱动  
  
    mDriverFD = -1;  
  
}  
  
}  
  
}
```

- **ProcessState** 的单例模式的惟一性，因此一个进程只打开 **binder** 设备一次,其中 **ProcessState** 的成员变量 **mDriverFD** 记录 **binder** 驱动的 fd, 用于访问 **binder** 设备。
- **BINDER_VM_SIZE = (1*1024*1024) - (4096 *2)**, **binder** 分配的默认内存大小为 1M-8k。
- **DEFAULT_MAX_BINDER_THREADS = 15**, **binder** 默认的最大可并发访问的线程数为 16。

2.3 open_driver

[-> ProcessState.cpp]

```
static int open_driver()  
{  
    // 打开/dev/binder 设备，建立与内核的 Binder 驱动的交流通道  
    int fd = open("/dev/binder", O_RDWR);  
    if (fd >= 0) {  
        fcntl(fd, F_SETFD, FD_CLOEXEC);  
        int vers = 0;  
        status_t result = ioctl(fd, BINDER_VERSION, &vers);  
        if (result == -1) {  
            close(fd);  
            fd = -1;  
        }  
    }
```

```
if (result != 0 || vers != BINDER_CURRENT_PROTOCOL_VERSION) {  
    close(fd);  
    fd = -1;  
}  
  
size_t maxThreads = DEFAULT_MAX_BINDER_THREADS;  
  
// 通过 ioctl 设置 binder 驱动，能支持的最大线程数  
result = ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads);  
  
if (result == -1) {  
    ALOGE("Binder ioctl to set max threads failed: %s", strerror(errno));  
}  
} else {  
    ALOGW("Opening '/dev/binder' failed: %s\n", strerror(errno));  
}  
  
return fd;  
}
```

open_driver 作用是打开/dev/binder 设备，设定 binder 支持的最大线程数。关于 binder 驱动的相应方法，见文章 [Binder Driver 初探](#)。

三. 获取 BpBinder 对象

3.1 getContextObject

[-> ProcessState.cpp]

```
sp<IBinder> ProcessState::getContextObject(const sp<IBinder>& /*caller*/)  
{  
    return getStrongProxyForHandle(0); // 【见小节 3.2】  
}
```

获取 handle=0 的 IBinder

3.2 getStrongProxyForHandle

[-> ProcessState.cpp]

```
sp<IBinder> ProcessState::getStrongProxyForHandle(int32_t handle)
{
    sp<IBinder> result;

    AutoMutex _l(mLock);

    //查找 handle 对应的资源项【见小节 3.3】
    handle_entry* e = lookupHandleLocked(handle);

    if (e != NULL) {
        IBinder* b = e->binder;

        if (b == NULL || !e->refs->attemptIncWeak(this)) {
            if (handle == 0) {
                Parcel data;

                //通过 ping 操作测试 binder 是否准备就绪
                status_t status = IPCThreadState::self()->transact(
                    0, IBinder::PING_TRANSACTION, data, NULL, 0);

                if (status == DEAD_OBJECT)
                    return NULL;
            }

            //当 handle 值所对应的 IBinder 不存在或弱引用无效时，则创建 BpBinder 对象【见
            小节 3.4】
            b = new BpBinder(handle);
            e->binder = b;

            if (b) e->refs = b->getWeakRefs();

            result = b;
        }
    }
}
```

```
    } else {  
        result.force_set(b);  
        e->refs->decWeak(this);  
    }  
}  
  
return result;  
}
```

当 handle 值所对应的 IBinder 不存在或弱引用无效时会创建 BpBinder，否则直接获取。针对 handle==0 的特殊情况，通过 PING_TRANSACTION 来判断是否准备就绪。如果在 context manager 还未生效前，一个 BpBinder 的本地引用就已经被创建，那么驱动将无法提供 context manager 的引用。

3.3 lookupHandleLocked

[-> ProcessState.cpp]

```
ProcessState::handle_entry* ProcessState::lookupHandleLocked(int32_t handle)  
{  
    const size_t N=mHandleToObject.size();  
    //当 handle 大于 mHandleToObject 的长度时，进入该分支  
    if (N <= (size_t)handle) {  
        handle_entry e;  
        e.binder = NULL;  
        e.refs = NULL;  
        //从 mHandleToObject 的第 N 个位置开始，插入(handle+1-N)个 e 到队列中  
        status_t err = mHandleToObject.insertAt(e, N, handle+1-N);  
        if (err < NO_ERROR) return NULL;  
    }  
    return &mHandleToObject.editItemAt(handle);  
}
```

根据 handle 值来查找对应的 `handle_entry`, `handle_entry` 是一个结构体, 里面记录 `IBinder` 和 `weakref_type` 两个指针。当 handle 大于 `mHandleToObject` 的 `Vector` 长度时, 则向该 `Vector` 中添加 `(handle+1-N)` 个 `handle_entry` 结构体, 然后再返回 handle 向对应位置的 `handle_entry` 结构体指针。

3.4 创建 BpBinder

[-> BpBinder.cpp]

```
BpBinder::BpBinder(int32_t handle)
    : mHandle(handle)
    , mAlive(1)
    , mObitsSent(0)
    , mObituaries(NULL)
{
    extendObjectLifetime(OBJECT_LIFETIME_WEAK); //延长对象的生命时间
    IPCThreadState::self()->incWeakHandle(handle); //handle 所对应的 bindle 弱引用
    + 1
}
```

创建 BpBinder 对象中会将 handle 相对应 Binder 的弱引用增加 1。

四. 获取 BpServiceManager

4.1 interface_cast

[-> IInterface.h]

```
template<typename INTERFACE> inline sp<INTERFACE> interface_cast(const sp<IBinder> & obj)
{
    return INTERFACE::asInterface(obj); //【见小节 4.2】
}
```

1

这是一个模板函数，可得出，`interface_cast<IServiceManager>()` 等价于 `IServiceManager::asInterface()`。接下来,再来说说 `asInterface()` 函数的具体功能。

4.2 IServiceManager::asInterface

对于 `asInterface()` 函数，通过搜索代码，你会发现根本找不到这个方法是在哪里定义这个函数的，其实是通过模板函数来定义的，通过下面两个代码完成的：

```
//位于 IServiceManager.h 文件 【见小节 4.3】  
  
DECLARE_META_INTERFACE(ServiceManager)//位于 IServiceManager.cpp 文件 【见小节 4.4】  
  
IMPLEMENT_META_INTERFACE(ServiceManager,"android.os.IServiceManager")
```

接下来，再说说这两行代码分别完成的功能：

4.3 DECLARE_META_INTERFACE

[-> IInterface.h]

```
#define DECLARE_META_INTERFACE(INTERFACE) \n\n    static const android::String16 descriptor; \n\n    static android::sp<I##INTERFACE> asInterface( \n\n        const android::sp<android::IBinder>& obj); \n\n    virtual const android::String16& getInterfaceDescriptor() const; \n\n    I##INTERFACE(); \n\n    virtual ~I##INTERFACE(); \n
```

位于 IServiceManager.h 文件中,INTERFACE=ServiceManager 展开即可得：

[-> IServiceManager.h]

```
static const android::String16 descriptor;  
  
static android::sp< IServiceManager > asInterface(const android::sp<android::IBinder>& obj)  
  
virtual const android::String16& getInterfaceDescriptor() const;  
  
IServiceManager ();virtual ~IServiceManager();
```

该过程主要是声明 `asInterface()`, `getInterfaceDescriptor()` 方法.

4.4 IMPLEMENT_META_INTERFACE

[-> IInterface.h]

```
#define IMPLEMENT_META_INTERFACE(INTERFACE, NAME) \n\n    const android::String16 I##INTERFACE::descriptor(NAME); \n\n    const android::String16& \n\n        I##INTERFACE::getInterfaceDescriptor() const { \n\n        return I##INTERFACE::descriptor; \n\n        } \n\n    android::sp<I##INTERFACE> I##INTERFACE::asInterface( \n\n        const android::sp<android::IBinder>& obj) \n\n    { \n\n        android::sp<I##INTERFACE> intr; \n\n        if (obj != NULL) { \n\n            intr = static_cast<I##INTERFACE*>( \n\n                obj->queryLocalInterface( \n\n                    I##INTERFACE::descriptor).get()); \n\n            if (intr == NULL) { \n\n                intr = new Bp##INTERFACE(obj); \n\n            } \n\n        } \n\n    }
```



```
return intr;\n\n}\n\nI##INTERFACE::I##INTERFACE() { } \n\nI##INTERFACE::~~I##INTERFACE() { } \n
```

位于 IServiceManager.cpp 文件中,INTERFACE=ServiceManager,
NAME="android.os.IServiceManager"展开即可得:

[-> IServiceManager.cpp]

```
const android::String16 IServiceManager::descriptor("android.os.IServiceManager");\n\nconst android::String16& IServiceManager::getInterfaceDescriptor() const\n{\n    return IServiceManager::descriptor;\n}\n\nandroid::sp<IServiceManager> IServiceManager::asInterface(const android::sp<android::IBinder>& obj)\n{\n    android::sp<IServiceManager> intr;\n\n    if(obj != NULL) {\n        intr = static_cast<IServiceManager *>(\n            obj->queryLocalInterface(IServiceManager::descriptor).get());\n\n        if (intr == NULL) {\n            intr = new BpServiceManager(obj); // 【见小节 4.5】\n        }\n    }\n\n    return intr;\n}\n\nIServiceManager::IServiceManager () { }
```

```
IServiceManager::~~ IServiceManager() { }
```

不难发现，[小节 4.2]的 `IServiceManager::asInterface()` 等价于 `new BpServiceManager()`。在这里，更确切地说应该是 `new BpServiceManager(BpBinder)`。

4.5 BpServiceManager 实例化

创建 BpServiceManager 对象的过程，会先初始化父类对象：

4.5.1 BpServiceManager 初始化

[-> IServiceManager.cpp]

```
BpServiceManager(const sp<IBinder>& impl)
    : BpInterface<IServiceManager>(impl)
{ }
```

4.5.2 BpInterface 初始化

[-> IInterface.h]

```
inline BpInterface<INTERFACE>::BpInterface(const sp<IBinder>& remote)
    : BpRefBase(remote)
{ }
```

4.5.3 BpRefBase 初始化

[-> Binder.cpp]

```
BpRefBase::BpRefBase(const sp<IBinder>& o)
```

```
        : mRemote(o.get()), mRefs(NULL), mState(0)
    {
        extendObjectLifetime(OBJECT_LIFETIME_WEAK);

        if (mRemote) {
            mRemote->incStrong(this);
            mRefs = mRemote->createWeak(this);
        }
    }
}
```

`new BpServiceManager()`，在初始化过程中，比较重要工作的是类 `BpRefBase` 的 `mRemote` 指向 `new BpBinder(0)`，从而 `BpServiceManager` 能够利用 `Binder` 进行通过通信。

五. 总结

`defaultServiceManager` 等价于 `new BpServiceManager(new BpBinder(0));`

`ProcessState::self()` 主要工作：

- 调用 `open()`，打开 `/dev/binder` 驱动设备；
- 再利用 `mmap()`，创建大小为 `1M-8K` 的内存地址空间；
- 设定当前进程最大的最大并发 `Binder` 线程个数为 `16`。

`BpServiceManager` 巧妙将通信层与业务层逻辑合为一体，

- 通过继承接口 `IServiceManager` 实现了接口中的业务逻辑函数；
- 通过成员变量 `mRemote = new BpBinder(0)` 进行 `Binder` 通信工作。
- `BpBinder` 通过 `handler` 来指向所对应 `BBinder`，在整个 `Binder` 系统中 `handle=0` 代表 `ServiceManager` 所对应的 `BBinder`。

5.1 模板函数

Native 层的 Binder 架构,通过如下两个宏,非常方便地创建了 new

Bp##INTERFACE(obj):

```
//用于申明 asInterface(),getInterfaceDescriptor()#define DECLARE_META_INTERFACE(I
NTERFACE)#define IMPLEMENT_META_INTERFACE(INTERFACE, NAME) //用于实现上述两个方法
```

例如:

```
// 实现 BpServiceManager 对象
IMPLEMENT_META_INTERFACE(ServiceManager,"android.os.IServiceManager")
```

等价于:

```
const android::String16 IServiceManager::descriptor("android.os.IServiceManage
r");const android::String16& IServiceManager::getInterfaceDescriptor() const
{
    return IServiceManager::descriptor;
}

android::sp<IServiceManager> IServiceManager::asInterface(const android::sp<an
droid::IBinder>& obj)
{
    android::sp<IServiceManager> intr;

    if(obj != NULL) {
        intr = static_cast<IServiceManager *>(
            obj->queryLocalInterface(IServiceManager::descriptor).get());
        if (intr == NULL) {
            intr = new BpServiceManager(obj);
        }
    }

    return intr;
}
```

```
IServiceManager::IServiceManager () { }  
IServiceManager::~~ IServiceManager() { }
```

第六节注册服务(addService)

基于 Android 6.0 的源码剖析， 本文讲解如何向 *ServiceManager* 注册 Native 层的服务的过程。

```
framework/native/libs/binder/  
- Binder.cpp  
- BpBinder.cpp  
- IPCThreadState.cpp  
- ProcessState.cpp  
- IServiceManager.cpp  
- IInterface.cpp  
- Parcel.cpp  
  
frameworks/native/include/binder/  
- IInterface.h (包括 BnInterface, BpInterface)
```

一.概述

1.1 media 服务注册

media 入口函数是 `main_mediaserver.cpp` 中的 `main()` 方法，代码如下：

```
int main(int argc __unused, char** argv)  
{  
    ...  
    InitializeIcuOrDie();  
}
```

```
//获得 ProcessState 实例对象【见小节 2.1】

sp<ProcessState> proc(ProcessState::self());

//获取 BpServiceManager 对象

sp<IServiceManager> sm = defaultServiceManager();

AudioFlinger::instantiate();

//注册多媒体服务 【见小节 3.1】

MediaPlayerService::instantiate();

ResourceManagerService::instantiate();

CameraService::instantiate();

AudioPolicyService::instantiate();

SoundTriggerHwService::instantiate();

RadioService::instantiate();

registerExtensions();

//启动 Binder 线程池

ProcessState::self()->startThreadPool();

//当前线程加入到线程池

IPCThreadState::self()->joinThreadPool();

}
```

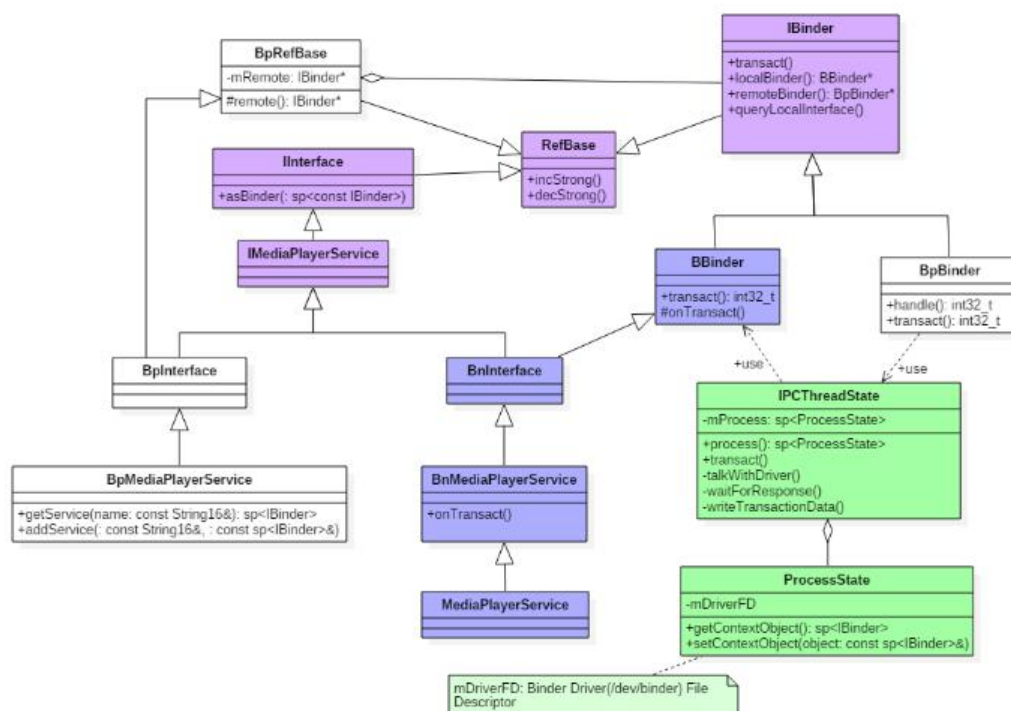
过程说明:

1. 获取 **ServiceManager**: 讲解了 defaultServiceManager() 返回的是 BpServiceManager 对象, 用于跟 servicemanager 进程通信;
2. 理解 **Binder** 线程池的管理, 讲解了 startThreadPool 和 joinThreadPool 过程.

本文的重点就是讲解 Native 层服务注册的过程.

1.2 类图

在 Native 层的服务以 media 服务为例, 来说一说服务注册过程, 先来看看 media 的整个的类关系图。 [点击查看大图](#)



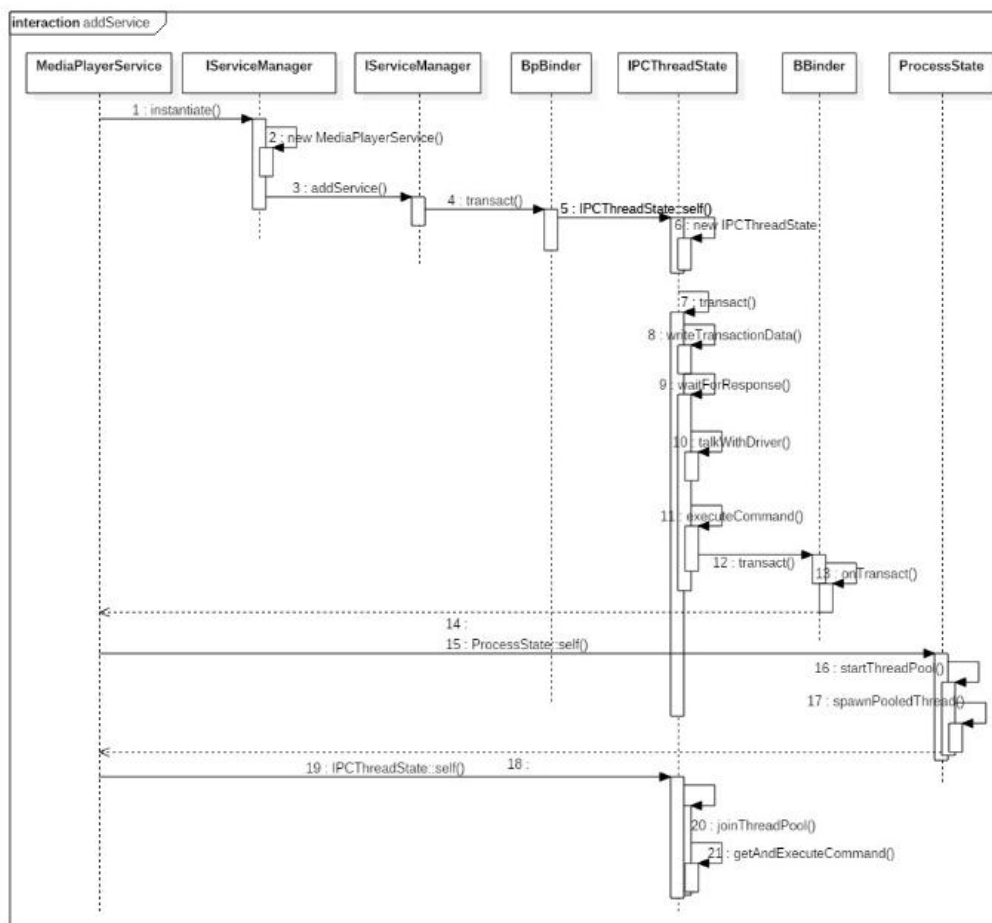
图解：

- 蓝色代表的是注册 MediaPlayerService 服务所涉及的类
- 绿色代表的是 Binder 架构中与 Binder 驱动通信过程中的最为核心的两个类；
- 紫色代表的是注册服务和获取服务的公共接口/父类；

1.3 时序图

先通过一幅图来说说，media 服务启动过程是如何向 servicemanager 注册服务的。

[点击查看大图](#)



二. ProcessState

2.1 ProcessState::self

[-> ProcessState.cpp]

```
sp<ProcessState> ProcessState::self()
{
    Mutex::Autolock _l(gProcessMutex);
    if (gProcess != NULL) {
        return gProcess;
    }
}
```



```
//实例化 ProcessState 【见小节 2.2】  
  
gProcess = new ProcessState;  
  
return gProcess;  
  
}
```

获得 ProcessState 对象: 这也是单例模式, 从而保证每一个进程只有一个 ProcessState 对象。其中 gProcess 和 gProcessMutex 是保存在 Static.cpp 类的全局变量。

2.2 ProcessState 初始化

[-> ProcessState.cpp]

```
ProcessState::ProcessState()  
  
: mDriverFD(open_driver()) // 打开 Binder 驱动【见小节 2.3】  
  
  , mVMStart(MAP_FAILED)  
  
  , mThreadCountLock(PTHREAD_MUTEX_INITIALIZER)  
  
  , mThreadCountDecrement(PTHREAD_COND_INITIALIZER)  
  
  , mExecutingThreadsCount(0)  
  
  , mMaxThreads(DEFAULT_MAX_BINDER_THREADS)  
  
  , mManagesContexts(false)  
  
  , mBinderContextCheckFunc(NULL)  
  
  , mBinderContextUserData(NULL)  
  
  , mThreadPoolStarted(false)  
  
  , mThreadPoolSeq(1)  
  
{  
  
    if (mDriverFD >= 0) {  
  
        //采用内存映射函数 mmap, 给 binder 分配一块虚拟地址空间【见小节 2.4】  
  
        mVMStart = mmap(0, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE | MAP_NORESERVE, mDriverFD, 0);  
  
    }
```

```
if (mVMStart == MAP_FAILED) {  
  
    close(mDriverFD); //没有足够空间分配给/dev/binder,则关闭驱动  
  
    mDriverFD = -1;  
  
}  
  
}  
  
}
```

- `ProcessState` 的单例模式的惟一性，因此一个进程只打开 binder 设备一次,其中 `ProcessState` 的成员变量 `mDriverFD` 记录 binder 驱动的 fd, 用于访问 binder 设备。
- `BINDER_VM_SIZE = (1*1024*1024) - (4096 *2)`, binder 分配的默认内存大小为 1M-8k。
- `DEFAULT_MAX_BINDER_THREADS = 15`, binder 默认的最大可并发访问的线程数为 16。

2.3 open_driver

[-> ProcessState.cpp]

```
static int open_driver()  
{  
    // 打开/dev/binder 设备，建立与内核的 Binder 驱动的交互通道  
    int fd = open("/dev/binder", O_RDWR);  
    if (fd >= 0) {  
        fcntl(fd, F_SETFD, FD_CLOEXEC);  
        int vers = 0;  
        status_t result = ioctl(fd, BINDER_VERSION, &vers);  
        if (result == -1) {  
            close(fd);  
            fd = -1;  
        }  
        if (result != 0 || vers != BINDER_CURRENT_PROTOCOL_VERSION) {  
            close(fd);  
        }  
    }  
}
```

```
        fd = -1;

    }

    size_t maxThreads = DEFAULT_MAX_BINDER_THREADS;

    // 通过 ioctl 设置 binder 驱动，能支持的最大线程数
    result = ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads);

    if (result == -1) {
        ALOGE("Binder ioctl to set max threads failed: %s", strerror(errno));
    }

    } else {
        ALOGW("Opening '/dev/binder' failed: %s\n", strerror(errno));
    }

    return fd;
}
```

open_driver 作用是打开/dev/binder 设备，设定 binder 支持的最大线程数。关于 binder 驱动的相应方法，见文章 [Binder Driver 初探](#)。

ProcessState 采用单例模式，保证每一个进程都只打开一次 Binder Driver。

2.4 mmap

```
//原型 void* mmap(void* addr, size_t size, int prot, int flags, int fd, off_t off
set) //此处 mmap(0, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE | MAP_NORESERVE, mDri
verFD, 0);
```

参数说明：

- **addr:** 代表映射到进程地址空间的起始地址，当值等于 0 则由内核选择合适地址，此处为 0；
- **size:** 代表需要映射的内存地址空间的大小，此处为 1M-8K；
- **prot:** 代表内存映射区的读写等属性值，此处为 PROT_READ(可读取)；
- **flags:** 标志位，此处为 MAP_PRIVATE(私有映射，多进程间不共享内容的改变)和 MAP_NORESERVE(不保留交换空间)

- fd: 代表 mmap 所关联的文件描述符，此处为 mDriverFD;
- offset: 偏移量，此处为 0。

mmap()经过系统调用，执行 `binder_mmap` 过程。

三. 服务注册

3.1 instantiate

[-> MediaPlayerService.cpp]

```
void MediaPlayerService::instantiate() {
    //注册服务【见小节 3.2】
    defaultServiceManager()->addService(String16("media.player"), new MediaPlayerService());
}
```

注册服务 MediaPlayerService: 由 `defaultServiceManager()` 返回的是 `BpServiceManager`，同时会创建 `ProcessState` 对象和 `BpBinder` 对象。故此处等价于调用 `BpServiceManager->addService`。其中 `MediaPlayerService` 位于 `libmediaplayerservice` 库。

3.2 BpSM.addService

[-> IServiceManager.cpp ::BpServiceManager]

```
virtual status_t addService(const String16& name, const sp<IBinder>& service, bool allowIsolated) {
    Parcel data, reply; //Parcel 是数据通信包
    //写入头信息"android.os.IServiceManager"
    data.writeInterfaceToken(IServiceManager::getInterfaceDescriptor());
    data.writeString16(name); // name 为 "media.player"
    data.writeStrongBinder(service); // MediaPlayerService 对象【见小节 3.2.1】
}
```

```
data.writeInt32(allowIsolated ? 1 : 0); // allowIsolated= false

//remote()指向的是 BpBinder 对象【见小节 3.3】

status_t err = remote()->transact(ADD_SERVICE_TRANSACTION, data, &reply);

return err == NO_ERROR ? reply.readExceptionCode() : err;

}
```

服务注册过程：向 ServiceManager 注册服务 MediaPlayerService，服务名为"media.player"；

3.2.1 writeStrongBinder

[-> parcel.cpp]

```
status_t Parcel::writeStrongBinder(const sp<IBinder>& val)

{

    return flatten_binder(ProcessState::self(), val, this);

}
```

3.2.2 flatten_binder

[-> parcel.cpp]

```
status_t flatten_binder(const sp<ProcessState>& /*proc*/,

    const sp<IBinder>& binder, Parcel* out)

{

    flat_binder_object obj;

    obj.flags = 0x7f | FLAT_BINDER_FLAG_ACCEPTS_FDS;

    if (binder != NULL) {

        IBinder *local = binder->localBinder(); //本地 Binder 不为空

        if (!local) {
```

```
BpBinder *proxy = binder->remoteBinder();

const int32_t handle = proxy ? proxy->handle() : 0;

obj.type = BINDER_TYPE_HANDLE;

obj.binder = 0;

obj.handle = handle;

obj.cookie = 0;

} else { //进入该分支

    obj.type = BINDER_TYPE_BINDER;

    obj.binder = reinterpret_cast<uintptr_t>(local->getWeakRefs());

    obj.cookie = reinterpret_cast<uintptr_t>(local);

}

} else {

    ...

}

//【见小节 3.2.3】

return finish_flatten_binder(binder, obj, out);

}
```

将 Binder 对象扁平化，转换成 flat_binder_object 对象。

- 对于 Binder 实体，则 cookie 记录 Binder 实体的指针；
- 对于 Binder 代理，则用 handle 记录 Binder 代理的句柄；

关于 localBinder，代码见 Binder.cpp。

```
BBinder* BBinder::localBinder()

{

    return this;

}

BBinder* IBinder::localBinder()
```

```
{  
    return NULL;  
}
```

3.2.3 finish_flatten_binder

```
inline static status_t finish_flatten_binder(  
    const sp<IBinder>& , const flat_binder_object& flat, Parcel* out)  
{  
    return out->writeObject(flat, false);  
}
```

将 flat_binder_object 写入 out。

3.3 BpBinder::transact

[-> BpBinder.cpp]

```
status_t BpBinder::transact(  
    uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags)  
{  
    if (mAlive) {  
        // code=ADD_SERVICE_TRANSACTION【见小节 3.4】  
        status_t status = IPCThreadState::self()->transact(  
            mHandle, code, data, reply, flags);  
        if (status == DEAD_OBJECT) mAlive = 0;  
        return status;  
    }  
    return DEAD_OBJECT;  
}
```

Binder 代理类调用 `transact()` 方法，真正工作还是交给 `IPCThreadState` 来进行 `transact` 工作。先来看看 `IPCThreadState::self` 的过程。

3.3.1 IPCThreadState::self

[-> IPCThreadState.cpp]

```
IPCThreadState* IPCThreadState::self()
{
    if (gHaveTLS) {
restart:
        const pthread_key_t k = gTLS;
        IPCThreadState* st = (IPCThreadState*)pthread_getspecific(k);
        if (st) return st;
        return new IPCThreadState; //初始 IPCThreadState 【见小节 3.3.2】
    }

    if (gShutdown) return NULL;

    pthread_mutex_lock(&gTLSMutex);

    if (!gHaveTLS) { //首次进入 gHaveTLS 为 false
        if (pthread_key_create(&gTLS, threadDestructor) != 0) { //创建线程的 TLS
            pthread_mutex_unlock(&gTLSMutex);
            return NULL;
        }
        gHaveTLS = true;
    }

    pthread_mutex_unlock(&gTLSMutex);
    goto restart;
}
```


TLS 是指 Thread local storage(线程本地储存空间), 每个线程都拥有自己的 TLS, 并且是私有空间, 线程之间不会共享。通过 `pthread_getspecific/pthread_setspecific` 函数可以获取/设置这些空间中的内容。从线程本地存储空间中获得保存在其中的 `IPCThreadState` 对象。

3.3.2 IPCThreadState 初始化

[-> IPCThreadState.cpp]

```
IPCThreadState::IPCThreadState()
: mProcess(ProcessState::self()),
  mMyThreadId(gettid()),
  mStrictModePolicy(0),
  mLastTransactionBinderFlags(0)
{
    pthread_setspecific(gTLS, this);
    clearCaller();
    mIn.setDataCapacity(256);
    mOut.setDataCapacity(256);
}
```

每个线程都有一个 `IPCThreadState`, 每个 `IPCThreadState` 中都有一个 `mIn`、一个 `mOut`。成员变量 `mProcess` 保存了 `ProcessState` 变量(每个进程只有一个)。

- `mIn` 用来接收来自 Binder 设备的数据, 默认大小为 256 字节;
- `mOut` 用来存储发往 Binder 设备的数据, 默认大小为 256 字节。

3.4 IPC::transact

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::transact(int32_t handle,
                                  uint32_t code, const Parcel& data,
```

```
Parcel* reply, uint32_t flags)
{
    status_t err = data.errorCheck(); //数据错误检查

    flags |= TF_ACCEPT_FDS;

    ....

    if (err == NO_ERROR) { // 传输数据 【见小节 3.5】

        err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, NULL);
    }

    ...

    if ((flags & TF_ONE_WAY) == 0) {
        if (reply) {
            //等待响应 【见小节 3.6】

            err = waitForResponse(reply);
        } else {
            Parcel fakeReply;

            err = waitForResponse(&fakeReply);
        }
    }

    //oneway, 则不需要等待 reply 的场景

    err = waitForResponse(NULL, NULL);
}

return err;
}
```

IPCThreadState 进行 transact 事务处理分 3 部分:

- errorCheck() //数据错误检查
- writeTransactionData() // 传输数据

- `waitForResponse()` //f 等待响应

3.5 IPC.writeTransactionData

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::writeTransactionData(int32_t cmd, uint32_t binderFlags,
int32_t handle, uint32_t code, const Parcel& data, status_t* statusBuffer)
{
    binder_transaction_data tr;

    tr.target.ptr = 0;

    tr.target.handle = handle; // handle = 0

    tr.code = code;           // code = ADD_SERVICE_TRANSACTION

    tr.flags = binderFlags;   // binderFlags = 0

    tr.cookie = 0;

    tr.sender_pid = 0;

    tr.sender_euid = 0;

    // data 为记录 Media 服务信息的 Parcel 对象

    const status_t err = data.errorCheck();

    if (err == NO_ERROR) {

        tr.data_size = data.ipcDataSize(); // mDataSize

        tr.data.ptr.buffer = data.ipcData(); //mData

        tr.offsets_size = data.ipcObjectsCount()*sizeof(binder_size_t); //mObjectsSize

        tr.data.ptr.offsets = data.ipcObjects(); //mObjects

    } else if (statusBuffer) {

        ...

    } else {

        return (mLastError = err);
    }
}
```

```
}  
  
    mOut.writeInt32(cmd);           //cmd = BC_TRANSACTION  
  
    mOut.write(&tr, sizeof(tr));    //写入 binder_transaction_data 数据  
  
    return NO_ERROR;  
}
```

其中 `handle` 的值用来标识目的端，注册服务过程的目的端为 `service manager`，此处 `handle=0` 所对应的是 `binder_context_mgr_node` 对象，正是 `service manager` 所对应的 `binder` 实体对象。`binder_transaction_data` 结构体是 `binder` 驱动通信的数据结构，该过程最终是把 `Binder` 请求码 `BC_TRANSACTION` 和 `binder_transaction_data` 结构体写入到 `mOut`。

`transact` 过程，先写完 `binder_transaction_data` 数据，其中 `Parcel data` 的重要成员变量：

- `mDataSize`:保存再 `data_size`，`binder_transaction` 的数据大小；
- `mData`: 保存在 `ptr.buffer`，`binder_transaction` 的数据的起始地址；
- `mObjectsSize`:保存在 `ptr.offsets_size`，记录着 `flat_binder_object` 结构体的个数；
- `mObjects`: 保存在 `offsets`，记录着 `flat_binder_object` 结构体在数据偏移量；

接下来执行 `waitForResponse()` 方法。

3.6 IPC.waitForResponse

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)  
{  
  
    int32_t cmd;  
  
    int32_t err;  
  
    while (1) {
```

```
        if ((err=talkWithDriver()) < NO_ERROR) break; // 【见小节 3.7】  
  
        ...  
  
        if (mIn.dataAvail() == 0) continue;  
  
        cmd = mIn.readInt32();  
  
        switch (cmd) {  
            case BR_TRANSACTION_COMPLETE: ...  
            case BR_DEAD_REPLY: ...  
            case BR_FAILED_REPLY: ...  
            case BR_ACQUIRE_RESULT: ...  
            case BR_REPLY: ...  
            goto finish;  
  
            default:  
                err = executeCommand(cmd); // 【见小节 3.x】  
                if (err != NO_ERROR) goto finish;  
                break;  
        }  
  
        ...  
  
        return err;  
    }  
}
```

在 `waitForResponse` 过程，首先执行 `BR_TRANSACTION_COMPLETE`；另外，目标进程收到事务后，处理 `BR_TRANSACTION` 事务。然后发送给当前进程，再执行 `BR_REPLY` 命令。

3.7 IPC.talkWithDriver

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::talkWithDriver(bool doReceive)
{
    ...

    binder_write_read bwr;

    const bool needRead = mIn.dataPosition() >= mIn.dataSize();

    const size_t outAvail = (!doReceive || needRead) ? mOut.dataSize() : 0;

    bwr.write_size = outAvail;

    bwr.write_buffer = (uintptr_t)mOut.data();

    if (doReceive && needRead) {
        //接收数据缓冲区信息的填充。如果以后收到数据，就直接填在 mIn 中了。

        bwr.read_size = mIn.dataCapacity();

        bwr.read_buffer = (uintptr_t)mIn.data();
    } else {
        bwr.read_size = 0;

        bwr.read_buffer = 0;
    }

    //当读缓冲和写缓冲都为空，则直接返回

    if ((bwr.write_size == 0) && (bwr.read_size == 0)) return NO_ERROR;

    bwr.write_consumed = 0;

    bwr.read_consumed = 0;

    status_t err;

    do {
        //通过 ioctl 不停的读写操作，跟 Binder Driver 进行通信

        if (ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr) >= 0)

            err = NO_ERROR;

        ...

    } while (err == -EINTR); //当被中断，则继续执行
```

```
...  
return err;  
}
```

`binder_write_read` 结构体用来与 Binder 设备交换数据的结构, 通过 `ioctl` 与 `mDriverFD` 通信, 是真正与 Binder 驱动进行数据读写交互的过程。主要是操作 `mOut` 和 `mIn` 变量。

`ioctl()` 经过系统调用后进入 Binder Driver.

四. Binder Driver

`ioctl` -> `binder_ioctl` -> `binder_ioctl_write_read`

4.1 binder_ioctl_write_read

[-> binder.c]

```
static int binder_ioctl_write_read(struct file *filp,  
    unsigned int cmd, unsigned long arg,  
    struct binder_thread *thread)  
{  
    struct binder_proc *proc = filp->private_data;  
    void __user *ubuf = (void __user *)arg;  
    struct binder_write_read bwr;  
  
    //将用户空间 bwr 结构体拷贝到内核空间  
    copy_from_user(&bwr, ubuf, sizeof(bwr));  
    ...  
  
    if (bwr.write_size > 0) {
```

```
//将数据放入目标进程【见小节 4.2】

ret = binder_thread_write(proc, thread,
                           bwr.write_buffer,
                           bwr.write_size,
                           &bwr.write_consumed);

...

}

if (bwr.read_size > 0) {
    //读取自己队列的数据 【见小节】

    ret = binder_thread_read(proc, thread, bwr.read_buffer,
                              bwr.read_size,
                              &bwr.read_consumed,
                              filp->f_flags & O_NONBLOCK);

    if (!list_empty(&proc->todo))
        wake_up_interruptible(&proc->wait);

    ...

}

//将内核空间 bwr 结构体拷贝到用户空间
copy_to_user(ubuf, &bwr, sizeof(bwr));

...

}
```

4.2 binder_thread_write

```
static int binder_thread_write(struct binder_proc *proc,
                               struct binder_thread *thread,
                               binder_uintptr_t binder_buffer, size_t size,
                               binder_size_t *consumed)
{
```



```
uint32_t cmd;

void __user *buffer = (void __user *) (uintptr_t) binder_buffer;

void __user *ptr = buffer + *consumed;

void __user *end = buffer + size;

while (ptr < end && thread->return_error == BR_OK) {
    //拷贝用户空间的 cmd 命令，此时为 BC_TRANSACTION
    if (get_user(cmd, (uint32_t __user *) ptr)) -EFAULT;

    ptr += sizeof(uint32_t);

    switch (cmd) {
        case BC_TRANSACTION:
        case BC_REPLY: {
            struct binder_transaction_data tr;

            //拷贝用户空间的 binder_transaction_data
            if (copy_from_user(&tr, ptr, sizeof(tr))) return -EFAULT;

            ptr += sizeof(tr);

            // 见小节 4.3】
            binder_transaction(proc, thread, &tr, cmd == BC_REPLY);

            break;
        }
        ...
    }

    *consumed = ptr - buffer;
}

return 0;
}
```

4.3 binder_transaction

```
static void binder_transaction(struct binder_proc *proc,
                               struct binder_thread *thread,
```

```
        struct binder_transaction_data *tr, int reply){  
    struct binder_transaction *t;  
    struct binder_work *tcomplete;  
    ...  
  
    if (reply) {  
        ...  
    } else {  
        if (tr->target.handle) {  
            ...  
        } else {  
            // handle=0 则找到 servicemanager 实体  
            target_node = binder_context_mgr_node;  
        }  
        //target_proc 为 servicemanager 进程  
        target_proc = target_node->proc;  
    }  
  
    if (target_thread) {  
        ...  
    } else {  
        //找到 servicemanager 进程的 todo 队列  
        target_list = &target_proc->todo;  
        target_wait = &target_proc->wait;  
    }  
  
    t = kzalloc(sizeof(*t), GFP_KERNEL);  
    tcomplete = kzalloc(sizeof(*tcomplete), GFP_KERNEL);  
  
    //非 oneway 的通信方式，把当前 thread 保存到 transaction 的 from 字段
```

```
if (!reply && !(tr->flags & TF_ONE_WAY))  
  
    t->from = thread;  
  
else  
  
    t->from = NULL;  
  
  
    t->sender_euid = task_euid(proc->tsk);  
    t->to_proc = target_proc; //此次通信目标进程为 servicemanager 进程  
    t->to_thread = target_thread;  
    t->code = tr->code; //此次通信 code = ADD_SERVICE_TRANSACTION  
    t->flags = tr->flags; // 此次通信 flags = 0  
    t->priority = task_nice(current);  
  
  
    //从 servicemanager 进程中分配 buffer  
    t->buffer = binder_alloc_buf(target_proc, tr->data_size,  
        tr->offsets_size, !reply && (t->flags & TF_ONE_WAY));  
  
  
    t->buffer->allow_user_free = 0;  
    t->buffer->transaction = t;  
    t->buffer->target_node = target_node;  
  
  
    if (target_node)  
        binder_inc_node(target_node, 1, 0, NULL); //引用计数加 1  
    offp = (binder_size_t *)(t->buffer->data + ALIGN(tr->data_size, sizeof(void  
*))) );  
  
  
    //分别拷贝用户空间的 binder_transaction_data 中 ptr.buffer 和 ptr.offsets 到内核  
    copy_from_user(t->buffer->data,  
        (const void __user *) (uintptr_t) tr->data.ptr.buffer, tr->data_size);  
    copy_from_user(offp,  
        (const void __user *) (uintptr_t) tr->data.ptr.offsets, tr->offsets_size);
```

```
    off_end = (void *)offp + tr->offsets_size;

    for (; offp < off_end; offp++) {
        struct flat_binder_object *fp;

        fp = (struct flat_binder_object *) (t->buffer->data + *offp);

        off_min = *offp + sizeof(struct flat_binder_object);

        switch (fp->type) {
            case BINDER_TYPE_BINDER:
            case BINDER_TYPE_WEAK_BINDER: {
                struct binder_ref *ref;

                // 【见 4.3.1】

                struct binder_node *node = binder_get_node(proc, fp->binder);

                if (node == NULL) {
                    //服务所在进程 创建 binder_node 实体【见 4.3.2】

                    node = binder_new_node(proc, fp->binder, fp->cookie);

                    ...
                }

                //servicemanager 进程 binder_ref【见 4.3.3】

                ref = binder_get_ref_for_node(target_proc, node);

                ...

                //调整 type 为 HANDLE 类型

                if (fp->type == BINDER_TYPE_BINDER)

                    fp->type = BINDER_TYPE_HANDLE;

                else

                    fp->type = BINDER_TYPE_WEAK_HANDLE;

                fp->binder = 0;

                fp->handle = ref->desc; //设置 handle 值

                fp->cookie = 0;

                binder_inc_ref(ref, fp->type == BINDER_TYPE_HANDLE,
```

```
        &thread->todo);  
  
        } break;  
  
        case :...  
    }  
  
    if (reply) {  
        ..  
    } else if (!(t->flags & TF_ONE_WAY)) {  
        //BC_TRANSACTION 且 非 oneway, 则设置事务栈信息  
        t->need_reply = 1;  
        t->from_parent = thread->transaction_stack;  
        thread->transaction_stack = t;  
    } else {  
        ...  
    }  
  
    //将 BINDER_WORK_TRANSACTION 添加到目标队列，本次通信的目标队列为 target_proc->to  
do  
    t->work.type = BINDER_WORK_TRANSACTION;  
    list_add_tail(&t->work.entry, target_list);  
  
    //将 BINDER_WORK_TRANSACTION_COMPLETE 添加到当前线程的 todo 队列  
    tcomplete->type = BINDER_WORK_TRANSACTION_COMPLETE;  
    list_add_tail(&tcomplete->entry, &thread->todo);  
  
    //唤醒等待队列，本次通信的目标队列为 target_proc->wait  
    if (target_wait)  
        wake_up_interruptible(target_wait);  
  
    return;  
}
```

注册服务的过程，传递的是 BBinder 对象，故[小节 3.2.1]的 writeStrongBinder() 过程中 localBinder 不为空，从而 flat_binder_object.type 等于 BINDER_TYPE_BINDER。

服务注册过程是在服务所在进程创建 binder_node，在 servicemanager 进程创建 binder_ref。对于同一个 binder_node，每个进程只会创建一个 binder_ref 对象。

向 servicemanager 的 binder_proc->todo 添加 BINDER_WORK_TRANSACTION 事务，接下来进入 ServiceManager 进程。

4.3.1 binder_get_node

```
static struct binder_node *binder_get_node(struct binder_proc *proc,
                                           binder_uintptr_t ptr)
{
    struct rb_node *n = proc->nodes.rb_node;
    struct binder_node *node;

    while (n) {
        node = rb_entry(n, struct binder_node, rb_node);

        if (ptr < node->ptr)
            n = n->rb_left;
        else if (ptr > node->ptr)
            n = n->rb_right;
        else
            return node;
    }
    return NULL;
}
```

从 binder_proc 来根据 binder 指针 ptr 值，查询相应的 binder_node。

4.3.2 binder_new_node

```
static struct binder_node *binder_new_node(struct binder_proc *proc,
                                           binder_uintptr_t ptr,
                                           binder_uintptr_t cookie)
{
    struct rb_node **p = &proc->nodes.rb_node;
    struct rb_node *parent = NULL;
    struct binder_node *node;
    ... //红黑树位置查找

    //给新创建的 binder_node 分配内核空间
    node = kzalloc(sizeof(*node), GFP_KERNEL);

    // 将新创建的 node 添加到 proc 红黑树:
    rb_link_node(&node->rb_node, parent, p);
    rb_insert_color(&node->rb_node, &proc->nodes);
    node->debug_id = ++binder_last_id;
    node->proc = proc;
    node->ptr = ptr;
    node->cookie = cookie;
    node->work.type = BINDER_WORK_NODE; //设置 binder_work 的 type
    INIT_LIST_HEAD(&node->work.entry);
    INIT_LIST_HEAD(&node->async_todo);

    return node;
}
```

4.3.3 binder_get_ref_for_node

```
static struct binder_ref *binder_get_ref_for_node(struct binder_proc *proc,
                                                  struct binder_node *node)
```

```
{
    struct rb_node *n;

    struct rb_node **p = &proc->refs_by_node.rb_node;

    struct rb_node *parent = NULL;

    struct binder_ref *ref, *new_ref;

    //从 refs_by_node 红黑树，找到 binder_ref 则直接返回。
    while (*p) {
        parent = *p;

        ref = rb_entry(parent, struct binder_ref, rb_node_node);

        if (node < ref->node)
            p = &(*p)->rb_left;
        else if (node > ref->node)
            p = &(*p)->rb_right;
        else
            return ref;
    }

    //创建 binder_ref
    new_ref = kzalloc_preempt_disabled(sizeof(*ref));

    new_ref->debug_id = ++binder_last_id;
    new_ref->proc = proc; //记录进程信息
    new_ref->node = node; //记录 binder 节点
    rb_link_node(&new_ref->rb_node_node, parent, p);
    rb_insert_color(&new_ref->rb_node_node, &proc->refs_by_node);

    //计算 binder 引用的 handle 值，该值返回给 target_proc 进程
    new_ref->desc = (node == binder_context_mgr_node) ? 0 : 1;
}
```



```
//从红黑树最最左边的 handle 对比，依次递增，直到红黑树遍历结束或者找到更大的 handle 则结束。

for (n = rb_first(&proc->refs_by_desc); n != NULL; n = rb_next(n)) {

    //根据 binder_ref 的成员变量 rb_node_desc 的地址指针 n，来获取 binder_ref 的首地址
    ref = rb_entry(n, struct binder_ref, rb_node_desc);

    if (ref->desc > new_ref->desc)

        break;

    new_ref->desc = ref->desc + 1;

}

// 将新创建的 new_ref 插入 proc->refs_by_desc 红黑树

p = &proc->refs_by_desc.rb_node;

while (*p) {

    parent = *p;

    ref = rb_entry(parent, struct binder_ref, rb_node_desc);

    if (new_ref->desc < ref->desc)

        p = &(*p)->rb_left;

    else if (new_ref->desc > ref->desc)

        p = &(*p)->rb_right;

    else

        BUG();

}

rb_link_node(&new_ref->rb_node_desc, parent, p);

rb_insert_color(&new_ref->rb_node_desc, &proc->refs_by_desc);

if (node) {

    hlist_add_head(&new_ref->node_entry, &node->refs);

}

return new_ref;

}
```

handle 值计算方法规律:

- 每个进程 binder_proc 所记录的 binder_ref 的 handle 值是从 1 开始递增的;
- 所有进程 binder_proc 所记录的 handle=0 的 binder_ref 都指向 service manager;
- 同一个服务的 binder_node 在不同进程的 binder_ref 的 handle 值可以不同;

五. ServiceManager

由 [Binder 系列 3—启动 ServiceManager](#) 已介绍其原理, 循环在 binder_loop() 过程, 会调用 binder_parse()方法。

5.1 binder_parse

[-> servicemanager/binder.c]

```
int binder_parse(struct binder_state *bs, struct binder_io *bio, uintptr_t ptr,
size_t size, binder_handler func) {
    int r = 1;
    uintptr_t end = ptr + (uintptr_t) size;

    while (ptr < end) {
        uint32_t cmd = *(uint32_t *) ptr;
        ptr += sizeof(uint32_t);
        switch(cmd) {
            case BR_TRANSACTION: {
                struct binder_transaction_data *txn = (struct binder_transaction_data
                *) ptr;
                ...
                binder_dump_txn(txn);
            }
            if (func) {
```

```
        unsigned rdata[256/4];

        struct binder_io msg;

        struct binder_io reply;

        int res;

        bio_init(&reply, rdata, sizeof(rdata), 4);

        bio_init_from_txn(&msg, txn); //从 txn 解析出 binder_io 信息

        // 收到 Binder 事务 【见小节 5.2】

        res = func(bs, txn, &msg, &reply);

        // 发送 reply 事件【见小节 5.4】

        binder_send_reply(bs, &reply, txn->data.ptr.buffer, res);

    }

    ptr += sizeof(*txn);

    break;

}

case : ...

}

return r;

}
```

5.2 svcmgr_handler

[-> service_manager.c]

```
int svcmgr_handler(struct binder_state *bs, struct binder_transaction_data *tx
n, struct binder_io *msg, struct binder_io *reply) {

    struct svcinfo *si;

    uint16_t *s;

    size_t len;

    uint32_t handle;
```

```
uint32_t strict_policy;

int allow_isolated;

...

strict_policy = bio_get_uint32(msg);

s = bio_get_string16(msg, &len);

...

switch(txn->code) {

case SVC_MGR_ADD_SERVICE:

    s = bio_get_string16(msg, &len);

    ...

    handle = bio_get_ref(msg); //获取 handle

    allow_isolated = bio_get_uint32(msg) ? 1 : 0;

    //注册指定服务 【见小节 5.3】

    if (do_add_service(bs, s, len, handle, txn->sender_euid,
        allow_isolated, txn->sender_pid))

        return -1;

    break;

case :...

}

bio_put_uint32(reply, 0);

return 0;

}
```

5.3 do_add_service

[-> service_manager.c]

```
int do_add_service(struct binder_state *bs,
```

```
const uint16_t *s, size_t len,
uint32_t handle, uid_t uid, int allow_isolated,
pid_t spid)
{
    struct svcinfo *si;

    if (!handle || (len == 0) || (len > 127))
        return -1;

    //权限检查
    if (!svc_can_register(s, len, spid)) {
        return -1;
    }

    //服务检索
    si = find_svc(s, len);

    if (si) {
        if (si->handle) {
            svcinfo_death(bs, si); //服务已注册时，释放相应的服务
        }
        si->handle = handle;
    } else {
        si = malloc(sizeof(*si) + (len + 1) * sizeof(uint16_t));
        if (!si) { //内存不足，无法分配足够内存
            return -1;
        }
        si->handle = handle;
        si->len = len;
        memcpy(si->name, s, (len + 1) * sizeof(uint16_t)); //内存拷贝服务信息
        si->name[len] = '\0';
    }
}
```

```
    si->death.func = (void*) svcinfo_death;

    si->death.ptr = si;

    si->allow_isolated = allow_isolated;

    si->next = svclist; // svclist 保存所有已注册的服务

    svclist = si;

}

//以 BC_ACQUIRE 命令, handle 为目标的信息, 通过 ioctl 发送给 binder 驱动
binder_acquire(bs, handle);

//以 BC_REQUEST_DEATH_NOTIFICATION 命令的信息, 通过 ioctl 发送给 binder 驱动, 主要用于清理内存等收尾工作。
binder_link_to_death(bs, handle, &si->death);

return 0;
}
```

svcinfo 记录着服务名和 handle 信息, 保存到 svclist 列表。

5.4 binder_send_reply

[-> servicemanager/binder.c]

```
void binder_send_reply(struct binder_state *bs, struct binder_io *reply, binder_uintptr_t buffer_to_free, int status) {

    struct {

        uint32_t cmd_free;

        binder_uintptr_t buffer;

        uint32_t cmd_reply;

        struct binder_transaction_data txn;

    } __attribute__((packed)) data;

    data.cmd_free = BC_FREE_BUFFER; //free buffer 命令
```

```
data.buffer = buffer_to_free;

data.cmd_reply = BC_REPLY; // reply 命令

data.txn.target.ptr = 0;

data.txn.cookie = 0;

data.txn.code = 0;

if (status) {
    ...
} else {

    data.txn.flags = 0;

    data.txn.data_size = reply->data - reply->data0;

    data.txn.offsets_size = ((char*) reply->offs) - ((char*) reply->offs0);

    data.txn.data.ptr.buffer = (uintptr_t)reply->data0;

    data.txn.data.ptr.offsets = (uintptr_t)reply->offs0;

}

//向 Binder 驱动通信

binder_write(bs, &data, sizeof(data));

}
```

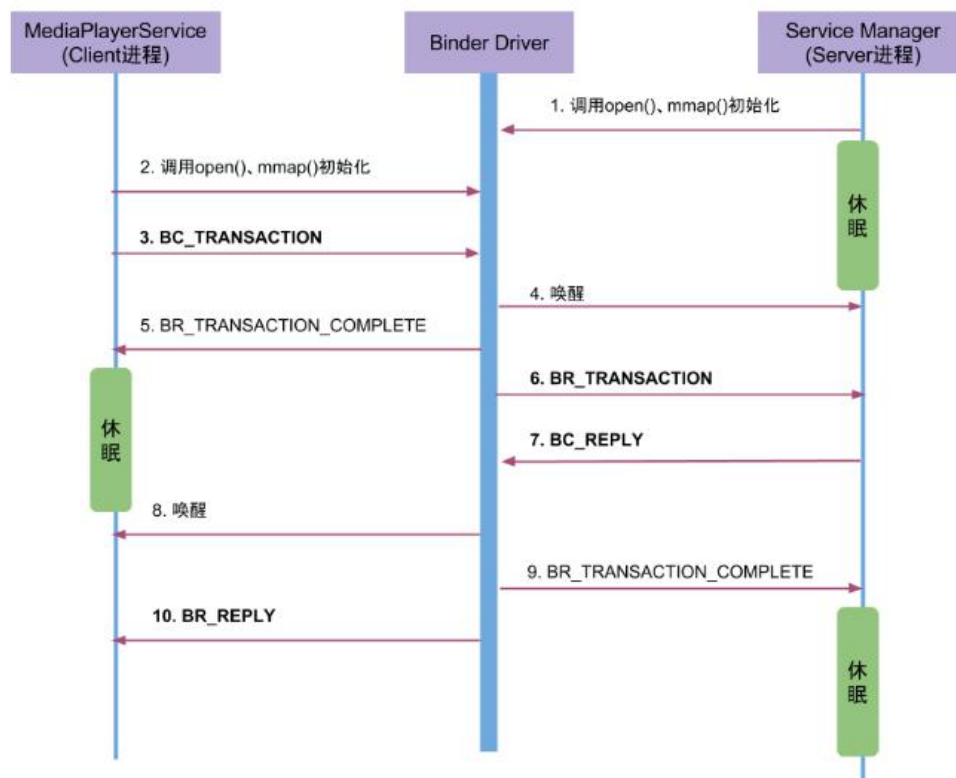
binder_write 进入 binder 驱动后，将 BC_FREE_BUFFER 和 BC_REPLY 命令协议发送给 Binder 驱动，向 client 端发送 reply。

六. 总结

服务注册过程(addService)核心功能：在服务所在进程创建 binder_node，在 servicemanager 进程创建 binder_ref。其中 binder_ref 的 desc 再同一个进程内是唯一的：

- 每个进程 binder_proc 所记录的 binder_ref 的 handle 值是从 1 开始递增的；
- 所有进程 binder_proc 所记录的 handle=0 的 binder_ref 都指向 service manager；
- 同一个服务的 binder_node 在不同进程的 binder_ref 的 handle 值可以不同；

Media 服务注册的过程涉及到 MediaPlayerService(作为 Client 进程)和 Service Manager(作为 Service 进程), 通信流程图如下所示:



过程分析:

1. MediaPlayerService 进程调用 `ioctl()` 向 Binder 驱动发送 IPC 数据, 该过程可以理解成一个事务 `binder_transaction` (记为 `T1`), 执行当前操作的线程 `binder_thread` (记为 `thread1`), 则 `T1->from_parent=NULL`, `T1->from=thread1`, `thread1->transaction_stack=T1`。其中 IPC 数据内容包含:

- Binder 协议为 `BC_TRANSACTION`;
- Handle 等于 0;
- RPC 代码为 `ADD_SERVICE`;
- RPC 数据为 "media.player"。

2.

Binder 驱动收到该 Binder 请求, 生成 `BR_TRANSACTION` 命令, 选择目标处理该请求的线程, 即 ServiceManager 的 binder 线程 (记为 `thread2`), 则

T1->to_parent = NULL, T1->to_thread = thread2。并将整个 binder_transaction 数据(记为 T2)插入到目标线程的 todo 队列;

3.

4.

Service Manager 的线程 thread2 收到 T2 后, 调用服务注册函数将服务"media.player"注册到服务目录中。当服务注册完成后, 生成 IPC 应答数据(BC_REPLY), T2->from_parent = T1, T2->from = thread2, thread2->transaction_stack = T2。

5.

6. Binder 驱动收到该 Binder 应答请求, 生成 BR_REPLY 命令, T2->to_parent = T1, T2->to_thread = thread1, thread1->transaction_stack = T2。在 MediaPlayerService 收到该命令后, 知道服务注册完成便可以正常使用。

整个过程中, BC_TRANSACTION 和 BR_TRANSACTION 过程是一个完整的事务过程; BC_REPLY 和 BR_REPLY 是一个完整的事务过程。到此, 其他进行便可以获取该服务, 使用服务提供的方法, 下一篇文章将会讲述[如何获取服务](#)。

第七节获取服务(getService)

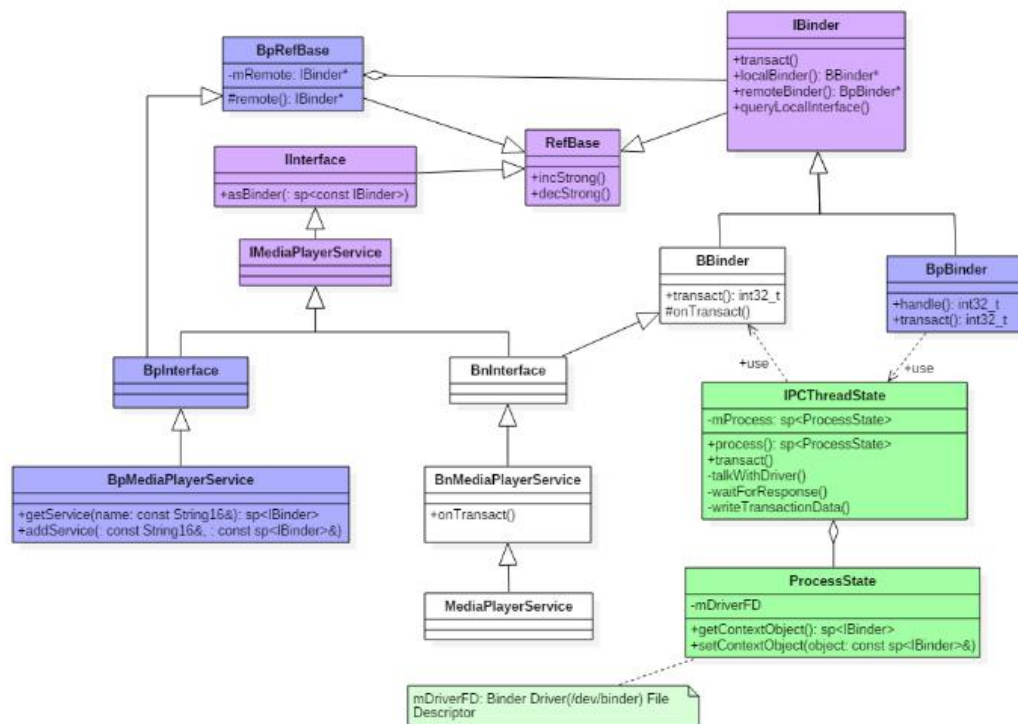
基于 Android 6.0 的源码剖析, 本文 Client 如何向 Server 获取服务的过程。

一、 获取服务

在 Native 层的服务注册, 我们选择以 media 为例来展开讲解, 先来看看 media 的类关系图。

1.1 类图

[点击查看大图](#)



图解:

- 蓝色: 代表获取 MediaPlayerService 服务相关的类;
- 绿色: 代表 Binder 架构中与 Binder 驱动通信过程中的最为核心的两个类;
- 紫色: 代表注册服务和获取服务的公共接口/父类;

二. 获取 Media 服务

2.1 getMediaPlayerService

```
[-> framework/av/media/libmedia/IMediaDeathNotifier.cpp]
```

```
sp<IMediaPlayerService>&  
IMediaDeathNotifier::getMediaPlayerService()  
{  
    Mutex::Autolock _l(sServiceLock);
```

```
if (sMediaPlayerService == 0) {  
    sp<IServiceManager> sm = defaultServiceManager(); //获取 ServiceManager  
    sp<IBinder> binder;  
    do {  
        //获取名为"media.player"的服务 【见 2.2】  
        binder = sm->getService(String16("media.player"));  
        if (binder != 0) {  
            break;  
        }  
        usleep(500000); // 0.5s  
    } while (true);  
  
    if (sDeathNotifier == NULL) {  
        sDeathNotifier = new DeathNotifier(); //创建死亡通知对象  
    }  
  
    //将死亡通知连接到 binder 【见流程 14】  
    binder->linkToDeath(sDeathNotifier);  
    sMediaPlayerService = interface_cast<IMediaPlayerService>(binder);  
}  
return sMediaPlayerService;  
}
```

其中 defaultServiceManager()过程在上一篇文章[获取 ServiceManager](#) 已讲过，返回 BpServiceManager。

在请求获取名为"media.player"的服务过程中，采用不断循环获取的方法。由于 MediaPlayerService 服务可能还没向 ServiceManager 注册完成或者尚未启动完成等情况，故则 binder 返回为 NULL，休眠 0.5s 后继续请求，直到获取服务为止。

2.2 BpSM.getService

更多录播视频+架构学习资料免费领取 QQ 3607409029

[-> IServiceManager.cpp ::BpServiceManager]

```
virtual sp<IBinder> getService(const String16& name) const
{
    unsigned n;
    for (n = 0; n < 5; n++){
        sp<IBinder> svc = checkService(name); // 【见 2.3】
        if (svc != NULL) return svc;
        sleep(1);
    }
    return NULL;
}
```

通过 BpServiceManager 来获取 MediaPlayer 服务: 检索服务是否存在, 当服务存在则返回相应的服务, 当服务不存在则休眠 1s 再继续检索服务。该循环进行 5 次。为什么是循环 5 次呢, 这估计跟 Android 的 ANR 时间为 5s 相关。如果每次都无法获取服务, 循环 5 次, 每次循环休眠 1s, 忽略 checkService() 的时间, 差不多就是 5s 的时间

2.3 BpSM.checkService

[-> IServiceManager.cpp ::BpServiceManager]

```
virtual sp<IBinder> checkService(const String16& name) const
{
    Parcel data, reply;
    //写入 RPC 头
    data.writeInterfaceToken(IServiceManager::getInterfaceDescriptor());
    //写入服务名
    data.writeString16(name);
    remote()->transact(CHECK_SERVICE_TRANSACTION, data, &reply); // 【见 2.4】
    return reply.readStrongBinder(); // 【见小节 2.9】
}
```

```
}
```

检索指定服务是否存在，其中 `remote()` 为 `BpBinder`。

2.4 BpBinder::transact

[-> BpBinder.cpp]

```
status_t BpBinder::transact(  
    uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags)  
{  
    if (mAlive) {  
        // 【见流程 2.5】  
        status_t status = IPCThreadState::self()->transact(  
            mHandle, code, data, reply, flags);  
        if (status == DEAD_OBJECT) mAlive = 0;  
        return status;  
    }  
    return DEAD_OBJECT;  
}
```

Binder 代理类调用 `transact()` 方法，真正工作还是交给 `IPCThreadState` 来进行 `transact` 工作，

2.4.1 IPCThreadState::self

[-> IPCThreadState.cpp]

```
IPCThreadState* IPCThreadState::self()  
{  
    if (gHaveTLS) {
```

```
restart:

    const pthread_key_t k = gTLS;

    IPCThreadState* st = (IPCThreadState*)pthread_getspecific(k);

    if (st) return st;

    return new IPCThreadState; //初始 IPCThreadState 【见小节 2.4.2】

}

if (gShutdown) return NULL;

pthread_mutex_lock(&gTLSMutex);

if (!gHaveTLS) { //首次进入 gHaveTLS 为 false

    if (pthread_key_create(&gTLS, threadDestructor) != 0) { //创建线程的 TLS

        pthread_mutex_unlock(&gTLSMutex);

        return NULL;

    }

    gHaveTLS = true;

}

pthread_mutex_unlock(&gTLSMutex);

goto restart;

}
```

TLS 是指 Thread local storage(线程本地储存空间), 每个线程都拥有自己的 TLS, 并且是私有空间, 线程之间不会共享。通过 `pthread_getspecific/pthread_setspecific` 函数可以获取/设置这些空间中的内容。从线程本地存储空间中获得保存在其中的 `IPCThreadState` 对象。

2.4.2 IPCThreadState 初始化

[-> IPCThreadState.cpp]

```
IPCThreadState::IPCThreadState()

: mProcess(ProcessState::self()),
```

```
mMyThreadId(gettid()),  
mStrictModePolicy(0),  
mLastTransactionBinderFlags(0)  
{  
    pthread_setspecific(gTLS, this);  
    clearCaller();  
    mIn.setDataCapacity(256);  
    mOut.setDataCapacity(256);  
}
```

每个线程都有一个 `IPCThreadState`，每个 `IPCThreadState` 中都有一个 `mIn`、一个 `mOut`。成员变量 `mProcess` 保存了 `ProcessState` 变量(每个进程只有一个)。

- `mIn` 用来接收来自 `Binder` 设备的数据，默认大小为 256 字节；
- `mOut` 用来存储发往 `Binder` 设备的数据，默认大小为 256 字节。

2.5 IPC::transact

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::transact(int32_t handle,  
                                uint32_t code, const Parcel& data,  
                                Parcel* reply, uint32_t flags)  
{  
    status_t err = data.errorCheck(); //数据错误检查  
    flags |= TF_ACCEPT_FDS;  
    ....  
    if (err == NO_ERROR) {  
        // 传输数据 【见流程 2.6】  
        err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, NULL);  
    }  
}
```

```
    if (err != NO_ERROR) {  
        if (reply) reply->setError(err);  
        return (mLastError = err);  
    }  
  
    if ((flags & TF_ONE_WAY) == 0) { //flags=0 进入该分支  
        if (reply) {  
            //等待响应 【见流程 2.7】  
            err = waitForResponse(reply);  
        } else {  
            Parcel fakeReply;  
            err = waitForResponse(&fakeReply);  
        }  
  
    } else {  
        //不需要响应消息的 binder 则进入该分支  
        err = waitForResponse(NULL, NULL);  
    }  
    return err;  
}
```

2.6 IPC.writeTransactionData

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::writeTransactionData(int32_t cmd, uint32_t binderFlag  
s,  
    int32_t handle, uint32_t code, const Parcel& data, status_t* statusBuffer)  
{
```



```
binder_transaction_data tr;

tr.target.ptr = 0;

tr.target.handle = handle; // handle = 0

tr.code = code;           // code = CHECK_SERVICE_TRANSACTION

tr.flags = binderFlags;   // binderFlags = 0

tr.cookie = 0;

tr.sender_pid = 0;

tr.sender_euid = 0;


// data 为记录 Media 服务信息的 Parcel 对象

const status_t err = data.errorCheck();

if (err == NO_ERROR) {

    tr.data_size = data.ipcDataSize(); // mDataSize

    tr.data.ptr.buffer = data.ipcData(); // mData

    tr.offsets_size = data.ipcObjectsCount()*sizeof(binder_size_t); // mObjectsSize

    tr.data.ptr.offsets = data.ipcObjects(); // mObjects

} else if (statusBuffer) {

    ...

} else {

    return (mLastError = err);

}


mOut.writeInt32(cmd); //cmd = BC_TRANSACTION

mOut.write(&tr, sizeof(tr)); //写入 binder_transaction_data 数据

return NO_ERROR;

}
```

其中 handle 的值用来标识目的端，注册服务过程的目的端为 service manager，此处 handle=0 所对应的是 binder_context_mgr_node 对象，正是 service manager 所对应的 binder 实体对象。binder_transaction_data 结构体是 binder

驱动通信的数据结构，该过程最终是把 Binder 请求码 BC_TRANSACTION 和 binder_transaction_data 结构体写入到 mOut。

2.7 IPC.waitForResponse

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)
{
    int32_t cmd;
    int32_t err;

    while (1) {
        if ((err=talkWithDriver()) < NO_ERROR) break; // 【见流程 2.8】
        err = mIn.errorCheck();
        if (err < NO_ERROR) break;
        if (mIn.dataAvail() == 0) continue;

        cmd = mIn.readInt32();
        switch (cmd) {
            case BR_TRANSACTION_COMPLETE: ...
            case BR_DEAD_REPLY: ...
            case BR_FAILED_REPLY: ...
            case BR_ACQUIRE_RESULT: ...
            case BR_REPLY:
            {
                binder_transaction_data tr;
                err = mIn.read(&tr, sizeof(tr));
                if (reply) {
                    if ((tr.flags & TF_STATUS_CODE) == 0) {
```

```
        reply->ipcSetDataReference(
            reinterpret_cast<const uint8_t*>(tr.data.ptr.buffer),
            tr.data_size,
            reinterpret_cast<const binder_size_t*>(tr.data.ptr.offsets),
            tr.offsets_size/sizeof(binder_size_t),
            freeBuffer, this);
    } else {
        ...
    }
}
}
}
goto finish;

default:
    err = executeCommand(cmd);
    if (err != NO_ERROR) goto finish;
    break;
}
}
...
return err;
}
```

2.8 IPC.talkWithDriver

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::talkWithDriver(bool doReceive)
{
```

```
...

binder_write_read bwr;

const bool needRead = mIn.dataPosition() >= mIn.dataSize();

const size_t outAvail = (!doReceive || needRead) ? mOut.dataSize() : 0;

bwr.write_size = outAvail;

bwr.write_buffer = (uintptr_t)mOut.data();

if (doReceive && needRead) {

    //接收数据缓冲区信息的填充。如果以后收到数据，就直接填在 mIn 中了。

    bwr.read_size = mIn.dataCapacity();

    bwr.read_buffer = (uintptr_t)mIn.data();

} else {

    bwr.read_size = 0;

    bwr.read_buffer = 0;

}

//当读缓冲和写缓冲都为空，则直接返回

if ((bwr.write_size == 0) && (bwr.read_size == 0)) return NO_ERROR;

bwr.write_consumed = 0;

bwr.read_consumed = 0;

status_t err;

do {

    //通过 ioctl 不停的读写操作，跟 Binder Driver 进行通信【2.8.1】

    if (ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr) >= 0)

        err = NO_ERROR;

    ...

} while (err == -EINTR); //当被中断，则继续执行

...

return err;
```

}

binder_write_read 结构体用来与 Binder 设备交换数据的结构, 通过 `ioctl` 与 `mDriverFD` 通信, 是真正与 Binder 驱动进行数据读写交互的过程。先向 `service manager` 进程发送查询服务的请求(`BR_TRANSACTION`), 见 [Binder 系列 3—启动 ServiceManager](#)。当 `service manager` 进程收到该命令后, 会执行 `do_find_service()` 查询服务所对应的 `handle`, 然后再 `binder_send_reply()` 应答发起者, 发送 `BC_REPLY` 协议, 然后调用 `binder_transaction()`, 再向服务请求者的 `Todo` 队列 插入事务。

接下来, 再看看 `binder_transaction` 过程。

2.8.1 binder_transaction

```
static void binder_transaction(struct binder_proc *proc,
                               struct binder_thread *thread,
                               struct binder_transaction_data *tr, int reply){
    //根据各种判定, 获取以下信息:

    struct binder_thread *target_thread; //目标线程
    struct binder_proc *target_proc;     //目标进程
    struct binder_node *target_node;     //目标 binder 节点
    struct list_head *target_list;       //目标 TODO 队列
    wait_queue_head_t *target_wait;     //目标等待队列
    ...

    //分配两个结构体内存
    struct binder_transaction *t = kzalloc(sizeof(*t), GFP_KERNEL);
    struct binder_work *tcomplete = kzalloc(sizeof(*tcomplete), GFP_KERNEL);

    //从 target_proc 分配一块 buffer
    t->buffer = binder_alloc_buf(target_proc, tr->data_size,

    for (; offp < off_end; offp++) {
        switch (fp->type) {
```

```
case BINDER_TYPE_BINDER: ...

case BINDER_TYPE_WEAK_BINDER: ...


case BINDER_TYPE_HANDLE:

case BINDER_TYPE_WEAK_HANDLE: {

    struct binder_ref *ref = binder_get_ref(proc, fp->handle,

        fp->type == BINDER_TYPE_HANDLE);

    ...

    //此时运行在 servicemanager 进程，故 ref->node 是指向服务所在进程的 binder 实
    体，

    //而 target_proc 为请求服务所在的进程，此时并不相等。

    if (ref->node->proc == target_proc) {

        if (fp->type == BINDER_TYPE_HANDLE)

            fp->type = BINDER_TYPE_BINDER;

        else

            fp->type = BINDER_TYPE_WEAK_BINDER;

        fp->binder = ref->node->ptr;

        fp->cookie = ref->node->cookie; //BBinder 服务的地址

        binder_inc_node(ref->node, fp->type == BINDER_TYPE_BINDER, 0, NULL);

    } else {

        struct binder_ref *new_ref;

        //请求服务所在进程并非服务所在进程，则为请求服务所在进程创建 binder_ref

        new_ref = binder_get_ref_for_node(target_proc, ref->node);

        fp->binder = 0;

        fp->handle = new_ref->desc; //重新赋予 handle 值

        fp->cookie = 0;

        binder_inc_ref(new_ref, fp->type == BINDER_TYPE_HANDLE, NULL);

    }

} break;
```

```
        case BINDER_TYPE_FD: ...
    }
}

//分别 target_list 和当前线程 TODO 队列插入事务
t->work.type = BINDER_WORK_TRANSACTION;
list_add_tail(&t->work.entry, target_list);
tcomplete->type = BINDER_WORK_TRANSACTION_COMPLETE;
list_add_tail(&tcomplete->entry, &thread->todo);

if (target_wait)
    wake_up_interruptible(target_wait);

return;
}
```

这个过程非常重要，分两种情况来说：

1. 当请求服务的进程与服务属于不同进程，则为请求服务所在进程创建 `binder_ref` 对象，指向服务进程中的 `binder_node`；
2. 当请求服务的进程与服务属于同一进程，则不再创建新对象，只是引用计数加 1，并且修改 `type` 为 `BINDER_TYPE_BINDER` 或 `BINDER_TYPE_WEAK_BINDER`。

2.8.2 binder_thread_read

```
binder_thread_read (...) {
    ...

    //当线程 todo 队列有数据则执行往下执行；当线程 todo 队列没有数据，则进入休眠等待状态
    ret = wait_event_freezable(thread->wait, binder_has_thread_work(thread));

    ...

    while (1) {
        uint32_t cmd;

        struct binder_transaction_data tr;
```

```
struct binder_work *w;

struct binder_transaction *t = NULL;

//先从线程 todo 队列获取事务数据

if (!list_empty(&thread->todo)) {

    w = list_first_entry(&thread->todo, struct binder_work, entry);

    // 线程 todo 队列没有数据，则从进程 todo 对获取事务数据

} else if (!list_empty(&proc->todo) && wait_for_proc_work) {

    ...

}

switch (w->type) {

    case BINDER_WORK_TRANSACTION:

        //获取 transaction 数据

        t = container_of(w, struct binder_transaction, work);

        break;

    case : ...

}

//只有 BINDER_WORK_TRANSACTION 命令才能继续往下执行

if (!t) continue;

if (t->buffer->target_node) {

    ...

} else {

    tr.target.ptr = NULL;

    tr.cookie = NULL;

    cmd = BR_REPLY; //设置命令为 BR_REPLY

}

tr.code = t->code;

tr.flags = t->flags;
```



```
tr.sender_euid = t->sender_euid;

if (t->from) {
    struct task_struct *sender = t->from->proc->tsk;
    //当非 oneway 的情况下,将调用者进程的 pid 保存到 sender_pid
    tr.sender_pid = task_tgid_nr_ns(sender, current->nsproxy->pid_ns);
} else {
    ...
}

tr.data_size = t->buffer->data_size;
tr.offsets_size = t->buffer->offsets_size;
tr.data.ptr.buffer = (void *)t->buffer->data +
    proc->user_buffer_offset;
tr.data.ptr.offsets = tr.data.ptr.buffer +
    ALIGN(t->buffer->data_size,
        sizeof(void *));

//将 cmd 和数据写回用户空间
put_user(cmd, (uint32_t __user *)ptr);
ptr += sizeof(uint32_t);
copy_to_user(ptr, &tr, sizeof(tr));
ptr += sizeof(tr);

list_del(&t->work.entry);
t->buffer->allow_user_free = 1;
if (cmd == BR_TRANSACTION && !(t->flags & TF_ONE_WAY)) {
    ...
} else {
    t->buffer->transaction = NULL;
```

```
kfree(t); //通信完成则运行释放

}

break;

}

done:

    *consumed = ptr - buffer;

    if (proc->requested_threads + proc->ready_threads == 0 &&
        proc->requested_threads_started < proc->max_threads &&
        (thread->looper & (BINDER_LOOPER_STATE_REGISTERED |
        BINDER_LOOPER_STATE_ENTERED))) {

        proc->requested_threads++;

        // 生成 BR_SPAWN_LOOPER 命令，用于创建新的线程

        put_user(BR_SPAWN_LOOPER, (uint32_t __user *)buffer);

    }

    return 0;

}
```

2.9 readStrongBinder

[-> Parcel.cpp]

```
sp<IBinder> Parcel::readStrongBinder() const
{
    sp<IBinder> val;

    //【见小节 2.9.1】

    unflatten_binder(ProcessState::self(), *this, &val);

    return val;
}
```

2.9.1 unflatten_binder

更多录播视频+架构学习资料免费领取 QQ 3607409029

[-> Parcel.cpp]

```
status_t unflatten_binder(const sp<ProcessState>& proc,
    const Parcel& in, sp<IBinder>* out)
{
    const flat_binder_object* flat = in.readObject(false);
    if (flat) {
        switch (flat->type) {
            case BINDER_TYPE_BINDER:
                // 当请求服务的进程与服务属于同一进程
                *out = reinterpret_cast<IBinder*>(flat->cookie);
                return finish_unflatten_binder(NULL, *flat, in);
            case BINDER_TYPE_HANDLE:
                //请求服务的进程与服务属于不同进程【见 2.9.2】
                *out = proc->getStrongProxyForHandle(flat->handle);
                //创建 BpBinder 对象
                return finish_unflatten_binder(
                    static_cast<BpBinder*>(out->get()), *flat, in);
        }
    }
    return BAD_TYPE;
}
```

2.9.2 getStrongProxyForHandle

[-> ProcessState.cpp]

```
sp<IBinder> ProcessState::getStrongProxyForHandle(int32_t handle)
{
    sp<IBinder> result;
```

```
AutoMutex _l(mLock);

//查找 handle 对应的资源项[2.9.3]

handle_entry* e = lookupHandleLocked(handle);

if (e != NULL) {
    IBinder* b = e->binder;

    if (b == NULL || !e->refs->attemptIncWeak(this)) {
        ...

        //当 handle 值所对应的 IBinder 不存在或弱引用无效时，则创建 BpBinder 对象
        b = new BpBinder(handle);
        e->binder = b;
        if (b) e->refs = b->getWeakRefs();
        result = b;
    } else {
        result.force_set(b);
        e->refs->decWeak(this);
    }
}

return result;
}
```

readStrongBinder 的功能是 flat_binder_object 解析并创建 BpBinder 对象。

2.9.3 lookupHandleLocked

```
ProcessState::handle_entry* ProcessState::lookupHandleLocked(int32_t handle)
{
    const size_t N=mHandleToObject.size();

    //当 handle 大于 mHandleToObject 的长度时，进入该分支

    if (N <= (size_t)handle) {
```

```
handle_entry e;

e.binder = NULL;

e.refs = NULL;

//从 mHandleToObject 的第 N 个位置开始, 插入(handle+1-N)个 e 到队列中
status_t err = mHandleToObject.insertAt(e, N, handle+1-N);

if (err < NO_ERROR) return NULL;

}

return &mHandleToObject.editItemAt(handle);

}
```

根据 handle 值来查找对应的 handle_entry.

二. 总结

请求服务(getService)过程, 就是向 servicemanager 进程查询指定服务, 当执行 binder_transaction()时, 会区分请求服务所属进程情况。

1. 当请求服务的进程与服务属于不同进程, 则为请求服务所在进程创建 binder_ref 对象, 指向服务进程中的 binder_node;
 - 最终 readStrongBinder(), 返回的是 BpBinder 对象;
2. 当请求服务的进程与服务属于同一进程, 则不再创建新对象, 只是引用计数加 1, 并且修改 type 为 BINDER_TYPE_BINDER 或 BINDER_TYPE_WEAK_BINDER。
 - 最终 readStrongBinder(), 返回的是 BBinder 对象的真实子类;

第八节 framework 层分析

主要分析 Binder 在 java framework 层的框架, 相关源码:

```
framework/base/core/java/android/os/

- IInterface.java

- IServiceManager.java
```

```
- ServiceManager.java

- ServiceManagerNative.java(内含 ServiceManagerProxy 类)

framework/base/core/java/android/os/

- IBinder.java
- Binder.java(内含 BinderProxy 类)
- Parcel.java

framework/base/core/java/com/android/internal/os/

- BinderInternal.java

framework/base/core/jni/

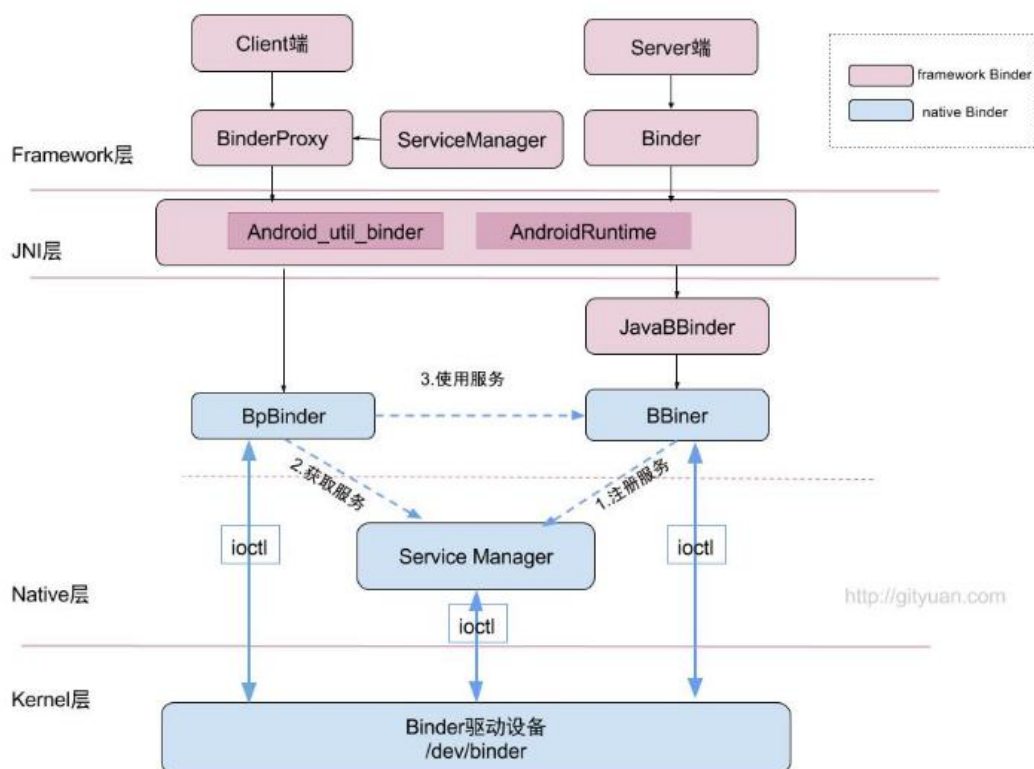
- AndroidRuntime.cpp
- android_os_Parcel.cpp
- android_util_Binder.cpp
```

一、概述

1.1 Binder 架构

binder 在 framework 层，采用 JNI 技术来调用 native(C/C++)层的 binder 架构，从而为上层应用程序提供服务。看过 binder 系列之前的文章，我们知道 native 层中，binder 是 C/S 架构，分为 Bn 端(Server)和 Bp 端(Client)。对于 java 层在命名与架构上非常相近，同样实现了一套 IPC 通信架构。

framework Binder 架构图：[查看大图](#)

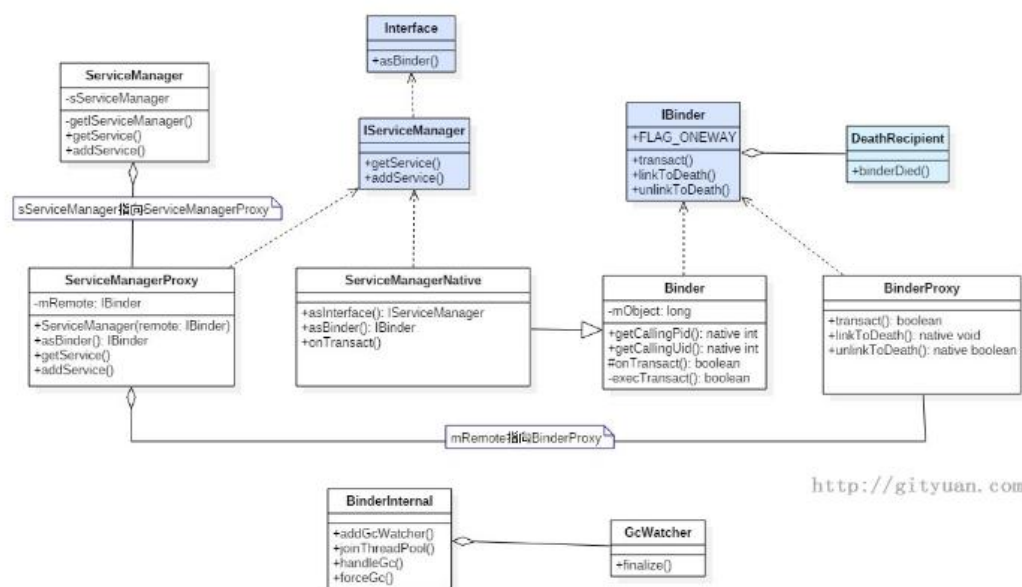


图解:

- 图中红色代表整个 framework 层 binder 架构相关组件;
 - Binder 类代表 Server 端, BinderProxy 类代码 Client 端;
- 图中蓝色代表 Native 层 Binder 架构相关组件;
- 上层 framework 层的 Binder 逻辑是建立在 Native 层架构基础之上的, 核心逻辑都是交予 Native 层方法来处理。
- framework 层的 ServiceManager 类与 Native 层的功能并不完全对应, framework 层的 ServiceManager 类的实现最终是通过 BinderProxy 传递给 Native 层来完成的, 后面会详细说明。

1.2 Binder 类图

下面列举 framework 的 binder 类关系图: 查看大图

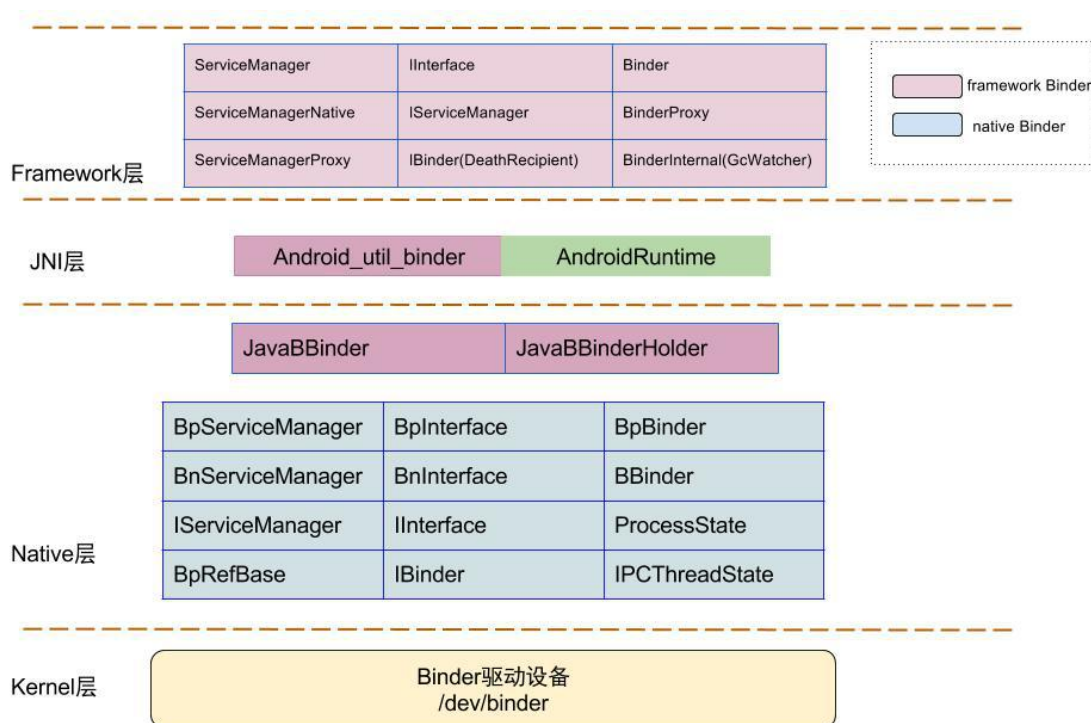


图解：(图中浅蓝色都是 Interface，其余都是 Class)

1. **ServiceManager**：通过 `getServiceManager` 方法获取的是 `ServiceManagerProxy` 对象；`ServiceManager` 的 `addService`, `getService` 实际工作都交由 `ServiceManagerProxy` 的相应方法来处理；
2. **ServiceManagerProxy**：其成员变量 `mRemote` 指向 `BinderProxy` 对象，`ServiceManagerProxy` 的 `addService`, `getService` 方法最终是交由 `mRemote` 来完成。
3. **ServiceManagerNative**：其方法 `asInterface()` 返回的是 `ServiceManagerProxy` 对象，`ServiceManager` 便是借助 `ServiceManagerNative` 类来找到 `ServiceManagerProxy`；
4. **Binder**：其成员变量 `mObject` 和方法 `execTransact()` 用于 native 方法
5. **BinderInternal**：内部有一个 `GcWatcher` 类，用于处理和调试与 `Binder` 相关的垃圾回收。
6. **IBinder**：接口中常量 `FLAG_ONEWAY`：客户端利用 `binder` 跟服务端通信是阻塞式的，但如果设置了 `FLAG_ONEWAY`，这成为非阻塞的调用方式，客户端能立即返回，服务端采用回调方式来通知客户端完成情况。另外 `IBinder` 接口有一个内部接口 `DeathDecipient`(死亡通告)。

1.3 Binder 类分层

整个 Binder 从 kernel 至，native，JNI，Framework 层所涉及的全部类

[点击查看大图](#)

二、初始化

在 Android 系统开机过程中，Zygote 启动时会有一个[虚拟机注册过程](#)，该过程调用 `AndroidRuntime::startReg` 方法来完成 jni 方法的注册。

2.1 startReg

==> `AndroidRuntime.cpp`

```
int AndroidRuntime::startReg(JNIEnv* env)
{
    androidSetCreateThreadFunc((android_create_thread_fn) javaCreateThreadEtc);

    env->PushLocalFrame(200);
```

```
//注册 jni 方法【见 2.2】  
  
if (register_jni_procs(gRegJNI, NELEM(gRegJNI), env) < 0) {  
    env->PopLocalFrame(NULL);  
    return -1;  
}  
  
env->PopLocalFrame(NULL);  
  
return 0;  
}
```

注册 JNI 方法，其中 `gRegJNI` 是一个数组，记录所有需要注册的 jni 方法，其中有一项便是 `REG_JNI(register_android_os_Binder)`，下面说说 `register_android_os_Binder` 过程。

2.2 register_android_os_Binder

==> android_util_Binder.cpp

```
int register_android_os_Binder(JNIEnv* env) {  
    // 注册 Binder 类的 jni 方法【见 2.3】  
    if (int_register_android_os_Binder(env) < 0)  
        return -1;  
  
    // 注册 BinderInternal 类的 jni 方法【见 2.4】  
    if (int_register_android_os_BinderInternal(env) < 0)  
        return -1;  
  
    // 注册 BinderProxy 类的 jni 方法【见 2.5】  
    if (int_register_android_os_BinderProxy(env) < 0)  
        return -1;  
    ...  
}
```

```
return 0;  
}
```

2.3 注册 Binder

==> android_util_Binder.cpp

```
static int int_register_android_os_Binder(JNIEnv* env) {  
    //其中 kBinderPathName = "android/os/Binder"; 查找 kBinderPathName 路径所属类  
    jclass clazz = FindClassOrDie(env, kBinderPathName);  
  
    //将 Java 层 Binder 类保存到 mClass 变量;  
    gBinderOffsets.mClass = MakeGlobalRefOrDie(env, clazz);  
  
    //将 Java 层 execTransact() 方法保存到 mExecTransact 变量;  
    gBinderOffsets.mExecTransact = GetMethodIDOrDie(env, clazz, "execTransact",  
    "(IJJI)Z");  
  
    //将 Java 层 mObject 属性保存到 mObject 变量  
    gBinderOffsets.mObject = GetFieldIDOrDie(env, clazz, "mObject", "J");  
  
    //注册 JNI 方法  
    return RegisterMethodsOrDie(env, kBinderPathName, gBinderMethods,  
    NELEM(gBinderMethods));  
}
```

注册 Binder 类的 jni 方法，其中：

- FindClassOrDie(env, kBinderPathName) 基本等价于 env->FindClass(kBinderPathName)
- MakeGlobalRefOrDie() 等价于 env->NewGlobalRef()
- GetMethodIDOrDie() 等价于 env->GetMethodID()
- GetFieldIDOrDie() 等价于 env->GetFieldID()
- RegisterMethodsOrDie() 等价于 Android::registerNativeMethods();

(1)gBinderOffsets

gBinderOffsets 是全局静态结构体(struct)，定义如下：

```
static struct bindernative_offsets_t
{
    jclass mClass; //记录 Binder 类
    jmethodID mExecTransact; //记录 execTransact()方法
    jfieldID mObject; //记录 mObject 属性
} gBinderOffsets;
```

gBinderOffsets 保存了 **Binder.java** 类本身以及其成员方法 **execTransact()** 和成员属性 **mObject**，这为 JNI 层访问 Java 层提供通道。另外通过查询获取 Java 层 binder 信息后保存到 **gBinderOffsets**，而不再需要每次查找 binder 类信息的方式能大幅度提高效率，是由于每次查询需要花费较多的 CPU 时间，尤其是频繁访问时，但用额外的结构体来保存这些信息，是以空间换时间的方法。

(2)gBinderMethods

```
static const JNINativeMethod gBinderMethods[] = {
    /* 名称，签名，函数指针 */
    { "getCallingPid", "()I", (void*)android_os_Binder_getCallingPid },
    { "getCallingUid", "()I", (void*)android_os_Binder_getCallingUid },
    { "clearCallingIdentity", "()J", (void*)android_os_Binder_clearCallingIdentity },
    { "restoreCallingIdentity", "(J)V", (void*)android_os_Binder_restoreCallingIdentity },
    { "setThreadStrictModePolicy", "(I)V", (void*)android_os_Binder_setThreadStrictModePolicy },
    { "getThreadStrictModePolicy", "()I", (void*)android_os_Binder_getThreadStrictModePolicy },
    { "flushPendingCommands", "()V", (void*)android_os_Binder_flushPendingCommands },
}
```

```
{ "init", "()V", (void*)android_os_Binder_init },  
  
{ "destroy", "()V", (void*)android_os_Binder_destroy },  
  
{ "blockUntilThreadAvailable", "()V", (void*)android_os_Binder_blockUntilThreadAvailable }  
  
};
```

通过 RegisterMethodsOrDie(), 将为 gBinderMethods 数组中的方法建立了一一映射关系, 从而为 Java 层访问 JNI 层提供通道。

总之, `int_register_android_os_Binder` 方法的主要功能:

- 通过 gBinderOffsets, 保存 Java 层 Binder 类的信息, 为 JNI 层访问 Java 层提供通道;
- 通过 RegisterMethodsOrDie, 将 gBinderMethods 数组完成映射关系, 从而为 Java 层访问 JNI 层提供通道。

也就是说该过程建立了 Binder 类在 Native 层与 framework 层之间的相互调用的桥梁。

2.4 注册 BinderInternal

==> android_util_Binder.cpp

```
static int int_register_android_os_BinderInternal(JNIEnv* env) {  
  
    //其中 kBinderInternalPathName = "com/android/internal/os/BinderInternal"  
  
    jclass clazz = FindClassOrDie(env, kBinderInternalPathName);  
  
    gBinderInternalOffsets.mClass = MakeGlobalRefOrDie(env, clazz);  
  
    gBinderInternalOffsets.mForceGc = GetStaticMethodIDOrDie(env, clazz, "forceBinderGc", "()V");  
  
    return RegisterMethodsOrDie(  
  
        env, kBinderInternalPathName,  
  
        gBinderInternalMethods, NELEM(gBinderInternalMethods));  
}
```

```
}
```

注册 `BinderInternal` 类的 jni 方法, `gBinderInternalOffsets` 保存了 `BinderInternal` 的 `forceBinderGc()` 方法。

下面是 `BinderInternal` 类的 JNI 方法注册:

```
static const JNINativeMethod gBinderInternalMethods[] = {  
    { "getContextObject", "()Landroid/os/IBinder;", (void*)android_os_BinderInternal_getContextObject },  
    { "joinThreadPool", "()V", (void*)android_os_BinderInternal_joinThreadPool },  
    { "disableBackgroundScheduling", "(Z)V", (void*)android_os_BinderInternal_disableBackgroundScheduling },  
    { "handleGc", "()V", (void*)android_os_BinderInternal_handleGc }  
};
```

该过程其【2.3】非常类似,也就是说该过程建立了是 `BinderInternal` 类在 Native 层与 framework 层之间的相互调用的桥梁。

2.5 注册 `BinderProxy`

==> `android_util_Binder.cpp`

```
static int int_register_android_os_BinderProxy(JNIEnv* env) {  
    //gErrorOffsets 保存了 Error 类信息  
    jclass clazz = FindClassOrDie(env, "java/lang/Error");  
    gErrorOffsets.mClass = MakeGlobalRefOrDie(env, clazz);  
  
    //gBinderProxyOffsets 保存了 BinderProxy 类的信息  
    //其中 kBinderProxyPathName = "android/os/BinderProxy"  
    clazz = FindClassOrDie(env, kBinderProxyPathName);
```

```
gBinderProxyOffsets.mClass = MakeGlobalRefOrDie(env, clazz);

gBinderProxyOffsets.mConstructor = GetMethodIDOrDie(env, clazz, "<init>", "()V");

gBinderProxyOffsets.mSendDeathNotice = GetStaticMethodIDOrDie(env, clazz, "sendDeathNotice", "(Landroid/os/IBinder$DeathRecipient;)V");

gBinderProxyOffsets.mObject = GetFieldIDOrDie(env, clazz, "mObject", "J");

gBinderProxyOffsets.mSelf = GetFieldIDOrDie(env, clazz, "mSelf", "Ljava/lang/ref/WeakReference;");

gBinderProxyOffsets.mOrgue = GetFieldIDOrDie(env, clazz, "mOrgue", "J");

//gClassOffsets 保存了 Class.getName() 方法
clazz = FindClassOrDie(env, "java/lang/Class");

gClassOffsets.mGetName = GetMethodIDOrDie(env, clazz, "getName", "()Ljava/lang/String;");

return RegisterMethodsOrDie(
    env, kBinderProxyPathName,
    gBinderProxyMethods, NELEM(gBinderProxyMethods));
}
```

注册 BinderProxy 类的 jni 方法，gBinderProxyOffsets 保存了 BinderProxy 的构造方法，sendDeathNotice(), mObject, mSelf, mOrgue 信息。

下面 BinderProxy 类的 JNI 方法注册：

```
static const JNINativeMethod gBinderProxyMethods[] = {
    /* 名称, 签名, 函数指针 */
    {"pingBinder",      "()Z", (void*)android_os_BinderProxy_pingBinder},
    {"isBinderAlive",   "()Z", (void*)android_os_BinderProxy_isBinderAlive},
    {"getInterfaceDescriptor", "()Ljava/lang/String;", (void*)android_os_BinderProxy_getInterfaceDescriptor},
    {"transactNative",  "(ILandroid/os/Parcel;Landroid/os/Parcel;I)Z", (void*)android_os_BinderProxy_transact},
}
```

```
        {"linkToDeath", "(Landroid/os/IBinder$DeathRecipient;I)V", (void*)android_os_BinderProxy_linkToDeath},  
  
        {"unlinkToDeath", "(Landroid/os/IBinder$DeathRecipient;I)Z", (void*)android_os_BinderProxy_unlinkToDeath},  
  
        {"destroy", "()V", (void*)android_os_BinderProxy_destroy},  
  
    };
```

该过程其【2.3】非常类似，也就是说该过程建立了是 BinderProxy 类在 Native 层与 framework 层之间的相互调用的桥梁。

三、注册服务

3.1 SM.addService

[-> ServiceManager.java]

```
public static void addService(String name, IBinder service, boolean allowIsolated) {  
  
    try {  
  
        //先获取 SMP 对象，则执行注册服务操作【见小节 3.2/3.4】  
  
        getIServiceManager().addService(name, service, allowIsolated);  
  
    } catch (RemoteException e) {  
  
        Log.e(TAG, "error in addService", e);  
  
    }  
  
}
```

先来看看 getIServiceManager()过程，如下：

3.2 getIServiceManager

[-> ServiceManager.java]


```
private static IServiceManager getIServiceManager() {  
    if (sServiceManager != null) {  
        return sServiceManager;  
    }  
    //【分别见 3.2.1 和 3.3】  
    sServiceManager = ServiceManagerNative.asInterface(BinderInternal.getContextObject());  
    return sServiceManager;  
}
```

采用了单例模式获取 ServiceManager getIServiceManager()返回的是 ServiceManagerProxy(简称 SMP)对象

3.2.1 getContextObject()

[-> android_util_binder.cpp]

```
static jobject android_os_BinderInternal_getContextObject(JNIEnv* env, jobject clazz)  
{  
    sp<IBinder> b = ProcessState::self()->getContextObject(NULL);  
    return javaObjectForIBinder(env, b); //【见 3.2.2】  
}
```

BinderInternal.java 中有一个 native 方法 getContextObject(), JNI 调用执行上述方法。

对于 ProcessState::self()->getContextObject(), 在[获取 ServiceManager](#)的第 3 节已详细解决, 即 `ProcessState::self()->getContextObject()`等价于 `new BpBinder(0);`

3.2.2 javaObjectForIBinder

[-> android_util_binder.cpp]

```
jobject javaObjectForIBinder(JNIEnv* env, const sp<IBinder>& val) {  
    if (val == NULL) return NULL;  
  
    if (val->checkSubclass(&gBinderOffsets)) { //返回 false  
        jobject object = static_cast<JavaBBinder*>(val.get())->object();  
        return object;  
    }  
  
    AutoMutex _l(mProxyLock);  
  
    jobject object = (jobject)val->findObject(&gBinderProxyOffsets);  
    if (object != NULL) { //第一次 object 为 null  
        jobject res = jniGetReferent(env, object);  
        if (res != NULL) {  
            return res;  
        }  
        android_atomic_dec(&gNumProxyRefs);  
        val->detachObject(&gBinderProxyOffsets);  
        env->DeleteGlobalRef(object);  
    }  
  
    //创建 BinderProxy 对象  
    object = env->NewObject(gBinderProxyOffsets.mClass, gBinderProxyOffsets.mConstructor);  
    if (object != NULL) {  
        //BinderProxy.mObject 成员变量记录 BpBinder 对象  
        env->SetLongField(object, gBinderProxyOffsets.mObject, (jlong)val.get());  
        val->incStrong((void*)javaObjectForIBinder);  
    }  
}
```

```
        jobject refObject = env->NewGlobalRef(  
            env->GetObjectField(object, gBinderProxyOffsets.mSelf));  
        //将 BinderProxy 对象信息附加到 BpBinder 的成员变量 mObjects 中  
        val->attachObject(&gBinderProxyOffsets, refObject,  
            JNIenv_to_javavm(env), proxy_cleanup);  
  
        sp<DeathRecipientList> drl = new DeathRecipientList;  
        drl->incStrong((void*)javaObjectForIBinder);  
        //BinderProxy.mOrgue 成员变量记录死亡通知对象  
        env->SetLongField(object, gBinderProxyOffsets.mOrgue, reinterpret_cast<j  
long>(drl.get()));  
  
        android_atomic_inc(&gNumProxyRefs);  
        incRefsCreated(env);  
    }  
    return object;  
}
```

根据 BpBinder(C++)生成 BinderProxy(Java)对象. 主要工作是创建 BinderProxy 对象,并把 BpBinder 对象地址保存到 BinderProxy.mObject 成员变量. 到此,可知 ServiceManagerNative.asInterface(BinderInternal.getContextObject()) 等价于

```
ServiceManagerNative.asInterface(new BinderProxy())
```

3.3 SMN.asInterface

[-> ServiceManagerNative.java]

```
static public IServiceManager asInterface(IBinder obj){
```

```
if (obj == null) { //obj 为 BpBinder
    return null;
}
//由于 obj 为 BpBinder，该方法默认返回 null
IServiceManager in = (IServiceManager)obj.queryLocalInterface(descriptor);
if (in != null) {
    return in;
}
return new ServiceManagerProxy(obj); //【见小节 3.3.1】
}
```

由此，可知 `ServiceManagerNative.asInterface(new BinderProxy())` 等价于 `new ServiceManagerProxy(new BinderProxy())`。为了方便，`ServiceManagerProxy` 简称为 SMP。

3.3.1 ServiceManagerProxy 初始化

[-> ServiceManagerNative.java ::ServiceManagerProxy]

```
class ServiceManagerProxy implements IServiceManager {
    public ServiceManagerProxy(IBinder remote) {
        mRemote = remote;
    }
}
```

`mRemote` 为 `BinderProxy` 对象，该 `BinderProxy` 对象对应于 `BpBinder(0)`，其作为 binder 代理端，指向 native 层大管家 `service Manager`。

`ServiceManager.getIServiceManager` 最终等价于 `new ServiceManagerProxy(new BinderProxy())`，意味着【3.1】中的 `getServiceManager().addService()`，等价于 `SMP.addService()`。

framework 层的 ServiceManager 的调用实际的工作确实交给 SMP 的成员变量 BinderProxy；而 BinderProxy 通过 jni 方式，最终会调用 BpBinder 对象；可见上层 binder 架构的核心功能依赖 native 架构的服务来完成的。

3.4 SMP.addService

[-> ServiceManagerNative.java ::ServiceManagerProxy]

```
public void addService(String name, IBinder service, boolean allowIsolated) throws RemoteException {  
    Parcel data = Parcel.obtain();  
    Parcel reply = Parcel.obtain();  
    data.writeInterfaceToken(IServiceManager.descriptor);  
    data.writeString(name);  
    // 【见小节 3.5】  
    data.writeStrongBinder(service);  
    data.writeInt(allowIsolated ? 1 : 0);  
    // mRemote 为 BinderProxy 【见小节 3.7】  
    mRemote.transact(ADD_SERVICE_TRANSACTION, data, reply, 0);  
    reply.recycle();  
    data.recycle();  
}
```

3.5 writeStrongBinder(Java)

[-> Parcel.java]

```
public writeStrongBinder(IBinder val){  
    // 此处为 Native 调用 【见 3.5.1】  
    nativeWriteStrongBinder(mNativePtr, val);  
}
```

3.5.1 android_os_Parcel_writeStrongBinder

[-> android_os_Parcel.cpp]

```
static void android_os_Parcel_writeStrongBinder(JNIEnv* env, jclass clazz, jlong nativePtr, jobject object) {  
  
    //将 java 层 Parcel 转换为 native 层 Parcel  
  
    Parcel* parcel = reinterpret_cast<Parcel*>(nativePtr);  
  
    if (parcel != NULL) {  
  
        //【见 3.5.2】  
  
        const status_t err = parcel->writeStrongBinder(ibinderForJavaObject(env, object));  
  
        if (err != NO_ERROR) {  
  
            signalExceptionForError(env, clazz, err);  
  
        }  
  
    }  
  
}
```

3.5.2 ibinderForJavaObject

[-> android_util_Binder.cpp]

```
sp<IBinder> ibinderForJavaObject(JNIEnv* env, jobject obj)  
{  
  
    if (obj == NULL) return NULL;  
  
    //Java 层的 Binder 对象  
  
    if (env->IsInstanceOf(obj, gBinderOffsets.mClass)) {  
  
        JavaBBinderHolder* jbh = (JavaBBinderHolder*)  
  
        env->GetLongField(obj, gBinderOffsets.mObject);  
  
        return jbh != NULL ? jbh->get(env, obj) : NULL; //【见 3.5.3】  
  
    }  
  
}
```

```
}  
  
//Java 层的 BinderProxy 对象  
if (env->IsInstanceOf(obj, gBinderProxyOffsets.mClass)) {  
    return (IBinder*)env->GetLongField(obj, gBinderProxyOffsets.mObject);  
}  
  
return NULL;  
}
```

根据 Binde(Java)生成 JavaBBinderHolder(C++)对象. 主要工作是创建 JavaBBinderHolder 对象,并把 JavaBBinderHolder 对象地址保存到 Binder.mObject 成员变量.

3.5.3 JavaBBinderHolder.get()

[-> android_util_Binder.cpp]

```
sp<JavaBBinder> get(JNIEnv* env, jobject obj){  
    AutoMutex _l(mLock);  
    sp<JavaBBinder> b = mBinder.promote();  
    if (b == NULL) {  
        //首次进来, 创建 JavaBBinder 对象【见 3.5.4】  
        b = new JavaBBinder(env, obj);  
        mBinder = b;  
    }  
    return b;  
}
```

JavaBBinderHolder 有一个成员变量 mBinder, 保存当前创建的 JavaBBinder 对象, 这是一个 wp 类型的, 可能会被垃圾回收器给回收, 所以每次使用前, 都需要先判断是否存在。

3.5.4 JavaBBinder 初始化

==> [-> android_util_Binder.cpp]

```
JavaBBinder(JNIEnv* env, jobject object)
: mVM(jnienv_to_javavm(env)), mObject(env->NewGlobalRef(object))
{
    android_atomic_inc(&gNumLocalRefs);
    incRefsCreated(env);
}
```

创建 JavaBBinder，该对象继承于 BBinder 对象。

data.writeStrongBinder(service)最终等价于 `parcel->writeStrongBinder(new JavaBBinder(env, obj));`

3.6 writeStrongBinder(C++)

[-> parcel.cpp]

```
status_t Parcel::writeStrongBinder(const sp<IBinder>& val)
{
    return flatten_binder(ProcessState::self(), val, this);
}
```

3.6.1 flatten_binder

[-> parcel.cpp]

```
status_t flatten_binder(const sp<ProcessState>& /*proc*/,
    const sp<IBinder>& binder, Parcel* out)
```



```
{
    flat_binder_object obj;

    obj.flags = 0x7f | FLAT_BINDER_FLAG_ACCEPTS_FDS;

    if (binder != NULL) {
        IBinder *local = binder->localBinder();

        if (!local) {
            BpBinder *proxy = binder->remoteBinder();

            const int32_t handle = proxy ? proxy->handle() : 0;

            obj.type = BINDER_TYPE_HANDLE; //远程 Binder

            obj.binder = 0;

            obj.handle = handle;

            obj.cookie = 0;

        } else {
            obj.type = BINDER_TYPE_BINDER; //本地 Binder，进入该分支

            obj.binder = reinterpret_cast<uintptr_t>(local->getWeakRefs());

            obj.cookie = reinterpret_cast<uintptr_t>(local);

        }

    } else {
        obj.type = BINDER_TYPE_BINDER; //本地 Binder

        obj.binder = 0;

        obj.cookie = 0;

    }

    //【见小节 3.6.2】

    return finish_flatten_binder(binder, obj, out);
}
```

将 Binder 对象扁平化，转换成 flat_binder_object 对象。

- 对于 Binder 实体，则 cookie 记录 Binder 实体的指针；
- 对于 Binder 代理，则用 handle 记录 Binder 代理的句柄；

更多录播视频+架构学习资料免费领取 QQ 3607409029

关于 localBinder，代码见 Binder.cpp。

```
BBinder* BBinder::localBinder()
{
    return this;
}

BBinder* IBinder::localBinder()
{
    return NULL;
}
```

3.6.2 finish_flatten_binder

```
inline static status_t finish_flatten_binder(
    const sp<IBinder>& , const flat_binder_object& flat, Parcel* out)
{
    return out->writeObject(flat, false);
}
```

再回到小节 3.4 的 addService 过程，则接下来进入 transact。

3.7 BinderProxy.transact

[-> Binder.java ::BinderProxy]

```
public boolean transact(int code, Parcel data, Parcel reply, int flags) throws
    RemoteException {
    //用于检测 Parcel 大小是否大于 800k
    Binder.checkParcel(this, code, data, "Unreasonably large binder buffer");
    return transactNative(code, data, reply, flags); //【见 3.8】
}
```

```
}
```

回到 `ServiceManagerProxy.addService`, 其成员变量 `mRemote` 是 `BinderProxy`。
`transactNative` 经过 `jni` 调用, 进入下面的方法

3.8 android_os_BinderProxy_transact

[-> android_util_Binder.cpp]

```
static jboolean android_os_BinderProxy_transact(JNIEnv* env, jobject obj,
    jint code, jobject dataObj, jobject replyObj, jint flags)
{
    ...

    //java Parcel 转为 native Parcel
    Parcel* data = parcelForJavaObject(env, dataObj);
    Parcel* reply = parcelForJavaObject(env, replyObj);
    ...

    //gBinderProxyOffsets.mObject 中保存的是 new BpBinder(0)对象
    IBinder* target = (IBinder*)
        env->GetLongField(obj, gBinderProxyOffsets.mObject);
    ...

    //此处便是 BpBinder::transact(), 经过 native 层, 进入 Binder 驱动程序
    status_t err = target->transact(code, *data, reply, flags);
    ...

    return JNI_FALSE;
}
```

Java 层的 `BinderProxy.transact()` 最终交由 Native 层的 `BpBinder::transact()` 完成。Native Binder 的注册服务(`addService`)中有详细说明 `BpBinder` 执行过程。另外，该方法可抛出 `RemoteException`。

3.9 小结

`addService` 的核心过程：

```
public void addService(String name, IBinder service, boolean allowIsolated) throws RemoteException {
    ...

    Parcel data = Parcel.obtain(); //此处还需要将 java 层的 Parcel 转为 Native 层的 Parcel
    data->writeStrongBinder(new JavaBBinder(env, obj));
    BpBinder::transact(ADD_SERVICE_TRANSACTION, *data, reply, 0); //与 Binder 驱动交互
    ...
}
```

注册服务过程就是通过 `BpBinder` 来发送 `ADD_SERVICE_TRANSACTION` 命令，与实现与 binder 驱动进行数据交互。

四、获取服务

4.1 SM.getService

[-> ServiceManager.java]

```
public static IBinder getService(String name) {
    try {
        IBinder service = sCache.get(name); //先从缓存中查看
        if (service != null) {
```

```
        return service;

    } else {

        return getIServiceManager().getService(name); 【见 4.2】

    }

    } catch (RemoteException e) {

        Log.e(TAG, "error in getService", e);

    }

    return null;

}
```

关于 `getIServiceManager()`，在前面 [小节 3.2](#) 已经讲述了，等价于 `new ServiceManagerProxy(new BinderProxy())`。其中 `sCache = new HashMap<String, IBinder>()` 以 `hashmap` 格式缓存已组成的名称。请求获取服务过程中，先从缓存中查询是否存在，如果缓存中不存在的话，再通过 `binder` 交互来查询相应的服务。

4.2 SMP.getService

[-> ServiceManagerNative.java ::ServiceManagerProxy]

```
class ServiceManagerProxy implements IServiceManager {

    public IBinder getService(String name) throws RemoteException {

        Parcel data = Parcel.obtain();

        Parcel reply = Parcel.obtain();

        data.writeInterfaceToken(IServiceManager.descriptor);

        data.writeString(name);

        //mRemote 为 BinderProxy 【见 4.3】

        mRemote.transact(GET_SERVICE_TRANSACTION, data, reply, 0);

        //从 reply 里面解析出获取的 IBinder 对象【见 4.8】

        IBinder binder = reply.readStrongBinder();

        reply.recycle();

        data.recycle();

    }

}
```

```
        return binder;
    }
}
```

4.3 BinderProxy.transact

[-> Binder.java]

```
final class BinderProxy implements IBinder {
    public boolean transact(int code, Parcel data, Parcel reply, int flags) throws RemoteException {
        Binder.checkParcel(this, code, data, "Unreasonably large binder buffer");
        return transactNative(code, data, reply, flags);
    }
}
```

4.4 android_os_BinderProxy_transact

[-> android_util_Binder.cpp]

```
static jboolean android_os_BinderProxy_transact(JNIEnv* env, jobject obj,
    jint code, jobject dataObj, jobject replyObj, jint flags)
{
    ...

    //java Parcel 转为 native Parcel
    Parcel* data = parcelForJavaObject(env, dataObj);
    Parcel* reply = parcelForJavaObject(env, replyObj);
    ...
}
```

```
//gBinderProxyOffsets.mObject 中保存的是 new BpBinder(0)对象
IBinder* target = (IBinder*)
    env->GetLongField(obj, gBinderProxyOffsets.mObject);
...

//此处便是 BpBinder::transact(), 经过 native 层[见小节 4.5]
status_t err = target->transact(code, *data, reply, flags);
...

return JNI_FALSE;
}
```

4.5 BpBinder.transact

[-> BpBinder.cpp]

```
status_t BpBinder::transact(
    uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags)
{
    if (mAlive) {
        // [见小节 4.6]
        status_t status = IPCThreadState::self()->transact(
            mHandle, code, data, reply, flags);
        if (status == DEAD_OBJECT) mAlive = 0;
        return status;
    }

    return DEAD_OBJECT;
}
```

4.6 IPC.transact

[-> IPCThreadState.cpp]

```
status_t IPCThreadState::transact(int32_t handle,
                                  uint32_t code, const Parcel& data,
                                  Parcel* reply, uint32_t flags)
{
    status_t err = data.errorCheck(); //数据错误检查

    flags |= TF_ACCEPT_FDS;

    ....

    if (err == NO_ERROR) {
        // 传输数据

        err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, NULL);
    }

    ...

    // 默认情况下,都是采用非 oneway 的方式,也就是需要等待服务端的返回结果
    if ((flags & TF_ONE_WAY) == 0) {
        if (reply) {
            //等待回应事件

            err = waitForResponse(reply);
        } else {
            Parcel fakeReply;

            err = waitForResponse(&fakeReply);
        }
    } else {
        err = waitForResponse(NULL, NULL);
    }
}
```



```
    return err;
}
```

4.7 IPC.waitForResponse

```
status_t IPCThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)
{
    int32_t cmd;
    int32_t err;

    while (1) {
        if ((err=talkWithDriver()) < NO_ERROR) break;
        ...

        cmd = mIn.readInt32();

        switch (cmd) {
            case BR_REPLY:
            {
                binder_transaction_data tr;

                err = mIn.read(&tr, sizeof(tr));

                if (reply) {
                    if ((tr.flags & TF_STATUS_CODE) == 0) {
                        //当 reply 对象回收时，则会调用 freeBuffer 来回收内存

                        reply->ipcSetDataReference(
                            reinterpret_cast<const uint8_t*>(tr.data.ptr.buffer),
                            tr.data_size,
                            reinterpret_cast<const binder_size_t*>(tr.data.ptr.offsets),
                            tr.offsets_size/sizeof(binder_size_t),
                            freeBuffer, this);
                    } else {
                        ...
                    }
                }
            }
        }
    }
}
```

```
    }  
    }  
    }  
    case :...  
    }  
    }  
    ...  
    return err;  
}
```

那么这个 reply 是哪来的呢，在文章 [Binder 系列 3—启动 ServiceManager](#)

4.7.1 binder_send_reply

[-> servicemanager/binder.c]

```
void binder_send_reply(struct binder_state *bs, struct binder_io *reply, binder  
_uintptr_t buffer_to_free, int status) {  
    struct {  
        uint32_t cmd_free;  
        binder_uintptr_t buffer;  
        uint32_t cmd_reply;  
        struct binder_transaction_data txn;  
    } __attribute__((packed)) data;  
  
    data.cmd_free = BC_FREE_BUFFER; //free buffer 命令  
    data.buffer = buffer_to_free;  
    data.cmd_reply = BC_REPLY; // reply 命令  
    data.txn.target.ptr = 0;  
    data.txn.cookie = 0;  
    data.txn.code = 0;
```

<https://ke.qq.com/course/341933?flowToken=1027385>

```

    if (status) {
        ...
    } else {=

        data.txn.flags = 0;

        data.txn.data_size = reply->data - reply->data0;

        data.txn.offsets_size = ((char*) reply->offs) - ((char*) reply->offs0);

        data.txn.data.ptr.buffer = (uintptr_t)reply->data0;

        data.txn.data.ptr.offsets = (uintptr_t)reply->offs0;

    }

    //向 Binder 驱动通信

    binder_write(bs, &data, sizeof(data));

}

```

binder_write 将 BC_FREE_BUFFER 和 BC_REPLY 命令协议发送给驱动，进入驱动。binder_ioctl -> binder_ioctl_write_read -> binder_thread_write，由于是 BC_REPLY 命令协议，则进入 binder_transaction，该方法会向请求服务的线程 Todo 队列插入事务。

接下来，请求服务的进程在执行 `talkWithDriver` 的过程执行到 `binder thread read()`，处理 `Todo` 队列的事务。

4.8 readStrongBinder

[-> Parcel.java]

readStrongBinder 的过程基本是 writeStrongBinder 逆过程。

```
static jobject android_os_Parcel_readStrongBinder(JNIEnv* env, jclass clazz, jlong nativePtr) {  
  
    Parcel* parcel = reinterpret_cast<Parcel*>(nativePtr);  
  
    if (parcel != NULL) {  
  
        // 【见小节 4.8.1】
```

```
        return javaObjectForIBinder(env, parcel->readStrongBinder());  
    }  
    return NULL;  
}
```

javaObjectForIBinder 将 native 层 BpBinder 对象转换为 Java 层 BinderProxy 对象。

4.8.1 readStrongBinder(C++)

[-> Parcel.cpp]

```
sp<IBinder> Parcel::readStrongBinder() const  
{  
    sp<IBinder> val;  
    // 【见小节 4.8.2】  
    unflatten_binder(ProcessState::self(), *this, &val);  
    return val;  
}
```

4.8.2 unflatten_binder

[-> Parcel.cpp]

```
status_t unflatten_binder(const sp<ProcessState>& proc,  
    const Parcel& in, sp<IBinder>* out){  
    const flat_binder_object* flat = in.readObject(false);  
    if (flat) {  
        switch (flat->type) {  
            case BINDER_TYPE_BINDER:  
                *out = reinterpret_cast<IBinder*>(flat->cookie);
```

```
        return finish_unflatten_binder(NULL, *flat, in);

    case BINDER_TYPE_HANDLE:

        //进入该分支【见 4.8.3】

        *out = proc->getStrongProxyForHandle(flat->handle);

        //创建 BpBinder 对象

        return finish_unflatten_binder(

            static_cast<BpBinder*>(out->get()), *flat, in);

    }

}

return BAD_TYPE;

}
```

4.8.3 getStrongProxyForHandle

[-> ProcessState.cpp]

```
sp<IBinder> ProcessState::getStrongProxyForHandle(int32_t handle)

{

    sp<IBinder> result;

    AutoMutex _l(mLock);

    //查找 handle 对应的资源项

    handle_entry* e = lookupHandleLocked(handle);

    if (e != NULL) {

        IBinder* b = e->binder;

        if (b == NULL || !e->refs->attemptIncWeak(this)) {

            ...

            //当 handle 值所对应的 IBinder 不存在或弱引用无效时，则创建 BpBinder 对象

            b = new BpBinder(handle);
```

```
e->binder = b;

if (b) e->refs = b->getWeakRefs();

result = b;

} else {

    result.force_set(b);

    e->refs->decWeak(this);

}

}

return result;

}
```

经过该方法，最终创建了指向 Binder 服务端的 BpBinder 代理对象。回到[小节 4.8] 经过 javaObjectForIBinder 将 native 层 BpBinder 对象转换为 Java 层 BinderProxy 对象。 也就是说通过 getService()最终获取了指向目标 Binder 服务端的代理对象 BinderProxy。

4.9 小结

getService 的核心过程：

```
public static IBinder getService(String name) {

    ...

    Parcel reply = Parcel.obtain(); //此处还需要将 java 层的 Parcel 转为 Native 层的 Parcel

    BpBinder::transact(GET_SERVICE_TRANSACTION, *data, reply, 0); //与 Binder 驱动交互

    IBinder binder = javaObjectForIBinder(env, new BpBinder(handle));

    ...

}
```

javaObjectForIBinder 作用是创建 BinderProxy 对象，并将 BpBinder 对象的地址保存到 BinderProxy 对象的 mObjects 中。 获取服务过程就是通过 BpBinder 来发送 GET_SERVICE_TRANSACTION 命令，与实现与 binder 驱动进行数据交互。

五. 实例

以 IWindowManager 为例

```
public interface IWindowManager extends android.os.IInterface {

    public static abstract class Stub extends android.os.Binder implements android.view.IWindowManager {

        private static final java.lang.String DESCRIPTOR = "android.view.IWindowManager";

        public Stub() {
            this.attachInterface(this, DESCRIPTOR);
        }

        public static android.view.IWindowManager asInterface(android.os.IBinder obj) {
            if ((obj == null)) {
                return null;
            }
            android.os.IInterface iin = obj.queryLocalInterface(DESCRIPTOR);
            if (((iin != null) && (iin instanceof android.view.IWindowManager))) {
                return ((android.view.IWindowManager) iin);
            }
            return new android.view.IWindowManager.Stub.Proxy(obj);
        }

        public android.os.IBinder asBinder() {
            return this;
        }
    }
}
```

```
private static class Proxy implements android.view.IWindowManager {  
    private android.os.IBinder mRemote;  
  
    Proxy(android.os.IBinder remote) {  
        mRemote = remote;  
    }  
  
    public android.os.IBinder asBinder() {  
        return mRemote;  
    }  
    ...  
}
```

5.1 Binder

[-> Binder.java]

```
public class Binder implements IBinder {  
    public void attachInterface(IInterface owner, String descriptor) {  
        mOwner = owner;  
        mDescriptor = descriptor;  
    }  
  
    public IInterface queryLocalInterface(String descriptor) {  
        if (mDescriptor.equals(descriptor)) {  
            return mOwner;  
        }  
    }  
}
```



```
        return null;
    }
}
```

5.2 BinderProxy

```
final class BinderProxy implements IBinder {
    public IInterface queryLocalInterface(String descriptor) {
        return null;
    }
}
```

第九节如何使用 Binder

自定义 binder 架构的 client/ server 组件

一、Native 层 Binder

源码结构：

- ClientDemo.cpp: 客户端程序
- ServerDemo.cpp: 服务端程序
- IMyService.h: 自定义的 MyService 服务的头文件
- IMyService.cpp: 自定义的 MyService 服务
- Android.mk: 源码 build 文件

1.1 服务端

```
#include "IMyService.h"

int main() {
    //获取 service manager 引用
    sp < IServiceManager > sm = defaultServiceManager();
}
```

```
//注册名为"service.myservice"的服务到 service manager  
  
sm->addService(String16("service.myservice"), new BnMyService());  
  
ProcessState::self()->startThreadPool(); //启动线程池  
  
IPCThreadState::self()->joinThreadPool(); //把主线程加入线程池  
  
return 0;  
  
}
```

将名为"service.myservice"的 BnMyService 服务添加到 ServiceManager，并启动服务

1.2 客户端

```
#include "IMyService.h"int main() {  
  
    //获取 service manager 引用  
  
    sp < IServiceManager > sm = defaultServiceManager();  
  
    //获取名为"service.myservice"的 binder 接口  
  
    sp < IBinder > binder = sm->getService(String16("service.myservice"));  
  
    //将 biner 对象转换为强引用类型的 IMyService  
  
    sp<IMyService> cs = interface_cast < IMyService > (binder);  
  
    //利用 binder 引用调用远程 sayHello()方法  
  
    cs->sayHello();  
  
    return 0;  
  
}
```

获取名为"service.myservice"的服务，再进行类型，最后调用远程方法 `sayHello()`

1.3 创建 MyService

(1)IMyService.h

```
namespace android
```

```
{  
  
    class IMyService : public IInterface  
    {  
    public:  
        DECLARE_META_INTERFACE(MyService); //使用宏，申明 MyService  
        virtual void sayHello()=0; //定义方法  
    };  
  
    //定义命令字段  
    enum  
    {  
        HELLO = 1,  
    };  
  
    //申明客户端 BpMyService  
    class BpMyService: public BpInterface<IMyService> {  
    public:  
        BpMyService(const sp<IBinder>& impl);  
        virtual void sayHello();  
    };  
  
    //申明服务端 BnMyService  
    class BnMyService: public BnInterface<IMyService> {  
    public:  
        virtual status_t onTransact(uint32_t code, const Parcel& data, Parcel* r  
eply, uint32_t flags = 0);  
        virtual void sayHello();  
    };  
}
```

主要功能:

- 申明 IMyService
- 申明 BpMyService (Binder 客户端)
- 申明 BnMyService (Binder 的服务端)

(2)IMyService.cpp

```
#include "IMyService.h" namespace android

{

    //使用宏, 完成 MyService 定义
    IMPLEMENT_META_INTERFACE(MyService, "android.demo.IMyService");

    //客户端
    BpMyService::BpMyService(const sp<IBinder>& impl) :
        BpInterface<IMyService>(impl) {

    }

    // 实现客户端 sayHello 方法
    void BpMyService::sayHello() {
        printf("BpMyService::sayHello\n");

        Parcel data, reply;

        data.writeInterfaceToken(IMyService::getInterfaceDescriptor());

        remote()->transact(HELLO, data, &reply);

        printf("get num from BnMyService: %d\n", reply.readInt32());
    }

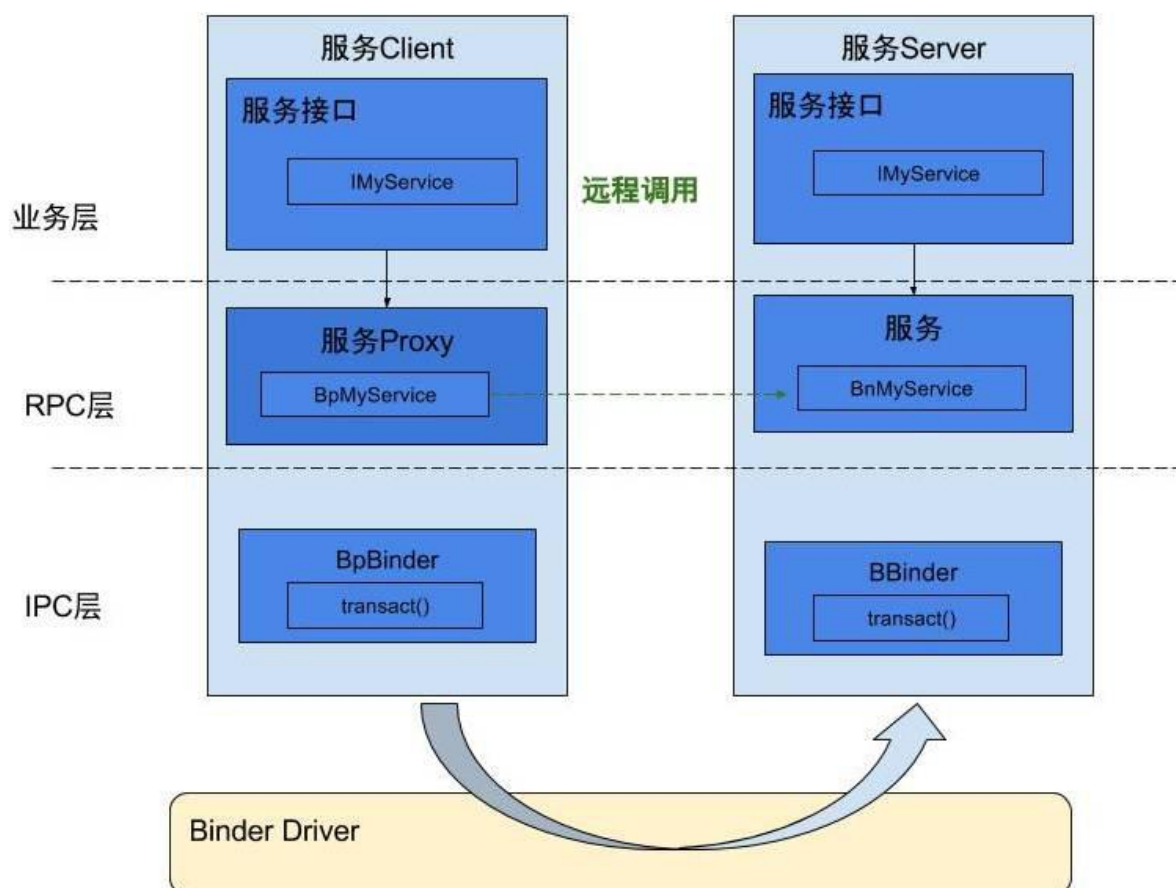
    //服务端, 接收远程消息, 处理 onTransact 方法
    status_t BnMyService::onTransact(uint_t code, const Parcel& data,
        Parcel* reply, uint32_t flags) {

        switch (code) {

            case HELLO: { //收到 HELLO 命令的处理流程
```

```
        printf("BnMyService:: got the client hello\n");  
        CHECK_INTERFACE(IMyService, data, reply);  
        sayHello();  
        reply->writeInt32(2015);  
        return NO_ERROR;  
    }  
    break;  
    default:  
    break;  
}  
return NO_ERROR;  
}  
  
// 实现服务端 sayHello 方法  
void BnMyService::sayHello() {  
    printf("BnMyService::sayHello\n");  
};  
}
```

1.4 原理图



1.5 运行

(1)编译生成 利用 Android.mk 编译上述代码，在 Android 的源码中，通过 mm 编译后，可生成两个可执行文件 ServerDemo，ClientDemo。

(2)执行

首先将这两个 ServerDemo，ClientDemo 可执行文件 push 到手机

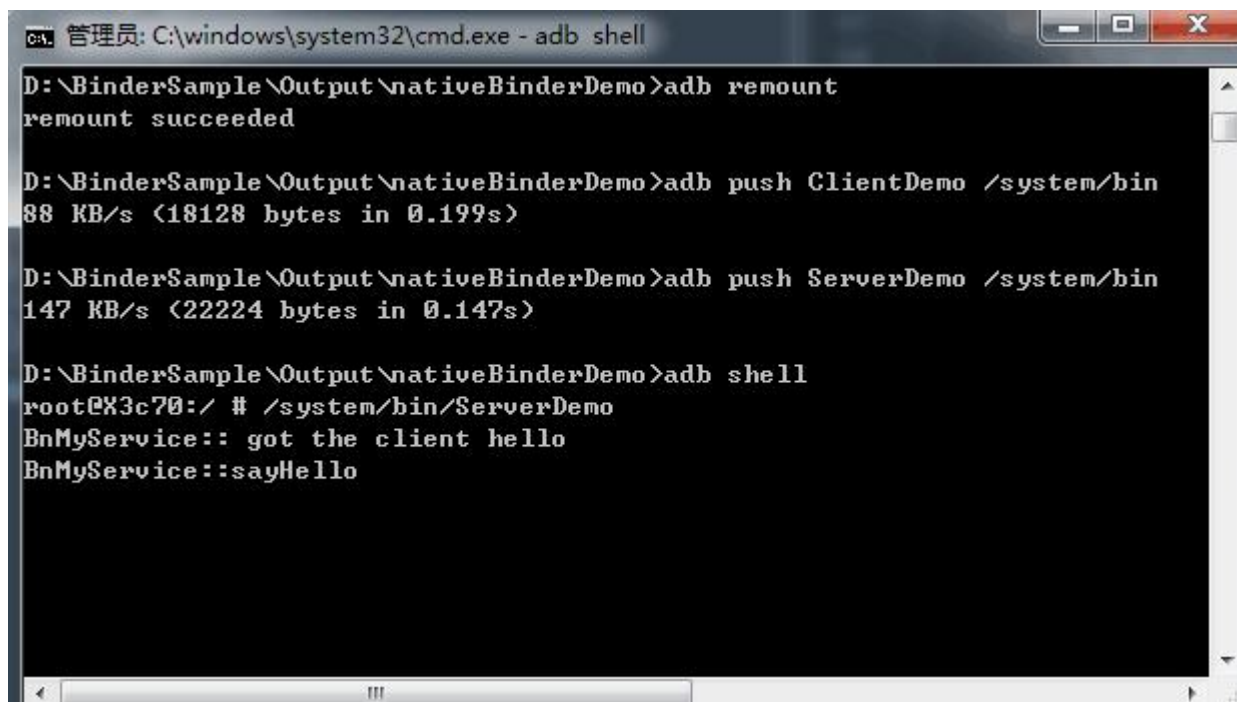
```
adb push ServerDemo /system/bin
adb push ClientDemo /system/bin
```

如果 push 不成功，那么先执行 `adb remount`，再执行上面的指令；如果还不成功，可能就是权限不够。

如果上述开启成功，通过开启两个窗口运行（一个运行 **client** 端，另一个运行 **server** 端）

(3)结果

服务端：



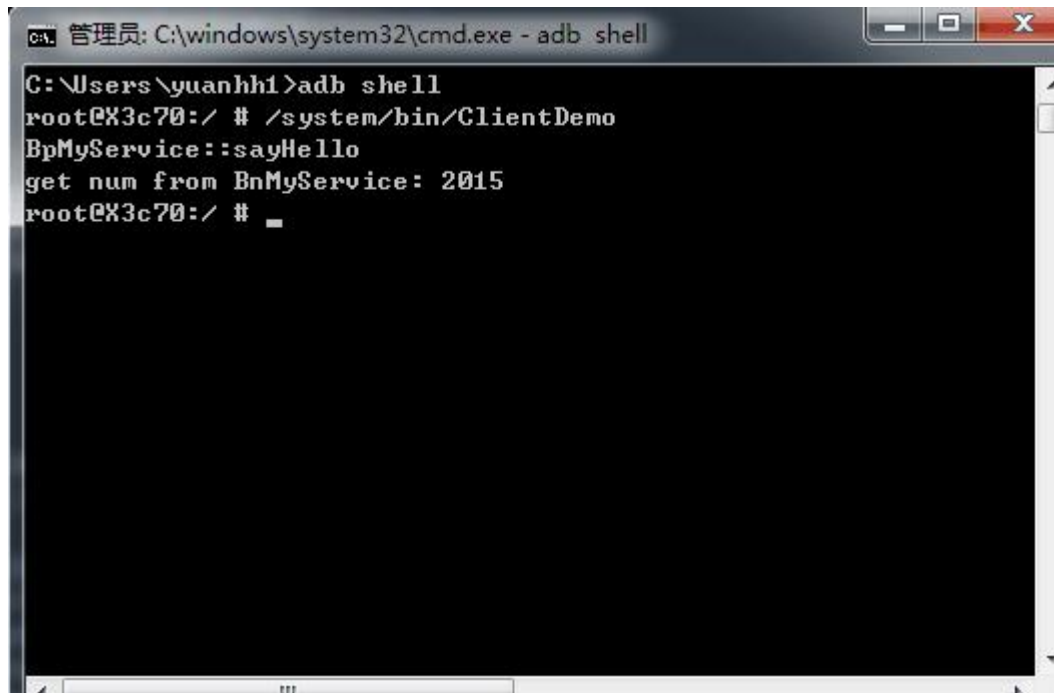
```
管理员: C:\windows\system32\cmd.exe - adb shell
D:\BinderSample\Output\nativeBinderDemo>adb remount
remount succeeded

D:\BinderSample\Output\nativeBinderDemo>adb push ClientDemo /system/bin
88 KB/s (18128 bytes in 0.199s)

D:\BinderSample\Output\nativeBinderDemo>adb push ServerDemo /system/bin
147 KB/s (22224 bytes in 0.147s)

D:\BinderSample\Output\nativeBinderDemo>adb shell
root@X3c70:/ # /system/bin/ServerDemo
BnMyService:: got the client hello
BnMyService::sayHello
```

客户端：



```
C:\Users\yuanhh1>adb shell
root@X3c70:/ # /system/bin/ClientDemo
BpMyService::sayHello
get num from BpMyService: 2015
root@X3c70:/ # _
```

二、Framework 层 Binder

源码结构:

Server 端

1. ServerDemo.java: 可执行程序
2. IMyService.java: 定义 IMyService 接口
3. MyService.java: 定义 MyService

Client 端

1. ClientDemo.java: 可执行程序
2. IMyService.java: 与 Server 端完全一致
3. MyServiceProxy.java: 定义 MyServiceProxy

2.1 Server 端

(1)ServerDemo.java

可执行程序

```
public class ServerDemo {  
  
    public static void main(String[] args) {  
  
        System.out.println("MyService Start");  
  
        //准备 Looper 循环执行  
  
        Looper.prepareMainLooper();  
  
        //设置为前台优先级  
  
        android.os.Process.setThreadPriority(android.os.Process.THREAD_PRIORITY_  
FOREGROUND);  
  
        //注册服务  
  
        ServiceManager.addService("MyService", new MyService());  
  
        Looper.loop();  
  
    }  
  
}
```

(2)IMyService.java

定义 sayHello()方法，DESCRIPTOR 属性

```
public interface IMyService extends IInterface {  
  
    static final java.lang.String DESCRIPTOR = "com.gityuan.frameworkBinder.MySe  
rver";  
  
    public void sayHello(String str) throws RemoteException ;  
  
    static final int TRANSACTION_say = android.os.IBinder.FIRST_CALL_TRANSACTIO  
N;  
  
}
```

(3)MyService.java

```
public class MyService extends Binder implements IMyService{
```

```
public MyService() {  
  
    this.attachInterface(this, DESCRIPTOR);  
  
}  
  
@Override  
public IBinder asBinder() {  
  
    return this;  
  
}  
  
/** 将 MyService 转换为 IMyService 接口 */  
public static com.gityuan.frameworkBinder.IMyService asInterface( android.os.  
IBinder obj) {  
  
    if ((obj == null)) {  
  
        return null;  
  
    }  
  
    android.os.IInterface iInterface = obj.queryLocalInterface(DESCRIPTOR);  
  
    if (((iInterface != null)&&(iInterface instanceof com.gityuan.frameworkB  
inder.IMyService)))){  
  
        return ((com.gityuan.frameworkBinder.IMyService) iInterface);  
  
    }  
  
    return null;  
  
}  
  
/** 服务端，接收远程消息，处理 onTransact 方法 */  
@Override  
protected boolean onTransact(int code, Parcel data, Parcel reply, int flag  
s) throws RemoteException {  
  
    switch (code) {  
  
        case INTERFACE_TRANSACTION: {  
  
            reply.writeString(DESCRIPTOR);  
  
            return true;  
  
        }  
  
    }  
  
}
```

```
    }  
  
    case TRANSACTION_say: {  
        data.enforceInterface(DESCRIPTOR);  
        String str = data.readString();  
        sayHello(str);  
        reply.writeNoException();  
        return true;  
    }  
  
    return super.onTransact(code, data, reply, flags);  
}  
  
/** 自定义 sayHello()方法 */  
@Override  
public void sayHello(String str) {  
    System.out.println("MyService:: Hello, " + str);  
}  
}
```

2.2 Client 端

(1)ClientDemo.java

可执行程序

```
public class ClientDemo {  
    public static void main(String[] args) throws RemoteException {  
        System.out.println("Client start");  
        IBinder binder = ServiceManager.getService("MyService"); //获取名为"MyService"的服务  
        IMyService myService = new MyServiceProxy(binder); //创建 MyServiceProxy对象
```

```
myService.sayHello("binder"); //通过 MyServiceProxy 对象调用接口的方法

System.out.println("Client end");

}

}
```

(2)IMyService.java

与 Server 端的 IMyService 是一致的，基本都是拷贝一份过来。

(3)MyServiceProxy.java

```
public class MyServiceProxy implements IMyService {

    private android.os.IBinder mRemote; //代表 BpBinder

    public MyServiceProxy(android.os.IBinder remote) {

        mRemote = remote;

    }

    public java.lang.String getInterfaceDescriptor() {

        return DESCRIPTOR;

    }

    /** 自定义的 sayHello()方法 */

    @Override

    public void sayHello(String str) throws RemoteException {

        android.os.Parcel _data = android.os.Parcel.obtain();

        android.os.Parcel _reply = android.os.Parcel.obtain();

        try {

            _data.writeInterfaceToken(DESCRIPTOR);

            _data.writeString(str);

            mRemote.transact(TRANSACTION_say, _data, _reply, 0);

        } catch (Exception e) {

            e.printStackTrace();

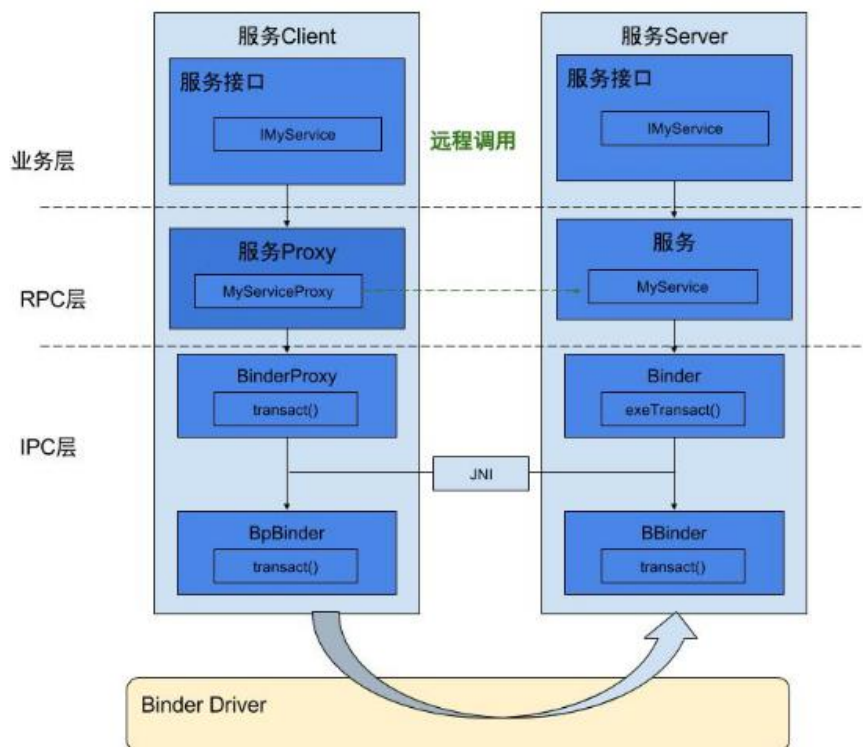
        }

    }

}
```

```
_reply.readException();  
  
} finally {  
  
    _reply.recycle();  
  
    _data.recycle();  
  
}  
  
}  
  
}  
  
@Override  
  
public IBinder asBinder() {  
  
    return mRemote;  
  
}  
  
}
```

2.3 原理图



2.4 运行

首先将 ServerDemo, ClientDemo 可执行文件, 以及 ServerDemo.jar, ClientDemo.jar 都 push 到手机

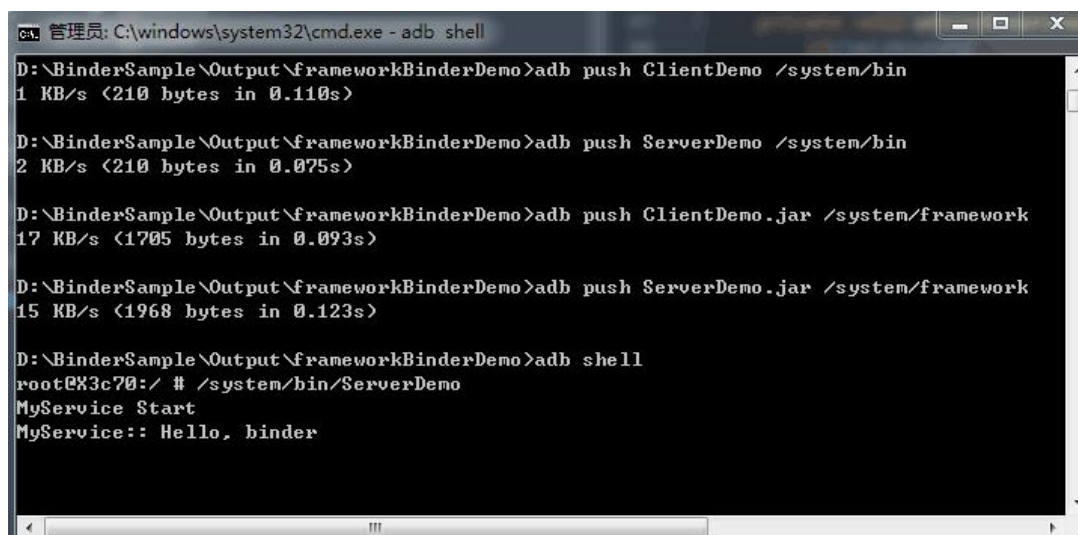
```
adb push ServerDemo /system/bin
adb push ClientDemo /system/bin
adb push ServerDemo.jar /system/framework
adb push ClientDemo.jar /system/framework
```

如果 push 不成功, 那么先执行 `adb remount`, 再执行上面的指令; 如果还不成功, 可能就是权限不够。

如果上述开启成功, 通过开启两个窗口运行 (一个运行 client 端, 另一个运行 server 端)

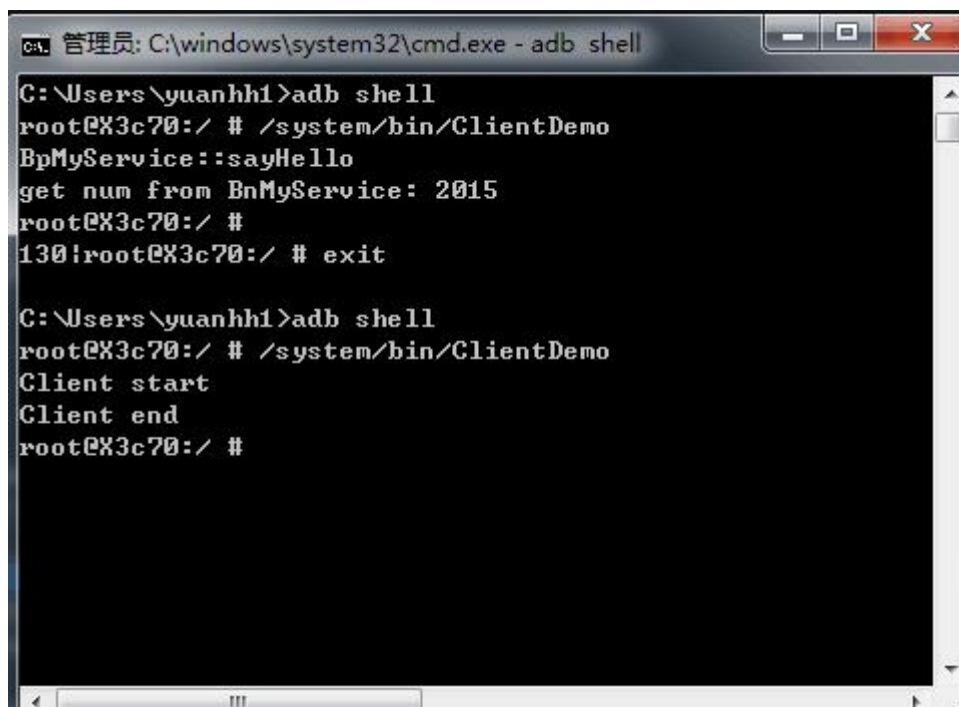
结果

服务端:



```
管理员: C:\windows\system32\cmd.exe - adb shell
D:\BinderSample\Output\frameworkBinderDemo>adb push ClientDemo /system/bin
1 KB/s <210 bytes in 0.110s>
D:\BinderSample\Output\frameworkBinderDemo>adb push ServerDemo /system/bin
2 KB/s <210 bytes in 0.075s>
D:\BinderSample\Output\frameworkBinderDemo>adb push ClientDemo.jar /system/framework
17 KB/s <1705 bytes in 0.093s>
D:\BinderSample\Output\frameworkBinderDemo>adb push ServerDemo.jar /system/framework
15 KB/s <1968 bytes in 0.123s>
D:\BinderSample\Output\frameworkBinderDemo>adb shell
root@X3c70:/ # /system/bin/ServerDemo
MyService Start
MyService:: Hello, binder
```

客户端:



```
C:\Users\yuanhh1>adb shell
root@X3c70:/ # /system/bin/ClientDemo
BpMyService::sayHello
get num from BnMyService: 2015
root@X3c70:/ #
130!root@X3c70:/ # exit

C:\Users\yuanhh1>adb shell
root@X3c70:/ # /system/bin/ClientDemo
Client start
Client end
root@X3c70:/ #
```

第十节如何使用 AIDL

自定义 binder 架构的 client/ server 组件

一、AIDL

1.1 Server 端

RemoteService.java

本例是为了演示进程间的通信机制，故需要将 Service 与 Activity 处于不同的进程，需要在 AndroidManifest.xml 中，把 service 配置成 `android:process=":remote"`，进程也可以命名成其他的。

```
public class RemoteService extends Service {

    private static final String TAG = "BinderSimple";

    MyData mMyData;
```

```
@Override

public void onCreate() {

    super.onCreate();

    Log.i(TAG, "[RemoteService] onCreate");

    initMyData();

}

@Override

public IBinder onBind(Intent intent) {

    Log.i(TAG, "[RemoteService] onBind");

    return mBinder;

}

@Override

public boolean onUnbind(Intent intent) {

    Log.i(TAG, "[RemoteService] onUnbind");

    return super.onUnbind(intent);

}

@Override

public void onDestroy() {

    Log.i(TAG, "[RemoteService] onDestroy");

    super.onDestroy();

}

/** * 实现 IRemoteService.aidl 中定义的方法 */

private final IRemoteService.Stub mBinder = new IRemoteService.Stub() {

    @Override
```



```
public int getPid() throws RemoteException {  
    Log.i(TAG, "[RemoteService] getPid()="+android.os.Process.myPid());  
    return android.os.Process.myPid();  
}  
  
@Override  
public MyData getMyData() throws RemoteException {  
    Log.i(TAG, "[RemoteService] getMyData() "+ mMyData.toString());  
    return mMyData;  
}  
  
/**此处可用于权限拦截**/  
@Override  
public boolean onTransact(int code, Parcel data, Parcel reply, int flags  
s) throws RemoteException {  
    return super.onTransact(code, data, reply, flags);  
}  
};  
  
/** * 初始化 MyData 数据 */  
private void initMyData() {  
    mMyData = new MyData();  
    mMyData.setData1(10);  
    mMyData.setData2(20);  
}  
}
```

1.2 Client 端

ClientActivity.java

```
public class ClientActivity extends AppCompatActivity {

    private static final String TAG = "BinderSimple";

    private IRemoteService mRemoteService;

    private boolean mIsBound;

    private TextView mCallBackTv;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        Log.i(TAG, "[ClientActivity] onCreate");

        setContentView(R.layout.activity_main);

        mCallBackTv = (TextView) findViewById(R.id.tv_callback);
        mCallBackTv.setText(R.string.remote_service_unattached);
    }

    /**
     * 用语监控远程服务连接的状态
     */

    private ServiceConnection mConnection = new ServiceConnection() {

        @Override

        public void onServiceConnected(ComponentName name, IBinder service) {

            mRemoteService = IRemoteService.Stub.asInterface(service);

            String pidInfo = null;

            try {

                MyData myData = mRemoteService.getMyData();
```

```
        pidInfo = "pid="+ mRemoteService.getPid() +  
            ", data1 = "+ myData.getData1() +  
            ", data2="+ myData.getData2();  
    } catch (RemoteException e) {  
        e.printStackTrace();  
    }  
  
    Log.i(TAG, "[ClientActivity] onServiceConnected "+pidInfo);  
    mCallBackTv.setText(pidInfo);  
  
    Toast.makeText(ClientActivity.this, R.string.remote_service_connecte  
d, Toast.LENGTH_SHORT).show();  
}  
  
@Override  
public void onServiceDisconnected(ComponentName name) {  
    Log.i(TAG, "[ClientActivity] onServiceDisconnected");  
    mCallBackTv.setText(R.string.remote_service_disconnected);  
    mRemoteService = null;  
    Toast.makeText(ClientActivity.this, R.string.remote_service_disconne  
cted, Toast.LENGTH_SHORT).show();  
}  
};  
  
public void clickHandler(View view){  
    switch (view.getId()){  
        case R.id.btn_bind:  
            bindRemoteService();  
            break;  
  
        case R.id.btn_unbind:  
            unbindRemoteService();  
    }  
}
```

```
        break;

        case R.id.btn_kill:
            killRemoteService();
            break;
    }
}

/**
 * 绑定远程服务
 */
private void bindRemoteService(){
    Log.i(TAG, "[ClientActivity] bindRemoteService");
    Intent intent = new Intent(ClientActivity.this, RemoteService.class);
    intent.setAction(IRemoteService.class.getName());
    bindService(intent, mConnection, Context.BIND_AUTO_CREATE);

    mIsBound = true;
    mCallBackTv.setText(R.string.binding);
}

/**
 * 解除绑定远程服务
 */
private void unbindRemoteService(){
    if(!mIsBound){
        return;
    }

    Log.i(TAG, "[ClientActivity] unbindRemoteService ==>");
    unbindService(mConnection);
}
```

```
mIsBound = false;

mCallBackTv.setText(R.string.unbinding);

}

/**
 * 杀死远程服务
 */

private void killRemoteService(){
    Log.i(TAG, "[ClientActivity] killRemoteService");
    try {
        android.os.Process.killProcess(mRemoteService.getPid());
        mCallBackTv.setText(R.string.kill_success);
    } catch (RemoteException e) {
        e.printStackTrace();
        Toast.makeText(ClientActivity.this, R.string.kill_failure, Toast.LENGTH_SHORT).show();
    }
}
}
```

1.3 AIDL 文件

(1)IRemoteService.aidl 定义远程通信的接口方法

```
interface IRemoteService {

    int getPid();

    MyData getMyData();

}
```

(2)MyData.aidl 定义远程通信的自定义数据

```
parcelable MyData;
```

1.4 Parcel 数据

MyData.java

```
public class MyData implements Parcelable {  
  
    private int data1;  
    private int data2;  
  
    public MyData(){  
  
    }  
  
    protected MyData(Parcel in) {  
        readFromParcel(in);  
    }  
  
    public static final Creator<MyData> CREATOR = new Creator<MyData>() {  
        @Override  
        public MyData createFromParcel(Parcel in) {  
            return new MyData(in);  
        }  
  
        @Override  
        public MyData[] newArray(int size) {  
            return new MyData[size];  
        }  
    };  
};
```

```
@Override

    public int describeContents() {

        return 0;

    }

    /** 将数据写入到 Parcel */

    @Override

    public void writeToParcel(Parcel dest, int flags) {

        dest.writeInt(data1);

        dest.writeInt(data2);

    }

    /** 从 Parcel 中读取数据 */

    public void readFromParcel(Parcel in){

        data1 = in.readInt();

        data2 = in.readInt();

    }


    public int getData2() {

        return data2;

    }


    public void setData2(int data2) {

        this.data2 = data2;

    }


    public int getData1() {

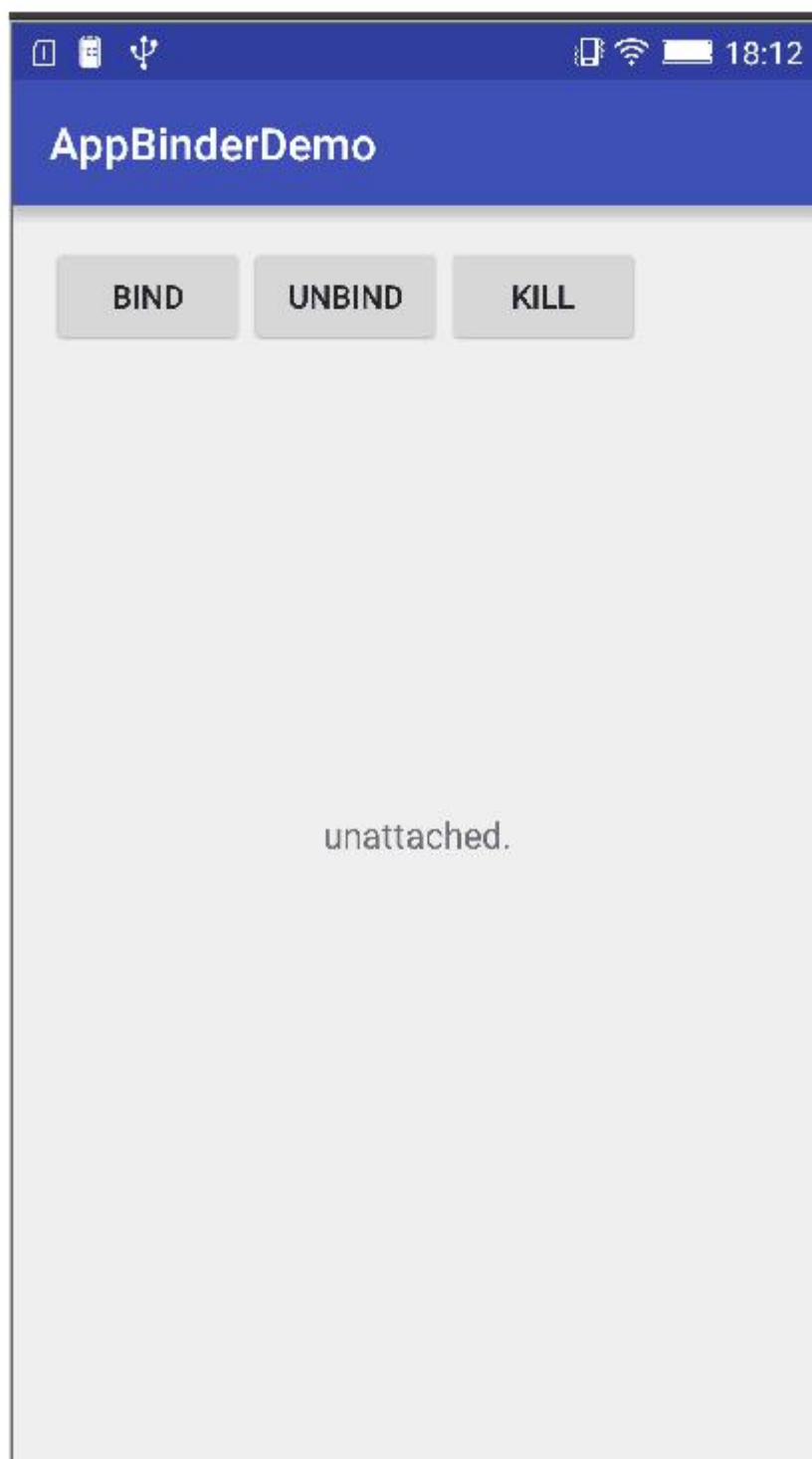
        return data1;

    }
```

```
public void setData1(int data1) {  
    this.data1 = data1;  
}  
  
@Override  
public String toString() {  
    return "data1 = " + data1 + ", data2=" + data2;  
}  
}
```

1.5 运行

该工程会生成一个 apk，安装到手机，打开 apk，界面如下：



界面上有三个按钮，分别是功能分别是 `bindService`(绑定 `Service`), `unbindService`(解除绑定 `Service`), `killProcess`(杀死 `Service` 进程)。

从左往右，依次点击界面，可得：

```
C:\windows\system32\cmd.exe - adb shell

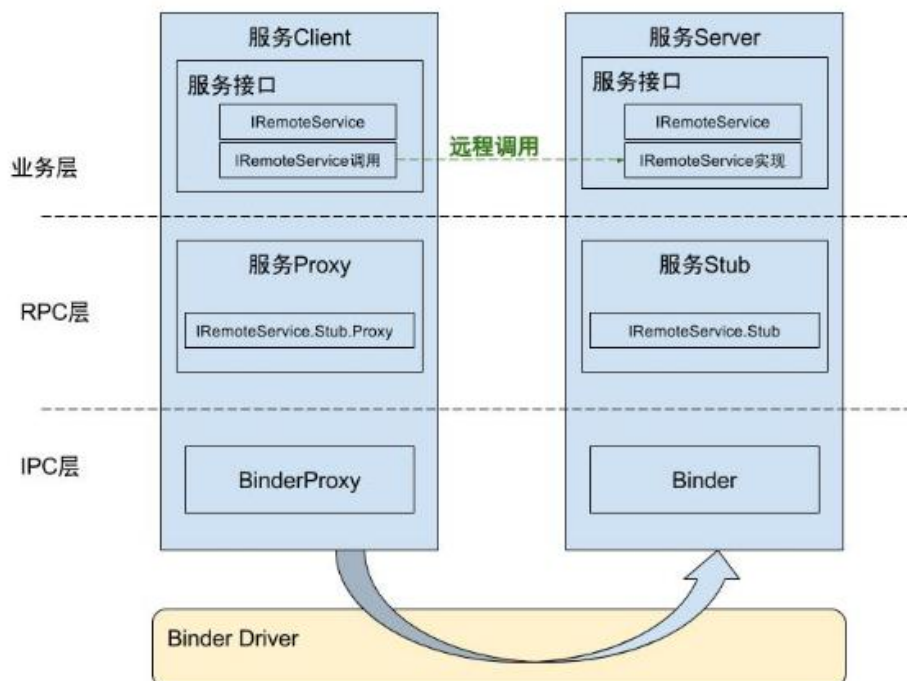
I/BinderSimple<16195>: [ClientActivity] bindRemoteService
I/BinderSimple<19867>: [RemoteService] onCreate
I/BinderSimple<19867>: [RemoteService] onBind
I/BinderSimple<19867>: [RemoteService] getMyData() data1 = 10, data2=20
I/BinderSimple<19867>: [RemoteService] getPid()=19867
I/BinderSimple<16195>: [ClientActivity] onServiceConnected pid=19867, data1 = 10, data2=20

I/BinderSimple<16195>: [ClientActivity] unbindRemoteService
I/BinderSimple<19867>: [RemoteService] onUnbind
I/BinderSimple<19867>: [RemoteService] onDestroy

I/BinderSimple<16195>: [ClientActivity] killRemoteService
I/BinderSimple<19867>: [RemoteService] getPid()=19867
```

二、原理分析

调用图：



2.1 源码

采用 AIDL 技术，是原理还是利用 framework binder 的架构。本文的实例 AIDL 会自动生成一个与之相对应的 IRemoteService.java 文件，如下：

```
package com.gityuan.appbinderdemo;public interface IRemoteService extends android.os.IInterface {

    public static abstract class Stub extends android.os.Binder implements com.gityuan.appbinderdemo.IRemoteService {

        private static final java.lang.String DESCRIPTOR = "com.gityuan.appbinderdemo.IRemoteService";

        /**
         * Stub 构造函数
         */
        public Stub() {

            this.attachInterface(this, DESCRIPTOR);

        }

        /**
         * 将 IBinder 转换为 IRemoteService interface
         */
        public static com.gityuan.appbinderdemo.IRemoteService asInterface(android.os.IBinder obj) {

            if ((obj == null)) {

                return null;

            }

            android.os.IInterface iin = obj.queryLocalInterface(DESCRIPTOR);

            if (((iin != null) && (iin instanceof com.gityuan.appbinderdemo.IRemoteService))) {

                return ((com.gityuan.appbinderdemo.IRemoteService) iin);

            }

            return new com.gityuan.appbinderdemo.IRemoteService.Stub.Proxy(obj);

        }

    }

}
```

```
@Override
public android.os.IBinder asBinder() {
    return this;
}

@Override
public boolean onTransact(int code, android.os.Parcel data, android.os.Parcel reply, int flags) throws android.os.RemoteException {
    switch (code) {
        case INTERFACE_TRANSACTION: {
            reply.writeString(DESCRIPTOR);
            return true;
        }
        case TRANSACTION_getPid: {
            data.enforceInterface(DESCRIPTOR);
            int _result = this.getPid();
            reply.writeNoException();
            reply.writeInt(_result);
            return true;
        }
        case TRANSACTION_getMyData: {
            data.enforceInterface(DESCRIPTOR);
            com.gityuan.appbinderdemo.MyData _result = this.getMyData();
            reply.writeNoException();
            if ((_result != null)) {
                reply.writeInt(1);
                _result.writeToParcel(reply, android.os.Parcelable.PARCELABLE_WRITE_RETURN_VALUE);
            } else {
                reply.writeInt(0);
            }
        }
    }
}
```

```
    }

    return true;
}

}

return super.onTransact(code, data, reply, flags);
}

private static class Proxy implements com.gityuan.appbinderdemo.IRemoteService {
    private android.os.IBinder mRemote;

    /**
     * Proxy 构造函数
     */
    Proxy(android.os.IBinder remote) {
        mRemote = remote;
    }

    @Override
    public android.os.IBinder asBinder() {
        return mRemote;
    }

    public java.lang.String getInterfaceDescriptor() {
        return DESCRIPTOR;
    }

    @Override
    public int getPid() throws android.os.RemoteException {
        android.os.Parcel _data = android.os.Parcel.obtain();
    }
}
```

```
        android.os.Parcel _reply = android.os.Parcel.obtain();

        int _result;

        try {

            _data.writeInterfaceToken(DESCRIPTOR);

            mRemote.transact(Stub.TRANSACTION_getPid, _data, _reply, 0);

            _reply.readException();

            _result = _reply.readInt();

        } finally {

            _reply.recycle();

            _data.recycle();

        }

        return _result;

    }

    @Override

    public com.gityuan.appbinderdemo.MyData getMyData() throws android.os.RemoteException {

        android.os.Parcel _data = android.os.Parcel.obtain();

        android.os.Parcel _reply = android.os.Parcel.obtain();

        com.gityuan.appbinderdemo.MyData _result;

        try {

            _data.writeInterfaceToken(DESCRIPTOR);

            mRemote.transact(Stub.TRANSACTION_getMyData, _data, _reply, 0);

            _reply.readException();

            if ((0 != _reply.readInt())) {

                _result = com.gityuan.appbinderdemo.MyData.CREATOR.createFromParcel(_reply);

            } else {

                _result = null;

            }

        }
```

```
        } finally {  
            _reply.recycle();  
            _data.recycle();  
        }  
        return _result;  
    }  
}  
  
    static final int TRANSACTION_getPid = (android.os.IBinder.FIRST_CALL_TRANSACTION + 0);  
    static final int TRANSACTION_getMyData = (android.os.IBinder.FIRST_CALL_TRANSACTION + 1);  
}  
  
    public int getPid() throws android.os.RemoteException;  
  
    public com.gityuan.appbinderdemo.MyData getMyData() throws android.os.RemoteException;  
}
```

第十一节 Binder 总结

基于 Android 6.0 的源码剖析， Binder 所涉及的源码目录

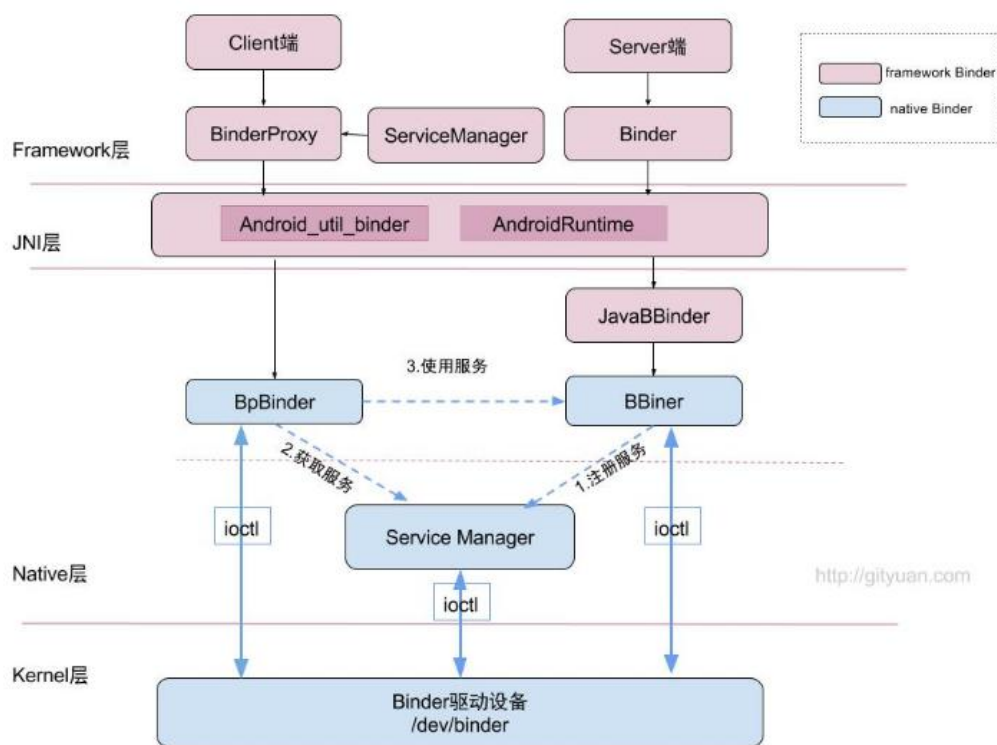
1. Binder 概述

1. 从 IPC 角度来说：Binder 是 Android 中的一种跨进程通信方式，该通信方式在 linux 中没有，是 Android 独有；
2. 从 Android Driver 层：Binder 还可以理解为一种虚拟的物理设备，它的设备驱动是/dev/binder；
3. 从 Android Native 层：Binder 是创建 Service Manager 以及 BpBinder/BBinder 模型，搭建与 binder 驱动的桥梁；
4. 从 Android Framework 层：Binder 是各种 Manager（ActivityManager、WindowManager 等）和相应 xxxManagerService 的桥梁；

更多录播视频+架构学习资料免费领取 QQ 3607409029

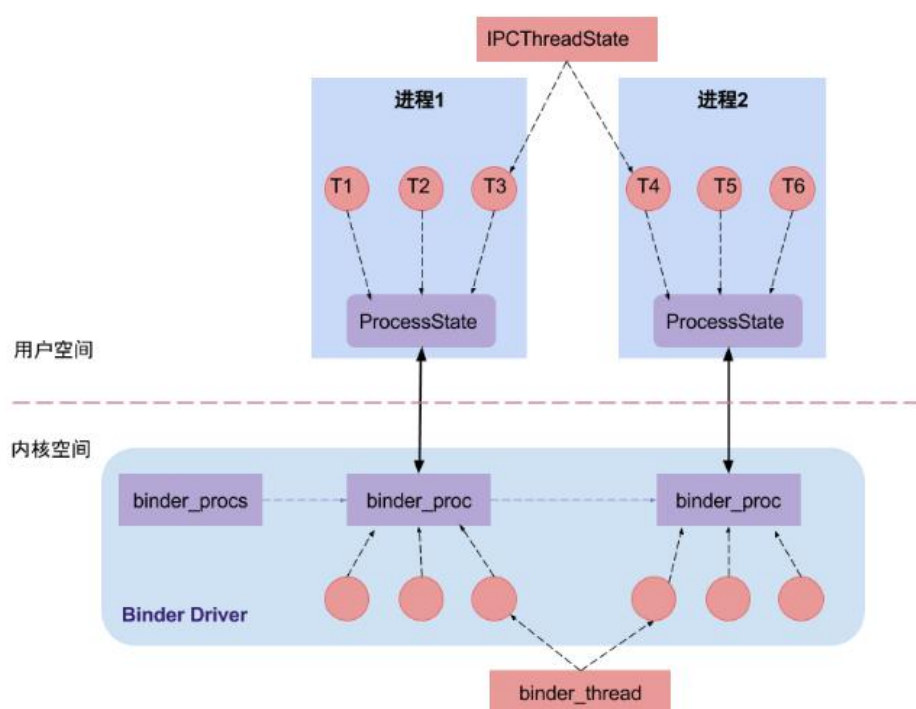
5. 从 Android APP 层：Binder 是客户端和服务端进行通信的媒介，当 `bindService` 的时候，服务端会返回一个包含了服务端业务调用的 `Binder` 对象，通过这个 `Binder` 对象，客户端就可以获取服务端提供的服务或者数据，这里的服务包括普通服务和基于 AIDL 的服务。

2. Binder 架构



Binder 在整个 Android 系统中有这举足轻重的地位，在 Native 层有一套完整的 binder 通信的 C/S 架构(图中的蓝色)，BpBinder 作为客户端，BBinder 作为服务端。基于 native 层的 Binder 框架，Java 也有一套镜像功能的 binder C/S 架构，通过 JNI 技术，与 native 层的 binder 对应，Java 层的 binder 功能最终都是交给 native 的 binder 来完成。从 kernel 到 native, jni, framework 层的架构所涉及的所有有关类和方法见 [Binder 类图](#)。

3. Binder 进程与线程



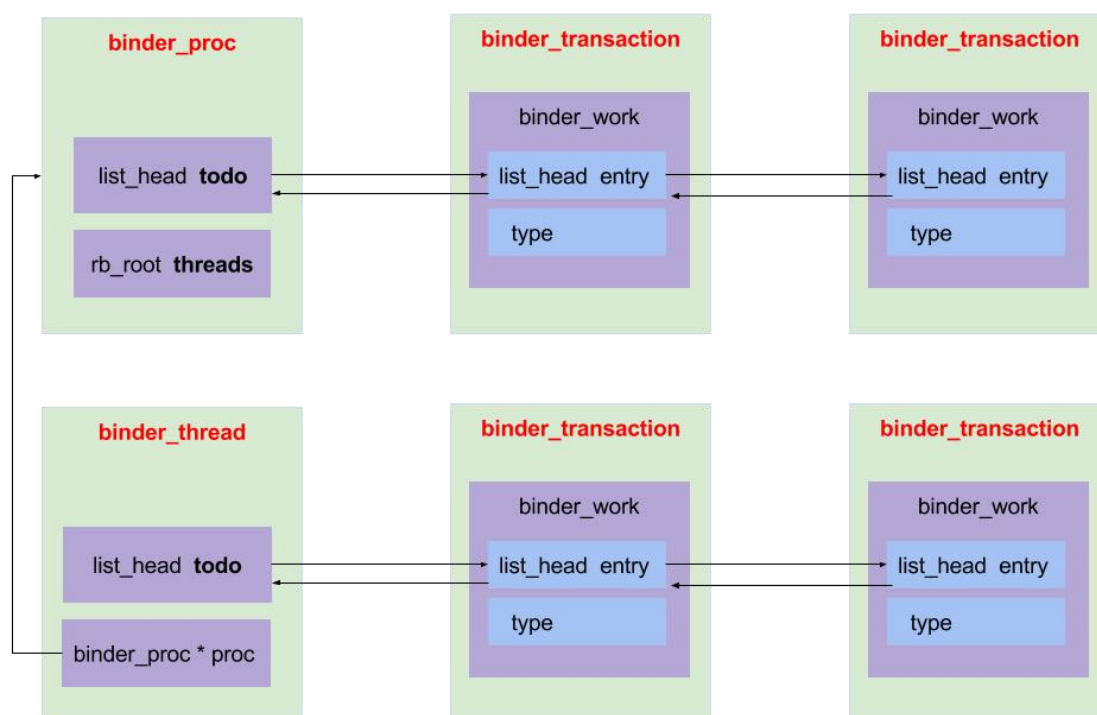
对于底层 Binder 驱动，通过 **binder_procs** 链表记录所有创建的 **binder_proc** 结构体，binder 驱动层的每一个 **binder_proc** 结构体都与用户空间的一个用于 binder 通信的进程一一对应，且每个进程有且只有一个 **ProcessState** 对象，这是通过单例模式来保证的。在每个进程中可以有多个线程，每个线程对应一个 **IPCThreadState** 对象，**IPCThreadState** 对象也是单例模式，即一个线程对应一个 **IPCThreadState** 对象，在 Binder 驱动层也有与之相对应的结构，那就是 **Binder_thread** 结构体。在 **binder_proc** 结构体中通过成员变量 **rb_root threads**，来记录当前进程内所有的 **binder_thread**。

Binder 线程池：每个 Server 进程在启动时会创建一个 binder 线程池，并向其中注册一个 Binder 线程；之后 Server 进程也可以向 binder 线程池注册新的线程，或者 Binder 驱动在探测到没有空闲 binder 线程时会主动向 Server 进程注册新的 binder 线程。对于一个 Server 进程有一个最大 Binder 线程数限制，默认为 16 个 binder 线程，例如 Android 的 **system_server** 进程就存在 16 个线程。对于所有 Client 端进程的 binder 请求都是交由 Server 端进程的 binder 线程来处理的。

4. Binder 传输过程

Binder IPC 机制，就是指在进程间传输数据（`binder_transaction_data`），一次数据的传输，称为事务（`binder_transaction`）。对于多个不同进程向同一个进程发送事务时，这个同一个进程或线程的事务需要串行执行，在 Binder 驱动中为 `binder_proc` 和 `binder_thread` 都有 `todo` 队列。

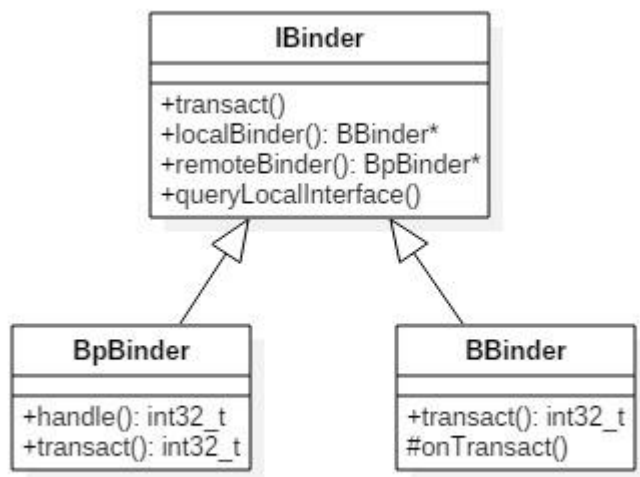
也就是说对于进程间的通信，就是发送端把 `binder_transaction` 节点，插入到目标进程或其子线程的 `todo` 队列中，等目标进程或线程不断循环地从 `todo` 队列中取出数据并进行相应的操作。



在 Binder 驱动层，每个接收端进程都有一个 `todo` 队列，用于保存发送端进程发送过来的 `binder` 请求，这类请求可以由接收端进程的任意一个空闲的 `binder` 线程处理；接收端进程存在一个或多个 `binder` 线程，在每个 `binder` 线程里都有一个 `todo` 队列，也是用于保存发送端进程发送过来的 `binder` 请求，这类请求只能由当前 `binder` 线程来处理。`binder` 线程在空闲时进入可中断的休眠状态，当自己的 `todo` 队列或所属进程的 `todo` 队列有新的请求到来时便会唤醒，如果是由所需进程唤醒的，那么进程会让其中一个线程处理响应的请求，其他线程再次进入休眠状态。

5. Binder 路由

先来看看 Native Binder IPC 的两个重量级对象: BpBinder(客户端)和 BBinder(服务端)都是 Android 中 Binder 通信相关的代表, 它们都从 IBinder 类中派生而来, 关系图如下:



- IBinder 有一个重要方法 queryLocalInterface, 默认返回值为 NULL;
 - BBinder/BpBinder 都没有实现, 默认返回 NULL; BnInterface 重写该方法;
 - BinderProxy(Java)默认返回 NULL; Binder(Java)重写该方法;
- IInterface 有一个重要方法 asBinder;
- IInterface 子类(服务端)会有一个方法 asInterface;

Native 层通过宏 IMPLEMENT_META_INTERFACE 来完成 asInterface 实现和 descriptor 的赋值过程;

对于 Java 层跟 Native 一样, 也有完全对应的一套对象和方法:

- 例如 ActivityManagerNative, 通过实现 asInterface 方法, 以及其通过其构造函数 调用 attachInterface(), 完成 descriptor 的赋值过程。
- 再如 AIDL 全自动生成 asInterface 和 descriptor 赋值过程。

同一个进程, 请求 binder 服务, 不需要创建 binder_ref, BpBinder 等这些对象, 但是是否需要经过 binder call, 取决于 descriptor 是否设置。这就涉及到 Java 服务 Native 使用, 或许 Native 服务在 Java 层使用, 需要格外注意。

binder 的路由原理：BpBinder 发送端，根据 handler，在当前 binder_proc 中，找到相应的 binder_ref，由 binder_ref 再找到目标 binder_node 实体，由目标 binder_node 再找到目标进程 binder_proc。简单地方式是直接把 binder_transaction 节点插入到 binder_proc 的 todo 队列中，完成传输过程。

对于 binder 驱动来说应尽可能地把 binder_transaction 节点插入到目标进程的某个线程的 todo 队列，效率更高。当 binder 驱动可以找到合适的线程，就会把 binder_transaction 节点插入到相应线程的 todo 队列中，如果找不到合适的线程，就把节点插入到 binder_proc 的 todo 队列。

第十二节 Binder 面试题全解析

Binder 是一种进程间通信机制，做 Android 开发肯定离不开跟 Binder 打交道。在面试中 Binder 也是经常被问的一个点，那么本篇文章就以问答的方式，带你了解一下关于 Binder 的重要知识点。

1. Binder 有什么优势

性能方面

- 共享内存 0 次数据拷贝
- Binder 1 次数据拷贝
- Socket/管道/消息队列 2 次数据拷贝

稳定性方面

- Binder：基于 C/S 架构，客户端（Client）有什么需求就丢给服务端（Server）去完成，架构清晰、职责明确又相互独立，自然稳定性更好
- 共享内存：虽然无需拷贝，但是控制复杂，难以使用
- 从稳定性的角度讲，Binder 机制是优于内存共享的。

安全性方面

- 传统的 IPC 没有任何安全措施，安全依赖上层协议来确保。
- 传统的 IPC 方法无法获得对方可靠的进程用户 ID/进程 UI（UID/PID），从而无法鉴别对方身份。
- 传统的 IPC 只能由用户在数据包中填入 UID/PID，容易被恶意程序利用。
- 传统的 IPC 访问接入点是开放的，无法阻止恶意程序通过猜测接收方地址获得连接。
- Binder 既支持实名 Binder，又支持匿名 Binder，安全性高。

2. Binder 是如何做到一次拷贝的

主要是因为 Linux 是使用的虚拟内存寻址方式，它有如下特性：

更多录播视频+架构学习资料免费领取 QQ 3607409029

- 用户空间的虚拟内存地址是映射到物理内存中的
- 对虚拟内存的读写实际上是对物理内存的读写，这个过程就是内存映射
- 这个内存映射过程是通过系统调用 `mmap()` 来实现的

Binder 借助了内存映射的方法，在内核空间和接收方用户空间的数据缓存区之间做了一层内存映射，就相当于直接拷贝到了接收方用户空间的数据缓存区，从而减少了一次数据拷贝

3.MMAP 的内存映射原理了解吗

MMAP 内存映射的实现过程，总的来说可以分为三个阶段：

（一）进程启动映射过程，并在虚拟地址空间中为映射创建虚拟映射区域

1. 进程在用户空间调用库函数 `mmap`，原型：`void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset);`
2. 在当前进程的虚拟地址空间中，寻找一段空闲的满足要求的连续的虚拟地址
3. 为此虚拟区分配一个 `vm_area_struct` 结构，接着对这个结构的各个域进行了初始化
4. 将新建的虚拟区结构（`vm_area_struct`）插入进程的虚拟地址区域链表或树中

（二）调用内核空间的系统调用函数 `mmap`（不同于用户空间函数），实现文件物理地址和进程虚拟地址的一一映射关系

1. 为映射分配了新的虚拟地址区域后，通过待映射的文件指针，在文件描述符表中找到对应的文件描述符，通过文件描述符，链接到内核“已打开文件集”中该文件的文件结构体（`struct file`），每个文件结构体维护着和这个已打开文件相关各项信息。
2. 通过该文件的文件结构体，链接到 `file_operations` 模块，调用内核函数 `mmap`，其原型为：`int mmap(struct file *filp, struct vm_area_struct *vma)`，不同于用户空间库函数。
3. 内核 `mmap` 函数通过虚拟文件系统 `inode` 模块定位到文件磁盘物理地址。
4. 通过 `remap_pfn_range` 函数建立页表，即实现了文件地址和虚拟地址区域的映射关系。此时，这片虚拟地址并没有任何数据关联到主存中。

（三）进程发起对这片映射空间的访问，引发缺页异常，实现文件内容到物理内存（主存）的拷贝

注：前两个阶段仅在于创建虚拟区间并完成地址映射，但是并没有将任何文件数据的拷贝至主存。真正的文件读取是当进程发起读或写操作时。

进程的读或写操作访问虚拟地址空间这一段映射地址，通过查询页表，发现这一段地址并不在物理页面上。因为目前只建立了地址映射，真正的硬盘数据还没有拷贝到内存中，因此引发缺页异常。

1. 缺页异常进行一系列判断，确定无非法操作后，内核发起请求调页过程。
2. 调页过程先在交换缓存空间（`swap cache`）中寻找需要访问的内存页，如果没有则调用 `nopage` 函数把所缺的页从磁盘装入到主存中。
3. 之后进程即可对这片主存进行读或者写的操作，如果写操作改变了其内容，一定时间后系统会自动回写脏页面到对应磁盘地址，也即完成了写入到文件的过程。

注：修改过的脏页面并不会立即更新回文件中，而是有一段时间的延迟，可以调用 `msync()` 来强制同步，这样所写的内容就能立即保存到文件里了。

4. Binder 机制是如何跨进程的

1. Binder 驱动

- 在内核空间创建一块接收缓存区，
- 实现地址映射：将内核缓存区、接收进程用户空间映射到同一接收缓存区

2. 发送进程通过系统调用（`copy_from_user`）将数据发送到内核缓存区。由于内核缓存区和接收进程用户空间存在映射关系，故相当于也发送了接收进程的用户空间，实现了跨进程通信。

5. 说说四大组件的通信机制

1. activity

- （1）一个 Activity 通常就是一个单独的屏幕（窗口）。
- （2）Activity 之间通过 Intent 进行通信。
- （3）android 应用中每一个 Activity 都必须要在 `AndroidManifest.xml` 配置文件中声明，否则系统将不识别也不执行该 Activity。

2. service

（1）service 用于在后台完成用户指定的操作。service 分为两种：

- **started（启动）**：当应用程序组件（如 activity）调用 `startService()` 方法启动服务时，服务处于 **started** 状态。
- **bound（绑定）**：当应用程序组件调用 `bindService()` 方法绑定到服务时，服务处于 **bound** 状态。

（2）`startService()` 与 `bindService()` 区别：

- **started service（启动服务）** 是由其他组件调用 `startService()` 方法启动的，这导致服务的 `onStartCommand()` 方法被调用。当服务是 **started** 状态时，其生命周期与启动它的组件无关，并且可以在后台无限期运行，即使启动服务的组件已经被销毁。因此，服务需要在完成任务后调用 `stopSelf()` 方法停止，或者由其他组件调用 `stopService()` 方法停止。
- 使用 `bindService()` 方法启用服务，调用者与服务绑定在了一起，调用者一旦退出，服务也就终止，大有“不求同时生，必须同时死”的特点。

（3）开发人员需要在应用程序配置文件中声明全部的 service，使用 `<service></service>` 标签。

（4）Service 通常位于后台运行，它一般不需要与用户交互，因此 Service 组件没有图形用户界面。Service 组件需要继承 Service 基类。Service 组件通常用于为其他组件提供后台服务或监控其他组件的运行状态。

3.content provider

(1) android 平台提供了 Content Provider 使一个应用程序的指定数据集提供给其他应用程序。其他应用可以通过 ContentResolver 类从该内容提供者中获取或存入数据。

(2) 只有需要在多个应用程序间共享数据是才需要内容提供者。例如，通讯录数据被多个应用程序使用，且必须存储在一个内容提供者中。它的好处是统一数据访问方式。

(3) ContentProvider 实现数据共享。ContentProvider 用于保存和获取数据，并使其对所有应用程序可见。这是不同应用程序间共享数据的唯一方式，因为 android 没有提供所有应用共同访问的公共存储区。

(4) 开发人员不会直接使用 ContentProvider 类的对象，大多数是通过 ContentResolver 对象实现对 ContentProvider 的操作。

(5) ContentProvider 使用 URI 来唯一标识其数据集，这里的 URI 以 content:// 作为前缀，表示该数据由 ContentProvider 来管理。

4.broadcast receiver

(1) 你的应用可以使用它对外部事件进行过滤，只对感兴趣的外部事件(如当电话呼入时，或者数据网络可用时)进行接收并做出响应。广播接收器没有用户界面。然而，它们可以启动一个 activity 或 service 来响应它们收到的信息，或者用 NotificationManager 来通知用户。通知可以用很多种方式来吸引用户的注意力，例如闪动背灯、震动、播放声音等。一般来说是在状态栏上放一个持久的图标，用户可以打开它并获取消息。

(2) 广播接收者的注册有两种方法，分别是程序动态注册和 AndroidManifest 文件中进行静态注册。

(3) 动态注册广播接收器特点是当用来注册的 Activity 关掉后，广播也就失效了。静态注册无需担忧广播接收器是否被关闭，只要设备是开启状态，广播接收器也是打开着的。也就是说哪怕 app 本身未启动，该 app 订阅的广播在触发时也会对它起作用。

7.为什么 Intent 不能传递大数据

Intent 携带信息的大小其实是受 Binder 限制。数据以 Parcel 对象的形式存放在 Binder 传递缓存中。如果数据或返回值比传递 buffer 大，则此次传递调用失败并抛出 TransactionTooLargeException 异常。

Binder 传递缓存有一个限定大小，通常是 1Mb。但同一个进程中所有的传输共享缓存空间。多个地方在进行传输时，即时它们各自传输的数据不超出大小限制，TransactionTooLargeException 异常也可能被抛出。在使用 Intent 传递数据时，1Mb 并不是安全上限。因为 Binder 中可能正在处理其它的传输工作。不同的机型和系统版本，这个上限值也可能不同。

第二章深入解析 Handler 消息机制

第一节 宏观理论分析与 Message 源码分析

预热

在写这篇文章前我不止一次的问自己,网上分析 Handler 机制原理的文章那么多,为啥还要画蛇添足啊?不是说前人们写的文章不好,我就是觉得他们写的不细,有些点不讲清楚,逻辑很难通顺的,每次我学个什么东西时遇到这种情况都贼难受。

我们处在这么一个被层层封装的世界里搞开发,被真相所蒙蔽本来就是一件很痛苦的事,如果分享知识的人再搞出了一堆天书般的“经验总结”,这得多打击求学者的的心啊!

此篇文章为我【Android Framework 层分析】系列的第一篇。说了这么一大堆就当做个序吧。作为在探索技术之路中饱经风霜中一员,希望我能谨记前人的经验教训,多走点心,思路清晰的分析,尽量写高质量的文章,既对得起自己也对得起他人。

本文所分析的内容大概有以下几个模块:

- 开发人员最初设计 Handler 时想要解决什么问题
- Handler 为我们提供了哪些功能以及如何使用
- Handler 实现原理的理论分析
- Handler 实现原理的源码分析
- Android UI 线程中 Handler 的特殊操作

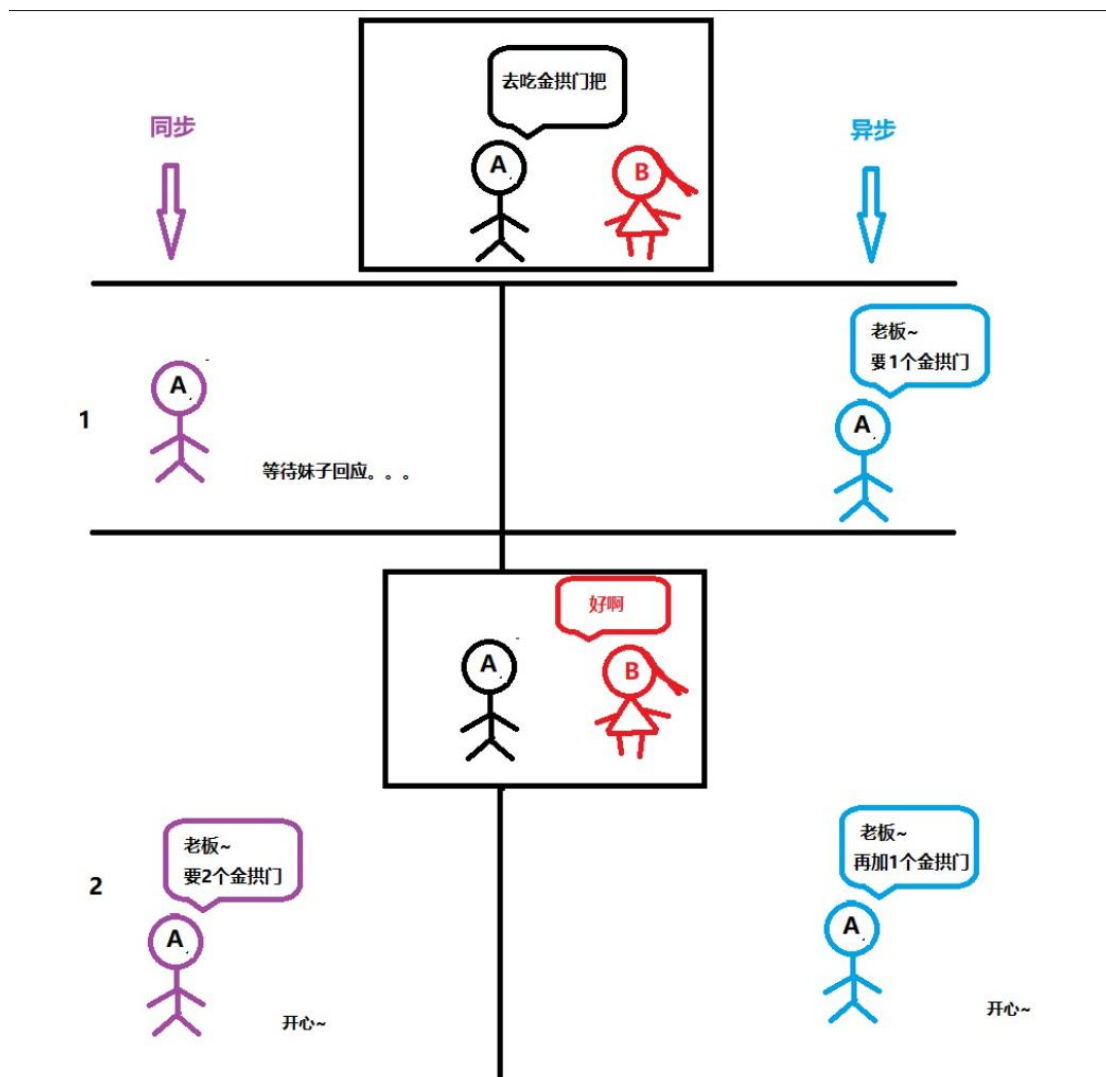
文章很长,但思路是循序渐进的,如果你能坚持读完我相信肯定不会让你失望,只要跟着文章的思路走,就不会有需要反复读好几遍加深理解的地方。建议刚读的时候先快速浏览一遍,在脑海中宏观的理清逻辑,有想不通的细节再仔细研究,如有不明白的地方或觉得我分析有误的地方欢迎留言评论。

设计 Handler 的初衷

在分析 Handler 之前,需要先搞清楚两个概念:

- 同步与异步的区别
- 线程与多线程的概念

讲道理上述两点中每个拿出来都涉及到很多东西，一方面我学的也不是很深不好意思献丑另一方面这不是本文重点，所以我就当老铁们都知道啦（嘿嘿一笑）。



同步与异步 - 金拱门篇

Java 多线程通信

Java 中有很多种方法实现线程之间相互通信访问数据，大概先简单的介绍两个典型的，就不上代码了。

通过 `synchronized` 关键字以“上锁”机制实现线程间的通信。多个线程持有同一个对象，他们可以访问同一个共享变量，利用 `synchronized`“上锁”机制，哪个线程拿到了锁，它就可以对共享变量进行修改，从而实现了通信。

使用 `Object` 类的 `wait/notify` 机制，执行代码 `obj.wait()`；后这个对象 `obj` 所在的线程进入阻塞状态，直到其他线程调用了 `obj.notify()`；方法后线程才会被唤醒。

Android 多线程的特殊性

在上面的两个 `Java` 多线程通信的方法中都有一个共同的特点，那就是线程的**阻塞**。利用 `synchronized` 机制拿不到锁的线程需要等拿到锁了才会继续执行操作，`obj.wait()`；需要等 `obj.notify()`；才会继续执行操作。

虽然 `Android` 系统是由 `Java` 封装的，但是由于 `Android` 系统的特殊性，`Google` 的开发人员对 `Android` 线程的设计进行了改造。他们把启动 `APP` 时运行的主线程定义为 **UI 线程**。

`UI` 线程负责所有你能想到的所有的跟界面相关的操作，例如分发绘制事件，分发交互事件等可多了。由于其特殊性 `Android` 系统强制要求以下两点：

为保持用户界面流畅 `UI` 线程不能被阻塞，如果线程阻塞界面会卡死，若干秒后 `Android` 系统抛出 `ANR`。

除 `UI` 线程外其他线程不可执行 `UI` 操作。

（此处只是简单介绍一下 `UI` 线程，后面会有专门一节分析 `Android UI` 线程。）

Android 多线程通信

既然 `UI` 线程中不能被阻塞，那么查询数据库和访问网络这类的耗时操作肯定就不能在 `UI` 线程中执行了，我们就需要单独开个线程操作。

但是除 `UI` 线程外其他线程又不可执行 `UI` 操作，最后还是要回到 `UI` 线程更新 `UI`，这就需要多线程之间的通信。

可 `Java` 中线程间通信又都是阻塞式方法，所以传统的 `Java` 多线程通信方式在 `Android` 中并不适用。

为此 `Google` 开发人员就不得不设计一套 `UI` 线程与 `Worker` 线程通信的方法。既能实现多线程之间的通信，又能完美解决 `UI` 线程不能被阻塞的问题。具体方法有以下几类：

`view.post(Runnable action)` 系列,通过 `View` 对象引用切换回 `UI` 线程。

`activity.runOnUiThread(Runnable action)`，通过 `Activity` 对象引用切换回 `UI` 线程。

AsyncTask，内部封装了 UI 线程与 Worker 线程切换的操作。

Handler，本文的主角，异步消息处理机制，多线程通信。

小结

说到这里应该大概明白了当初设计 Handler 的初衷。

由于 Android 系统的特殊性创造了 UI 线程。

由于 UI 线程的特殊性创造了若干个 UI 线程与 Worker 线程通信的方法。

在这若干个线程通信方法中就包含了 Handler。

Handler 就是针对 Android 系统中与 UI 线程通信而专门设计的多线程通信机制。

Handler 提供的一些方法

Handler API 方法相对其他 Android API 方法来说算少了的，不过要是都拿出挨个介绍还是很多，所以这里给出官方的 API 文档地址：[不用搭梯子就能看的 Android 官网 - Handler API](#)。

在介绍 Handler 的消息处理前还有一件事，为了线程传递数据时方便处理，开发人员为 Handler 专门设计了一个传递消息的载体 **Message**，这样就能让传输数据比较规范化，它有两个十分重要且常用的属性：

- `int what`；开发者自定义的消息标识，我们可以根据它来区分不同的消息，例如 `switch(message.what)`。
- `Object obj`；开发者想要传递的数据，具体什么类型的都可以。

同样，此处就是简单介绍一下 **Message**，后面会详细分析的。

Handler 提供的方法有些我们是用不到的，能用到的方法大体分为发送消息，处理消息和切换线程三类。

发送消息类方法

1. sendEmptyMessage

```
boolean sendEmptyMessage (int what)
```

发送一个只有消息标识 `what` 的空消息。该方法适用于不需要传递具体消息只是单独的发通知时。

2. sendMessageAtTime

```
boolean sendMessageAtTime (int what, long uptimeMillis)
```

在具体指定的时间 `uptimeMillis` 发送一个只有消息标识 `what` 的空消息。
`uptimeMillis` 为系统开机到当前的时间（毫秒）。

3. sendMessageDelayed

```
boolean sendMessageDelayed (int what, long delayMillis)
```

在过了 `delayMillis` 毫秒之后发送一个只有消息标识 `what` 的空消息。

4. sendMessage

```
boolean sendMessage (Message msg)
```

发送一条消息。

5. sendMessageAtTime

```
boolean sendMessageAtTime (Message msg, long uptimeMillis)
```

在具体指定的时间 `uptimeMillis` 发送一条消息。`uptimeMillis` 为系统开机到当前的时间（毫秒）。

6. sendMessageDelayed

```
boolean sendMessageDelayed (Message msg, long delayMillis)
```

在过了 `delayMillis` 毫秒之后发送一条消息。

处理消息类方法

handleMessage

```
void handleMessage (Message msg)
```

负责接受消息,所有发送的消息都会返回该方法，注意！必须 `Override` 这个方法才能接收消息。

切换线程类方法

1. post

```
boolean post (Runnable r)
```

`Runnable r` 会运行在 `handler` 对象被创建的线程上。当我们在 UI 线程创建了 `Handler` 对象，在 `Worker` 线程调用 `handler.post()` 方法时，`Runnable` 就会运行在 UI 线程中。

2. postAtTime

```
boolean postAtTime (Runnable r, long uptimeMillis)
```

在具体指定的时间 `uptimeMillis` 让 `Runnable` 运行在 `Handler` 对象被创建的线程中。

3. postDelayed

```
boolean postDelayed(Runnable r, long delayMillis)
```

在具体指定的时间 `delayMillis` 之后让 `Runnable` 运行在 `Handler` 对象被创建的线程中。

使用 Handler

在上节方法介绍中出现了 `XXXAtTime(long uptimeMillis)` 和 `XXxDelayed(long delayMillis)` 这两类控制时间的方法，两类方法的时间参数虽然都是毫秒，但是代表的意义却不一样：

- `XXxDelayed(long delayMillis)` 中的时间参数是指从当前时间开始 `delayMillis` 毫秒后。
- `XXXAtTime(long uptimeMillis)` 中的时间参数是指从系统开机算起 `uptimeMillis` 毫秒后。

利用静态方法 `SystemClock.uptimeMillis()` 可以得到从系统开机到现在的毫秒数，所以，下面两个语句执行的时间是相等的：

- `XXxDelayed(1000);`
- `XXXAtTime(SystemClock.uptimeMillis() + 1000)`

知道了这些后就可以随意使用 `Handler` 了。下面是使用 `Handler` 的一个小 Demo，代码有点长但大部分都是注释，代码共分为三块：

1. 创建 `Handler`，实现处理消息逻辑
2. 定义了 `Worker` 线程，在 `Worker` 线程内部使用 `Handler` 提供的方法。
3. 在 `activity.onCreate(Bundle savedInstanceState)` 初始化控件，启动 `Worker` 线程。

```
public class HandlerActivity extends AppCompatActivity {
```

```
    TextView mTextView;
```

```
    // 实现 Handler
```

```
    Handler mHandler = new Handler() {
```

```
        @Override
```

```
        public void handleMessage(Message msg) {
```

```
            super.handleMessage(msg);
```

```
            switch (msg.what) {
```

```
                case 1:
```

```

        mTextView.setText("接收到一条从 Worker Thread 线程发来
空消息");
        break;
    case 2:
        mTextView.setText("接收到一条从 Worker Thread 线程发来
指定时间的空消息");
        break;
    case 3:
        mTextView.setText("接收到一条从 Worker Thread 线程发来
指定时间之后的空消息");
        break;
    case 4:
        String data = (String) msg.obj;
        mTextView.setText("接收到一条从 Worker Thread 线程发来
消息, 消息中包含数据:" + data);
        break;
    case 5:
        String data1 = (String) msg.obj;
        mTextView.setText("接收到一条从 Worker Thread 线程发来
指定时间的消息, 消息中包含数据:" + data1);
        break;
    case 6:
        String data2 = (String) msg.obj;
        mTextView.setText("接收到一条从 Worker Thread 线程发来
指定时间之后消息, 消息中包含数据:" + data2);
        break;
    }
}

};

// 定义 Worker 线程
class WorkerThread extends Thread{

    @Override
    public void run() {
        super.run();

        // 在 Worker Thread 向 UI Thread 发送一条只有 what 的空消息
        mHandler.sendMessage(1);

        // 在指定的时间 Worker Thread 向 UI Thread 发送一条只有 what
的空消息
        mHandler.sendMessageAtTime(2,
SystemClock.elapsedMillis() + 1000);

```

```
// 在指定的时间之后 Worker Thread 向 UI Thread 发送一条只有
what 的空消息
mHandler.sendMessageDelayed(3, 1000);

// 创建一个 Message
Message msg1 = new Message();
msg1.what = 4;
msg1.obj = "这是一个从 Worker Thread 发送的普通信息";
// 在 Worker Thread 向 UI Thread 发送一条消息
mHandler.sendMessage(msg1);

// 创建一个 Message
Message msg2 = new Message();
msg2.what = 5;
msg2.obj = "这是一个在指定的时间从 Worker Thread 发送的信息";
// 在指定的时间 Worker Thread 向 UI Thread 发送一条消息
mHandler.sendMessageAtTime(msg2, SystemClock.uptimeMillis()
+ 1000);

// 创建一个 Message
Message msg3 = new Message();
msg3.what = 6;
msg3.obj = "这是一个在指定的时间之后从 Worker Thread 发送的信
息";

// 在指定的时间之后 Worker Thread 向 UI Thread 发送一条消息
mHandler.sendMessageDelayed(msg3, 1000);

// post 方法可以让 Runnable 直接运行在 UI Thread 中
mHandler.post(new Runnable() {
    @Override
    public void run() {
        mTextView.setText("通过 post 方法，从 Worker Thread 回
到了 UI Thread");
    }
});

// postAtTime 方法可以让 Runnable 在指定的时间直接运行在 UI
Thread 中
mHandler.postAtTime(new Runnable() {
    @Override
    public void run() {
        mTextView.setText("通过 postAtTime 方法，从 Worker
Thread 回到了 UI Thread");
    }
});
```

```

    }
    }, SystemClock.uptimeMillis() + 1000);

    // postDelayed 方法可以让 Runnable 在指定的时间之后直接运行在
    UI Thread 中
    mHandler.postDelayed(new Runnable() {
        @Override
        public void run() {
            mTextView.setText("通过 postDelayed 方法，从 Worker
Thread 回到了 UI Thread");
        }
    }, 1000);

}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_handler);
    // 绑定控件
    mTextView = (TextView)findViewById(R.id.tv);
    // 启动 worker 线程
    new WorkerThread().start();
}
}

```

小结

到这里我们已经熟练的掌握了 **Handler** 提供的操作，是不是很简单啊？其实 **Handler** 中还有一些比较特别方法这里没有介绍，在后面看源码分析实现原理里会介绍。

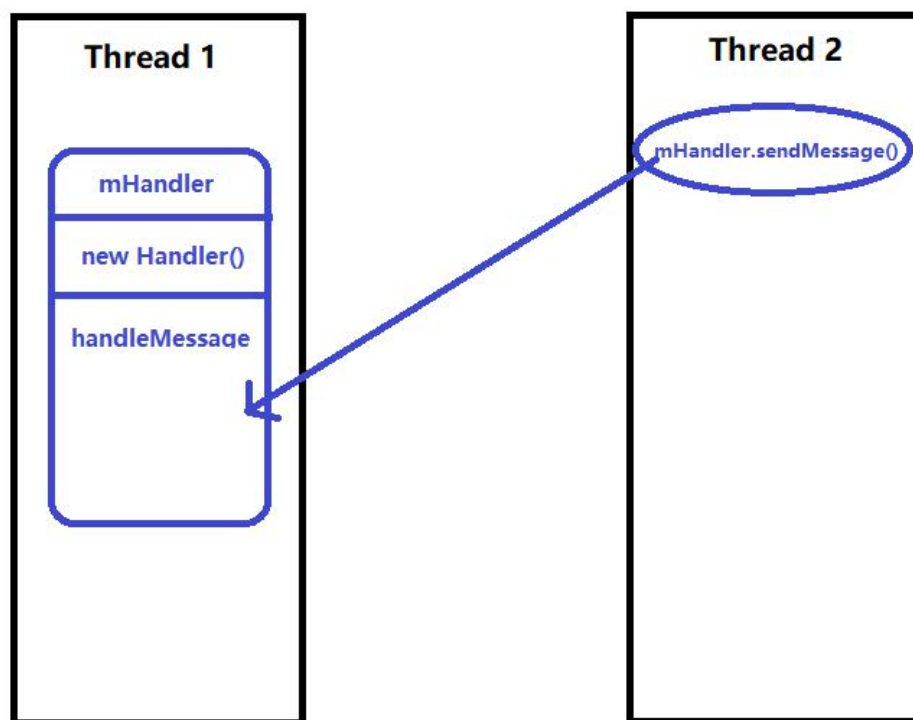
Message 的用法绝不是实例化出来赋值两个属性那么简单，本节只介绍了一点，同样在分析源码的时候会详细的介绍。

Handler 实现原理 - 理论分析

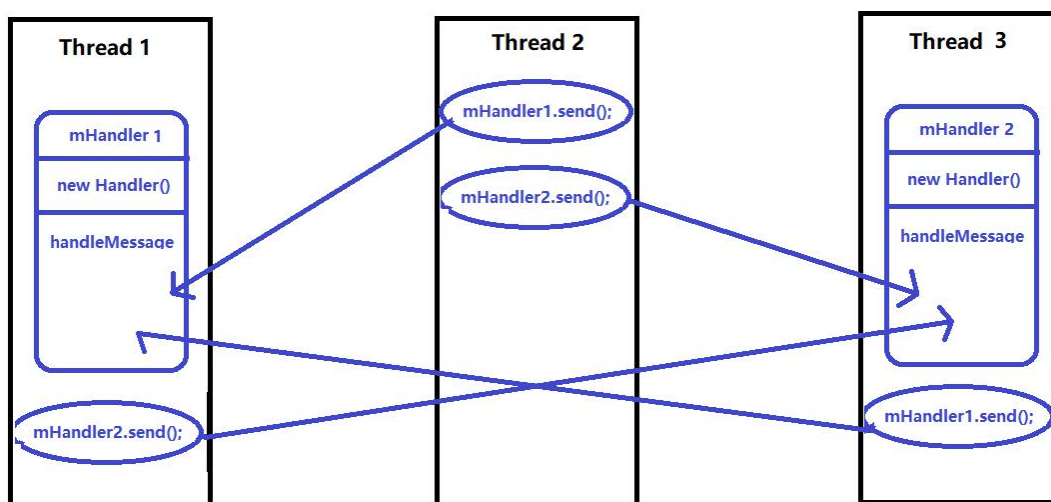
关于源码分析的文章我真是有一肚子吐槽的话想说，总的来说就是介绍源码的作者嗨的不行，而看文章的人一脸懵逼。吸取了前辈们的教训，这里分析源码前先来一波理论上的分析。

线程中接收消息端的特殊性

首先我们得知道理想状态下使用 **Handler** 是希望它被实例化在哪个线程，哪个线程就是消息的接收端，虽然在其他线程内发送消息时调用的同样是这个 **Handler** 的引用。这没错吧？



两个线程通信



多线程通信

根据上面的结论可以知道 **Handler** 接收消息端是线程独立的，不管 **handler** 的引用在哪个线程发送消息都会传回自己被实例化的那个线程中。

但显而易见的是 **Handler** 不可能是线程独立的，因为它的引用会在别的线程作为消息的发送端，也就是说它本身就是多线程共享的引用，不可能独立存在于某个线程内。

所以！**Handler** 需要一个独立存在于线程内部且私有使用的类帮助它接收消息！这个类就是 **Looper**！

Looper - 线程独立

通过上节分析我们已经知道设计 **Looper** 就是为了辅助 **Handler** 接收消息且仅独立于线程内部。那如何才能实现线程独立的呢？

好消息是 **Java** 早就考虑到了这一点，早在 **JDK 1.2** 的版本中就提供 **ThreadLocal** 这么一个工具类来帮助开发者实现线程独立。这里简单分析一下 **ThreadLocal** 的使用方法，不分析实现原理了。[Android 官网 - ThreadLocal API](#)

ThreadLocal 支持泛型，用于定义线程私有化变量的类型，实例化对象时可选 **Override** 一个初始化方法 **initialValue()**，这个方法的作用就是给你的引用变量赋初始值，如果没有 **Override** 这个方法那么默认你的引用变量就是 **null** 的：

```
//定义一个线程私有的 String 类型变量
private static final ThreadLocal<String> local = new
ThreadLocal<String>() {
```

```
// 设置引用变量的初始化值
@Override
protected String initialValue() {
    return super.initialValue();
}
};
```

定义好了 `ThreadLocal` 之后还需要了解三个方法：

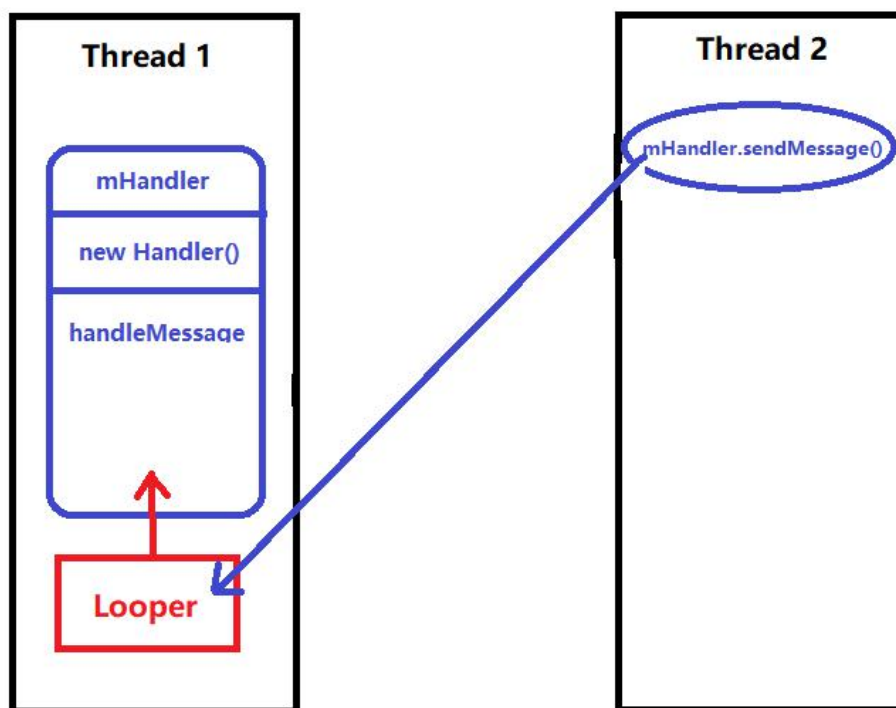
- `get()` 得到你的本地线程引用变量。
- `set(T value)` 为你的本地线程引用变量赋值。
- `remove()` 删除本地线程引用变量。

是不是很简单呢？有了 `ThreadLocal` 之后我们只需要把 `Looper` 存进去就能实现线程独立了。

```
private static final ThreadLocal<Looper> mLooper = new
ThreadLocal<Looper>();
```

到这里再梳理一下流程：

1. `Handler` 引用可以多线程间共享。
2. 当 `Handler` 对象在其他线程发送消息时，通过 `Handler` 的引用找到它所在线程的 `Looper` 接收消息。
3. `Looper` 负责接收消息再分发给 `Handler` 的接收消息方法。



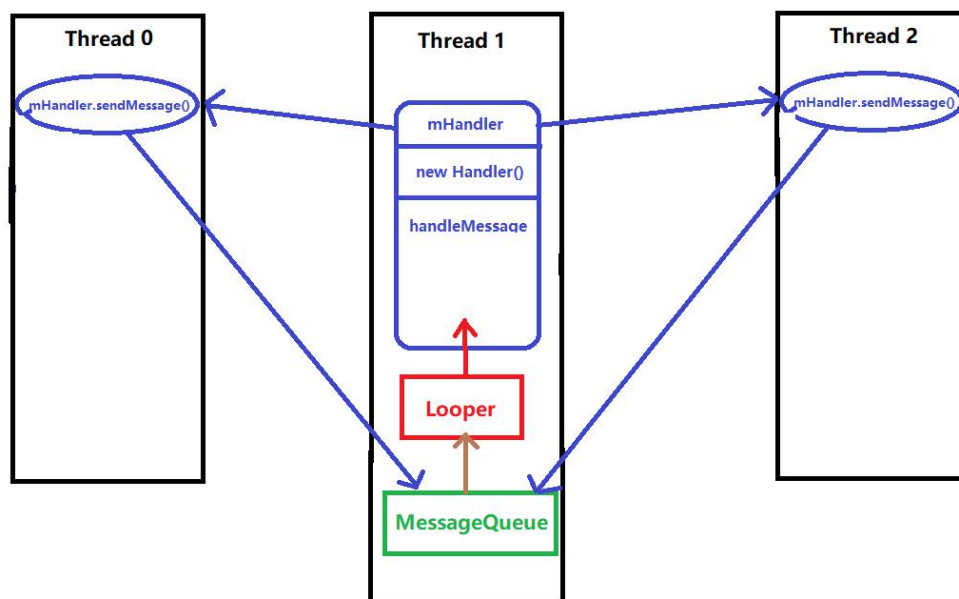
Looper

但是！这样还会有一个问题，如果多个线程同时使用一个 Handler 发消息，Looper 该怎么办？给接收消息的方法上锁吗？显然不能这样做啊！于是就设计了 **MessageQueue** 来解决这个问题。

MessageQueue - 多线程同时发消息

为了防止多个线程同时发送消息 Looper 一下着忙不过来，于是设计一个 **MessageQueue** 类以队列的方式保存着待发送的消息，这样 Looper 就可以一个个的有序的从 **MessageQueue** 中取出消息处理了。

既然 **MessageQueue** 是为 Looper 服务的，而 Looper 又是线程独立的，所以 **MessageQueue** 也是线程独立的。



MessageQueue

小结

现在我们已经知道为了完成异步消息功能需要有 Handler 家族的四位成员共同合作：

- **Handler:** 负责发送消息，为开发者提供发送消息与接收消息的方法。
- **Message:** 消息载体，负责保存消息具体的数据。
- **MessageQueue:** 消息队列，以队列形式保存着所有待处理的消息。
- **Looper:** 消息接受端，负责不断从 MessageQueue 中取出消息分发给 Handler 接受消息端。

这四位成员哪个都不是平白无故出现的。因为要规范化消息传递格式而定义了 Message；为了实现消息接收端只存在线程内部私有化使用而定义了 Looper；为了解决多线程同时发送数据 Looper 分发消息处理时会产生问题而设计 MessageQueue 队列化消息。

到这里你应该知道了 Handler 家族四位成员各自负责的是什么工作，以及他们自身的特点特殊性，比如 Handler 是线程间共享的而 Looper 是线程独立的，MessageQueue 跟 Looper 又是一对一的。

接下来我们就可以开始读源码了！

Message 源码分析

本文出现的源码版本均为 Android 7.1.1(Nougat) - API 25 版本。

Message 作为消息传递的载体，源码主要分为以下几个部分：

1. 操作数据相关，类似 `getter()` 和 `setter()` 这种方法还有之前提到过的 `what` 和 `obj` 这类属性。
2. 创建与回收对象实例相关，除了用关键字 `new` 外，其他得到对象实例的方法。
3. 其他工具类性质的扩展方法。

Message 中的数据属性与方法

首先说一个本篇文章忽略的属性及相关方法：`public Messenger replyTo;。`

为什么要忽略过去呢？因为 `Messenger` 类是基于 `Message` 上实现**进程间通信**的类。注意，是进程间通信，不是线程间通信。一方面进程间通信不是本文分析的重点，另一方面进程间通信需要掌握 **AIDL** 方面的知识。

接下来就让我们看看 **Message** 源码有哪些可供我们使用的属性吧：

- `public int what;` 开发者可自定义的消息标识代码，用于区分不同的消息。
- `public int arg1;` 如果要传递的消息只有少量的 `integer` 型数据，可以使用这个属性。
- `public int arg2;` 同上面 `arg1`。
- `public Object obj;` 开发者可自定义类型的传输数据。

上面四个属性作为常用的消息传递的数据载体可直接赋值，例如 `msg.arg1 = 100;`。基本可以满足我们日常开发中简单消息传递。

如果上面几个数据属性不能满足我们的需求，可以使用**扩展数据：Bundle** 来传递（*Bundle 是啥应该都知道吧？*）：

```
Bundle data;

// 得到 Bundle 数据，如果 data 是空的就 new 一个
public Bundle getData() {
    if (data == null) {
        data = new Bundle();
    }
    return data;
}

// 得到 Bundle 数据，如果 data 是空的就返回 null
public Bundle peekData() {
    return data;
}

// 设置 Bundle 数据
```

```
public void setData(Bundle data) {  
    this.data = data;  
}
```

这段代码也没什么逻辑好分析的，值得一提就是 `Bundle data` 不是 `public` 的，所以我们不能直接操作这个属性，需要通过上面三个方法操作数据。使用 `Bundle` 数据也非常简单：

```
Bundle bundle = new Bundle();  
bundle.putString("String", "value");  
bundle.putFloat("float", 0.1f);  
  
Message msg = Message.obtain();  
msg.setData(bundle);
```

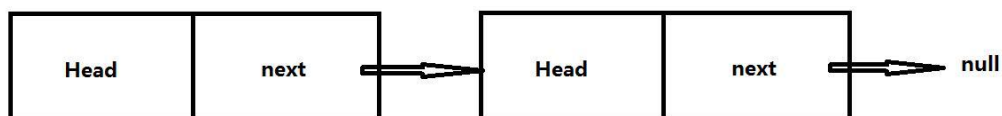
创建与回收 Message 对象的基本方法

先看一下源码中 `Message` 的构造方法：

```
/** Constructor (but the preferred way  
    to get a Message is to call {@link #obtain() Message.obtain()}).  
 */  
public Message() {  
}
```

没错，这货的构造方法里什么也没有，不过它的注释却告诉我们想要得到 `Message` 对象首选的方法应该是调用静态方法 `Message.obtain()`。那这个 `obtain()` 方法干了什么呢？其实就是内部维持了一个链表形式的 `Message` 对象缓存池，这样会节省重复实例化对象产生的开销成本。

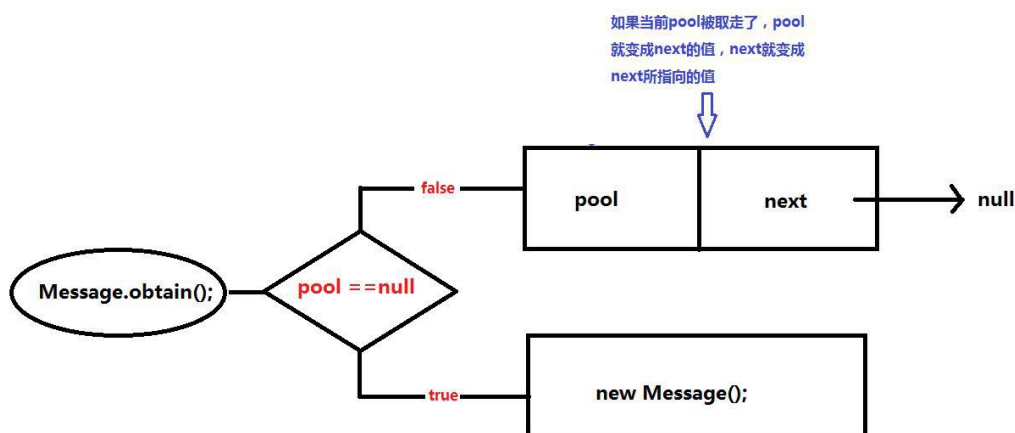
老样子还是理论分析一波，数据结构中的链表一个单元有两个值，当前单元的值（`head`）和下一个单元的地址指针（`next`），如果下一个单元不存在那么 `next` 就是 `null` 的。



链表结构

所以，想要实现 **Message** 对象链表式缓存池就需要额外的两个 **Message** 类型的引用 **head** 和 **next**，都说了叫缓存池，所以把 **head** 叫 **pool** 更合适一点。

有了链表的基础结构我们再想实例化对象的时候就可以先去链表缓存池中看看有没有，有的话直接从缓存池中拿出来用，没有再 **new** 一个。



从缓存池中取对象

由于代码多起来逻辑有些复杂，这样不太好分析，所以我在源码中加了许多自己的注释，下面代码看上去很长，其实把注释都去掉后并没有多少。

```
// 用于标识当前对象是否存在于缓存池，0 代表不在缓存池中
int flags;

/**
 * 这个常量是供上面的 flags 使用的，它表示 in use（正在使用）状态
 */
```


* 如果 Message 对象被存入了 MessageQueue 消息队列排队等待 Looper 处理或者被回收到缓存池中等待重复利用时，那么它就是 in use（正在使用）状态

*

* 只有在 new Message() 和 Message.obtain() 时候才可以清除掉 flags 上的 in use 状态

*

* 你不可以让一个 in use 状态的 Message 对象去传递消息。

*

* 1<< 0 还是 1，真不知道为啥要这么写，直接写等于 1 不就得了

*/

```
static final int FLAG_IN_USE = 1 << 0;
```

/** 静态常量对象，通过 synchronized (sPoolSync) 让它作为线程并发操作时的锁

* 确保同一时刻只有一个线程可以访问当前对象的引用

*/

```
private static final Object sPoolSync = new Object();
```

// 当前链表缓存池的入口，装载着缓存池中第一个可用的对象

```
private static Message sPool;
```

// 链表缓存池中指向下一个对象引用的 next 指针

```
Message next;
```

// 当前链表缓存池中对象的数量

```
private static int sPoolSize = 0;
```

/**

* 从缓存池中拿出来一个 Message 对象给你

* 可以让我们在许多情况下避免分配新对象。

*/

```
public static Message obtain() {
```

// 上锁，这期间只有一个线程可以执行这段代码

```
synchronized (sPoolSync) {
```

// pool 不等于空就说明缓存池中还有可用的对象，直接取出来

```
if (sPool != null) {
```

// 声明一个 Message 引用指向缓存池中的 pool 对象

```
Message m = sPool;
```

// 让缓存池中 pool 引用指向它的 next 引用的对象

```
sPool = m.next;
```

// 因为该对象已经从缓存池中被取出，所以将 next 指针置空

```
m.next = null;
```

// 将从缓存池中取出的对象的 flags 的 in use 标识清除掉

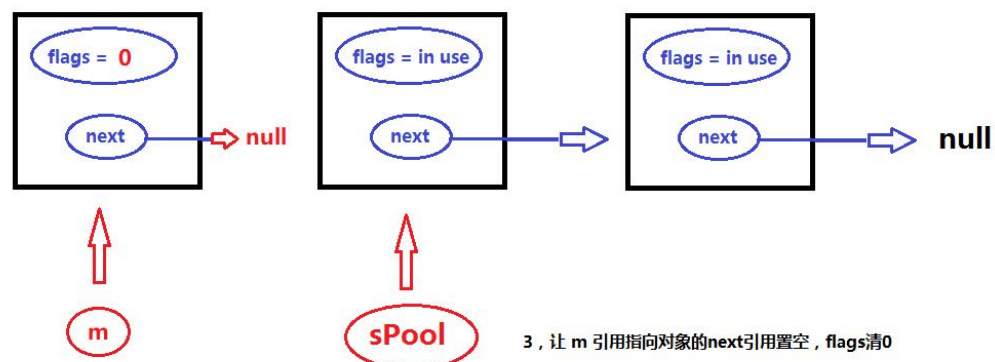
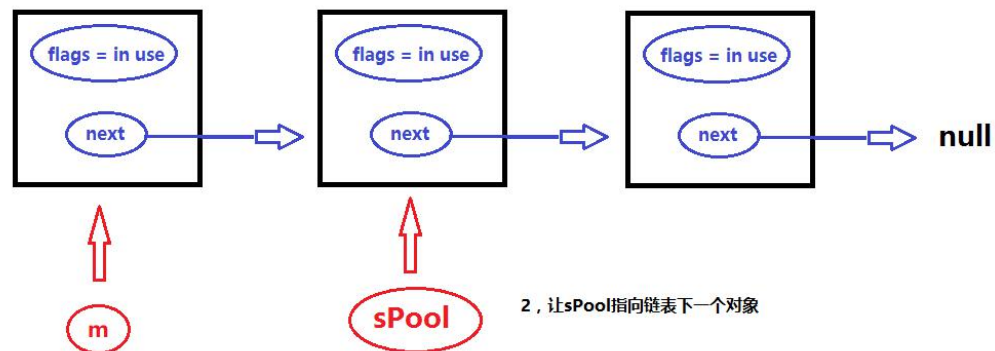
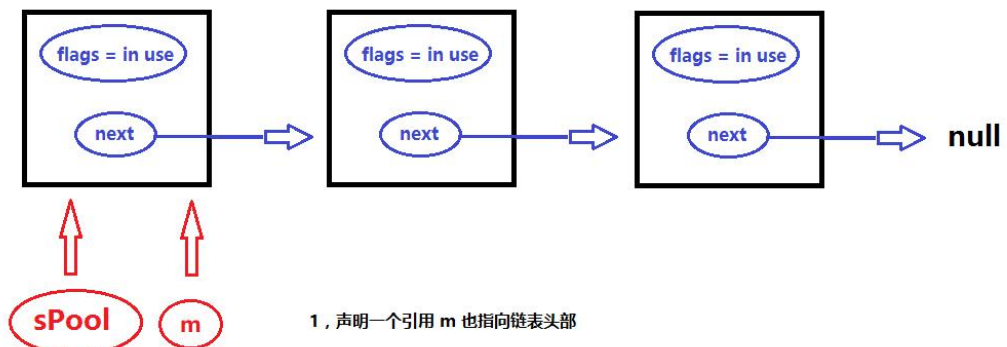
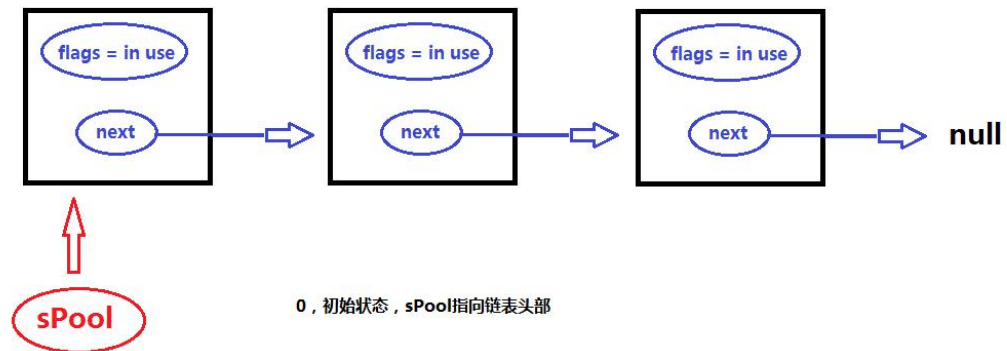
```
        m.flags = 0;
        // 缓存池中 Message 对象数量减去一个
        sPoolSize--;
        return m;
    }
}
// 如果缓存池中没有可用的对象就 new 一个吧
return new Message();
}
```

理论上我们希望 sPool 引用指向了链表缓存池中的第一个对象，让它作为整个缓存池的出入口。所以我们把它设置成 static 的，这样它就与实例化出来的对象无关，也就是说无论我们在哪个 Message 对象中进行操作，sPool 还是 sPool。

静态方法 obtain() 的代码逻辑流程：

先判断缓存池是不是空的：if(sPool != null),如果是空的就直接：return new Message();，不是空的就声明一个引用让它指向缓存池第一个对象：Message m = sPool;，而缓存池的链表头部 sPool 引用就指向了链表中下一个对象：sPool = m.next;，因为这个时候缓存池中第一个对象已经取出交给了引用 Message m，所以需要清除掉这个对象身上的特殊标识，包括缓存池中的 next 引用和用来标记对象状态的 flags 值：m.next = null; m.flags = 0;，最后将缓存池中的对象数量减一：sPoolSize--;

逻辑理清了整个流程就显得很简单了，再看看图解逻辑流程：



从缓存池中取出一个对象

分析到这里我们知道了为什么官方推荐我们使用 `Message.obtain()` 得到对象了, 因为它是在缓存池中取出来重复利用的, 但是通过上面代码也可以看到,

只有缓存池里有东西时也就是 `sPool != null` 的时候才可以取，**Message** 是怎么把对象回收到缓存池中的呢？

回收 **Message** 对象到缓存池的方法

阅读源码后发现有一个 `public void recycle()` 方法用于回收 **Message** 对象，但是它也牵扯出了一堆其他方法与属性：

```
// 缓存池最大存储值
private static final int MAX_POOL_SIZE = 50;

// 区分当前 Android 版本是否大于或者等于 LOLLIPOP 版本的全局静态变量，默认初始值为 true
private static boolean gCheckRecycle = true;

/**
 * 用于区分当前 Android 版本是否大于或者等于 LOLLIPOP 版本
 * 内部隐藏方法，在 APP 启动时就会执行该方法，开发者是不可见的
 * @hide
 */
public static void updateCheckRecycle(int targetSdkVersion) {
    if (targetSdkVersion < Build.VERSION_CODES.LOLLIPOP) {
        gCheckRecycle = false;
    }
}

/**
 * 判断当前对象的 flags 是否为 in-use 状态
 */
boolean isInUse() {
    return ((flags & FLAG_IN_USE) == FLAG_IN_USE);
}

/**
 * 调用这个方法后，当前对象就会被回收入缓存池中。
 * 你不能回收一个在 MessageQueue 排队等待处理或者正在交付给 Handler 处理的 Message 对象
 * 说白了就是 in-use 状态的不可回收
 */
public void recycle() {
    // 判断当前对象是否为 in-use 状态
    if (isInUse()) {
        // 如果当前版本大于或者等于 LOLLIPOP 则抛出异常
        if (gCheckRecycle) {
```

```

        throw new IllegalStateException("This message cannot be
recycled because it "
        + "is still in use.");
    }
    // 如果当前版本小于 LOLLIPOP 什么也不干直接结束方法
    return;
}
// 回收 Message 对象
recycleUnchecked();
}

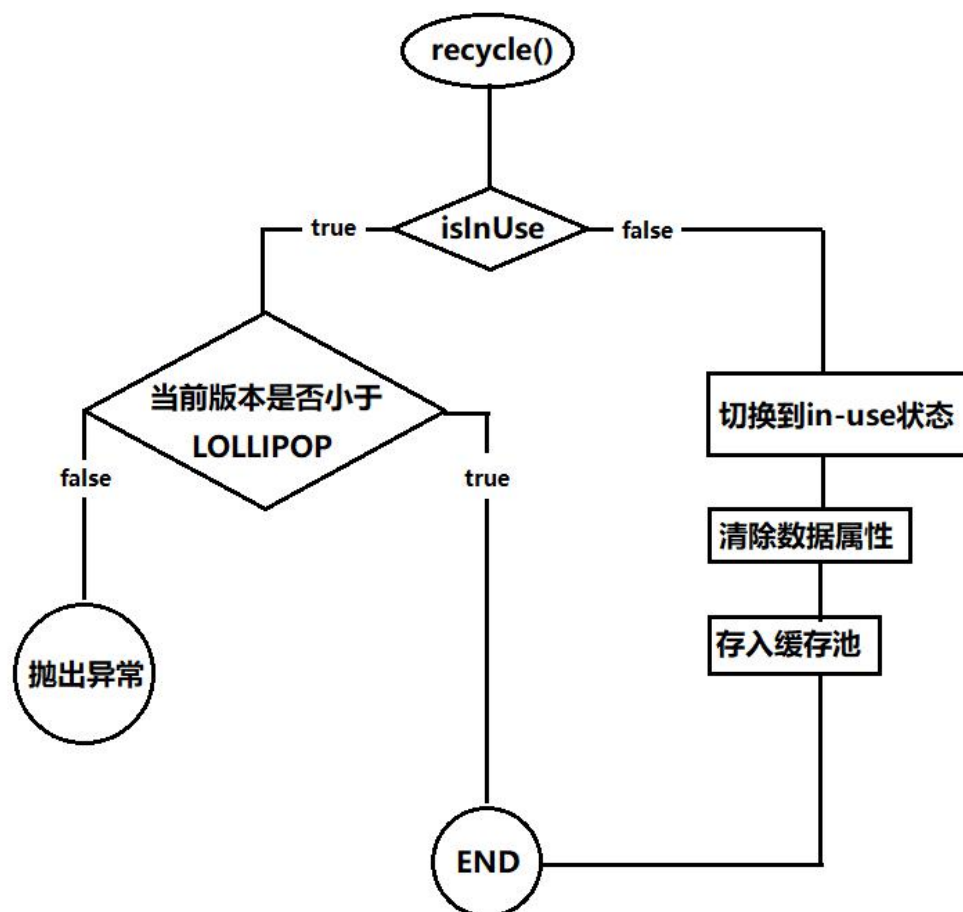
/**
 * 回收一个可能是 in use 状态的 Message 对象
 * 在 MessageQueue 和 Looper 内部处理排队 Message 时也会使用这个方法
 */
void recycleUnchecked() {
    // 将当前 Message 对象置为 in-use 状态
    flags = FLAG_IN_USE;

    // 清除当前 Message 对象的所有数据属性
    what = 0;
    arg1 = 0;
    arg2 = 0;
    obj = null;
    replyTo = null;
    sendingUid = -1;
    when = 0;
    target = null;
    callback = null;
    data = null;

    // 上锁
    synchronized (sPoolSync) {
        // 如果当前缓存池对象中的数量小于缓存池最大存储值（50）就存
        // 入缓存池中
        if (sPoolSize < MAX_POOL_SIZE) {
            // 存入缓存池
            next = sPool;
            sPool = this;
            // 缓存池数量加 1
            sPoolSize++;
        }
    }
}

```

上面代码的逻辑很清晰，执行 `recycle()` 方法后先判断当前对象是否为 `in-use` 状态：`if (isInUse())`，如果是 `in-use` 状态的话当前 Android 版本是 LOLLIPOP(5.0) 版本之前直接结束程序，LOLLIPOP 及之后版本抛出异常。如果当前对象不是 `in-use` 状态，那么就执行 `recycleUnchecked()` 方法先将它切换到 `in-use` 状态：`flags = FLAG_IN_USE;`，再把所有的数据属性全部清除，最后把对象存入缓存池链表中。



回收 Message 对象

为什么要区分 Android LOLLIPOP(5.0)前后版本？

源码刚开始就有两个用于区分 Android 版本的全局属性和方法：

- `private static boolean gCheckRecycle = true;`
- `public static void updateCheckRecycle(int targetSdkVersion)`

通过查看源码发现 `Message` 类在 LOLLIPOP 版本进行了一次更新也就是我们现在看到的源码，在 LOLLIPOP 版本之前虽然 `recycle()` 方法的注释上同样警告了我们不能回收 `in-use` 对象，但是如果你坚持让 `in-use` 状态的对象调用 `recycle()` 的话也会也会被回收：

```
/**
 * Android Lollipop 版本源码
 *
 * Return a Message instance to the global pool. You MUST NOT touch
 * the Message after calling this function -- it has effectively been
 * freed.
 */
public void recycle() {
    // 清除数据
    clearForRecycle();
    // 存入缓存池
    synchronized (sPoolSync) {
        if (sPoolSize < MAX_POOL_SIZE) {
            next = sPool;
            sPool = this;
            sPoolSize++;
        }
    }
}
```

所以在 Lollipop 版本的时候 Google 进行了改进，强制要求不可以回收 in-use 状态的对象否则抛出异常，但是为了兼容之前的版本，所以新增加了个内部私有的区分 Android 版本的方法。

我们需要手动回收吗

现在我们知道了通过执行 `recycle()` 方法回收 `Message` 对象，但是如果要为每个 `Message` 对象都进行手动回收岂不是很麻烦？

庆幸的是开发人员也想到了这一点，从源码中可以看到其实最终真正执行回收操作的调用 `recycleUnchecked()` 方法，且注释中告诉我们 `MessageQueue` 和 `Looper` 内部也会调用该方法执行回收。

这里先说一个结论，`MessageQueue` 和 `Looper` 内部分发处理消息时，当它们得知当前这个 `Message` 对象已经使用完毕后就会直接调用 `recycleUnchecked()` 方法将它回收掉，等分析到 `MessageQueue` 和 `Looper` 再具体讲这个地方。

所以，如果我们用实例化 `Message` 对象是放入 `Handler` 中去传消息的，那么我们就不需要手动回收，他们内部自己就回收了。如果我们使用的 `Message` 对象跟 `Handler`，`Looper`，`MessageQueue` 一点交互都没有，那我们就自己去回收。

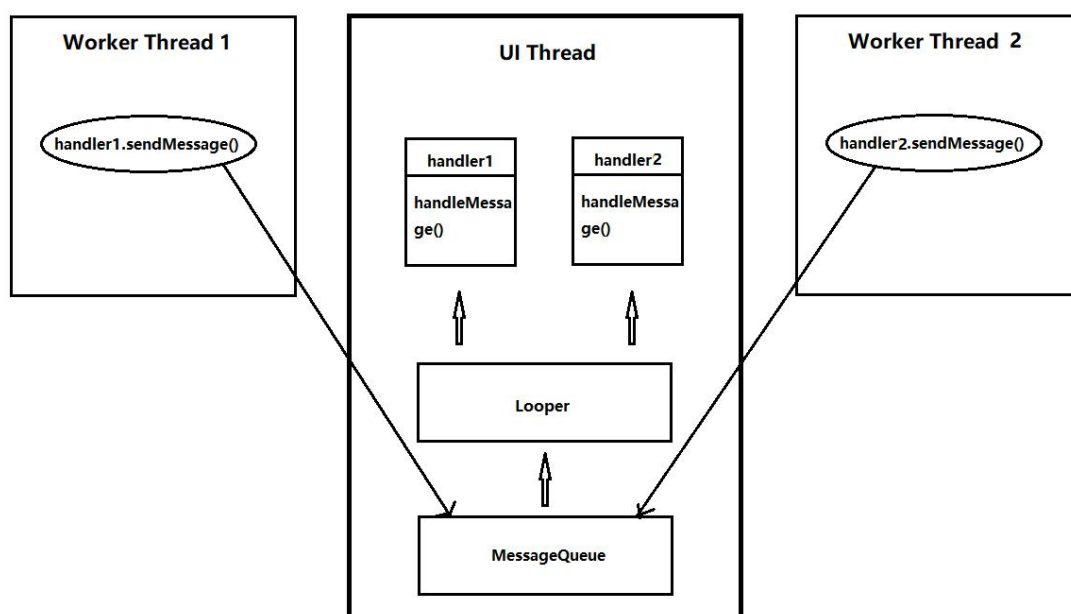
包含 Handler 参数的 obtain() 方法

给 `Message` 内部装了一个 `Handler` 起了什么作用呢？首先，我们可以通过上面讲解我们可以得出以下已知的结论：

1. 表面上看我们使用 **Handler** 发送消息后，消息直接传回到了 **Handler** 内部的 `handleMessage(Message msg)` 方法中。
2. 实际上是先把消息传入了 **MessageQueue** 中，**Looper** 再从 **MessageQueue** 依次取出消息分发给 **Handler**。
3. **Looper** 是线程独立的，**Looper** 和 **MessageQueue** 是一对一的。

但是，你有没有想过 **Looper** 和 **Handler** 是不是一对一的？答案当然是否定的，**MessageQueue** 只负责队列消息，**Looper** 只负责取出消息分发。他们的功能很明确而且通用。

所以，无论当前线程有多少个 **Handler**，同样都只有一个 **Looper** 和一个 **MessageQueue**。



一个线程存在多个 **Handler**

既然每个线程只有一个 **Looper** 和 **MessageQueue** 的话那么 **Looper** 分发消息的时候要如何判断当前这个 **Message** 是哪个 **Handler** 的呢？所以开发人员就给 **Message** 内部配置了一个 **Handler** 属性，这样 **Looper** 分发消息时直接调用 **Message** 内部的 **Handler** 属性就能找到它对应的 `handleMessage(Message msg)` 接收消息的方法了。

源码很简单，就是在空参方法 `obtain()` 基础上加了个 **Handler** 属性，还有它的 `getter()` 和 `setter()`：

```
Handler target;
```

```
public static Message obtain(Handler h) {
    Message m = obtain();
```

更多录播视频+架构学习资料免费领取 QQ 3607409029


```
m.target = h;
return m;
}

public void setTarget(Handler target) {
    this.target = target;
}

public Handler getTarget() {
    return target;
}
```

包含 Runnable 参数的 obtain()方法

跟上面类似，该方法就是在上面基础加了个 Runnable 参数，源码如下：

```
Runnable callback;

public static Message obtain(Handler h, Runnable callback) {
    Message m = obtain();
    m.target = h;
    m.callback = callback;
    return m;
}

public Runnable getCallback() {
    return callback;
}
```

这个 Runnable 的作用是：在 **Looper** 分发消息时如果 Runnable callback 不是空的，那么就不调用 **Handler** 的 `handleMessage(Message msg)` 方法，直接运行这个 Runnable callback。注意，这里的运行是已经回到了 **Handler** 被创建的线程上，也就是说 Runnable 会运行在 **Handler** 被创建的线程上。

更多包含参数的 obtain()方法

下面这些带参的 obtain() 方法我相信不用介绍大家也都能看的懂：

```
public static Message obtain(Handler h, int what) {
    Message m = obtain();
    m.target = h;
    m.what = what;
    return m;
}
```

```
public static Message obtain(Handler h, int what, Object obj) {
    Message m = obtain();
    m.target = h;
    m.what = what;
    m.obj = obj;
    return m;
}

public static Message obtain(Handler h, int what, int arg1, int arg2)
{
    Message m = obtain();
    m.target = h;
    m.what = what;
    m.arg1 = arg1;
    m.arg2 = arg2;
    return m;
}

public static Message obtain(Handler h, int what,
    int arg1, int arg2, Object obj) {
    Message m = obtain();
    m.target = h;
    m.what = what;
    m.arg1 = arg1;
    m.arg2 = arg2;
    m.obj = obj;
    return m;
}
```

扩展方法

特殊属性 long when;

```
long when;

public long getWhen() {
    return when;
}
```

既然这个属性的名字都叫 when 了那肯定就是跟时间有关了。还记得 Handler 给我们提供的方法中有几个可以控制时间的方法吗？例如 `XXXAtTime()` 和 `XXxDelayed()`，when 这个属性就是就用存储当前这个 Message 应该被处理的时间。当我们讲 Handler 和 MessageQueue 时会在提到它。

序列化对象

Message 支持对象的序列化，就是可以把对象转为字节形式，可以保存到本地也可以用于网络传输。如果不了解这方面的知识建议先查阅相关文章。

为了实现对象序列化，我们需要实现 Parcelable 接口，实例化 Parcelable.Creator 接口，并重写 describeContents() 和 writeToParcel() 方法。先看源码，再讲他们都是干啥的。

```
// 实现 Parcelable 接口 public final class Message implements Parcelable
{

    ...

    // 实例化 Parcelable.Creator 接口，完成 Parcel 对象转 Message 对象的操作
    public static final Parcelable.Creator<Message> CREATOR
        = new Parcelable.Creator<Message>() {

        public Message createFromParcel(Parcel source) {
            Message msg = Message.obtain();
            msg.readFromParcel(source);
            return msg;
        }

        public Message[] newArray(int size) {
            return new Message[size];
        }
    };

    // 序列化对象时的特殊种类对象描述，这里开发人员没有修改，就是默认的 0
    public int describeContents() {
        return 0;
    }

    // 重写 Parcelable 接口的 writeToParcel 方法，将 Message 对象转为 Parcel 对象，
    public void writeToParcel(Parcel dest, int flags) {

        // 以下代码均是将 Message 对象中的属性写入 Parcel 对象中

        if (callback != null) {
            throw new RuntimeException(
                "Can't marshal callbacks across processes.");
        }
    }
}
```

更多录播视频+架构学习资料免费领取 QQ 3607409029

```
}
dest.writeInt(what);
dest.writeInt(arg1);
dest.writeInt(arg2);
if (obj != null) {
    try {
        Parcelable p = (Parcelable)obj;
        dest.writeInt(1);
        dest.writeParcelable(p, flags);
    } catch (ClassCastException e) {
        throw new RuntimeException(
            "Can't marshal non-Parcelable objects across
processes.");
    }
} else {
    dest.writeInt(0);
}
dest.writeLong(when);
dest.writeBundle(data);
Messenger.writeMessengerOrNullToParcel(replyTo, dest);
dest.writeInt(sendingUid);
}

// 从 Parcel 对象中读取数据转为当前对象的属性
private void readFromParcel(Parcel source) {
    what = source.readInt();
    arg1 = source.readInt();
    arg2 = source.readInt();
    if (source.readInt() != 0) {
        obj = source.readParcelable(getClass().getClassLoader());
    }
    when = source.readLong();
    data = source.readBundle();
    replyTo = Messenger.readMessengerOrNullFromParcel(source);
    sendingUid = source.readInt();
}
}
```

代码看上去很长,其实很简单,就是实现了 Parcelable 接口,接着重写将 Message 对象转为 Parcel 对象的方法 writeToParcel(), 再重写了接口 Parcelable.Creator<T>完成 Parcel 对象转 Message 对象的方法。

这里用到的主要都是序列化对象 Parcelable 接口相关的知识。

设置 Message 是异步传输还是同步传输

更多录播视频+架构学习资料免费领取 QQ 3607409029

正常情况下，我们的消息其实是同步处理的，为什么这么说呢？

Looper 的工作就是把消息队列 **MessageQueue** 中的消息依次取出然后分发，每个消息传输都是有时间顺序的，这个动作都是可控制的。

然而，将消息设置成异步传输后那么 **Message** 对象将不再受 **Looper** 的控制，传输的顺序可能会被打断，不一定哪个消息先传过来。

所以，请谨慎使用异步传输。

```
// 该常量代表为异步传输方式
static final int FLAG_ASYNCHRONOUS = 1 << 1;

// 判断是否为异步传输
public boolean isAsynchronous() {
    return (flags & FLAG_ASYNCHRONOUS) != 0;
}

// 设置当前对象是否为异步传输
public void setAsynchronous(boolean async) {
    if (async) {
        flags |= FLAG_ASYNCHRONOUS;
    } else {
        flags &= ~FLAG_ASYNCHRONOUS;
    }
}
```

值得一提的是，用于标记是否为异步传输的标识跟用于判断是否为 **in-use** 状态的标识是共用的一个属性 **flags**。

小结

到这里整个 **Message** 源码就已经分析的差不多了，如果你有认真看到这个相信你打开电脑中的 **Message** 源码看起来将对其中的代码了如指掌（除了本文没介绍的 **Messenger** 相关代码）。

在简单的总结一下 **Message** 源码，他们大致分为三类：

利用链表式缓存池避免操作对象的相关方法：

- **obtain()** 方法
- 其他包含参数的 **obtain()** 方法
- 手动回收对象到缓存池的 **recycle()** 方法

- `Looper` 与 `MessageQueue` 内部也会使用的回收对象到缓存池的 `recycleUnchecked()` 方法

保存数据的相关属性及方法

- 消息标识 `int what`
- 简单整数型属性 `int arg1` 和 `int arg2`
- 自定义类型数据 `Object obj`
- 扩展数据 `Bundle data`

其他功能相关属性及方法

- 控制消息被处理的时间属性 `int when`
- 序列化对象相关方法
- 设置异步传输方法相关方法

第二节 MessageQueue 的源码分析

看源码有一段时间了，越来越能从代码中感觉到工程师们满满的激情，无论是基础 Java 语法还是高级的语言特性都被发挥的淋漓尽致，写的恰到好处。分析源码的过程，何尝不是与大神们进行灵魂沟的过程。

`MessageQueue` 属于低层类且依附于 `Looper`，`Looper` 外其他类不应该单独创建它，如果想使用 `MessageQueue` 可以从 `Looper` 类中得到它。

消息队列存储原理

再上一章 `Message` 源码分析中我们知道了 `Message` 内部维持了一个链表缓存池来避免重复创建 `Message` 对象造成的额外消耗，以静态属性 `private static Message sPool` 作为缓存池链表头，`Message next;` 作为链表的 `next` 指针。

有意思的是 `Message` 对象中 `next` 指针的不止用于链表缓存池，在 `MessageQueue` 中也采用同样的方法存储消息对象：

```
public final class MessageQueue {  
    .....  
    Message mMessages;  
    .....  
}
```

上面代码中的 `mMessages` 就是 `MessageQueue` 用来维持消息队列的链表头，至于它是如何存储的，后面再说。

使用 JNI 实现的 native 方法

`MessageQueue` 的源码调用了多个的 C/C++ 方法，这类方法使用前都会用关键字 `native` 声明一下。

这些方法所属的底层 C++ 代码创建了属于 `native` 层自己的 `NativeMessageQueue` 和 `NativeLooper` 消息模型。它们对 Java 层作用其实就是控制线程是否阻塞。

当我们想要从 `MessageQueue` 中取出消息时，碰巧队列是空的或即将取出的消息还没到被处理时间，那么我们就需要将线程阻塞掉等待队列中有消息时再取出。

下面就是 `MessageQueue` 中的 `native` 方法：

```
// 初始化
private native static long nativeInit();
// 注销
private native static void nativeDestroy(long ptr);
// 让线程阻塞 timeoutMillis 毫秒
private native void nativePollOnce(long ptr, int timeoutMillis);
// 立刻唤醒线程
private native static void nativeWake(long ptr);
// 线程是否处于阻塞状态
private native static boolean nativeIsPolling(long ptr);
// 设置文件描述符
private native static void nativeSetFileDescriptorEvents(long ptr,
int fd, int events);
```

文件描述符与 `epoll` 指令

消息队列里控制线程阻塞状态的 `native` 代码本质是用 Linux 指令 `epoll` 完成的，在这之前需要先了解一点，Linux 内核依靠“文件描述符（file descriptor）”来完成所有的文件读写访问操作，它更像是一个文件的索引值。而我们用到的 `epoll` 指令就是用来监听文件描述符是否有可 I/O 操作的。（这都是一些 Linux 相关知识）

创建与销毁

上面说了 `MessageQueue` 是依附于 `Looper` 的，所以本节分析的创建与销毁方法其实都是给 `Looper` 调用的，`MessageQueue` 只提供了一个带参的构造方法来创建对象：

```
// 当前 MessageQueue 是否可以退出
private final boolean mQuitAllowed;

// native 层中 NativeMessageQueue 队列指针的地址
// mPtr 等于 0 时表示退出队列
private long mPtr;

// native 层代码，创建 native 层中 NativeMessageQueue
private native static long nativeInit();

// 构造方法
MessageQueue(boolean quitAllowed) {
    mQuitAllowed = quitAllowed;
    // 执行 native 层方法
    mPtr = nativeInit();
}
```

构造方法中 `boolean quitAllowed` 参数的意思是当前这个 `MessageQueue` 是否可以手动退出，为什么要控制能否手动退出呢？这里先说一个结论：**Android 系统中要求 UI 线程不可手动退出，而其他 Worker 线程则全部都是可以的。**（具体的操作在 *Looper* 和 *UI 线程* 中）

那么退出是什么意思呢？

退出就是当前这个 `MessageQueue` 停止服务，将队列中已存在的所有消息全部清空，看看源码中退出方法都做了什么：

```
// 是否已经退出了
private boolean mQuitting;

// native 方法，退出队列
private native static void nativeDestroy(long ptr);

// 退出队列
void quit(boolean safe) {

    // 如果不是可以手动退出的，抛出异常
    if (!mQuitAllowed) {
        throw new IllegalStateException("Main thread not allowed to
quit.");
    }

    synchronized (this) {
        // 如果已经退出了直接结束方法
```



```

    if (mQuitting) {
        return;
    }
    // 标记为已退出状态
    mQuitting = true;

    // 两种清除队列中消息的方法
    if (safe) {
        removeAllFutureMessagesLocked();
    } else {
        removeAllMessagesLocked();
    }

    // 注销
    nativeWake(mPtr);
}
}

```

方法 `quit(boolean safe)` 中的参数 `safe` 决定了到底执行哪种清除消息的方法：

- `removeAllMessagesLocked()`，简单暴力直接清除掉队列中所有的消息。
- `removeAllFutureMessagesLocked()`，清除掉可能还没有被处理的消息。

`removeAllMessagesLocked()` 方法的逻辑很简单，从队列头中取消息，有一个算一个，全部拿出来回收掉。：

```

private void removeAllMessagesLocked() {
    Message p = mMessages;
    while (p != null) {
        Message n = p.next;
        p.recycleUnchecked();
        p = n;
    }
    mMessages = null;
}

```

然而，`removeAllFutureMessagesLocked()` 方法的逻辑稍微多一点：

```

private void removeAllFutureMessagesLocked() {

    // 获取当前系统时间
    final long now = SystemClock.uptimeMillis();
    Message p = mMessages;
    if (p != null) {

```

```
// 判断当前消息对象的预处理时间是否晚于当前时间
if (p.when > now) {
    // 如果当前消息对象的预处理时间晚于当前时间直接全部暴力
```

清除

```
        removeAllMessagesLocked();
    } else {
        Message n;
        // 如果当前消息对象的预处理时间并不晚于当前时间
        // 说明有可能这个消息正在被分发处理
        // 那么就跳过这个消息往后找晚于当前时间的消息
        for (;;) {
            n = p.next;
            if (n == null) {
                return;
            }
            if (n.when > now) {
                // 如果找到了晚于当前时间的消息结束循环
                break;
            }
            p = n;
        }
        p.next = null;
        do {
            // n 就是那个晚于当前时间的消息
            // 从 n 开始之后的消息全部回收
            p = n;
            n = p.next;
            p.recycleUnchecked();
        } while (n != null);
    }
}
```

这么看来这个方法名字起的还挺靠谱的，很好的解释了是要删除还没有被处理的消息。

消息入队管理 enqueueMessage() 方法

```
// 消息入队
// 参数 when 就是此消息应该被处理的时间
boolean enqueueMessage(Message msg, long when) {
    // 如果此消息的 target 也就是宿主 handler 是空的抛异常
    if (msg.target == null) {
```

更多录播视频+架构学习资料免费领取 QQ 3607409029

```
throw new IllegalArgumentException("Message must have a
target.");
}

// 如果此消息是 in-use 状态抛异常，in-use 的消息不可拿来使用
if (msg.isInUse()) {
    throw new IllegalStateException(msg + " This message is already
in use.");
}

//上锁
synchronized (this) {

    // 如果当前 MessageQueue 已经退出了抛异常并释放掉此消息
    if (mQuitting) {
        IllegalStateException e = new IllegalStateException(
            msg.target + " sending message to a Handler on a dead
thread");
        Log.w(TAG, e.getMessage(), e);
        msg.recycle();
        return false;
    }

    // 将消息标记为 in-use 状态
    msg.markInUse();
    // 设置应该被处理的时间
    msg.when = when;
    // 拿到队列头
    Message p = mMessages;

    // 是否需要唤醒线程
    boolean needWake;

    // p 等于空说明队列是空的
    // when 等于 0 表示强制把此消息插入队列头部，最先处理
    // when 小于队列头的 when 说明此消息应该被处理的时间比队列中
    第一个要处理的时间还早
    // 以上情况满足任意一种直接将消息插入队列头部
    if (p == null || when == 0 || when < p.when) {
        // 将消息插入队列头部
        msg.next = p;
        mMessages = msg;
        needWake = mBlocked;
    }
}
```

```
} else {

    // 线程已经被阻塞&&消息存在宿主 Handler&&消息是异步的
    needWake = mBlocked && p.target == null &&
msg.isAsynchronous();

    //如果上述条件都不满足就要按照消息应该被处理的时间插入
队列中
    Message prev;
    for (;;) {
        // 两根相邻的引用一前一后从队列头开始依次向后移动
        prev = p;
        p = p.next;
        // 如果队列到尾部了或者找到了处理时间早于自身的消息
就结束循环
        if (p == null || when < p.when) {
            break;
        }

        // 如果入队的消息是异步的而排在它前面的消息有异步的
就不需要唤醒
        if (needWake && p.isAsynchronous()) {
            needWake = false;
        }
    }
    // 将新消息插在这一前一后两个引用中间，完成入队
    msg.next = p; // invariant: p == prev.next
    prev.next = msg;
}

// 判断是否需要唤醒线程
if (needWake) {
    nativeWake(mPtr);
}
}
return true;
}
```

总结一下消息入队的逻辑大致分为如下几步：

检查消息合法性，包括宿主 `target` 是否为空，是否为 **in-use** 状态，队列是否还存活。

如果满足条件【队列为空、when 等于 0、此消息应被处理的时间比队列中第一个要处理的时间还早】中的任意一个直接将此消息插在队列头部最先被处理。

如果以上三个条件均不满足，那么就从头遍历队列根据被处理时间找到它的位置。

同步消息拦截器

除了 `enqueueMessage()` 方法可以向队列中添加消息外，还有一个 `postSyncBarrier()` 方法也可以向队列添加消息，但它不是添加普通的消息，我们将它添加的特殊 `Message` 称为**同步消息拦截器**。

顾名思义，该拦截器只会影响同步消息。复习一下上节中分析到的东西，我们默认发送的消息都是同步的，只有某个 `Message` 被调用了 `setAsynchronous(true)` 后才是异步消息。同步消息受队列限制依次有序的等待处理，异步消息也不受限制。

消息拦截器与普通消息的差异在于拦截器的 `target` 是空的，正常我们通过 `enqueueMessage()` 方法入队的消息由于限制 `target` 是不能为空的。

```
// 标识拦截器的 token
private int mNextBarrierToken;

private int postSyncBarrier(long when) {

    synchronized (this) {
        // 得到拦截器 token
        final int token = mNextBarrierToken++;
        // 实例化一个消息对象
        final Message msg = Message.obtain();
        // 将对象设置为 in-use 状态
        msg.markInUse();
        // 设置时间
        msg.when = when;
        // 将 token 存于消息的常用属性 arg1 中
        msg.arg1 = token;

        Message prev = null;
        Message p = mMessages;

        // 如果 when 不等于 0 就在队列中按时间找到它的位置
        if (when != 0) {
            while (p != null && p.when <= when) {
```

```

        prev = p;
        p = p.next;
    }
}
// 如果 prev 不等于空就把拦截器插入
// 如果 prev 等于空直接插入队列头部
if (prev != null) { // invariant: p == prev.next
    msg.next = p;
    prev.next = msg;
} else {
    msg.next = p;
    mMessages = msg;
}

// 拦截器入队成功，返回对应 token
return token;
}
}

```

总体来说添加拦截器的方法跟正常消息入队差不多，值得一提的就是 **Message** 的 **target** 是空的，然后 **arg1** 保存着拦截器的唯一标识 **token**。

token 的作用是找到对应的拦截器删除，看看删除拦截器的方法。

```

public void removeSyncBarrier(int token) {
    synchronized (this) {
        Message prev = null;
        Message p = mMessages;
        // 遍历队列找到指定拦截器
        // 查找条件: target 为空, arg1 等于指定 token 值
        while (p != null && (p.target != null || p.arg1 != token)) {
            prev = p;
            p = p.next;
        }
        // 如果 p 等于空说明没找到
        if (p == null) {
            throw new IllegalStateException("The specified message
queue synchronization "
                + " barrier token has not been posted or has already
been removed.");
        }

        // 是否需要唤醒线程
        final boolean needWake;
    }
}

```

```
// 在队列中移除掉拦截器
if (prev != null) {
    prev.next = p.next;
    // 如果 prev 不等于空说明拦截器前面还有别的消息，就不需要
唤醒
    needWake = false;
} else {
    mMessages = p.next;
    // 拦截器在队列头部，移除它之后如果队列空了或者它的下一个消息是个正常消息就需要唤醒
    needWake = mMessages == null || mMessages.target != null;
}

// 回收
p.recycleUnchecked();

// 判断是否需要唤醒
if (needWake && !mQuitting) {
    nativeWake(mPtr);
}
}
}
```

队列空闲处理器 IdleHandler

由于在从队列中取出消息时队里可能是空的，这时候就会阻塞线程等待消息到来。每次队列中没有消息而进入的阻塞状态，我们叫它为“空闲状态”。

讲道理实际使用中队列空闲状态的情况还是很常见的，为了更好的利用资源，也为了更好的掌握线程的状态，开发人员就设计了这么一个“队列空闲处理器 IdleHandler”。

IdleHandler 是 MessageQueue 类下的一个子接口，只包含了一个方法：

```
public static interface IdleHandler {
    /**
     * 当线程的 MessageQueue 等待更多消息时会调用该方法。
     *
     * 返回值：true 代表只执行一次，false 代表会一直执行它
     */
    boolean queueIdle();
}
```

MessageQueue 为我们提供了添加和删除 **IdleHandler** 的方法：

```
//使用一个 ArrayList 存储 IdleHandler
private final ArrayList<IdleHandler> mIdleHandlers = new
ArrayList<IdleHandler>();

// 添加一个 IdleHandler
public void addIdleHandler(@NonNull IdleHandler handler) {
    if (handler == null) {
        throw new NullPointerException("Can't add a null
IdleHandler");
    }
    synchronized (this) {
        mIdleHandlers.add(handler);
    }
}

// 删除一个 IdleHandler
public void removeIdleHandler(@NonNull IdleHandler handler) {
    synchronized (this) {
        mIdleHandlers.remove(handler);
    }
}
```

消息出队管理 next () 方法

next () 方法很长，先大致看一下源码：

```
private IdleHandler[] mPendingIdleHandlers;

Message next () {

    // mPtr 是从 native 方法中得到的 NativeMessageQueue 地址
    // 如果 mPtr 等于 0 说明队列不存在或被清除掉了
    final long ptr = mPtr;
    if (ptr == 0) {
        return null;
    }

    // 待处理的 IdleHandler 数量，因为代表数量，所以只有第一次初始化
    时为-1
    int pendingIdleHandlerCount = -1;
```



```
// 线程将被阻塞的时间
// -1: 一直阻塞
// 0: 不阻塞
// >0:阻塞 nextPollTimeoutMillis 毫秒
int nextPollTimeoutMillis = 0;

// 开始死循环，下面的代码都是在循环中，贼长！
for (;;) {

    // 如果 nextPollTimeoutMillis 不等于 0 说明要阻塞线程了
    if (nextPollTimeoutMillis != 0) {
        // 为即将长时间阻塞做准备把该释放的对象都释放了
        Binder.flushPendingCommands();
    }

    // 阻塞线程操作
    nativePollOnce(ptr, nextPollTimeoutMillis);

    synchronized (this) {
        // 得到当前时间
        final long now = SystemClock.uptimeMillis();

        Message prevMsg = null;
        Message msg = mMessages;

        // 判断队列头是不是同步拦截器
        if (msg != null && msg.target == null) {
            // 如果是拦截器就向后找一个异步消息
            do {
                prevMsg = msg;
                msg = msg.next;
            } while (msg != null && !msg.isAsynchronous());
        }

        // 判断队列是否有可以取出的消息
        if (msg != null) {

            if (now < msg.when) {
                // 如果待取出的消息还没有到应该被处理的时间就让线程阻塞到应该被处理的时间
                nextPollTimeoutMillis = (int) Math.min(msg.when - now, Integer.MAX_VALUE);
            }
        }
    }
}
```

```

    } else {
        // 直接就能取出消息，所以不用阻塞线程
        mBlocked = false;

        将消息从队列中剥离出来
        if (prevMsg != null) {
            prevMsg.next = msg.next;
        } else {
            mMessages = msg.next;
        }
        // 让消息脱离队列
        msg.next = null;

        if (DEBUG) Log.v(TAG, "Returning message: " + msg);

        // 设置为 in-use 状态
        msg.markInUse();
        // 返回取出的消息，结束循环，结束 next() 方法
        return msg;
    }
} else {
    // 队列中没有可取出的消息，nextPollTimeoutMillis 等于
    -1 让线程一直阻塞
    nextPollTimeoutMillis = -1;
}

// 如果队列已经退出了直接注销和结束方法
if (mQuitting) {
    dispose();
    return null;
}

// IdleHandler 初始化为-1，所以在本循环中该条件成立的次数
<= 1
    if (pendingIdleHandlerCount < 0 && (mMessages == null ||
now < mMessages.when)) {
        // 得到 IdleHandler 的数量
        pendingIdleHandlerCount = mIdleHandlers.size();
    }

    // pendingIdleHandlerCount 小于或等于 0 说明既没有合适的
    消息也没有合适的闲时处理
    if (pendingIdleHandlerCount <= 0) {
        // 直接进入下次循环阻塞线程

```

```
        mBlocked = true;
        continue;
    }

    // 代码执行到此处就说明线程中有待处理的 IdleHandler
    // 那么就从 IdleHandler 集合列表中取出待处理的 IdleHandler
    if (mPendingIdleHandlers == null) {
        // 初始化待处理 IdleHandler 数组，最小长度为 4
        mPendingIdleHandlers = new
IdleHandler[Math.max(pendingIdleHandlerCount, 4)];
    }

    // 从 IdleHandler 集合中获取待处理的 IdleHandler
    mPendingIdleHandlers =
mIdleHandlers.toArray(mPendingIdleHandlers);
    }

    // =====>到此处同步代码块已经结束=====

    for (int i = 0; i < pendingIdleHandlerCount; i++) {
        // 取出一个 IdleHandler
        final IdleHandler idler = mPendingIdleHandlers[i];
        // 释放掉引用
        mPendingIdleHandlers[i] = null;

        // IdleHandler 的执行模式，true=执行一次，false=总是执行
        boolean keep = false;

        try {
            // 执行 IdleHandler 的 queueIdle() 代码，得到执行模式
            keep = idler.queueIdle();
        } catch (Throwable t) {
            Log.wtf(TAG, "IdleHandler threw exception", t);
        }

        // 通过执行模式判断是否需要移除掉对应的 IdleHandler
        if (!keep) {
            synchronized (this) {
                mIdleHandlers.remove(idler);
            }
        }
    }
}
```

```
// 处理完了所有 IdleHandler 把数量清 0
pendingIdleHandlerCount = 0;

// 因为执行了 IdleHandler 的代码块，有可能已经有新的消息入队
了

// 所以到这里就不阻塞线程，直接去查看有没有新消息
nextPollTimeoutMillis = 0;
}
}
```

消息出队的核心代码的逻辑都在一个庞大的死循环 `for(;;)` 中，其流程如下：

0，循环开始。

1，根据 `nextPollTimeoutMillis` 值阻塞线程，初始值为 0：不阻塞线程。

2，将【待取出消息指针】指向队列头。

3，如果队列头是同步拦截器的话就将【待取出消息指针】指向队列头后面最近的一个异步消息。

4，如果【待取出消息指针】不可用 (`msg == null`) 说明队列中没有可取出的消息，让 `nextPollTimeoutMillis` 等于 -1 让线程一直阻塞，等待新消息到来时唤醒它。

5，如果【待取出消息指针】可用 (`msg != null`) 再判断一下消息的待处理时间。

如果消息的待处理时间大于当前时间 (`now < msg.when`) 说明当前消息还没到要处理的时间，让线程阻塞到消息待处理的指定时间。

如果消息的待处理时间小于当前时间 (`now > msg.when`) 就直接从队列中取出消息返回给调用处。（此处会直接结束整个循环，结束 `next()` 方法。）

6，如果队列已经退出了直接结束 `next()` 方法。

7，如果是第一次循环就初始化 `IdleHandler` 数量的局部变量 `pendingIdleHandlerCount`。

8，如果 `IdleHandler` 数量小于等于 0 说明没有合适的 `IdleHandler`，直接进入下次循环阻塞线程。（此处会直接结束本次循环。）

9，初始化 `IdleHandler` 数组，里面保存着本地待处理的 `IdleHandler`。

10，遍历 `IdleHandler` 数组，执行对应的 `queueIdle()` 方法。

11，执行完所有 `IdleHandler` 之后，将 `IdleHandler` 数量清 0。

12, 因为执行了 `IdleHandler` 的代码块, 有可能已经有新的消息入队了, 所以让 `nextPollTimeoutMillis` 等于 0 不阻塞线程, 直接去查看有没有新消息。

13, 本次循环结束, 开始新一轮循环。

总结

`MessageQueue` 队列消息是有序的, 按消息待处理时间依次排序。

同步拦截器可以拦截它之后的所有同步消息, 直到这个拦截器被移除。

取出消息时如果没有合适的消息线程会阻塞。

第三节 Looper 的源码分析

刚看源码的时候: “这 TM 写的是啥? 那写的又 TM 是啥?”

研究明白了之后: “奥, 原来就这点玩意儿啊, 太简单了。”

`Looper` 的职责很单一, 就是单纯的从 `MessageQueue` 中取出消息分发给消息对应的宿主 `Handler`, 因此它的代码不多 (300 行左右)。

`Looper` 是线程独立的且每个线程只能存在一个 `Looper`。

`Looper` 会根据自己的存活情况来创建和退出属于它自己的 `MessageQueue`。

创建与退出 Looper

上面的结论中提到了 `Looper` 是线程独立的且每个线程只能存在一个 `Looper`。所以构造 `Looper` 实例的方法类似于单例模式。隐藏构造方法, 对外提供了两个指定的获取实例方法 `prepare()` 和 `prepareMainLooper()`。

```
// 应用主线程 (UI 线程) Looper 实例
private static Looper sMainLooper;

// Worker 线程 Looper 实例, 用 ThreadLocal 保存的对象都是线程独立的
static final ThreadLocal<Looper> sThreadLocal = new
ThreadLocal<Looper>();

// 与当前 Looper 对应的消息队列
final MessageQueue mQueue;
```

```
// 当前 Looper 所在的线程
final Thread mThread;

/**
 * 对外公开初始化方法
 *
 * 在普通线程中初始化 Looper 调用此方法
 */
public static void prepare() {
    // 初始化一个可以退出的 Looper
    prepare(true);
}

/**
 * 对外公开初始化方法
 *
 * 在应用主线程（UI 线程）中初始化 Looper 调用此方法
 */
public static void prepareMainLooper() {

    // 因为是主线程，初始化一个不允许退出的 Looper
    prepare(false);

    synchronized (Looper.class) {
        // 如果 sMainLooper 不等于空说明已经创建过主线程 Looper 了，不
        // 应该重复创建
        if (sMainLooper != null) {
            throw new IllegalStateException("The main Looper has
already been prepared.");
        }
        sMainLooper = myLooper();
    }
}

/**
 * 内部私有初始化方法
 * @param quitAllowed 是否允许退出 Looper
 */
private static void prepare(boolean quitAllowed) {
    // 每个线程只能有一个 Looper
    if (sThreadLocal.get() != null) {
        throw new RuntimeException("Only one Looper may be created per
thread");
    }
}
```

```
}  
// 保存实例  
sThreadLocal.set(new Looper(quitAllowed));  
}  
  
/**  
 * 私有构造方法  
 * @param quitAllowed 是否允许退出 Looper  
 */  
private Looper(boolean quitAllowed) {  
    // 初始化 MessageQueue  
    mQueue = new MessageQueue(quitAllowed);  
    // 得到当前线程实例  
    mThread = Thread.currentThread();  
}
```

真正创建 **Looper** 实例的构造方法中其实很简单，就是创建了对应的 **MessageQueue** 实例，然后得到当前线程，值得注意的是 **MessageQueue** 和线程实例都是被 **final** 关键字修饰的，只能被赋值一次。

对外公开初始化方法 `prepareMainLooper()` 是为应用主线程（UI 线程）准备的，应用刚被创建就会调用该方法，所以我们不该再去调用它。

开发者可以通过调用对外公开初始化方法 `prepare()` 对自己的 **worker** 线程创建 **Looper**，但是要注意只能初始化一次。

调用 `Looper.prepare()` 方法初始化完成后，可以调用 `myLooper()` 和 `myQueue()` 方法得到当前线程对应的实例。

```
public static @Nullable Looper myLooper() {  
    return sThreadLocal.get();  
}  
  
public static @NonNull MessageQueue myQueue() {  
    return myLooper().mQueue;  
}
```

退出 Looper

退出 **Looper** 有安全与不安全两种退出方法，其实对应的就是 **MessageQueue** 的安全与不安全方法：

```
public void quit() {
```

```
mQueue.quit(false);  
}  
  
public void quitSafely() {  
    mQueue.quit(true);  
}
```

什么安全退出，什么是不安全退出，在 `MessageQueue` 源码中分析过。

运行 `Looper` 处理消息

调用 `Looper.prepare()` 方法初始化完成 `Looper` 后就可以让 `Looper` 去工作了，只需要调用 `Looper.loop()` 方法即可。

```
public static void loop() {  
    // 得到当前线程下的 Looper  
    final Looper me = myLooper();  
  
    // 如果还没初始化过抛异常  
    if (me == null) {  
        throw new RuntimeException("No Looper; Looper.prepare()  
wasn't called on this thread.");  
    }  
  
    // 得到当前线程下与 Looper 对应的消息队列  
    final MessageQueue queue = me.mQueue;  
  
    // 得到当前线程的唯一标识 (uid+pid)，作用是下面每次循环都判断  
    一下线程有没有被切换  
    // 不知道为什么调用两次该方法  
    Binder.clearCallingIdentity();  
    final long ident = Binder.clearCallingIdentity();  
  
    // 进入死循环不断取出消息  
    for (;;) {  
  
        // 从队列中取出一个消息，这可能会阻塞线程  
        Message msg = queue.next();  
  
        // 如果消息是空的，说明队列已经退出了，直接结束循环，结束方  
        法  
        if (msg == null) {  
            return;  
        }  
    }  
}
```



```
}

// 打印日志
final Printer logging = me.mLogging;
if (logging != null) {
    logging.println(">>>>> Dispatching to " + msg.target + "
" +
        msg.callback + ": " + msg.what);
}

// 性能分析相关的东西
final long traceTag = me.mTraceTag;
if (traceTag != 0 && Trace.isTagEnabled(traceTag)) {
    Trace.traceBegin(traceTag,
msg.target.getTraceName(msg));
}

try {
    //尝试将消息分发给宿主 (Handler)
    //dispatchMessage 为宿主 Handler 的接收消息方法
    msg.target.dispatchMessage(msg);
} finally {
    // 性能分析相关的东西
    if (traceTag != 0) {
        Trace.traceEnd(traceTag);
    }
}

//打印日志
if (logging != null) {
    logging.println("<<<<< Finished to " + msg.target + " " +
msg.callback);
}

//得到当前线程的唯一标识
final long newIdent = Binder.clearCallingIdentity();

//如果本次循环所在的线程与最开始不一样，打印日志记录
if (ident != newIdent) {
    Log.wtf(TAG, "Thread identity changed from 0x"
        + Long.toHexString(ident) + " to 0x"
        + Long.toHexString(newIdent) + " while dispatching
to "
```

```

        + msg.target.getClass().getName() + " "
        + msg.callback + " what=" + msg.what);
    }

    //消息分发完毕，回收消息到缓存池
    msg.recycleUnchecked();
}
}

```

总结

Looper 的功能很简单，核心方法 `Looper.loop()` 就是不断的从消息队列中取出消息分发给对应的宿主 `Handler`，它与对应 `MessageQueue` 息息相关，一起创建，一起退出。

Looper 更想强调的是线程的独立性与唯一性，利用 `ThreadLocal` 保证每个线程只有一个 `Looper` 实例的存在。利用静态构造实例方法保证不能重复创建 `Looper`。

`Looper.prepareMainLooper()` 是比较特殊的方法，它是给 UI 线程准备，理论上开发者在任何情况下都不应该调用它。

第四节 handler 的源码分析

`Handler` 本身可在多线程之间调用，不管它在哪个线程发送消息，都会回到它被初始化的哪个线程中接收到消息。

初始化

`Handler` 有 7 个构造方法，分别对应不同的参数来初始化不同的 `Handler` 属性，但是真正完成初始化操作的只有两个构造方法：

```

// 是否需要查找潜在的漏洞
private static final boolean FIND_POTENTIAL_LEAKS = false;

/**
 * 将 Callback 接口作为构造方法参数，可以用作接收消息的回调
 * 这样就可以省去自己重写 Handler 自身的 handleMessage 方法
 *
 * @param msg 接收到的消息
 * @return 是否需要进一步处理，即调用 Handler 自身的 handleMessage
方法

```

```
*/
public interface Callback {
    public boolean handleMessage(Message msg);
}

final Looper mLooper;
final MessageQueue mQueue;
final Callback mCallback;

// 发送的消息是否为异步的, 默认是 false
final boolean mAsynchronous;

/**
 * @hide 隐藏的构造方法, 外部不可见
 */
public Handler(Callback callback, boolean async) {

    // 检查是否存在内存泄漏的可能
    if (FIND_POTENTIAL_LEAKS) {
        final Class<? extends Handler> klass = getClass();
        if ((klass.isAnonymousClass() || klass.isMemberClass() ||
klass.isLocalClass()) &&
            (klass.getModifiers() & Modifier.STATIC) == 0) {
            Log.w(TAG, "The following Handler class should be static
or leaks might occur: " +
                klass.getCanonicalName());
        }
    }

    // 得到当前线程的 Looper
    mLooper = Looper.myLooper();

    // 如果 Looper 还没初始化抛出异常
    if (mLooper == null) {
        throw new RuntimeException(
            "Can't create handler inside thread that has not called
Looper.prepare()");
    }

    // 得到当前线程的 MessageQueue
    mQueue = mLooper.mQueue;

    mCallback = callback;
}
```

```
mAsynchronous = async;
}

public Handler(Looper looper, Callback callback, boolean async) {
    mLooper = looper;
    mQueue = looper.mQueue;
    mCallback = callback;
    mAsynchronous = async;
}
```

从上面代码的得知，构造方法初始化的工作就是给 `mLooper`, `mQueue`, `mCallback` 和 `mAsynchronous` 这几个关键的属性赋值。`mLooper` 和 `mQueue` 自然就是当前线程下的 **Looper** 和 **MessageQueue** 了，如果传递了 **Looper** 参数就直接赋值，如果没传递就调用 `Looper.myLooper()`；得到当前线程的 **Looper**。

`mCallback` 是 **Handler** 内部定义的一个简单接口，其目的是为了**替代传统的接收消息方法**。当然使用 `mCallback` 的同时并不会影响正常的 **Handler** 消息分发。此处解释从后面接收消息时的逻辑就可以看到。

`mAsynchronous` 的意思是**该 Handler 发送的消息是否是异步的**，从前面 **Message** 源码的文章中我们知道 **Message** 中有一个设置消息是否为异步消息的方法，**MessageQueue** 对异步消息的处理也与同步消息不同。此处如果设置了 `mAsynchronous` 为 `true`，那么这个 **Handler** 发送的所有消息就都是异步消息。

在有两个参数的构造方法中我们会发现有一段检查是否存在内存泄漏的代码，为什么会这样呢？在分析完发送消息和接收消息后再说这个。

当然，除了上面两个构造方法外还有其它几个构造方法，但均是调用上面两个方法：

```
public Handler() {
    this(null, false);
}

public Handler(Callback callback) {
    this(callback, false);
}

public Handler(Looper looper) {
    this(looper, null, false);
}

public Handler(Looper looper, Callback callback) {
```

```
this(looper, callback, false);  
}  
  
/**  
 * @hide 隐藏的构造方法，外部不可见  
 */  
public Handler(boolean async) {  
    this(null, async);  
}
```

发送消息

使用 **Handler** 发送消息时我们知道它分为两类：

- `postXXX()` 方法切换回原线程。
- `sendMessageXXX()` 方法发送消息到原线程。

其实这两种方法本质都是发送一个 **Message** 对象到原线程，只不过 `PostXXX()` 方法是发送了一个只有 `Runnable callback` 属性的 **Message** 对象。

先来看一下 `sendMessageXXX()` 类的方法：

```
// 发送一条普通消息  
public final boolean sendMessage(Message msg) {  
    return sendMessageDelayed(msg, 0);  
}  
  
// 发送一条空消息  
public final boolean sendEmptyMessage(int what) {  
    return sendEmptyMessageDelayed(what, 0);  
}  
  
// 发送一条空的延时消息  
public final boolean sendEmptyMessageDelayed(int what, long  
delayMillis) {  
    Message msg = Message.obtain();  
    msg.what = what;  
    return sendMessageDelayed(msg, delayMillis);  
}  
  
// 发送一条空的定时消息  
public final boolean sendEmptyMessageAtTime(int what, long  
uptimeMillis) {
```

```

    Message msg = Message.obtain();
    msg.what = what;
    return sendMessageAtTime(msg, uptimeMillis);
}

// 发送一个普通的延时消息
public final boolean sendMessageDelayed(Message msg, long delayMillis)
{
    if (delayMillis < 0) {
        delayMillis = 0;
    }
    return sendMessageAtTime(msg, SystemClock.uptimeMillis() +
delayMillis);
}

// 发送一个普通的定时消息
public boolean sendMessageAtTime(Message msg, long uptimeMillis) {
    //得到消息队列
    MessageQueue queue = mQueue;
    // 如果消息队列是空的记录日志然后结束方法
    if (queue == null) {
        RuntimeException e = new RuntimeException(
            this + " sendMessageAtTime() called with no mQueue");
        Log.w("Looper", e.getMessage(), e);
        return false;
    }
    // 执行消息入队操作
    return enqueueMessage(queue, msg, uptimeMillis);
}

```

从上面的代码中发现，不管调用何种发送消息的方法，最后真正调用的都是 `sendMessageAtTime()` 方法。而真正发送的核心方法也就是入队方法是 **Handler** 的 `enqueueMessage()` 方法。

```

private boolean enqueueMessage(MessageQueue queue, Message msg, long
uptimeMillis) {
    // 将消息的宿主设置为当前 Handler 自身
    msg.target = this;

    //如果 Handler 被设置成了异步就把消息也设置成异步的
    if (mAsynchronous) {
        msg.setAsynchronous(true);
    }
}

```

```
// 执行消息队列的入队操作
return queue.enqueueMessage(msg, uptimeMillis);
}
```

看到这里终于明白了为啥 **Looper** 和 **MessageQueue** 一直在使用 **Message** 的 **target** 属性而我们却从来没有给它赋值过，是 **Handler** 在发送消息前自己赋值上去的。

看完了发送消息类的方法在看看切换线程类的方法干了什么：

```
private static Message getPostMessage(Runnable r) {
    Message m = Message.obtain();
    m.callback = r;
    return m;
}

private static Message getPostMessage(Runnable r, Object token) {
    Message m = Message.obtain();
    m.obj = token;
    m.callback = r;
    return m;
}

public final boolean post(Runnable r) {
    return sendMessageDelayed(getPostMessage(r), 0);
}

public final boolean postAtTime(Runnable r, long uptimeMillis) {
    return sendMessageAtTime(getPostMessage(r), uptimeMillis);
}

public final boolean postAtTime(Runnable r, Object token, long
uptimeMillis) {
    return sendMessageAtTime(getPostMessage(r, token),
uptimeMillis);
}

public final boolean postDelayed(Runnable r, long delayMillis) {
    return sendMessageDelayed(getPostMessage(r), delayMillis);
}

public final boolean postAtFrontOfQueue(Runnable r) {
    return sendMessageAtFrontOfQueue(getPostMessage(r));
}
```

上本节开头讲的一样，`postXXX()` 类方法就是构造了一个只有 `Runnable` callback 的 `Message` 对象，然后走正常发送消息的方法。唯一有一个特例就是 `postAtFrontOfQueue()` 方法，它调用了 `sendMessageAtFrontOfQueue()` 方法是之前发送消息没有用到过的：

```
public final boolean sendMessageAtFrontOfQueue(Message msg) {
    MessageQueue queue = mQueue;
    if (queue == null) {
        RuntimeException e = new RuntimeException(
            this + " sendMessageAtTime() called with no mQueue");
        Log.w("Looper", e.getMessage(), e);
        return false;
    }
    return enqueueMessage(queue, msg, 0);
}
```

原来这个方法特殊的地方就是在入队的时候时间参数为 0，我们在 `MessageQueue` 源码知道如果入队消息的时间参数为 0 那么这个消息会被直接放在队列头。所以，`postAtFrontOfQueue()` 方法就是直接在消息队列头部插入了一个消息。

接收消息

在 `Looper` 的源码中我们知道每当从 `MessageQueue` 中取出一个消息时就会调用这个消息的宿主 `target` 中分发消息的方法：

```
// Looper 分发消息
msg.target.dispatchMessage(msg);
```

而这个宿主 `target` 也就是我们的 `Handler`，所有 `Handler` 接收消息就是在这个 `dispatchMessage()` 方法中了：

```
public void dispatchMessage(Message msg) {
    // 如果 Message 的 callback 不为空，说明它是一个通过 postXXX() 方法
    发送的消息
    if (msg.callback != null) {
        // 直接运行这个 callback
        handleCallback(msg);
    } else {
        //如果 mCallback 不为空说明 Handler 设置了 Callback 接口
        // 先执行接口处理消息的方法
        if (mCallback != null) {
```



```

        // 如果 callback 接口处理完消息返回 true 说明它将消息拦截
        // 不再执行 Handler 自身的处理消息方法，直接结束方法
        if (mCallback.handleMessage(msg)) {
            return;
        }
    }
    // 调用 Handler 自身处理消息的方法
    handleMessage(msg);
}

private static void handleCallback(Message message) {
    // 运行 callback，也就是这个 Runnable 接口
    message.callback.run();
}

// Handler 自身处理消息的方法，开发者需要重新该方法来实现接收消息
public void handleMessage(Message msg) {
}

```

由此可见，如果是单纯的 `PostXXX()` 方法发送的消息，**Handler** 接收到了之后直接运行 **Message** 对象的 **Runnable** 接口，不会将它当做一个消息进行处理。

而我们的 `mCallback` 接口是完全可以替代 **Handler** 自身接收消息的方法，因为其高优先处理等级，它甚至可以选择拦截掉 **Handler** 自身的接收消息方法。

内存泄漏的可能

我们在使用 **Handler** 的时候写法一般如下：

```

private final Handler handler = new Handler() {

    @Override
    public void handleMessage(Message msg) {

    }

};

```

由于重写了 `handleMessage()` 方法相当于生成了一个匿名内部类，也就相当于如下代码：

```

private final Handler handler = new MyHandler ();

```

```
class MyHandler extends Handler{

    @Override
    public void handleMessage(Message msg) {

    }

}
```

可是你有没有想过内部类凭什么能够调用外部类的属性和方法呢？答案就是内部类隐式的持有着外部类的引用，编译器在创建内部类时把外部类的引用传入了其中，只不过是你看不到而已。

既然 **Handler** 作为内部类持有着外部类（多数情况为 **Activity**）的引用，而 **Handler** 对应的一般都是耗时操作。当我们在子线程执行一项耗时操作时，用户退出程序，**Activity** 需要被销毁，而 **Handler** 还在持有 **Activity** 的引用导致无法回收，就会引发内存泄漏。

解决方法分为两步

1. 生成内部类时把内部类声明为静态的。
2. 使用弱引用来持有外部类引用。

静态内部类不会持有外部类的引用，且弱引用不会阻止 **JVM** 回收对象。

```
static class MyHandler extends Handler{

    WeakReference<Activity> mActivity ;

    public MyHandler(Activity activity){
        mActivity = new WeakReference<Activity>(activity);
    }

    @Override
    public void handleMessage(Message msg) {
        Activity activity = mActivity.get();

        if (activity == null){
            return;
        }

        // do something

    }

}
```

```
}
```

所以，在文章刚开始初始化方法中检查漏洞的代码其实就是检查这个内存泄漏的可能性：

```
// 检查是否存在内存泄漏的可能
if (FIND_POTENTIAL_LEAKS) {
    final Class<? extends Handler> klass = getClass();
    // 是否为匿名类，内部类以及是否为静态类
    if ((klass.isAnonymousClass() || klass.isMemberClass() ||
klass.isLocalClass()) &&
        (klass.getModifiers() & Modifier.STATIC) == 0) {
        Log.w(TAG, "The following Handler class should be static
or leaks might occur: " +
            klass.getCanonicalName());
    }
}
```

第五节 Handler 机制实现原理总结

Message 缓存池

Android 的工程师们充分利用了 Java 的高级语言特性，即类中持有着一个类自身的属性作为经典数据结构中的链表 next 指针，以静态属性属于类本身的特性实现了链表的表头。这种模式给了我很大的启发，让我这种渣渣每逢想起都会惊讶“还有这种操作？”。

为什么要有缓存池

了解完 Handler 整体机制后我猜测，Message 功能十分单一且状态很少，它只是一个具体发送消息的载体，但是使用数量十分庞大，回收用过的 Message 不仅可以有效的减少重复消耗系统资源且回收它的成本很低，所以何乐而不为呢？

谁负责回收 Message

我们使用 Message 时候知道调用 Message.obtain()；方法可以从缓存池中取出一个 Message，有存才能有取，我们什么时候回收它呢？从源码中发现，Looper 在分发 Message 给宿主 Handler 之后，确定了 Message 已经完成了它的使命直接

就会将它回收。所以我们完全不用担心这个，我们发送的每个消息最后都会被回收。

真正的阻塞发生在 MessageQueue

MessageQueue 维持的消息队列也是靠跟 Message 缓存池同样的原理生成的，每次消息出队时如果没有合适的待取出消息就会阻塞线程等待有合适的消息。

非常奇怪的是，MessageQueue 线程的方式不是传统使用 java 实现的，而是通过 JNI 调用 native 层的 C++代码实现的，C++代码中也实现了一套 Looper+MessageQueue+Handler，阻塞线程的方式是调用 Linux 的监听文件描述符 ePoll 实现的。

我的猜测是因为 Java 代码需要经过 JVM 的帮助才能跟系统接触，这一过程会消耗性能，而 C++代码则直接可以绕过这一个环节。所以，使用 C++代码实现线程阻塞可能是性能上的需求。

为什么推荐使用 Handler 实现线程间通信

在没有真正了解 Handler 的时候以为 Google 的工程师们在 Handler 上使用了什么了不起的技术呢，所以才推荐开发者们使用 Handler 来实现线程间通信。

其实呢？Android 是事件型驱动的系统，刚创建一个应用程序的主线程里就会被创建一个 Looper 来不断接受各种事件，所以说如果我们打开一个程序什么都不操作，这个程序就有可能是阻塞状态的，因为他没有任何事件需要去处理。反之，我们在自己的 UI 线程里执行一项耗时操作，主线程 Looper 一直在处理这个任务而无法分身处理其它的事件这时候就有可能 ANR 了。

所以，不是 Handler 的技术多牛逼，是主线程用了 Handler 来通信，你是用别的方法通信有可能会影响主线程 Looper 的正常工作。

第六节 Handler 面试题全解析

做 Android 开发肯定离不开跟 Handler 打交道，它通常被我们用来做主线程与子线程之间的通信工具，而 Handler 作为 Android 中消息机制的重要一员也确实给我们的开发带来了极大的便利。

可以说只要有异步线程与主线程通信的地方就一定要有 Handler。

在面试中 Handler 也是经常被问的一个点，那么本篇文章就以问答的方式，带你了解一下关于 Handler 的重要知识点。

1. 一个线程有几个 **Looper**? 几个 **Handler**?

一个 **Thread** 只能有一个 **Looper**，一个 **MessageQueue**，可以有多个 **Handler**
以一个线程为基准，他们的数量级关系是：**Thread(1) : Looper(1) : MessageQueue(1) : Handler(N)**

2. **Handler** 内存泄漏原因? 以及最佳解决方案?

泄露原因:

Handler 允许我们发送延时消息，如果在延时期间用户关闭了 **Activity**，那么该 **Activity** 会泄露。这个泄露是因为 **Message** 会持有 **Handler**，而又因为 **Java** 的特性，内部类会持有外部类，使得 **Activity** 会被 **Handler** 持有，这样最终就导致 **Activity** 泄露。

解决方案

1. 最直接的思路就是避免使用非静态内部类。使用 **Handler** 的时候，放在一个新建的文件中来继承 **Handler** 或者使用静态的内部类来替代。静态内部类不会隐含的持有外部类的引用，因此这个 **activity** 也就不会出现内存泄漏问题。
2. 如果你需要在 **Handler** 内部调用外部 **Activity** 的方法，你可以让这个 **Handler** 持有这个 **Activity** 的弱引用，这样便不会出现内存泄漏的问题了。
3. 另外，对于匿名类 **Runnable**，我们同样可以设置成静态的，因为静态内部类不会持有外部类的引用。
4. 注意：如果使用 **Handler** 发送循环消息，最好是在 **Activity** 的 **OnDestroy** 方法中调用 **mLeakHandler.removeCallbacksAndMessages(null)**; 移除消息。（这不是解决内存泄漏的方法）

5. 两种解决办法如下:

弱引用 (**WeakReference**)

```
public class SampleActivity extends Activity {

    /**
     * Instances of static inner classes do not hold an implicit
     * reference to their outer class.
     * 弱引用的方式
     */
    private static class MyHandler extends Handler {
        private final WeakReference<SampleActivity> mActivity;
        public MyHandler(SampleActivity activity) {
            mActivity = new WeakReference<SampleActivity>(activity);
        }

        @Override
        public void handleMessage(Message msg) {
```

```

SampleActivity activity = mActivity.get();
if (activity != null) {
    //to Something
}
}
}

```

静态

```

//定义成 static 的，因为静态内部类不会持有外部类的引用
private final MyHandler mHandler = new MyHandler(this);
private static final Runnable sRunnable = new Runnable() {
    @Override
    public void run() { //to something }
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    mHandler.postDelayed(sRunnable, 1000 * 60 * 10);
    finish();
}
}

```

3.为何主线程可以 new Handler？如果想要在子线程中 new Handler 要做些什么准备？

每一个 handler 必须要对应一个 looper，主线程会自动创建 Looper 对象，不需要我们手动创建，所以主线程可以直接创建 handler。

在 new handler 的时候没有传入指定的 looper 就会默认绑定当前创建 handler 的线程的 looper，如果没有 looper 就报错。

因为在主线程中，Activity 内部包含一个 Looper 对象，它会自动管理 Looper，处理子线程中发送过来的消息。而对于子线程而言，没有任何对象帮助我们维护 Looper 对象，所以需要我自己手动维护。

所以要在子线程开启 Handler 要先创建 Looper，并开启 Looper 循环

如果在子线程中创建了一个 Handler，那么就必须做三个操作：

1. prepare();
2. loop();
3. quit();

4.子线程中维护的 **Looper**，消息队列无消息的时候的处理方案是什么？有什么用？

在 Handler 机制里面有一个 **Looper**，在 **Looper** 机制里面有一个函数，叫做 **quitSafely()**和 **quit()**函数，这两个函数是调用的 **MessageQueue** 的 **quit()**。

```
/**
 * Quits the looper.
 * <p>
 * Causes the {@link #loop} method to terminate without processing
any
 * more messages in the message queue.
 * </p><p>
 * Any attempt to post messages to the queue after the looper is asked
to quit will fail.
 * For example, the {@link Handler#sendMessage(Message)} method will
return false.
 * </p><p class="note">
 * Using this method may be unsafe because some messages may not be
delivered
 * before the looper terminates. Consider using {@link #quitSafely}
instead to ensure
 * that all pending work is completed in an orderly manner.
 * </p>
 *
 * @see #quitSafely
 */
public void quit() {
    mQueue.quit(false);
}

/**
 * Quits the looper safely.
 * <p>
 * Causes the {@link #loop} method to terminate as soon as all
remaining messages
 * in the message queue that are already due to be delivered have been
handled.
 * However pending delayed messages with due times in the future will
not be
 * delivered before the loop terminates.
 * </p><p>
 * Any attempt to post messages to the queue after the looper is asked
to quit will fail.
```

* For example, the {@link Handler#sendMessage(Message)} method will return false.

```
* </p>
*/
public void quitSafely() {
    mQueue.quit(true);
}
```

再进入到 **MessageQueue** 的 **quit()**函数。

```
void quit(boolean safe) {
    if (!mQuitAllowed) {
        throw new IllegalStateException("Main thread not allowed to
quit.");
    }

    synchronized (this) {
        if (mQuitting) {
            return;
        }
        mQuitting = true;

        if (safe) {
            removeAllFutureMessagesLocked();
        } else {
            removeAllMessagesLocked();
        }

        // We can assume mPtr != 0 because mQuitting was previously
false.
        nativeWake(mPtr);
    }
}
```

它会 **remove** 消息，把消息队列中的全部消息给干掉。
把消息全部干掉，也就**释放了内存**。

```
private void removeAllFutureMessagesLocked() {
    final long now = SystemClock.uptimeMillis();
    Message p = mMessages;
    if (p != null) {
        if (p.when > now) {
            removeAllMessagesLocked();
        } else {
            Message n;
```



```
for (;;) {
    n = p.next;
    if (n == null) {
        return;
    }
    if (n.when > now) {
        break;
    }
    p = n;
}
p.next = null;
do {
    p = n;
    n = p.next;
    p.recycleUnchecked();
} while (n != null);
}
}
```

而在 `quit()` 函数的最后一行，有一个 **`nativeWake()`** 函数。

```
// We can assume mPtr != 0 because mQuitting was previously false.
nativeWake(mPtr);
```

这个函数的调用，就会叫醒等待的地方，醒来之后，就接着往下执行。

```
//native 的方法，在没有消息的时候阻塞管道读取端，只有
nativePollOnce 返回之后才能往下执行
//阻塞操作，等待 nextPollTimeoutMillis 时长
nativePollOnce(ptr, nextPollTimeoutMillis);
```

往下执行后，发现 `Message msg = mMessages;` 是空的，然后就执行了这个，就接着往下走。

```
if (msg != null) {

    .....

} else {
    // No more messages.
    //没有消息，nextPollTimeoutMillis 复位
    nextPollTimeoutMillis = -1;
}
```

然后又调用了这个方法，并且 `return` 了 `null`。

```
// Process the quit message now that all pending messages
have been handled.
```

//如果消息队列正在处于退出状态返回 `null`，调用 `dispose()`；
释放该消息队列

```
if (mQuitting) {
    dispose();
    return null;
}
```

所以说，这个时候 `Looper` 就结束了(跳出了死循环)，则达成了第二个作用：**释放线程**。

4.既然可以存在多个 `Handler` 往 `MessageQueue` 中添加数据（发消息时各个 `Handler` 可能处于不同线程），那它内部是如何确保线程安全的？

这里主要关注 `MessageQueue` 的消息存取即可，看源码内部的话，在往消息队列里面存储消息时，会拿当前的 `MessageQueue` 对象作为锁对象，这样通过加锁就可以确保操作的原子性和可见性了。

消息的读取也是同理，也会拿当前的 `MessageQueue` 对象作为锁对象，来保证多线程读写的一个安全性。

```
boolean enqueueMessage(Message msg, long when) {
    if (msg.target == null) {
        throw new IllegalArgumentException("Message must have a target.");
    }
    if (msg.isInUse()) {
        throw new IllegalStateException(msg + " This message is already in use.");
    }

    synchronized (this) { ←
        if (mQuitting) {
            IllegalStateException e = new IllegalStateException(
                msg.target + " sending message to a Handler on a dead thread");
            Log.w(TAG, e.getMessage(), e);
            msg.recycle();
            return false;
        }
    }
}
```

5.我们使用 `Message` 时应该如何创建它？

创建的它的方式有两种：

一种是直接 `new` 一个 `Message` 对象，

另一种是通过调用 `Message.obtain()` 的方式去复用一個已经被回收的

`Message`，

当然日常使用者是推荐使用后者来拿到一个 `Message`，因为不断的去创建新对象的话，可能会导致垃圾回收区域中新生代被占满，从而触发 `GC`。

Message 中的 sPool 就是用来存放被回收的 Message，当我们调用 obtain 后，会先查看是否有可复用的对象，如果真的没有才会去创建一个新的 Message 对象。

补充：主要的 Message 回收时机是：

- 1.在 MQ 中 remove Message 后；
- 2.单次 loop 结束后；
- 3.我们主动调用 Message 的 recycle 方法后

6.Looper 死循环为什么不会导致应用卡死？

Launch 桌面的图标第一次启动 Activity 时，会最终走到 ActivityThread 的 main 方法，在 main 方法里面创建 Looper 和 MessageQueue 处理主线程的消息，然后 Looper.loop()方法进入死循环，我们的 Activity 的生命周期都是通过 Handler 机制处理的，包括 onCreate、onResume 等方法，下面是 loop 方法循环。

主线程的主要方法就是消息循环，一旦退出消息循环，那么你的应用也就退出了，Looper.loop()方法可能会引起主线程的阻塞，但只要它的消息循环没有被阻塞，能一直处理事件就不会产生 ANR 异常。

造成 ANR 的不是主线程阻塞，而是主线程的 Looper 消息处理过程发生了任务阻塞，无法响应手势操作，不能及时刷新 UI。

阻塞与程序无响应没有必然关系，虽然主线程在没有消息可处理的时候是阻塞的，但是只要保证有消息的时候能够立刻处理，程序是不会无响应的。

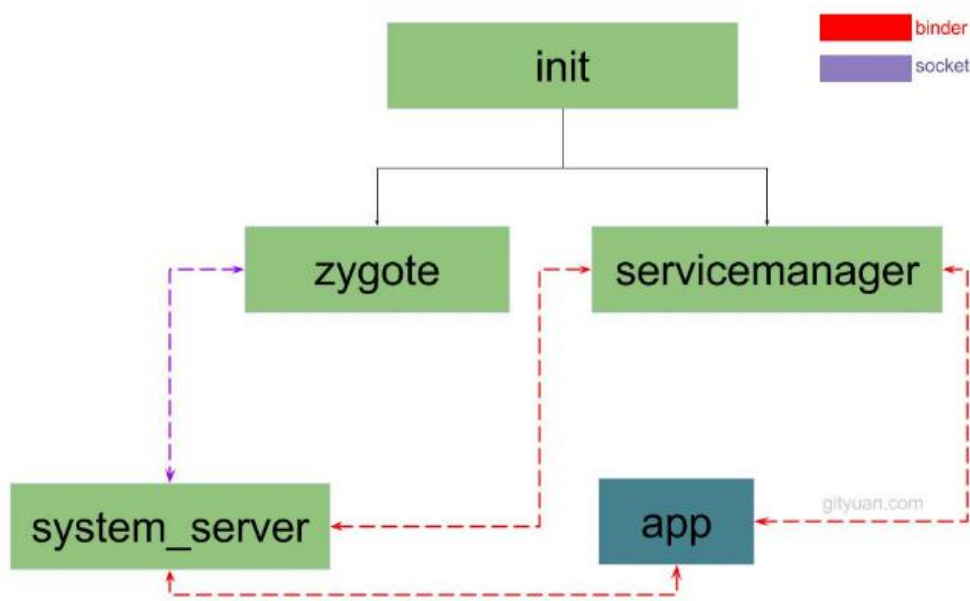
总结：应用卡死压根与这个 Looper 没有关系，应用在没有消息需要处理的时候，它是在睡眠，释放线程；卡死是 ANR，而 Looper 是睡眠。

第三章 Dalvik VM 进程系统

第一节 Android 系统启动-综述

一. 概述

Android 系统底层基于 Linux Kernel，当 Kernel 启动过程会创建 init 进程，该进程是所有用户空间的鼻祖，init 进程会启动 servicemanager(binder 服务管家)，Zygote 进程(Java 进程的鼻祖)。Zygote 进程会创建 system_server 进程以及各种 app 进程，下图是这几个系统重量级进程之间的层级关系。



二. init

init 是 Linux 系统中用户空间的第一个进程(pid=1), Kerner 启动后会调用 `/system/core/init/lnit.cpp` 的 `main()` 方法.

2.1 Init.main

```
int main(int argc, char** argv) {  
    ...  
    klog_init(); //初始化 kernel log  
    property_init(); //创建一块共享的内存空间, 用于属性服务  
    signal_handler_init(); //初始化子进程退出的信号处理过程  
  
    property_load_boot_defaults(); //加载/default.prop 文件  
    start_property_service(); //启动属性服务器(通过 socket 通信)  
    init_parse_config_file("/init.rc"); //解析 init.rc 文件
```

```
//执行 rc 文件中触发器为 on early-init 的语句
action_for_each_trigger("early-init", action_add_queue_tail);

//执行 rc 文件中触发器为 on init 的语句
action_for_each_trigger("init", action_add_queue_tail);

//执行 rc 文件中触发器为 on late-init 的语句
action_for_each_trigger("late-init", action_add_queue_tail);


while (true) {
    if (!waiting_for_exec) {
        execute_one_command();
        restart_processes();
    }

    int timeout = -1;

    if (process_needs_restart) {
        timeout = (process_needs_restart - gettime()) * 1000;

        if (timeout < 0)
            timeout = 0;
    }

    if (!action_queue_empty() || cur_action) {
        timeout = 0;
    }

    epoll_event ev;

    //循环 等待事件发生
    int nr = TEMP_FAILURE_RETRY(epoll_wait(epoll_fd, &ev, 1, timeout));

    if (nr == -1) {
        ERROR("epoll_wait failed: %s\n", strerror(errno));
    } else if (nr == 1) {
        ((void (*)(void*)) ev.data.ptr)();
    }
}
```

```
}  
  
return 0;  
  
}
```

init 进程的主要功能点:

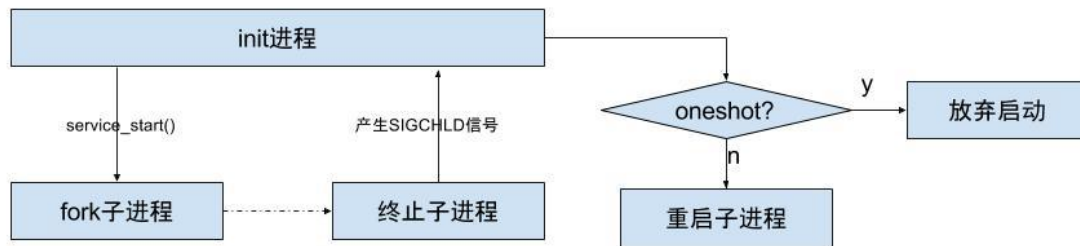
- 分析和运行所有的 init.rc 文件;
- 生成设备驱动节点; (通过 rc 文件创建)
- 处理子进程的终止(signal 方式);
- 提供属性服务 property service。

2.2 Zygote 自动重启机制

当 init 解析到下面这条语句,便会启动 Zygote 进程

```
service zygote /system/bin/app_process -Xzygote /system/bin --zygote --start-system-server  
  
    class main                                //伴随着 main class 的启动而启动  
  
        socket zygote stream 660 root system //创建 socket  
  
        onrestart write /sys/android_power/request_state wake  
  
        onrestart write /sys/power/state on  
  
        onrestart restart media                //当 zygote 重启时,则会重启 media  
  
        onrestart restart netd                 // 当 zygote 重启时,则会重启 netd
```

当 init 子进程(Zygote)退出时,会产生 SIGCHLD 信号,并发送给 init 进程,通过 socket 套接字传递数据,调用到 wait_for_one_process()方法,根据是否是 oneshot,来决定是重启子进程,还是放弃启动。由于缺省模式 oneshot=false,因此 Zygote 一旦被杀便会再次由 init 进程拉起。



接下来,便是进入了 Zygote 进程.

三. Zygote

当 Zygote 进程启动后, 便会执行到 frameworks/base/cmds/app_process/App_main.cpp 文件的 main()方法. 整个调用流程:

```
App_main.main
  AndroidRuntime.start
  AndroidRuntime.startVm
  AndroidRuntime.startReg
  ZygoteInit.main (首次进入 Java 世界)
  registerZygoteSocket
  preload
  startSystemServer
  runSelectLoop
```

3.1 App_main.main

```
int main(int argc, char* const argv[]){
    AppRuntime runtime(argv[0], computeArgBlockSize(argc, argv));
    while (i < argc) {
        ...//参数解析
    }
}
```

```
//设置进程名

if (!niceName.isEmpty()) {

    runtime.setArgv0(niceName.string());

    set_process_name(niceName.string());

}

if (zygote) {

    // 启动 AppRuntime, 见小节[3.2]

    runtime.start("com.android.internal.os.ZygoteInit", args, zygote);

} else if (className) {

    runtime.start("com.android.internal.os.RuntimeInit", args, zygote);

}

}
```

在 app_process 进程启动过程，有两个分支：

- 当 zygote 为 true 时，则执行 ZygoteInit.main()
- 当 zygote 为 false 时，则执行 RuntimeInit.main()

3.2 AndroidRuntime::start

[-> AndroidRuntime.cpp]

```
void AndroidRuntime::start(const char* className, const Vector<String8>& options)

{

    ...

    // 虚拟机创建

    if (startVm(&mJavaVM, &env, zygote) != 0) {

        return;

    }

}
```



```
}  
  
onVmCreated(env);  
  
// JNI 方法注册  
if (startReg(env) < 0) {  
    return;  
}  
  
...  
  
// 调用 ZygoteInit.main() 方法[见小节 3.3]  
env->CallStaticVoidMethod(startClass, startMeth, strArray);
```

3.3 ZygoteInit.main

[→ZygoteInit.java]

```
public static void main(String argv[]) {  
    try {  
        ...  
        registerZygoteSocket(socketName); //为 Zygote 注册 socket  
        preload(); // 预加载类和资源[见小节 3.4]  
        ...  
        if (startSystemServer) {  
            startSystemServer(abiList, socketName); //启动 system_server[见小节 3.5]  
        }  
        Log.i(TAG, "Accepting command socket connections");  
        runSelectLoop(abiList); //进入循环模式[见小节 3.6]  
        ...  
    } catch (MethodAndArgsCaller caller) {  
        caller.run(); //启动 system_server 中会讲到。  
    }  
}
```

```
...  
}
```

3.4 Zygotelnit.preload

[→Zygotelnit.java]

```
static void preload() {  
    Log.d(TAG, "begin preload");  
    preloadClasses();  
    preloadResources();  
    preloadOpenGL();  
    preloadSharedLibraries();  
    WebViewFactory.prepareWebViewInZygote();  
    Log.d(TAG, "end preload");  
}
```

3.5 Zygotelnit.startSystemServer

[→Zygotelnit.java]

```
private static boolean startSystemServer(String abilist, String socketName)  
    throws MethodAndArgsCaller, RuntimeException {  
    ...  
  
    // fork 子进程 system_server  
    pid = Zygote.forkSystemServer(  
        parsedArgs.uid, parsedArgs.gid,  
        parsedArgs.gids,  
        parsedArgs.debugFlags,
```

```
        null,  
        parsedArgs.permittedCapabilities,  
        parsedArgs.effectiveCapabilities);  
        ...  
  
        if (pid == 0) {  
            if (hasSecondZygote(abilist)) {  
                waitForSecondaryZygote(socketName);  
            }  
            //进入 system_server 进程[见小节 4.1]  
            handleSystemServerProcess(parsedArgs);  
        }  
        return true;  
    }  
}
```

3.6 ZygoteInit.runSelectLoop

[->ZygoteInit.java]

```
private static void runSelectLoop(String abilist) throws MethodAndArgsCaller {  
    ArrayList<FileDescriptor> fds = new ArrayList<FileDescriptor>();  
    ArrayList<ZygoteConnection> peers = new ArrayList<ZygoteConnection>();  
  
    //sServerSocket 是 socket 通信中的服务端，即 zygote 进程  
    fds.add(sServerSocket.getFileDescriptor());  
    peers.add(null);  
  
    while (true) {  
        StructPollfd[] pollFds = new StructPollfd[fds.size()];  
        for (int i = 0; i < pollFds.length; ++i) {
```

```
pollFds[i] = new StructPollfd();

pollFds[i].fd = fds.get(i);

pollFds[i].events = (short) POLLIN;

}

...

Os.poll(pollFds, -1);

for (int i = pollFds.length - 1; i >= 0; --i) {
    //采用 I/O 多路复用机制，当客户端发出 连接请求或者数据处理请求时，则执行 continue
    if ((pollFds[i].revents & POLLIN) == 0) {
        continue;
    }

    if (i == 0) {
        //创建客户端连接
        ZygoteConnection newPeer = acceptCommandPeer(abilist);
        peers.add(newPeer);
        fds.add(newPeer.getFileDescriptor());
    } else {
        //处理客户端数据事务
        boolean done = peers.get(i).runOnce();
        if (done) {
            peers.remove(i);
            fds.remove(i);
        }
    }
}

}
```

Zygote 进程创建 Java 虚拟机,并注册 JNI 方法, 真正成为 Java 进程的母体, 用于孵化 Java 进程. 在创建完 system_server 进程后,zygote 功成身退, 调用 runSelectLoop(), 随时待命, 当接收到请求创建新进程请求时立即唤醒并执行相应工作。

四. system_server

Zygote 通过 fork 后创建 system_server 进程, 在小节[3.5]执行完 startSystemServer()方法后, 进入到了 handleSystemServerProcess()方法, 如下所示。

4.1 handleSystemServerProcess

[→ZygoteInit.java]

```
private static void handleSystemServerProcess( ZygoteConnection.Arguments parse
dArgs) throws ZygoteInit.MethodAndArgsCaller {

    ...

    if (parsedArgs.niceName != null) {

        //设置当前进程名为"system_server"

        Process.setArgV0(parsedArgs.niceName);

    }

    final String systemServerClasspath = Os.getenv("SYSTEMSERVERCLASSPATH");

    if (systemServerClasspath != null) {

        //执行 dex 优化操作,比如 services.jar

        performSystemServerDexOpt(systemServerClasspath);

    }

    if (parsedArgs.invokeWith != null) {

        ...

    } else {
```

```
ClassLoader cl = null;

if (systemServerClasspath != null) {

    cl = new PathClassLoader(systemServerClasspath, ClassLoader.getSystemClassLoader());

    Thread.currentThread().setContextClassLoader(cl);

}

//[见小节 4.2]

RuntimeInit.zygoteInit(parsedArgs.targetSdkVersion, parsedArgs.remainingArgs, cl);

}
```

system_server 进程创建 PathClassLoader 类加载器.

4.2 RuntimeInit.zygoteInit

[→ RuntimeInit.java]

```
public static final void zygoteInit(int targetSdkVersion, String[] argv, ClassLoader classLoader) throws ZygoteInit.MethodAndArgsCaller {

    Trace.traceBegin(Trace.TRACE_TAG_ACTIVITY_MANAGER, "RuntimeInit");

    redirectLogStreams(); //重定向 log 输出

    commonInit(); // 通用的一些初始化

    nativeZygoteInit(); // zygote 初始化

    applicationInit(targetSdkVersion, argv, classLoader); // [见小节 3.4]

}
```

Binder 线程池启动

nativeZygoteInit()方法经过层层调用,会进入 app_main.cpp 中的 onZygoteInit()方法, Binder 线程池的创建也是在这个过程,如下:

```
virtual void onZygoteInit() {  
  
    sp<ProcessState> proc = ProcessState::self();  
  
    proc->startThreadPool(); //启动新 binder 线程池  
  
}
```

捕获特殊异常

applicationInit()方法经过层层调用,会抛出异常
ZygoteInit.MethodAndArgsCaller(m, argv), 具体过程如下:

```
protected static Runnable applicationInit(int targetSdkVersion, String[] argv,  
    ClassLoader classLoader) {  
  
    ...  
  
    VMRuntime.getRuntime().setTargetHeapUtilization(0.75f);  
    VMRuntime.getRuntime().setTargetSdkVersion(targetSdkVersion);  
  
    final Arguments args = new Arguments(argv);  
  
    //找到目标类的静态 main()方法  
    invokeStaticMain(args.startClass, args.startArgs, classLoader);  
  
}  
  
private static void invokeStaticMain(String className, String[] argv, ClassLoader classLoader)  
  
    throws ZygoteInit.MethodAndArgsCaller {  
  
    //此处的 className 等于 SystemServer  
  
    Class<?> cl = Class.forName(className, true, classLoader);  
  
    Method m = cl.getMethod("main", new Class[] { String[].class });  
  
    //抛出异常 Runnable 对象  
  
    throw new ZygoteInit.MethodAndArgsCaller(m, argv);  
  
}
```

设置虚拟机的堆利用率 0.75 和置 TargetSdk 版本；并抛出异常，然后由 ZygoteInit.main()捕获该异常，见下文

4.3 ZygoteInit.main

[→ZygoteInit.java]

```
public static void main(String argv[]) {  
    try {  
        startSystemServer(abiList, socketName); //抛出 MethodAndArgsCaller 异常  
        ....  
    } catch (MethodAndArgsCaller caller) {  
        caller.run(); //此处通过反射,会调用 SystemServer.main()方法 [见小节 4.4]  
    } catch (RuntimeException ex) {  
        ...  
    }  
}  
  
static class MethodAndArgsCaller implements Runnable {  
    private final Method mMethod;  
    private final String[] mArgs;  
  
    public MethodAndArgsCaller(Method method, String[] args) {  
        mMethod = method;  
        mArgs = args;  
    }  
  
    public void run() {  
        //执行 SystemServer.main()  
        mMethod.invoke(null, new Object[] { mArgs });  
    }  
}
```



```
}
```

采用抛出异常的方式,用于栈帧清空,提供利用率,以至于现在大家看到的每个 Java 进程的调用栈如下:

```
...  
    at com.android.server.SystemServer.main(SystemServer.java:175)  
    at java.lang.reflect.Method.invoke!(Native method)  
    at com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:738)  
    at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:628)
```

4.4 SystemServer.main

[→SystemServer.java]

```
public final class SystemServer {  
    ...  
    public static void main(String[] args) {  
        //先初始化 SystemServer 对象，再调用对象的 run() 方法  
        new SystemServer().run();  
    }  
}
```

4.5 SystemServer.run

[→SystemServer.java]

```
private void run() {  
    if (System.currentTimeMillis() < EARLIEST_SUPPORTED_TIME) {  
        Slog.w(TAG, "System clock is before 1970; setting to 1970.");  
    }  
}
```

```
        SystemClock.setCurrentTimeMillis(EARLIEST_SUPPORTED_TIME);
    }
    ...

    Slog.i(TAG, "Entered the Android system server!");

    EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_SYSTEM_RUN, SystemClock.uptimeMillis());

    Looper.prepareMainLooper(); // 准备主线程 looper

    //加载 android_servers.so 库，该库包含的源码在 frameworks/base/services/目录下
    System.loadLibrary("android_servers");

    //检测上次关机过程是否失败，该方法可能不会返回
    performPendingShutdown();

    createSystemContext(); //初始化系统上下文

    //创建系统服务管理
    mSystemServiceManager = new SystemServiceManager(mSystemContext);
    LocalServices.addService(SystemServiceManager.class, mSystemServiceManager);

    //启动各种系统服务
    try {
        startBootstrapServices(); // 启动引导服务
        startCoreServices();      // 启动核心服务
        startOtherServices();      // 启动其他服务[见小节 4.6]
    } catch (Throwable ex) {
        Slog.e("System", "***** Failure starting system services", ex);
        throw ex;
    }
}
```

```
//一直循环执行
Looper.loop();

throw new RuntimeException("Main thread loop unexpectedly exited");
}
```

4.6 服务启动

```
public final class SystemServer {

    private void startBootstrapServices() {
        ...

        //phase100

        mSystemServiceManager.startBootPhase(SystemService.PHASE_WAIT_FOR_DEFAULT_
DISPLAY);
        ...
    }

    private void startOtherServices() {
        ...

        //phase480 和 phase500

        mSystemServiceManager.startBootPhase(SystemService.PHASE_LOCK_SETTINGS_R
EADY);

        mSystemServiceManager.startBootPhase(SystemService.PHASE_SYSTEM_SERVICES
_READY);

        ...

        //[见小节 4.7]

        mActivityManagerService.systemReady(new Runnable() {

            @Override
            public void run() {

                //phase550

                mSystemServiceManager.startBootPhase(

                    SystemService.PHASE_ACTIVITY_MANAGER_READY);
            }
        });
    }
}
```

```
...  
//phase600  
mSystemServiceManager.startBootPhase(  
    SystemService.PHASE_THIRD_PARTY_APPS_CAN_START);  
}  
}  
}  
}
```

- start: 创建 AMS, PMS, LightsService, DMS.
- phase100: 进入 Phase100, 创建 PKMS, WMS, IMS, DBMS, LockSettingsService, JobSchedulerService, MmsService 等服务;
- phase480 && 500: 进入 Phase480, 调用 WMS, PMS, PKMS, DisplayManagerService 这 4 个服务的 systemReady();
- Phase550: 进入 phase550, 执行 AMS.systemReady(), 启动 SystemUI, WebViewFactory, Watchdog.
- Phase600: 进入 phase600, 执行 AMS.systemReady(), 执行各种服务的 systemRunning().
- Phase1000: 进入 1000, 执行 finishBootting, 启动启动 on-hold 进程.

4.7 AMS.systemReady

```
public final class ActivityManagerService extends ActivityManagerNative implements Watchdog.Monitor, BatteryStatsImpl.BatteryCallback {  
  
    public void systemReady(final Runnable goingCallback) {  
        ... //update 相关  
        mSystemReady = true;  
  
        //杀掉所有非 persistent 进程  
        removeProcessLocked(proc, true, false, "system update done");  
        mProcessesReady = true;  
  
        goingCallback.run(); //见小节 1.6.2
```

```
addAppLocked(info, false, null); //启动所有的 persistent 进程

mBooting = true;

//启动 home

startHomeActivityLocked(mCurrentUserId, "systemReady");

//恢复栈顶的 Activity

mStackSupervisor.resumeTopActivitiesLocked();

}

}
```

System_server 主线程的启动工作,总算完成, 进入 `Looper.loop()`状态,等待其他线程通过 handler 发送消息再处理.

五. app

对于普通的 app 进程,跟 system_server 进程的启动过来有些类似.不同的是 app 进程是向发消息给 system_server 进程, 由 system_server 向 zygote 发出创建进程的请求.

理解 [Android 进程创建流程](#), 可知进程创建后 接下来会进入 `ActivityThread.main()`过程。

5.1 ActivityThread.main

```
public static void main(String[] args) {

    ...

    Environment.initForCurrentUser();

    ...

    Process.setArgV0("<pre-initialized>");

    //创建主线程 looper

    Looper.prepareMainLooper();

}
```

```
        ActivityThread thread = new ActivityThread();  
        thread.attach(false); //attach 到系统进程  
  
        if (sMainThreadHandler == null) {  
            sMainThreadHandler = thread.getHandler();  
        }  
  
        //主线程进入循环状态  
        Looper.loop();  
  
        throw new RuntimeException("Main thread loop unexpectedly exited");  
    }  
}
```

5.2 调用栈对比

App 进程的主线程调用栈的栈底如下：

```
...  
at android.app.ActivityThread.main(ActivityThread.java:5442)  
at java.lang.reflect.Method.invoke!(Native method)  
at com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:738)  
at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:628)
```

跟前面介绍的 `system_server` 进程调用栈对比：

```
at com.android.server.SystemServer.main(SystemServer.java:175)  
at java.lang.reflect.Method.invoke!(Native method)  
at com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:738)  
at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:628)
```

六. 启动日志分析

以下列举启动部分重要进程以及关键节点会打印出的 log

```
/system/bin/vold: 383
/system/bin/lmkd: 432
/system/bin/surfaceflinger: 434
/system/bin/debuggerd64: 537
/system/bin/mediaserver: 540
/system/bin/installld: 541
/system/vendor/bin/thermal-engine: 552

zygote64: 557
zygote: 558
system_server: 1274
```

1. before zygote 日志

```
//启动 vold，再列举当前系统所支持的文件系统。 执行到 system/vold/main.cpp 的 main()11-23 14:36:47.474 383 383 I vold : Vold 3.0 (the awakening) firing up 11-23 14:36:47.475 383 383 V vold : Detected support for: ext4 vfat //使用内核的 lmk 策略11-23 14:36:47.927 432 432 I lowmemorykiller: Using in-kernel low memory killer interface

//启动 SurfaceFlinger
11-23 14:36:48.041 434 434 I SurfaceFlinger: SurfaceFlinger is starting
11-23 14:36:48.042 434 434 I SurfaceFlinger: SurfaceFlinger's main thread ready to run. Initializing graphics H/W...

// 开机动画
11-23 14:36:48.583 508 508 I BootAnimation: bootanimation launching ...

// debuggerd
11-23 14:36:50.306 537 537 I : debuggerd: starting

// installld 启动
```

```
11-23 14:36:50.311 541 541 I installd: installd firing up
// thermal 守护进程
11-23 14:36:50.369 552 552 I ThermalEngine: Thermal daemon started
```

2. zygote 日志

```
// Zygote64 进程(Zygote): AndroidRuntime::start11-23 14:36:51.260 557 557 D
AndroidRuntime: >>>>> START com.android.internal.os.ZygoteInit uid 0 <<<<<<// Z
ygote64 进程: AndroidRuntime::startVm11-23 14:36:51.304 557 557 D AndroidRun
time: CheckJNI is OFF

// 执行 ZygoteInit.preload()11-23 14:36:52.134 557 557 D Zygote : begin prel
oad// 执行 ZygoteInit.preloadClasses(), 预加载 3860 个 classes, 花费时长 746ms11-23
14:36:52.134 557 557 I Zygote : Preloading classes...11-23 14:36:52.881 5
57 557 I Zygote : ...preloaded 3860 classes in 746ms.

// 执行 ZygoteInit.preloadClasses(), 预加载 86 组资源, 花费时长 179ms11-23 14:36:53.
114 557 557 I Zygote : Preloading resources...11-23 14:36:53.293 557 557
I Zygote : ...preloaded 86 resources in 179ms.

// 执行 ZygoteInit.preloadSharedLibraries()11-23 14:36:53.494 557 557 I Zygote
: Preloading shared libraries...11-23 14:36:53.503 557 557 D Zygote : end
preload

// 执行 com_android_internal_os_Zygote_nativeForkSystemServer(), 成功 fork 出 system
_server 进程 11-23 14:36:53.544 557 557 I Zygote : System server process 1274
has been created// Zygote 开始进入 runSelectLoop()11-23 14:36:53.546 557 557
I Zygote : Accepting command socket connections
```

3. system_server 日志

```
//进入 system_server, 建立跟 Zygote 进程的 socket 通道 11-23 14:36:53.586 1274 1274
I Zygote : Process: zygote socket opened, supported ABIS: armeabi-v7a,armeabi//
执行 SystemServer.run()11-23 14:36:53.618 1274 1274 I SystemServer: Entered th
e Android system server! <==> boot_progress_system_run// 等待 installd 准备就绪
11-23 14:36:53.707 1274 1274 I Installer: Waiting for installd to be ready.

//服务启动 11-23 14:36:53.732 1274 1274 I ActivityManager: Memory class: 192

//phase10011-23 14:36:53.883 1274 1274 I SystemServiceManager: Starting phase
10011-23 14:36:53.902 1274 1274 I SystemServer: Package Manager11-23 14:37:03.
816 1274 1274 I SystemServer: User Service
```



```
...11-23 14:37:03.940 1274 1274 I SystemServer: Init Watchdog11-23 14:37:03.94
1 1274 1274 I SystemServer: Input Manager11-23 14:37:03.946 1274 1274 I Syst
emServer: Window Manager

...11-23 14:37:04.081 1274 1274 I SystemServiceManager: Starting com.android.s
erver.MountService$Lifecycle11-23 14:37:04.088 1274 2717 D MountService: Think
ing about reset, mSystemReady=false, mDaemonConnected=true11-23 14:37:04.088 12
74 1274 I SystemServiceManager: Starting com.android.server.UiModeManagerServic
e11-23 14:37:04.520 1274 1274 I SystemServer: NetworkTimeUpdateService

//phase480 && 50011-23 14:37:05.056 1274 1274 I SystemServiceManager: Starting
phase 48011-23 14:37:05.061 1274 1274 I SystemServiceManager: Starting phase
50011-23 14:37:05.231 1274 1274 I ActivityManager: System now ready <==> boot
_progress_ams_ready11-23 14:37:05.234 1274 1274 I SystemServer: Making service
s ready11-23 14:37:05.243 1274 1274 I SystemServer: WebViewFactory preparation

//phase55011-23 14:37:05.234 1274 1274 I SystemServiceManager: Starting phase
55011-23 14:37:05.237 1274 1288 I ActivityManager: Force stopping com.android.
providers.media appid=10010 user=-1: vold reset

//Phase60011-23 14:37:06.066 1274 1274 I SystemServiceManager: Starting phase
60011-23 14:37:06.236 1274 1274 D MountService: onStartUser 0
```

4. logcat 小技巧

通过 adb bugreport 抓取 log 信息.先看 zygote 是否起来, 再看 system_server 主线程的运行情况,再看 ActivityManager 情况

```
adb logcat -s Zygote

adb logcat -s SystemServer

adb logcat -s SystemServiceManager

adb logcat | grep "1359 1359" //system_server 情况

adb logcat -s ActivityManager
```

现场调试命令

1. cat proc/[pid]/stack ==> 查看 kernel 调用栈
2. debuggerd -b [pid] ==> 也不可以不带参数 -b, 则直接输出到 /data/tombstones/目录
3. kill -3 [pid] ==> 生成/data/anr/traces.txt 文件

4. lsof [pid] ==> 查看进程所打开的文件

七. 总结

各大核心进程启动后，都会进入各种对象所相应的 `main()` 方法，如下

进程 main 方法

进程	主方法
init 进程	<code>Init.main()</code>
zygote 进程	<code>ZygoteInit.main()</code>
app_process 进程	<code>RuntimeInit.main()</code>
system_server 进程	<code>SystemServer.main()</code>
app 进程	<code>ActivityThread.main()</code>

注意 `app_process` 进程是指通过 `/system/bin/app_process` 启动的进程，且后面跟的参数不带 `-zygote`，即并非启动 `zygote` 进程。比如常见的有通过 `adb shell` 方式来执行 `am,pm` 等命令，便是这种方式。

重启相关进程

关于重要进程重启的过程，会触发哪些关联进程重启名单：

- `zygote`: 触发 `media`、`netd` 以及子进程(包括 `system_server` 进程)重启；
- `system_server`: 触发 `zygote` 重启；
- `surfaceflinger`: 触发 `zygote` 重启；
- `servicemanager`: 触发 `zygote`、`healthd`、`media`、`surfaceflinger`、`drm` 重启

所以, surfaceflinger, servicemanager, zygote 自身以及 system_server 进程被杀都会触发 Zygote 重启。

第二节 Init 篇

一、概述

init 进程是 Linux 系统中用户空间的第一个进程, 进程号固定为 1。Kernel 启动后, 在用户空间启动 init 进程, 并调用 init 中的 main()方法执行 init 进程的职责。对于 init 进程的功能分为 4 部分:

- 解析并运行所有的 init.rc 相关文件
- 根据 rc 文件, 生成相应的设备驱动节点
- 处理子进程的终止(signal 方式)
- 提供属性服务的功能

接下来从 main()方法说起。

1.1 main

[-> init.cpp]

```
static int epoll_fd = -1;

int main(int argc, char** argv) {
    ...

    //设置文件属性 0777
    umask(0);

    //初始化内核 log, 位于节点/dev/kmsg【见小节 1.2】
    klog_init();

    //设置输出的 log 级别
    klog_set_level(KLOG_NOTICE_LEVEL);
```

```
//创建一块共享的内存空间，用于属性服务【见小节 5.1】

property_init();

//初始化 epoll 功能

epoll_fd = epoll_create1(EPOOL_CLOEXEC);

//初始化子进程退出的信号处理函数，并调用 epoll_ctl 设置 signal fd 可读的回调函数【见
小节 2.1】

signal_handler_init();


//加载 default.prop 文件

property_load_boot_defaults();

//启动属性服务器，此处会调用 epoll_ctl 设置 property fd 可读的回调函数【见小节 5.2】

start_property_service();

//解析 init.rc 文件

init_parse_config_file("/init.rc");


//执行 rc 文件中触发器为 on early-init 的语句

action_for_each_trigger("early-init", action_add_queue_tail);

//等冷插拔设备初始化完成

queue_builtin_action(wait_for_coldboot_done_action, "wait_for_coldboot_done
");

queue_builtin_action(mix_hwrng_into_linux_rng_action, "mix_hwrng_into_linux
_rng");

//设备组合键的初始化操作，此处会调用 epoll_ctl 设置 keychord fd 可读的回调函数

queue_builtin_action(keychord_init_action, "keychord_init");


// 屏幕上显示 Android 静态 Logo 【见小节 1.3】

queue_builtin_action(console_init_action, "console_init");


//执行 rc 文件中触发器为 on init 的语句

action_for_each_trigger("init", action_add_queue_tail);
```

```
queue_builtin_action(mix_hwrng_into_linux_rng_action, "mix_hwrng_into linux
_rng");

char bootmode[PROP_VALUE_MAX];

//当处于充电模式，则 charger 加入执行队列；否则 late-init 加入队列。

if (property_get("ro.bootmode", bootmode) > 0 && strcmp(bootmode, "charger")
== 0)
{
    action_for_each_trigger("charger", action_add_queue_tail);
} else {
    action_for_each_trigger("late-init", action_add_queue_tail);
}

//触发器为属性是否设置

queue_builtin_action(queue_property_triggers_action, "queue_property_trigge
rs");

while (true) {
    if (!waiting_for_exec) {
        execute_one_command();

        //根据需要重启服务【见小节 1.4】

        restart_processes();
    }

    int timeout = -1;

    if (process_needs_restart) {
        timeout = (process_needs_restart - gettime()) * 1000;

        if (timeout < 0)
            timeout = 0;
    }

    if (!action_queue_empty() || cur_action) {
        timeout = 0;
    }
}
```

```
    epoll_event ev;

    //循环等待事件发生

    int nr = TEMP_FAILURE_RETRY(epoll_wait(epoll_fd, &ev, 1, timeout));

    if (nr == -1) {

        ERROR("epoll_wait failed: %s\n", strerror(errno));

    } else if (nr == 1) {

        ((void (*)(void)) ev.data.ptr)();

    }

}

return 0;

}
```

init 进程执行完成后进入循环等待 `epoll_wait` 的状态。

1.2 log 系统

此时 android 的 log 系统还没有启动，采用 kernel 的 log 系统，打开的设备节点 `/dev/kmsg`，那么可通过 `cat /dev/kmsg` 来获取内核 log。

接下来，设置 log 的输出级别为 `KLOG_NOTICE_LEVEL(5)`，当 log 级别小于 5 时则会输出到 kernel log，默认值为 3。

```
#define KLOG_ERROR_LEVEL 3#define KLOG_WARNING_LEVEL 4#define KLOG_NOTICE_LEVEL 5
#define KLOG_INFO_LEVEL 6#define KLOG_DEBUG_LEVEL 7#define KLOG_DEFAULT_LEVEL 3 //默认为 3
```

1.3 console_init_action

[-> init.cpp]

```
static int console_init_action(int nargs, char **args) {
```

```
char console[PROP_VALUE_MAX];

if (property_get("ro.boot.console", console) > 0) {
    snprintf(console_name, sizeof(console_name), "/dev/%s", console);
}

int fd = open(console_name, O_RDWR | O_CLOEXEC);

if (fd >= 0)
    have_console = 1;

close(fd);

fd = open("/dev/tty0", O_WRONLY | O_CLOEXEC);

if (fd >= 0) {
    const char *msg;

    msg = "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        "\n" // console is 40 cols x 30 lines
        "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        "\n"
        " A N D R O I D ";

    write(fd, msg, strlen(msg));

    close(fd);
}
```

```
}
```

```
return 0;
```

```
}
```

这便是开机显示的底部带 ANDROID 字样的画面。

1.4 restart_processes

[-> init.cpp]

```
static void restart_processes() {  
    process_needs_restart = 0;  
    service_for_each_flags(SVC_RESTARTING,  
                           restart_service_if_needed);  
}
```

检查 service_list 中的所有服务，对于带有 SVC_RESTARTING 标志的服务，则都会调用其相应的 restart_service_if_needed。

```
static void restart_service_if_needed(struct service *svc) {  
    time_t next_start_time = svc->time_started + 5;  
  
    if (next_start_time <= gettime()) {  
        svc->flags &= (~SVC_RESTARTING);  
        service_start(svc, NULL);  
        return;  
    }  
  
    if ((next_start_time < process_needs_restart) ||  
        (process_needs_restart == 0)) {
```



```
process_needs_restart = next_start_time;  
  
}  
  
}
```

之后再调用 `service_start` 来启动服务。

接下来，解读 `init` 的 `main` 方法中的 4 大块核心知识点：信号处理、`rc` 文件语法、启动服务以及属性服务。

二、信号处理

在小节[1.1]的 `init.cpp` 的 `main()` 方法中通过 `signal_handler_init()` 来初始化信号处理过程。

主要工作：

- 初始化 `signal` 句柄；
- 循环处理子进程；
- 注册 `epoll` 句柄；
- 处理子进程的终止；

2.1 `signal_handler_init`

[-> `signal_handler.cpp`]

```
void signal_handler_init() {  
  
    int s[2];  
  
    // 创建 socket pair  
  
    if (socketpair(AF_UNIX, SOCK_STREAM | SOCK_NONBLOCK | SOCK_CLOEXEC, 0, s) ==  
        -1) {  
  
        exit(1);  
  
    }  
  
    signal_write_fd = s[0];  
}
```

```
    signal_read_fd = s[1];

    //当捕获信号 SIGCHLD, 则写入 signal_write_fd

    struct sigaction act;

    memset(&act, 0, sizeof(act));

    act.sa_handler = SIGCHLD_handler;

    //SA_NOCLDSTOP 使 init 进程只有在其子进程终止时才会受到 SIGCHLD 信号

    act.sa_flags = SA_NOCLDSTOP;

    sigaction(SIGCHLD, &act, 0);


    //进入 waitpid 来处理子进程是否退出的情况【见小节 2.2】

    reap_any_outstanding_children();

    //调用 epoll_ctl 方法来注册 epoll 的回调函数【见小节 2.3】

    register_epoll_handler(signal_read_fd, handle_signal);
}
```

每个进程在处理其他进程发送的 **signal** 信号时都需要先注册, 当进程的运行状态改变或终止时会产生某种 **signal** 信号, **init** 进程是所有用户空间进程的父进程, 当其子进程终止时产生 **SIGCHLD** 信号, **init** 进程调用信号安装函数 **sigaction()**, 传递参数给 **sigaction** 结构体, 便完成信号处理的过程。

这里有两个重要的函数: **SIGCHLD_handler** 和 **handle_signal**, 如下:

```
//写入数据 static void SIGCHLD_handler(int) {

    //向 signal_write_fd 写入 1, 直到成功为止

    if (TEMP_FAILURE_RETRY(write(signal_write_fd, "1", 1)) == -1) {

        ERROR("write(signal_write_fd) failed: %s\n", strerror(errno));

    }

}

//读取数据 static void handle_signal() {

    char buf[32];

    //读取 signal_read_fd 中的数据, 并放入 buf

    read(signal_read_fd, buf, sizeof(buf));
}
```

```
reap_any_outstanding_children(); 【见小节 2.2】  
}
```

2.2 reap_any_outstanding_children

[-> signal_handler.cpp]

```
static void reap_any_outstanding_children() {  
    while (wait_for_one_process()) { }  
}  
  
static bool wait_for_one_process() {  
    int status;  
  
    //等待任意子进程，如果子进程没有退出则返回 0，否则返回该子进程 pid。  
  
    pid_t pid = TEMP_FAILURE_RETRY(waitpid(-1, &status, WNOHANG));  
  
    if (pid == 0) {  
        return false;  
    } else if (pid == -1) {  
        return false;  
    }  
  
    //根据 pid 查找到相应的 service  
    service* svc = service_find_by_pid(pid);  
  
    std::string name;  
  
    if (!svc) {  
        return true;  
    }  
  
    //当 flags 为 RESTART，且不是 ONESHOT 时，先 kill 进程组内所有的子进程或子线程  
    if (!(svc->flags & SVC_ONESHOT) || (svc->flags & SVC_RESTART)) {  
        kill(-pid, SIGKILL);  
    }  
}
```

```
}

//移除当前服务 svc 中的所有创建过的 socket
for (socketinfo* si = svc->sockets; si; si = si->next) {
    char tmp[128];
    snprintf(tmp, sizeof(tmp), ANDROID_SOCKET_DIR"/%s", si->name);
    unlink(tmp);
}

//当 flags 为 EXEC 时，释放相应的服务
if (svc->flags & SVC_EXEC) {
    waiting_for_exec = false;
    list_remove(&svc->slist);
    free(svc->name);
    free(svc);
    return true;
}

svc->pid = 0;
svc->flags &= (~SVC_RUNNING);

//对于 ONESHOT 服务，使其进入 disabled 状态
if ((svc->flags & SVC_ONESHOT) && !(svc->flags & SVC_RESTART)) {
    svc->flags |= SVC_DISABLED;
}

//禁用和重置的服务，都不再自动重启
if (svc->flags & (SVC_DISABLED | SVC_RESET)) {
    svc->NotifyStateChange("stopped"); //设置相应的 service 状态为 stopped
    return true;
}
```

```
//服务在 4 分钟内重启次数超过 4 次，则重启手机进入 recovery 模式

time_t now = gettimeofday();

if ((svc->flags & SVC_CRITICAL) && !(svc->flags & SVC_RESTART)) {

    if (svc->time_crashed + CRITICAL_CRASH_WINDOW >= now) {

        if (++svc->nr_crashed > CRITICAL_CRASH_THRESHOLD) {

            android_reboot(ANDROID_RB_RESTART2, 0, "recovery");

            return true;

        }

    } else {

        svc->time_crashed = now;

        svc->nr_crashed = 1;

    }

}

svc->flags &= (~SVC_RESTART);

svc->flags |= SVC_RESTARTING;

//执行当前 service 中所有 onrestart 命令

struct listnode* node;

list_for_each(node, &svc->onrestart.commands) {

    command* cmd = node_to_item(node, struct command, clist);

    cmd->func(cmd->nargs, cmd->args);

}

//设置相应的 service 状态为 restarting

svc->NotifyStateChange("restarting");

return true;

}
```

另外：通过 `getprop | grep init.svc` 可查看所有 service 运行状态。状态总共分为：running, stopped, restarting

2.3 register_epoll_handler

[-> signal_handler.cpp]

```
void register_epoll_handler(int fd, void (*fn)()) {  
    epoll_event ev;  
    ev.events = EPOLLIN;  
    ev.data.ptr = reinterpret_cast<void*>(fn);  
    //将 fd 的可读事件加入到 epoll_fd 的监听队列中  
    if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, fd, &ev) == -1) {  
        ERROR("epoll_ctl failed: %s\n", strerror(errno));  
    }  
}
```

当 fd 可读，则会触发调用(*fn)函数。

三、rc 文件语法

rc 文件语法是以行尾单位，以空格间隔的语法，以#开始代表注释行。rc 文件主要包含 Action、Service、Command、Options，其中对于 Action 和 Service 的名称都是唯一的，对于重复的命名视为无效。

3.1 Action

Action: 通过触发器 trigger，即以 on 开头的语句来决定执行相应的 service 的时机，具体有如下时机：

- on early-init; 在初始化早期阶段触发；
- on init; 在初始化阶段触发；
- on late-init; 在初始化晚期阶段触发；
- on boot/charger: 当系统启动/充电时触发，还包含其他情况，此处不一一列举；
- on property:<key>=<value>; 当属性值满足条件时触发；

3.2 Service

服务 **Service**，以 **service** 开头，由 **init** 进程启动，一般运行在 **init** 的一个子进程，所以启动 **service** 前需要判断对应的可执行文件是否存在。**init** 生成的子进程，定义在 **rc** 文件，其中每一个 **service** 在启动时会通过 **fork** 方式生成子进程。

例如：`service servicemanager /system/bin/servicemanager` 代表的是服务名为 **servicemanager**，服务执行的路径为 **/system/bin/servicemanager**。

3.3 Command

下面列举常用的命令

- **class_start <service_class_name>**: 启动属于同一个 **class** 的所有服务;
- **start <service_name>**: 启动指定的服务，若已启动则跳过;
- **stop <service_name>**: 停止正在运行的服务
- **setprop <name> <value>**: 设置属性值
- **mkdir <path>**: 创建指定目录
- **symlink <target> <sym_link>**: 创建连接到<target>的<sym_link>符号链接;
- **write <path> <string>**: 向文件 **path** 中写入字符串;
- **exec**: **fork** 并执行，会阻塞 **init** 进程直到程序完毕;
- **export <name> <name>**: 设定环境变量;
- **loglevel <level>**: 设置 **log** 级别

3.4 Options

Options 是 **Service** 的可选项，与 **service** 配合使用

- **disabled**: 不随 **class** 自动启动，只有根据 **service** 名才启动;
- **oneshot**: **service** 退出后不再重启;
- **user/group**: 设置执行服务的用户/用户组，默认都是 **root**;
- **class**: 设置所属的类名，当所属类启动/退出时，服务也启动/停止，默认为 **default**;
- **onrestart**: 当服务重启时执行相应命令;
- **socket**: 创建名为 **/dev/socket/<name>** 的 **socket**
- **critical**: 在规定时间内该 **service** 不断重启，则系统会重启并进入恢复模式

default: 意味着 disabled=false, oneshot=false, critical=false。

四、启动服务

4.1 启动顺序

```
on early-init

on init

on late-init

    trigger post-fs

    trigger load_system_props_action

    trigger post-fs-data

    trigger load_persist_props_action

    trigger firmware_mounts_complete

    trigger boot


on post-fs      //挂载文件系统

    start logd

    mount rootfs rootfs / ro remount

    mount rootfs rootfs / shared rec

    mount none /mnt/runtime/default /storage slave bind rec

    ...

on post-fs-data //挂载 data

    start logd

    start vold    //启动 vold

    ...

on boot        //启动核心服务

    ...

class_start core //启动 core class
```


触发器的执行顺序为 on early-init -> init -> late-init，从上面的代码可知，在 late-init 触发器中会触发文件系统挂载以及 on boot。再 on boot 过程会触发启动 core class。至于 main class 的启动是由 vold.decrypt 的以下 4 个值的设置所决定的，该过程位于 system/vold/cryptfs.c 文件。

```
on nonencrypted
    class_start main
    class_start late_start
on property:vold.decrypt=trigger_restart_min_framework
    class_start main
on property:vold.decrypt=trigger_restart_framework
    class_start main
    class_start late_start
on property:vold.decrypt=trigger_reset_main
    class_reset main
on property:vold.decrypt=trigger_shutdown_framework
    class_reset late_start
    class_reset main
```

4.2 服务启动(Zygote)

在 init.zygote.rc 文件中，zygote 服务定义如下：

```
service zygote /system/bin/app_process -Xzygote /system/bin --zygote --start-system-server
    class main
    socket zygote stream 660 root system
    onrestart write /sys/android_power/request_state wake
    onrestart write /sys/power/state on
    onrestart restart media
    onrestart restart netd
```

通过 `init_parser.cpp` 完成整个 `service` 解析工作，此处就不详细展开讲解解析过程，该过程主要工作是：

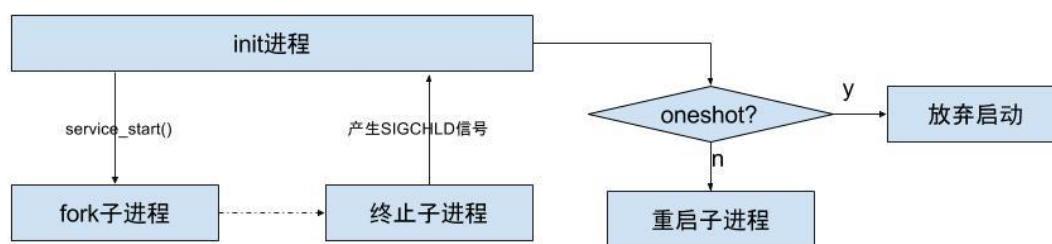
- 创建一个名叫“zygote”的 `service` 结构体；
- 创建一个用于 `socket` 通信的 `socketinfo` 结构体；
- 创建一个包含 4 个 `onrestart` 的 `action` 结构体。

Zygote 服务会随着 `main class` 的启动而启动，退出后会由 `init` 重启 `zygote`，即使多次重启也不会进入 `recovery` 模式。`zygote` 所对应的可执行文件是 `/system/bin/app_process`，通过调用 `pid = fork()` 创建子进程，通过 `execve(svc->args[0], (char**)svc->args, (char**) ENV)`，进入 `App_main.cpp` 的 `main()` 函数。故 `zygote` 是通过 `fork` 和 `execv` 共同创建的。

流程如下：

而关于 `Zygote` 重启在前面的信号处理过程中讲过，是处理 `SIGCHLD` 信号，`init` 进程重启 `zygote` 进程，更多关于 `Zygote` 内容见 [Zygote 篇](#)。

4.3 服务重启



当 `init` 子进程退出时，会产生 `SIGCHLD` 信号，并发送给 `init` 进程，通过 `socket` 套接字传递数据，调用到 `wait_for_one_process()` 方法，根据是否是 `oneshot`，来决定是重启子进程，还是放弃启动。

所有的 `Service` 里面只有 `servicemanager`，`zygote`，`surfaceflinger` 这 3 个服务有 `onrestart` 关键字来触发其他 `service` 启动过程。

```
//zygote 可触发 media、netd 重启

service zygote /system/bin/app_process -Xzygote /system/bin --zygote --start-system-server

    class main

    socket zygote stream 660 root system

    onrestart write /sys/android_power/request_state wake

    onrestart write /sys/power/state on

    onrestart restart media

    onrestart restart netd


//servicemanager 可触发 healthd、zygote、media、surfaceflinger、drm 重启

service servicemanager /system/bin/servicemanager

    class core

    user system

    group system

    critical

    onrestart restart healthd

    onrestart restart zygote

    onrestart restart media

    onrestart restart surfaceflinger

    onrestart restart drm


//surfaceflinger 可触发 zygote 重启

service surfaceflinger /system/bin/surfaceflinger

    class core

    user system

    group graphics drmrpc

    onrestart restart zygote
```

由上可知：

- zygote: 触发 media、netd 以及子进程(包括 system_server 进程)重启;
- system_server: 触发 zygote 重启;
- surfaceflinger: 触发 zygote 重启;
- servicemanager: 触发 zygote、healthd、media、surfaceflinger、drm 重启

所以, surfaceflinger,servicemanager,zygote 自身以及 system_server 进程被杀都会触发 Zygote 重启。

五、属性服务

当某个进程 A, 通过 `property_set()` 修改属性值后, `init` 进程会检查访问权限, 当权限满足要求后, 则更改相应的属性值, 属性值一旦改变则会触发相应的触发器 (即 `rc` 文件中的 `on` 开头的语句), 在 **Android Shared Memmory** (共享内存区域) 中有一个 `_system_property_area` 区域, 里面记录着所有的属性值。对于进程 A 通过 `property_get()` 方法, 获取的也是该共享内存区域的属性值。

5.1 property_init

[-> `property_service.cpp`]

```
void property_init() {  
    //用于保证只初始化_system_property_area_区域一次  
    if (property_area_initialized) {  
        return;  
    }  
    property_area_initialized = true;  
    //创建共享内存  
    if (__system_property_area_init()) {  
        return;  
    }  
    pa_workspace.size = 0;  
    pa_workspace.fd = open(PROP_FILENAME, O_RDONLY | O_NOFOLLOW | O_CLOEXEC);
```



该方法核心功能在执行__system_property_area_init()方法，创建用于跨进程的共享内存。主要工作如下：

- 执行 open()，打开名为“/dev/properties”的共享内存文件，并设置大小为 128KB；
- 执行 mmap()，将该内存映射到 init 进程；
- 将该内存的首地址保存在全局变量__system_property_area__，后续的增加或者修改属性都基于该变量来计算位置。

关于加载的 prop 文件

通过 load_all_load_all_propsprops()方法，加载以下：

1. /system/build.prop;
2. /vendor/build.prop;
3. /factory/factory.prop;
4. /data/local.prop;
5. /data/property 路径下的 persist 属性

5.2 start_property_service

[-> property_service.cpp]

```
void start_property_service() {  
  
    property_set("ro.property_service.version", "2");  
  
    property_set_fd = CreateSocket(PROP_SERVICE_NAME, SOCK_STREAM | SOCK_CLOEXEC  
    | SOCK_NONBLOCK,  
  
    false, 0666, 0, 0, nullptr, sehandle);  
  
    listen(property_set_fd, 8);  
  
    //设置 property 文件描述符可读的回调函数【见小节 2.3】  
  
    register_epoll_handler(property_set_fd, handle_property_set_fd);  
  
}
```

创建并监听名叫“property_service”的 socket，再利用 epoll_ctl 设置 property 文件描述符触发可读时的回调函数为 handle_property_set_fd，接下来看看该函数的实现。

5.3 handle_property_set_fd

```
static void handle_property_set_fd() {  
  
    static constexpr uint32_t kDefaultSocketTimeout = 2000; /* ms */  
  
    int s = accept4(property_set_fd, nullptr, nullptr, SOCK_CLOEXEC);  
  
    struct ucred cr;  
    socklen_t cr_size = sizeof(cr);  
    getsockopt(s, SOL_SOCKET, SO_PEERCRED, &cr, &cr_size) < 0;  
  
    SocketConnection socket(s, cr);  
    uint32_t timeout_ms = kDefaultSocketTimeout; //设置 2 秒超时  
  
    uint32_t cmd = 0;  
    if (!socket.RecvUint32(&cmd, &timeout_ms)) {  
        socket.SendUint32(PROP_ERROR_READ_CMD);  
        return;  
    }  
  
    switch (cmd) {  
        case PROP_MSG_SETPROP: {  
            char prop_name[PROP_NAME_MAX];  
            char prop_value[PROP_VALUE_MAX];  
  
            if (!socket.RecvChars(prop_name, PROP_NAME_MAX, &timeout_ms) ||  
                !socket.RecvChars(prop_value, PROP_VALUE_MAX, &timeout_ms)) {
```

```
        return;
    }

    prop_name[PROP_NAME_MAX-1] = 0;
    prop_value[PROP_VALUE_MAX-1] = 0;

    //设置 property【见小节 5.4】
    handle_property_set(socket, prop_value, prop_value, true);

    break;
}

case PROP_MSG_SETPROP2: {
    std::string name;
    std::string value;

    if (!socket.RecvString(&name, &timeout_ms) ||
        !socket.RecvString(&value, &timeout_ms)) {
        socket.SendUint32(PROP_ERROR_READ_DATA);
        return;
    }

    //设置 property【见小节 5.4】
    handle_property_set(socket, name, value, false);

    break;
}

default:
    socket.SendUint32(PROP_ERROR_INVALID_CMD);

    break;
}
}
```

这里针对 **socket** 接收事件设置 2 秒超时，也就是说 **property** 的设置过程有可能耗时。

5.4 handle_property_set

```
static void handle_property_set(SocketConnection& socket,
                                const std::string& name,
                                const std::string& value,
                                bool legacy_protocol) {
    const char* cmd_name = legacy_protocol ? "PROP_MSG_SETPROP" : "PROP_MSG_SETPROP2";

    if (!is_legal_property_name(name)) { //检查属性名是否合规
        socket.SendUint32(PROP_ERROR_INVALID_NAME);
        return;
    }

    struct ucred cr = socket.cred();
    char* source_ctx = nullptr;
    getpeercon(socket.socket(), &source_ctx);

    if (android::base::StartsWith(name, "ctl.")) {
        if (check_control_mac_perms(value.c_str(), source_ctx, &cr)) {
            //处理以 ctl.开头的属性
            handle_control_message(name.c_str() + 4, value.c_str());
        }
        ...
    } else {
        if (check_perms(name, source_ctx, &cr)) {
            //设置属性名和属性值
            uint32_t result = property_set(name, value);
        }
    }
}
```



```
...  
}  
  
freecon(source_ctx);  
}
```

这里会检测属性名是否合规，具体检查规范如下：

```
bool is_legal_property_name(const std::string& name) {  
    size_t namelen = name.size();  
  
    if (namelen < 1) return false;  
    if (name[0] == '.') return false;  
    if (name[namelen - 1] == '.') return false;  
  
    /* Only allow alphanumeric, plus '.', '-', '@', ':', or '_' */  
    /* Don't allow ".." to appear in a property name */  
    for (size_t i = 0; i < namelen; i++) {  
        if (name[i] == '.') {  
            // i=0 is guaranteed to never have a dot. See above.  
            if (name[i-1] == '.') return false;  
            continue;  
        }  
        if (name[i] == '_' || name[i] == '-' || name[i] == '@' || name[i] == ':')  
            continue;  
        if (name[i] >= 'a' && name[i] <= 'z') continue;  
        if (name[i] >= 'A' && name[i] <= 'Z') continue;  
        if (name[i] >= '0' && name[i] <= '9') continue;  
        return false;  
    }  
  
    return true;  
}
```

```
}

uint32_t property_set(const std::string& name, const std::string& value) {
    return PropertySetImpl(name, value);
}

static uint32_t PropertySetImpl(const std::string& name, const std::string& value) {
    size_t valuelen = value.size();

    if (!is_legal_property_name(name)) {
        return PROP_ERROR_INVALID_NAME;
    }

    if (valuelen >= PROP_VALUE_MAX) { //属性名不可过长
        return PROP_ERROR_INVALID_VALUE;
    }

    prop_info* pi = (prop_info*) __system_property_find(name.c_str());
    if (pi != nullptr) {
        // 以 ro.开头的属性不可更改
        if (android::base::StartsWith(name, "ro.")
            && strcmp(name.c_str(), "ro.build.software.version")) {
            return PROP_ERROR_READ_ONLY_PROPERTY;
        }
        //更新属性
        __system_property_update(pi, value.c_str(), valuelen);
    } else {
        //添加属性
        int rc = __system_property_add(name.c_str(), name.size(), value.c_str(),
            valuelen);
    }
}
```

```
//以 persist.开头的属性需要持久化

if (persistent_properties_loaded && android::base::StartsWith(name, "persist.
")) {

    write_persistent_property(name.c_str(), value.c_str());

}

//属性值改变的通知过程

property_changed(name, value);

return PROP_SUCCESS;

}
```

不同属性执行逻辑有所不同，主要区分如下：

- 属性名以 **ctl.** 开头，则表示是控制消息，控制消息用来执行一些命令。
例如：
 - setprop ctl.start bootanim 查看开机动画；
 - setprop ctl.stop bootanim 关闭开机动画；
 - setprop ctl.start pre-recovery 进入 recovery 模式；
- 属性名以 **ro.** 开头，则表示是只读的，不能设置，所以直接返回；
- 属性名以 **persist.** 开头，则需要把这些值写到对应文件；需要注意的是，**persist** 用于持久化保存某些属性值，当同时也带来了额外的 IO 操作。

六、总结

init 进程(pid=1)是 Linux 系统中用户空间的第一个进程，主要工作如下：

- 创建一块共享的内存空间，用于属性服务器；
- 解析各个 rc 文件，并启动相应属性服务进程；
- 初始化 epoll，依次设置 signal、property、keychord 这 3 个 fd 可读时相对应的回调函数；
- 进入无限循环状态，执行如下流程：
 - 检查 action_queue 列表是否为空，若不为空则执行相应的 action；
 - 检查是否需要重启的进程，若有则将其重新启动；

- 进入 `epoll_wait` 等待状态，直到系统属性变化事件(`property_set` 改变属性值)，或者收到子进程的信号 `SIGCHLD`，又或者 `keychord` 键盘输入事件，则会退出等待状态，执行相应的回调函数。

可见 `init` 进程在开机之后的核心工作就是响应 `property` 变化事件和回收僵尸进程。当某个进程调用 `property_set` 来改变一个系统属性值时，系统会通过 `socket` 向 `init` 进程发送一个 `property` 变化的事件通知，那么 `property fd` 会变成可读，`init` 进程采用 `epoll` 机制监听该 `fd` 则会 触发回调 `handle_property_set_fd()` 方法。回收僵尸进程，在 `Linux` 内核中，如父进程不等待子进程的结束直接退出，会导致子进程在结束后变成僵尸进程，占用系统资源。为此，`init` 进程专门安装了 `SIGCHLD` 信号接收器，当某些子进程退出时发现其父进程已经退出，则会向 `init` 进程发送 `SIGCHLD` 信号，`init` 进程调用回调方法 `handle_signal()` 来回收僵尸子进程。

第三节 zygote 篇

一. 概述

`Zygote` 是由 `init` 进程通过解析 `init.zygote.rc` 文件而创建的，`zygote` 所对应的可执行程序 `app_process`，所对应的源文件是 `App_main.cpp`，进程名为 `zygote`。

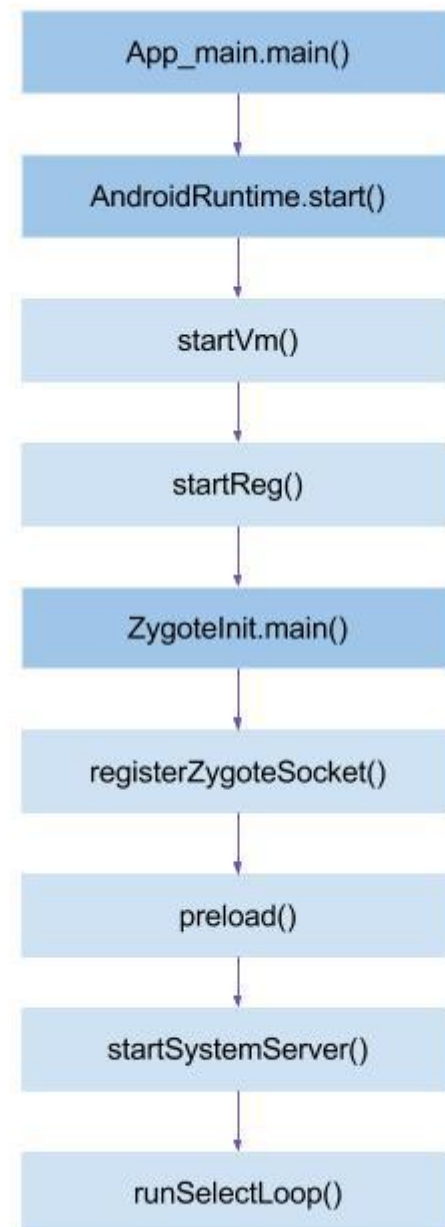
```
service zygote /system/bin/app_process -Xzygote /system/bin --zygote --start-system-server
    class main
        socket zygote stream 660 root system
        onrestart write /sys/android_power/request_state wake
        onrestart write /sys/power/state on
        onrestart restart media
        onrestart restart netd
```

`Zygote` 进程能够重启的地方：

- `servicemanager` 进程被杀; (`onresart`)
- `surfaceflinger` 进程被杀; (`onresart`)
- `Zygote` 进程自己被杀; (`oneshot=false`)

- system_server 进程被杀; (waitpid)

从 App_main()开始, Zygote 启动过程的函数调用类大致流程如下:



二、Zygote 启动过程

2.1 App_main.main

[-> App_main.cpp]

```
int main(int argc, char* const argv[]){  
    //传到的参数 argv 为“-Xzygote /system/bin --zygote --start-system-server”  
    AppRuntime runtime(argv[0], computeArgBlockSize(argc, argv));  
    argc--; argv++; //忽略第一个参数  
  
    int i;  
    for (i = 0; i < argc; i++) {  
        if (argv[i][0] != '-') {  
            break;  
        }  
        if (argv[i][1] == '-' && argv[i][2] == 0) {  
            ++i;  
            break;  
        }  
        runtime.addOption(strdup(argv[i]));  
    }  
    //参数解析  
    bool zygote = false;  
    bool startSystemServer = false;  
    bool application = false;  
    String8 niceName;  
    String8 className;  
    ++i;  
    while (i < argc) {  
        const char* arg = argv[i++];  
        if (strcmp(arg, "--zygote") == 0) {  
            zygote = true;  
            //对于 64 位系统 nice_name 为 zygote64; 32 位系统为 zygote
```

```
        niceName = ZYGOTE_NICE_NAME;

    } else if (strcmp(arg, "--start-system-server") == 0) {

        startSystemServer = true;

    } else if (strcmp(arg, "--application") == 0) {

        application = true;

    } else if (strncmp(arg, "--nice-name=", 12) == 0) {

        niceName.setTo(arg + 12);

    } else if (strncmp(arg, "--", 2) != 0) {

        className.setTo(arg);

        break;

    } else {

        --i;

        break;

    }

}

Vector<String8> args;

if (!className.isEmpty()) {

    // 运行 application 或 tool 程序

    args.add(application ? String8("application") : String8("tool"));

    runtime.setClassNameAndArgs(className, argc - i, argv + i);

} else {

    //进入 zygote 模式，创建 /data/dalvik-cache 路径

    maybeCreateDalvikCache();

    if (startSystemServer) {

        args.add(String8("start-system-server"));

    }

    char prop[PROP_VALUE_MAX];

    if (property_get(ABI_LIST_PROPERTY, prop, NULL) == 0) {

        return 11;

    }

}
```

```
String8 abiFlag("--abi-list=");

abiFlag.append(prop);

args.add(abiFlag);


for (; i < argc; ++i) {
    args.add(String8(argv[i]));
}

}

//设置进程名

if (!niceName.isEmpty()) {
    runtime.setArgv0(niceName.string());
    set_process_name(niceName.string());
}

if (zygote) {
    // 启动 AppRuntime 【见小节 2.2】
    runtime.start("com.android.internal.os.ZygoteInit", args, zygote);
} else if (className) {
    runtime.start("com.android.internal.os.RuntimeInit", args, zygote);
} else {
    //没有指定类名或 zygote，参数错误

    return 10;
}

}
```

2.2 start

[-> AndroidRuntime.cpp]


```
void AndroidRuntime::start(const char* className, const Vector<String8>& options,
    bool zygote)
{
    static const String8 startSystemServer("start-system-server");

    for (size_t i = 0; i < options.size(); ++i) {
        if (options[i] == startSystemServer) {
            const int LOG_BOOT_PROGRESS_START = 3000;

        }
    }

    const char* rootDir = getenv("ANDROID_ROOT");
    if (rootDir == NULL) {
        rootDir = "/system";
        if (!hasDir("/system")) {
            return;
        }
        setenv("ANDROID_ROOT", rootDir, 1);
    }

    JniInvocation jni_invocation;
    jni_invocation.Init(NULL);
    JNIEnv* env;
    // 虚拟机创建【见小节 2.3】
    if (startVm(&mJavaVM, &env, zygote) != 0) {
        return;
    }
    onVmCreated(env);
    // JNI 方法注册【见小节 2.4】
    if (startReg(env) < 0) {
        return;
    }
}
```

```
jclass stringClass;

jobjectArray strArray;

jstring classNameStr;


//等价 strArray= new String[options.size() + 1];
stringClass = env->FindClass("java/lang/String");
strArray = env->NewObjectArray(options.size() + 1, stringClass, NULL);


//等价 strArray[0] = "com.android.internal.os.ZygoteInit"
classNameStr = env->NewStringUTF(className);
env->SetObjectArrayElement(strArray, 0, classNameStr);


//等价 strArray[1] = "start-system-server";
// strArray[2] = "--abi-list=xxx";
//其中 xxx 为系统响应的 cpu 架构类型, 比如 arm64-v8a.
for (size_t i = 0; i < options.size(); ++i) {
    jstring optionsStr = env->NewStringUTF(options.itemAt(i).string());
    env->SetObjectArrayElement(strArray, i + 1, optionsStr);
}


//将"com.android.internal.os.ZygoteInit"转换为"com/android/internal/os/ZygoteInit"
char* slashClassName = toSlashClassName(className);
jclass startClass = env->FindClass(slashClassName);
if (startClass == NULL) {
    ...
} else {
    jmethodID startMeth = env->GetStaticMethodID(startClass, "main",
        "([Ljava/lang/String;)V");
```

```
// 调用 ZygoteInit.main()方法【见小节 3.1】
env->CallStaticVoidMethod(startClass, startMeth, strArray);
}

//释放相应对象的内存空间
free(slashClassName);

mJavaVM->DetachCurrentThread();

mJavaVM->DestroyJavaVM();
}
```

2.3 startVm

[→ AndroidRuntime.cpp]

创建 Java 虚拟机方法的主要篇幅是关于虚拟机参数的设置，下面只列举部分在调试优化过程中常用参数。

```
int AndroidRuntime::startVm(JavaVM** pJavaVM, JNIEnv** pEnv, bool zygote)
{
    // JNI 检测功能，用于 native 层调用 jni 函数时进行常规检测，比较弱字符串格式是否符合
    // 要求，资源是否正确释放。该功能一般用于早期系统调试或手机 Eng 版，对于 User 版往往不会开启，
    // 引用该功能比较消耗系统 CPU 资源，降低系统性能。
    bool checkJni = false;
    property_get("dalvik.vm.checkjni", propBuf, "");
    if (strcmp(propBuf, "true") == 0) {
        checkJni = true;
    } else if (strcmp(propBuf, "false") != 0) {
        property_get("ro.kernel.android.checkjni", propBuf, "");
        if (propBuf[0] == '1') {
            checkJni = true;
        }
    }
}
```

```
    if (checkJni) {
        addOption("-Xcheck:jni");
    }

    //虚拟机产生的 trace 文件，主要用于分析系统问题，路径默认为/data/anr/traces.txt
    parseRuntimeOption("dalvik.vm.stack-trace-file", stackTraceFileBuf, "-Xstack
    tracefile:");

    //对于不同的软硬件环境，这些参数往往需要调整、优化，从而使系统达到最佳性能
    parseRuntimeOption("dalvik.vm.heapstartsize", heapstartsizeOptsBuf, "-Xms",
    "4m");
    parseRuntimeOption("dalvik.vm.heapsize", heapsizeOptsBuf, "-Xmx", "16m");
    parseRuntimeOption("dalvik.vm.heapgrowthlimit", heapgrowthlimitOptsBuf, "-XX:
    HeapGrowthLimit=");
    parseRuntimeOption("dalvik.vm.heapminfree", heapminfreeOptsBuf, "-XX:HeapMin
    Free=");
    parseRuntimeOption("dalvik.vm.heapmaxfree", heapmaxfreeOptsBuf, "-XX:HeapMax
    Free=");
    parseRuntimeOption("dalvik.vm.heaptargetutilization",
        heaptargetutilizationOptsBuf, "-XX:HeapTargetUtilization=
    ");
    ...

    //preloaded-classes 文件内容是由 WritePreloadedClassFile.java 生成的，
    //在 ZygoteInit 类中会预加载工作将其中的 classes 提前加载到内存，以提高系统性能
    if (!hasFile("/system/etc/preloaded-classes")) {
        return -1;
    }

    //初始化虚拟机
    if (JNI_CreateJavaVM(pJavaVM, pEnv, &initArgs) < 0) {
        ALOGE("JNI_CreateJavaVM failed\n");
    }
}
```

```
    return -1;
}
}
```

2.4 startReg

[-> AndroidRuntime.cpp]

```
int AndroidRuntime::startReg(JNIEnv* env)
{
    //设置线程创建方法为 javaCreateThreadEtc 【见小节 2.4.1】
    androidSetCreateThreadFunc((android_create_thread_fn) javaCreateThreadEtc);

    env->PushLocalFrame(200);

    //进程 NI 方法的注册【见小节 2.4.2】
    if (register_jni_procs(gRegJNI, NELEM(gRegJNI), env) < 0) {
        env->PopLocalFrame(NULL);
        return -1;
    }

    env->PopLocalFrame(NULL);

    return 0;
}
```

2.4.1 androidSetCreateThreadFunc

[-> Threads.cpp]

```
void androidSetCreateThreadFunc(android_create_thread_fn func) {
    gCreateThreadFn = func;
}
```

虚拟机启动后 `startReg()` 过程，会设置线程创建函数指针 `gCreateThreadFn` 指向 `javaCreateThreadEtc`。

2.4.2 register_jni_procs

```
static int register_jni_procs(const RegJNIRec array[], size_t count, JNIEnv* env) {
    for (size_t i = 0; i < count; i++) {
        // 【见小节 2.4.3】
        if (array[i].mProc(env) < 0) {
            return -1;
        }
    }
    return 0;
}
```

2.4.3 gRegJNI.mProc

```
static const RegJNIRec gRegJNI[] = {
    REG_JNI(register_com_android_internal_os_RuntimeInit),
    REG_JNI(register_android_os_Binder),
    ...
};
```

`array[i]` 是指 `gRegJNI` 数组，该数组有 100 多个成员。其中每一项成员都是通过 `REG_JNI` 宏定义的：

```
#define REG_JNI(name) { name } struct RegJNIRec {
    int (*mProc)(JNIEnv*);
};
```

可见，调用 `mProc`，就等价于调用其参数名所指向的函数。例如 `REG_JNI(register_com_android_internal_os_RuntimeInit).mProc` 也就是指进入 `register_com_android_internal_os_RuntimeInit` 方法，接下来就继续以此为例来说明：

```
int register_com_android_internal_os_RuntimeInit(JNIEnv* env) {

    return jniRegisterNativeMethods(env, "com/android/internal/os/RuntimeInit",
        gMethods, NELEM(gMethods));
}

//gMethods: java 层方法名与 jni 层的方法的一一映射关系 static JNINativeMethod gMethods[] = {

    { "nativeFinishInit", "()V",
        (void*) com_android_internal_os_RuntimeInit_nativeFinishInit },
    { "nativeZygoteInit", "()V",
        (void*) com_android_internal_os_RuntimeInit_nativeZygoteInit },
    { "nativeSetExitWithoutCleanup", "(Z)V",
        (void*) com_android_internal_os_RuntimeInit_nativeSetExitWithoutCleanup
    },
};
```

三. 进入 Java 层

前面[小节 2.2]AndroidRuntime.start()执行到最后通过反射调用到 `ZygoteInit.main()`,见下文：

3.1 ZygoteInit.main

[→ZygoteInit.java]

```
public static void main(String argv[]) {

    try {

        RuntimeInit.enableDdms(); //开启 DDMS 功能
```

```
SamplingProfilerIntegration.start();

boolean startSystemServer = false;

String socketName = "zygote";

String abiList = null;

for (int i = 1; i < argv.length; i++) {

    if ("start-system-server".equals(argv[i])) {

        startSystemServer = true;

    } else if (argv[i].startsWith(ABI_LIST_ARG)) {

        abiList = argv[i].substring(ABI_LIST_ARG.length());

    } else if (argv[i].startsWith(SOCKET_NAME_ARG)) {

        socketName = argv[i].substring(SOCKET_NAME_ARG.length());

    } else {

        throw new RuntimeException("Unknown command line argument: " + argv[i]);

    }

}

...

registerZygoteSocket(socketName); //为 Zygote 注册 socket【见小节 3.2】

preload(); // 预加载类和资源【见小节 3.3】

SamplingProfilerIntegration.writeZygoteSnapshot();

gcAndFinalize(); //GC 操作

if (startSystemServer) {

    startSystemServer(abiList, socketName); //启动 system_server【见小节 3.4】

}

runSelectLoop(abiList); //进入循环模式【见小节 3.5】

closeServerSocket();

} catch (MethodAndArgsCaller caller) {

    caller.run(); //启动 system_server 中会讲到。

} catch (RuntimeException ex) {
```



```
        closeServerSocket();  
        throw ex;  
    }  
}
```

在异常捕获后调用的方法 `caller.run()`，会在后续的 `system_server` 文章会讲到。

3.2 registerZygoteSocket

[→ZygoteInit.java]

```
private static void registerZygoteSocket(String socketName) {  
    if (sServerSocket == null) {  
        int fileDesc;  
        final String fullSocketName = ANDROID_SOCKET_PREFIX + socketName;  
        try {  
            String env = System.getenv(fullSocketName);  
            fileDesc = Integer.parseInt(env);  
        } catch (RuntimeException ex) {  
            ...  
        }  
  
        try {  
            FileDescriptor fd = new FileDescriptor();  
            fd.setInt$(fileDesc); //设置文件描述符  
            sServerSocket = new LocalServerSocket(fd); //创建 Socket 的本地服务端  
        } catch (IOException ex) {  
            ...  
        }  
    }  
}
```

```
}
```

3.3 preload

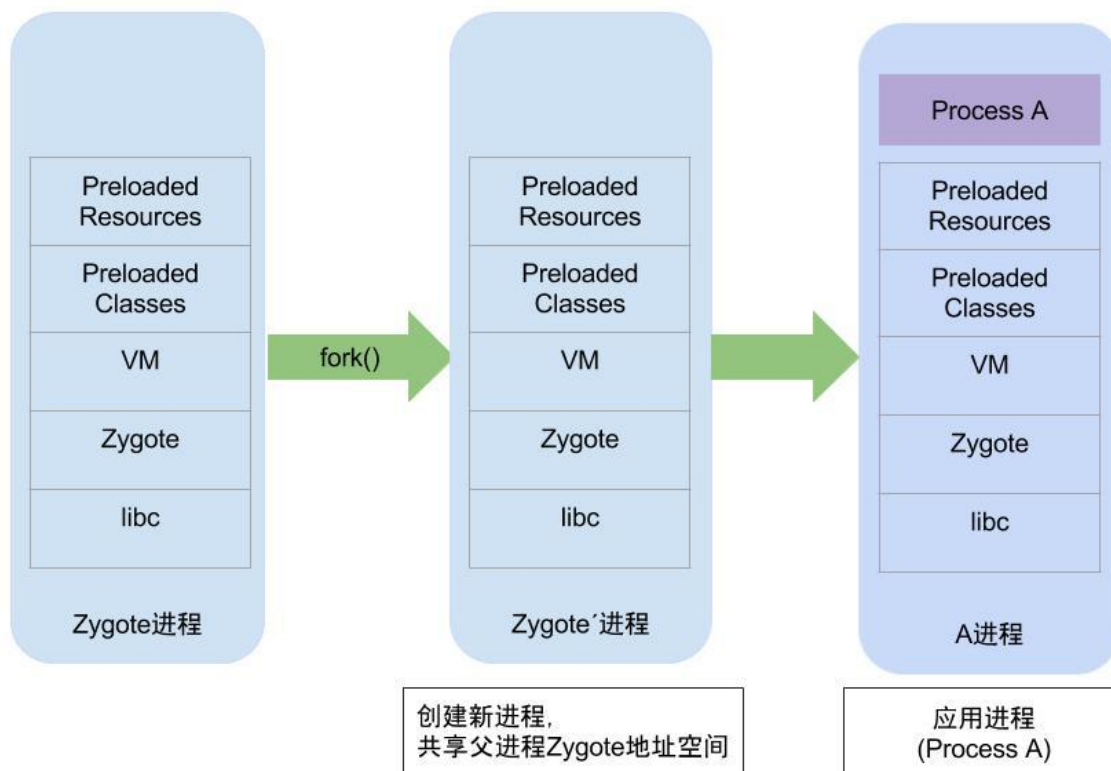
[→ZygoteInit.java]

```
static void preload() {  
    //预加载位于/system/etc/preloaded-classes 文件中的类  
    preloadClasses();  
  
    //预加载资源，包含 drawable 和 color 资源  
    preloadResources();  
  
    //预加载 OpenGL  
    preloadOpenGL();  
  
    //通过 System.loadLibrary() 方法，  
    //预加载"android","compiler_rt","jnigraphics"这 3 个共享库  
    preloadSharedLibraries();  
  
    //预加载 文本连接符资源  
    preloadTextResources();  
  
    //仅用于 zygote 进程，用于内存共享的进程  
    WebViewFactory.prepareWebViewInZygote();  
}
```

执行 Zygote 进程的初始化,对于类加载，采用反射机制 `Class.forName()`方法来加载。对于资源加载，主要是 `com.android.internal.R.array.preloaded_drawables` 和

com.android.internal.R.array.preloaded_color_state_lists, 在应用程序中以 com.android.internal.R.xxx 开头的资源, 便是此时由 Zygote 加载到内存的。

zygote 进程内加载了 preload()方法中的所有资源, 当需要 fork 新进程时, 采用 copy on write 技术, 如下:



3.4 startSystemServer

[→ZygoteInit.java]

```
private static boolean startSystemServer(String abiList, String socketName) throws
    MethodAndArgsCaller, RuntimeException {
    long capabilities = posixCapabilitiesAsBits(
        OsConstants.CAP_BLOCK_SUSPEND,
        OsConstants.CAP_KILL,
        OsConstants.CAP_NET_ADMIN,
        OsConstants.CAP_NET_BIND_SERVICE,
```

```
OsConstants.CAP_NET_BROADCAST,
OsConstants.CAP_NET_RAW,
OsConstants.CAP_SYS_MODULE,
OsConstants.CAP_SYS_NICE,
OsConstants.CAP_SYS_RESOURCE,
OsConstants.CAP_SYS_TIME,
OsConstants.CAP_SYS_TTY_CONFIG
);

//参数准备
String args[] = {
    "--setuid=1000",
    "--setgid=1000",
    "--setgroups=1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1018,1021,1032,3001,3002,3003,3006,3007",
    "--capabilities=" + capabilities + "," + capabilities,
    "--nice-name=system_server",
    "--runtime-args",
    "com.android.server.SystemServer",
};

ZygoteConnection.Arguments parsedArgs = null;

int pid;

try {
    //用于解析参数，生成目标格式
    parsedArgs = new ZygoteConnection.Arguments(args);
    ZygoteConnection.applyDebuggerSystemProperty(parsedArgs);
    ZygoteConnection.applyInvokeWithSystemProperty(parsedArgs);

    // fork 子进程，用于运行 system_server
    pid = Zygote.forkSystemServer(
```

```
        parsedArgs.uid, parsedArgs.gid,  
        parsedArgs.gids,  
        parsedArgs.debugFlags,  
        null,  
        parsedArgs.permittedCapabilities,  
        parsedArgs.effectiveCapabilities);  
    } catch (IllegalArgumentException ex) {  
        throw new RuntimeException(ex);  
    }  
  
    //进入子进程 system_server  
    if (pid == 0) {  
        if (hasSecondZygote(abilist)) {  
            waitForSecondaryZygote(socketName);  
        }  
  
        // 完成 system_server 进程剩余的工作  
        handleSystemServerProcess(parsedArgs);  
    }  
  
    return true;  
}
```

准备参数并 fork 新进程，从上面可以看出 system server 进程参数信息为 uid=1000,gid=1000,进程名为 sytem_server，从 zygote 进程 fork 新进程后，需要关闭 zygote 原有的 socket。另外，对于有两个 zygote 进程情况，需等待第 2 个 zygote 创建完成。更多详情见 [Android 系统启动-systemServer 上篇](#)。

3.5 runSelectLoop

[→ZygoteInit.java]

```
private static void runSelectLoop(String abilist) throws MethodAndArgsCaller {
```

```
ArrayList<FileDescriptor> fds = new ArrayList<FileDescriptor>();

ArrayList<ZygoteConnection> peers = new ArrayList<ZygoteConnection>();

//sServerSocket 是 socket 通信中的服务端，即 zygote 进程。保存到 fds[0]

fds.add(sServerSocket.getFileDescriptor());

peers.add(null);

while (true) {

    StructPollfd[] pollFds = new StructPollfd[fds.size()];

    for (int i = 0; i < pollFds.length; ++i) {

        pollFds[i] = new StructPollfd();

        pollFds[i].fd = fds.get(i);

        pollFds[i].events = (short) POLLIN;

    }

    try {

        //处理轮询状态，当 pollFds 有事件到来则往下执行，否则阻塞在这里

        Os.poll(pollFds, -1);

    } catch (ErrnoException ex) {

        ...

    }

    for (int i = pollFds.length - 1; i >= 0; --i) {

        //采用 I/O 多路复用机制，当接收到客户端发出连接请求 或者数据处理请求到来，则
        往下执行：

        // 否则进入 continue，跳出本次循环。

        if ((pollFds[i].revents & POLLIN) == 0) {

            continue;

        }

        if (i == 0) {

            //即 fds[0]，代表的是 sServerSocket，则意味着有客户端连接请求：

            // 则创建 ZygoteConnection 对象，并添加到 fds。
```

```
ZygoteConnection newPeer = acceptCommandPeer(abiList);

peers.add(newPeer);

fds.add(newPeer.getFileDescriptor()); //添加到 fds.

} else {

    //i>0, 则代表通过 socket 接收来自对端的数据, 并执行相应操作【见小节 3.6】

    boolean done = peers.get(i).runOnce();

    if (done) {

        peers.remove(i);

        fds.remove(i); //处理完则从 fds 中移除该文件描述符

    }

}

}

}
```

Zygote 采用高效的 I/O 多路复用机制, 保证在没有客户端连接请求或数据处理时休眠, 否则响应客户端的请求。

3.6 runOnce

[-> ZygoteConnection.java]

```
boolean runOnce() throws ZygoteInit.MethodAndArgsCaller {

    String args[];

    Arguments parsedArgs = null;

    FileDescriptor[] descriptors;

    try {

        //读取 socket 客户端发送过来的参数列表

        args = readArgumentList();

    }
```

```
        descriptors = mSocket.getAncillaryFileDescriptors();
    } catch (IOException ex) {
        ...
        return true;
    }
    ...

    try {
        //将 binder 客户端传递过来的参数，解析成 Arguments 对象格式
        parsedArgs = new Arguments(args);
        ...
        //【见小节 7】
        pid = Zygote.forkAndSpecialize(parsedArgs.uid, parsedArgs.gid, parsedArgs.gids,
            parsedArgs.debugFlags, rlimits, parsedArgs.mountExternal, parsedArgs.seInfo,
            parsedArgs.niceName, fdsToClose, parsedArgs.instructionSet,
            parsedArgs.appDataDir);
    } catch (Exception e) {
        ...
    }

    try {
        if (pid == 0) {
            //子进程执行
            IoUtils.closeQuietly(serverPipeFd);
            serverPipeFd = null;
            //进入子进程流程
            handleChildProc(parsedArgs, descriptors, childPipeFd, newStderr);
            return true;
        } else {
```



```
//父进程执行

IoUtils.closeQuietly(childPipeFd);

childPipeFd = null;

return handleParentProc(pid, descriptors, serverPipeFd, parsedArgs);

}

} finally {

IoUtils.closeQuietly(childPipeFd);

IoUtils.closeQuietly(serverPipeFd);

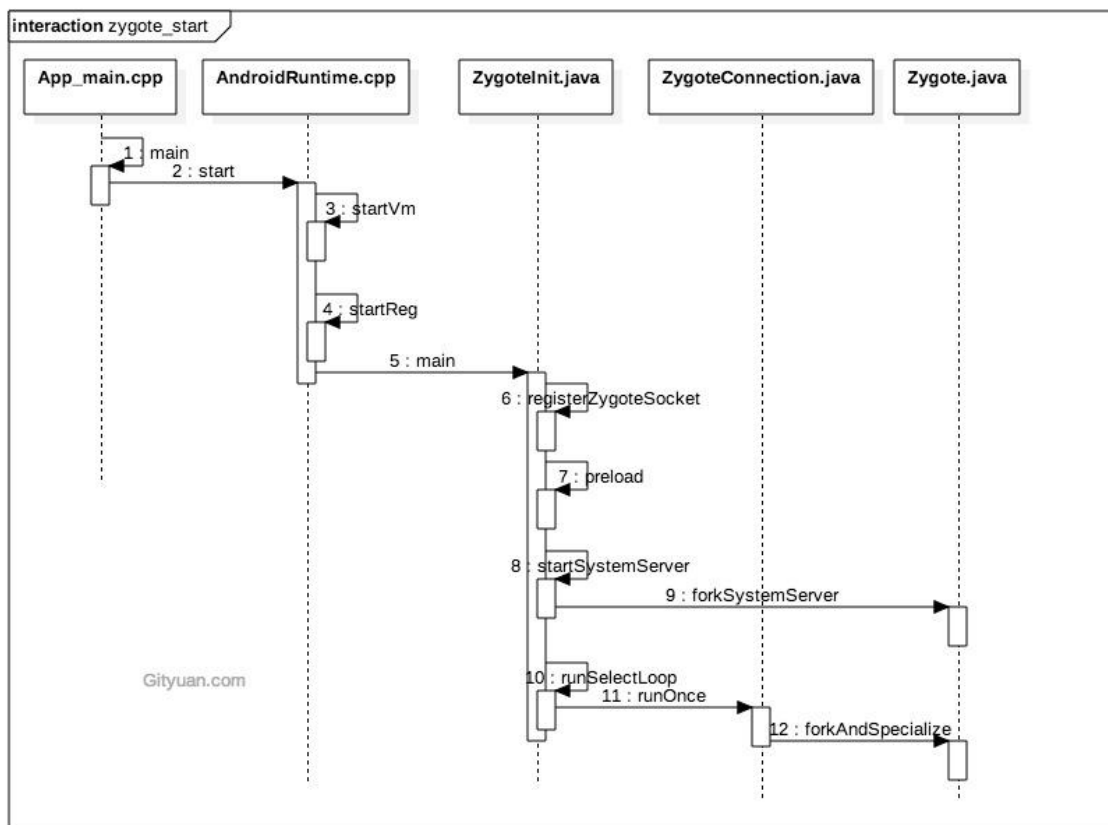
}

}
```

更多内容，见[理解 Android 进程创建流程](#)

四、总结

Zygote 启动过程的调用流程图：



1. 解析 `init.zygote.rc` 中的参数, 创建 `AppRuntime` 并调用 `AppRuntime.start()` 方法;
2. 调用 `AndroidRuntime` 的 `startVM()` 方法创建虚拟机, 再调用 `startReg()` 注册 JNI 函数;
3. 通过 JNI 方式调用 `ZygoteInit.main()`, 第一次进入 Java 世界;
4. `registerZygoteSocket()` 建立 socket 通道, `zygote` 作为通信的服务端, 用于响应客户端请求;
5. `preload()` 预加载通用类、drawable 和 color 资源、OpenGL 以及共享库以及 `WebView`, 用于提高 app 启动效率;
6. `zygote` 完毕大部分工作, 接下来再通过 `startSystemServer()`, `fork` 帮手 `system_server` 进程, 也是上层 `framework` 的运行载体。
7. `zygote` 功成身退, 调用 `runSelectLoop()`, 随时待命, 当接收到请求创建新进程请求时立即唤醒并执行相应工作。

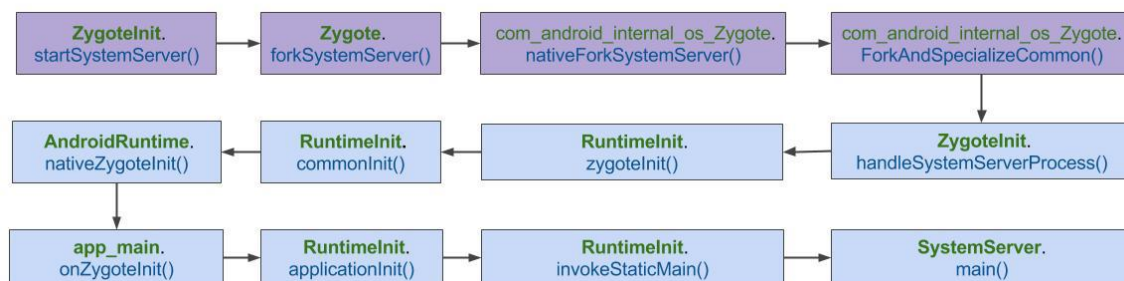
最后, 介绍给通过 `cmd` 命令, 来 `fork` 新进程来执行类中 `main` 方法的方式: (启动后进入 `RuntimeInit.main`)

```
app_process [可选参数] 命令所在路径 启动的类名 [可选参数]
```

第四节 SystemServer 上篇

启动流程

SystemServer 的在 Android 体系中所处的地位，SystemServer 由 Zygote fork 生成的，进程名为 `system_server`，该进程承载着 framework 的核心服务。Android 系统启动-zygote 篇中讲到 Zygote 启动过程中会调用 `startSystemServer()`，可知 `startSystemServer()` 函数是 `system_server` 启动流程的起点，启动流程图如下：



上图前 4 步骤（即颜色为紫色的流程）运行在是 `Zygote` 进程，从第 5 步（即颜色为蓝色的流程）`ZygoteInit.handleSystemServerProcess` 开始是运行在新创建的 `system_server`，这是 fork 机制实现的（fork 会返回 2 次）。下面从 `startSystemServer()` 开始讲解详细启动流程。

1. startSystemServer

[→ZygoteInit.java]

```
private static boolean startSystemServer(String abilist, String socketName) throws MethodAndArgsCaller, RuntimeException {
    ...
    // 参数准备
    String args[] = {
        "--setuid=1000",
        "--setgid=1000",
    }
```

```
--setgroups=1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1018,1021,
1032,3001,3002,3003,3006,3007",

"--capabilities=" + capabilities + "," + capabilities,

"--nice-name=system_server",

"--runtime-args",

"com.android.server.SystemServer",

};

ZygoteConnection.Arguments parsedArgs = null;

int pid;

try {

    //用于解析参数，生成目标格式

    parsedArgs = new ZygoteConnection.Arguments(args);

    ZygoteConnection.applyDebuggerSystemProperty(parsedArgs);

    ZygoteConnection.applyInvokeWithSystemProperty(parsedArgs);

    // fork 子进程，该进程是 system_server 进程【见小节 2】

    pid = Zygote.forkSystemServer(

        parsedArgs.uid, parsedArgs.gid,

        parsedArgs.gids,

        parsedArgs.debugFlags,

        null,

        parsedArgs.permittedCapabilities,

        parsedArgs.effectiveCapabilities);

} catch (IllegalArgumentException ex) {

    throw new RuntimeException(ex);

}

//进入子进程 system_server

if (pid == 0) {
```

```
        if (hasSecondZygote(abilist)) {  
            waitForSecondaryZygote(socketName);  
        }  
        // 完成 system_server 进程剩余的工作 【见小节 5】  
        handleSystemServerProcess(parsedArgs);  
    }  
    return true;  
}
```

准备参数并 fork 新进程，从上面可以看出 system server 进程参数信息为 uid=1000,gid=1000,进程名为 sytem_server，从 zygote 进程 fork 新进程后，需要关闭 zygote 原有的 socket。另外，对于有两个 zygote 进程情况，需等待第 2 个 zygote 创建完成。

2 forkSystemServer

[→Zygote.java]

```
public static int forkSystemServer(int uid, int gid, int[] gids, int debugFlags,  
    int[][] rlimits, long permittedCapabilities, long effectiveCapabilities) {  
    VM_HOOKS.preFork();  
    // 调用 native 方法 fork system_server 进程【见小节 3】  
    int pid = nativeForkSystemServer(  
        uid, gid, gids, debugFlags, rlimits, permittedCapabilities, effective  
        Capabilities);  
    if (pid == 0) {  
        Trace.setTracingEnabled(true);  
    }  
    VM_HOOKS.postForkCommon();  
    return pid;  
}
```

nativeForkSystemServer()方法在 AndroidRuntime.cpp 中注册的，调用 com_android_internal_os_Zygote.cpp 中的 register_com_android_internal_os_Zygote()方法建立 native 方法的映射关系，所以接下来进入如下方法。

3. nativeForkSystemServer

[->com_android_internal_os_Zygote.cpp]

```
static jint com_android_internal_os_Zygote_nativeForkSystemServer(  
    JNIEnv* env, jclass, uid_t uid, gid_t gid, jintArray gids,  
    jint debug_flags, jobjectArray rlimits, jlong permittedCapabilities,  
    jlong effectiveCapabilities) {  
    //fork 子进程，见【见小节 4】  
    pid_t pid = ForkAndSpecializeCommon(env, uid, gid, gids,  
        debug_flags, rlimits,  
        permittedCapabilities, effectiveCapabilities,  
        MOUNT_EXTERNAL_DEFAULT, NULL, NULL, true, NULL,  
        NULL, NULL);  
    if (pid > 0) {  
        // zygote 进程，检测 system_server 进程是否创建  
        gSystemServerPid = pid;  
        int status;  
        if (waitpid(pid, &status, WNOHANG) == pid) {  
            //当 system_server 进程死亡后，重启 zygote 进程  
            RuntimeAbort(env);  
        }  
    }  
    return pid;  
}
```

当 `system_server` 进程创建失败时，将会重启 `zygote` 进程。这里需要注意，对于 Android 5.0 以上系统，有两个 `zygote` 进程，分别是 `zygote`、`zygote64` 两个进程，`system_server` 的父进程，一般来说 64 位系统其父进程是 `zygote64` 进程

1. 当 kill `system_server` 进程后，只重启 `zygote64` 和 `system_server`，不重启 `zygote`;
2. 当 kill `zygote64` 进程后，只重启 `zygote64` 和 `system_server`，也不重启 `zygote`;
3. 当 kill `zygote` 进程，则重启 `zygote`、`zygote64` 以及 `system_server`。

4. ForkAndSpecializeCommon

[->com_android_internal_os_Zygote.cpp]

```
static pid_t ForkAndSpecializeCommon(JNIEnv* env, uid_t uid, gid_t gid, jintArray
javaGids, jint debug_flags, jobjectArray javaRlimits, jlong permittedCapabilities,
jlong effectiveCapabilities, jint mount_external, jstring java_se_info, jstring
java_se_name, bool is_system_server, jintArray fdsToClose, jstring instructionSet,
jstring dataDir) {
    SetSigChldHandler(); //设置子进程的 signal 信号处理函数

    pid_t pid = fork(); //fork 子进程

    if (pid == 0) {
        //进入子进程

        DetachDescriptors(env, fdsToClose); //关闭并清除文件描述符

        if (!is_system_server) {
            //对于非 system_server 子进程，则创建进程组

            int rc = createProcessGroup(uid, getpid());
        }

        SetGids(env, javaGids); //设置设置 group

        SetRLimits(env, javaRlimits); //设置资源 limit

        int rc = setresgid(gid, gid, gid);

        rc = setresuid(uid, uid, uid);
    }
}
```

```
SetCapabilities(env, permittedCapabilities, effectiveCapabilities);

SetSchedulerPolicy(env); //设置调度策略

//selinux 上下文

rc = selinux_android_setcontext(uid, is_system_server, se_info_c_str, se_name_c_str);

if (se_info_c_str == NULL && is_system_server) {
    se_name_c_str = "system_server";
}

if (se_info_c_str != NULL) {
    SetThreadName(se_name_c_str); //设置线程名为 system_server, 方便调试
}

UnsetSigChldHandler(); //设置子进程的 signal 信号处理函数为默认函数

//等价于调用 zygote.callPostForkChildHooks()

env->CallStaticVoidMethod(gZygoteClass, gCallPostForkChildHooks, debug_flags,
                           is_system_server ? NULL : instructionSet);
...

} else if (pid > 0) {
    //进入父进程, 即 zygote 进程
}

return pid;
}
```

fork()创建新进程, 采用 copy on write 方式, 这是 linux 创建进程的标准方法, 会有两次 return, 对于 pid==0 为子进程的返回, 对于 pid>0 为父进程的返回。到此 system_server 进程已完成了创建的所有工作, 接下来开始了 system_server 进程的真正工作。在前面 startSystemServer()方法中, zygote 进程执行完 forkSystemServer()后, 新创建出来的 system_server 进程便进入

handleSystemServerProcess()方法。关于fork(),可查看另一个文章[理解Android进程创建流程](#)。

5. handleSystemServerProcess

[→ZygoteInit.java]

```
private static void handleSystemServerProcess( ZygoteConnection.Arguments parse
dArgs) throws ZygoteInit.MethodAndArgsCaller {

    closeServerSocket(); //关闭父进程 zygote 复制而来的 Socket

    Os.umask(S_IRWXG | S_IRWXO);

    if (parsedArgs.niceName != null) {
        Process.setArgV0(parsedArgs.niceName); //设置当前进程名为"system_server"
    }

    final String systemServerClasspath = Os.getenv("SYSTEMSERVERCLASSPATH");
    if (systemServerClasspath != null) {
        //执行 dex 优化操作【见小节6】
        performSystemServerDexOpt(systemServerClasspath);
    }

    if (parsedArgs.invokeWith != null) {
        String[] args = parsedArgs.remainingArgs;

        if (systemServerClasspath != null) {
            String[] amendedArgs = new String[args.length + 2];
            amendedArgs[0] = "-cp";
            amendedArgs[1] = systemServerClasspath;
```

```
        System.arraycopy(parsedArgs.remainingArgs, 0, amendedArgs, 2, parsed
        Args.remainingArgs.length);

    }

    //启动应用进程

    WrapperInit.execApplication(parsedArgs.invokeWith,

        parsedArgs.niceName, parsedArgs.targetSdkVersion,

        VMRuntime.getCurrentInstructionSet(), null, args);

    } else {

        ClassLoader cl = null;

        if (systemServerClasspath != null) {

            // 创建类加载器，并赋予当前线程

            cl = new PathClassLoader(systemServerClasspath, ClassLoader.getSystemClassLoader());

            Thread.currentThread().setContextClassLoader(cl);

        }

        //system_server 故进入此分支【见小节 7】

        RuntimeInit.zygoteInit(parsedArgs.targetSdkVersion, parsedArgs.remaining
        Args, cl);

    }

    /* should never reach here */

}
```

此处 `systemServerClasspath` 环境变量主要有 `/system/framework/` 目录下的 `services.jar`, `ethernet-service.jar`, `wifi-service.jar` 这 3 个文件

6. performSystemServerDexOpt

[→ZygoteInit.java]

```
private static void performSystemServerDexOpt(String classPath) {
```

更多视频+架构学习资料免费领取 QQ 3607409029

```
final String[] classPathElements = classPath.split(":");

//创建一个与 installd 的建立 socket 连接

final InstallerConnection installer = new InstallerConnection();

//执行 ping 操作，直到与 installd 服务端连通为止

installer.waitForConnection();

final String instructionSet = VMRuntime.getRuntime().vmInstructionSet();

try {

    for (String classPathElement : classPathElements) {

        final int dexoptNeeded = DexFile.getDexOptNeeded(

            classPathElement, "*", instructionSet, false /* defer */);

        if (dexoptNeeded != DexFile.NO_DEXOPT_NEEDED) {

            //以 system 权限，执行 dex 文件优化

            installer.dexopt(classPathElement, Process.SYSTEM_UID, false,

                instructionSet, dexoptNeeded);

        }

    }

} catch (IOException ioe) {

    throw new RuntimeException("Error starting system_server", ioe);

} finally {

    installer.disconnect(); //断开与 installd 的 socket 连接

}

}
```

将 classPath 字符串中的 apk，分别进行 dex 优化操作。真正执行优化工作通过 socket 通信将相应的命令参数，发送给 installd 来完成。

7. zygoteInit

[→RuntimeInit.java]

```
public static final void zygoInit(int targetSdkVersion, String[] argv, ClassL
oader classLoader) throws ZygoInit.MethodAndArgsCaller {

    Trace.traceBegin(Trace.TRACE_TAG_ACTIVITY_MANAGER, "RuntimeInit");

    redirectLogStreams(); //重定向 log 输出

    commonInit(); // 通用的一些初始化【见小节 8】

    nativeZygoInit(); // zygo 初始化 【见小节 9】

    applicationInit(targetSdkVersion, argv, classLoader); // 应用初始化【见小节 10】

}
```

8. commonInit

[→RuntimeInit.java]

```
private static final void commonInit() {

    // 设置默认的未捕捉异常处理方法

    Thread.setDefaultUncaughtExceptionHandler(new UncaughtHandler());

    // 设置市区，中国时区为"Asia/Shanghai"

    TimezoneGetter.setInstance(new TimezoneGetter() {

        @Override

        public String getId() {

            return SystemProperties.get("persist.sys.timezone");

        }

    });

    TimeZone.setDefault(null);

    //重置 log 配置

    LogManager.getLogManager().reset();

}
```

```
new AndroidConfig();

// 设置默认的 HTTP User-agent 格式,用于 HttpURLConnection。

String userAgent = getDefaultUserAgent();

System.setProperty("http.agent", userAgent);

// 设置 socket 的 tag, 用于网络流量统计

NetworkManagementSocketTagger.install();
}
```

默认的 HTTP User-agent 格式, 例如:

```
"Dalvik/1.1.0 (Linux; U; Android 6.0.1; LenovoX3c70 Build/LMY47V)".
```

9. nativeZygoteInit

nativeZygoteInit()方法在 AndroidRuntime.cpp 中, 进行了 jni 映射, 对应下面的方法。

[→AndroidRuntime.cpp]

```
static void com_android_internal_os_RuntimeInit_nativeZygoteInit(JNIEnv* env, j
object clazz) {

    //此处的 gCurRuntime 为 AppRuntime, 是在 AndroidRuntime.cpp 中定义的

    gCurRuntime->onZygoteInit();

}
```

[→app_main.cpp]

```
virtual void onZygoteInit() {

    sp<ProcessState> proc = ProcessState::self();
```

```
proc->startThreadPool(); //启动新 binder 线程  
}
```

ProcessState::self()是单例模式,主要工作是调用 open()打开/dev/binder 驱动设备,再利用 mmap()映射内核的地址空间,将 Binder 驱动的 fd 赋值 ProcessState 对象中的变量 mDriverFD,用于交互操作。startThreadPool()是创建一个新的 binder 线程,不断进行 talkWithDriver(),在 binder 系列文章中的[注册服务\(addService\)](#)详细这两个方法的执行原理。

10. applicationInit

[->RuntimeInit.java]

```
private static void applicationInit(int targetSdkVersion, String[] argv, ClassLoader classLoader) throws ZygoteInit.MethodAndArgsCaller {  
    //true 代表应用程序退出时不调用 AppRuntime.onExit(), 否则会在退出前调用  
    nativeSetExitWithoutCleanup(true);  
  
    //设置虚拟机的内存利用率参数值为 0.75  
    VMRuntime.getRuntime().setTargetHeapUtilization(0.75f);  
    VMRuntime.getRuntime().setTargetSdkVersion(targetSdkVersion);  
  
    final Arguments args;  
    try {  
        args = new Arguments(argv); //解析参数  
    } catch (IllegalArgumentException ex) {  
        return;  
    }  
  
    Trace.traceEnd(Trace.TRACE_TAG_ACTIVITY_MANAGER);  
  
    //调用 startClass 的 static 方法 main() 【见小节 11】
```

```
        invokeStaticMain(args.startClass, args.startArgs, classLoader);  
    }  
}
```

在 `startSystemServer()` 方法中通过硬编码初始化参数, 可知此处 `args.startClass` 为“`com.android.server.SystemServer`”。

11. invokeStaticMain

[→RuntimeInit.java]

```
private static void invokeStaticMain(String className, String[] argv, ClassLoader classLoader) throws ZygoteInit.MethodAndArgsCaller {  
    Class<?> cl = Class.forName(className, true, classLoader);  
    ...  
  
    Method m;  
    try {  
        m = cl.getMethod("main", new Class[] { String[].class });  
    } catch (NoSuchMethodException ex) {  
        ...  
    } catch (SecurityException ex) {  
        ...  
    }  
    ...  
  
    int modifiers = m.getModifiers();  
    if (! (Modifier.isStatic(modifiers) && Modifier.isPublic(modifiers))) {  
        ...  
    }  
  
    //通过抛出异常, 回到 ZygoteInit.main()。这样做好处是能清空栈帧, 提高栈帧利用率。【见  
    小节 12】  
}
```

```
throw new ZygoteInit.MethodAndArgsCaller(m, argv);  
}
```

12. MethodAndArgsCaller

在 [Android 系统启动-zygote](#) 篇中遗留了一个问题没有讲解，如下：

[→ZygoteInit.java]

```
public static void main(String argv[]) {  
    try {  
        startSystemServer(abiList, socketName); //启动 system_server  
        ....  
    } catch (MethodAndArgsCaller caller) {  
        caller.run(); //【见小节 13】  
    } catch (RuntimeException ex) {  
        closeServerSocket();  
        throw ex;  
    }  
}
```

现在已经很明显了，是 `invokeStaticMain()` 方法中抛出的异常 `MethodAndArgsCaller`，从而进入 `caller.run()` 方法。

[→ZygoteInit.java]

```
public static class MethodAndArgsCaller extends Exception implements Runnable {  
  
    public void run() {  
        try {  
            //根据传递过来的参数，可知此处通过反射机制调用的是 SystemServer.main() 方法
```



```
mMethod.invoke(null, new Object[] { mArgs });  
  
} catch (IllegalAccessException ex) {  
  
    throw new RuntimeException(ex);  
  
} catch (InvocationTargetException ex) {  
  
    Throwable cause = ex.getCause();  
  
    if (cause instanceof RuntimeException) {  
  
        throw (RuntimeException) cause;  
  
    } else if (cause instanceof Error) {  
  
        throw (Error) cause;  
  
    }  
  
    throw new RuntimeException(ex);  
  
}  
  
}  
  
}
```

到此，总算是进入到了 SystemServer 类的 main()方法， 在文章 [Android 系统启动-SystemServer](#) 下篇中会紧接着这里开始讲述。

第五节 SystemServer 下篇

一. SystemServer 启动

上篇文章 [Android 系统启动-systemServer](#) 上篇 从 Zygote 一路启动到 SystemServer 的过程。简单回顾下，在 RuntimeInit.java 中 invokeStaticMain 方法通过创建并抛出异常 ZygoteInit.MethodAndArgsCaller, 在 [ZygoteInit.java](#) 中的 main()方法会捕捉该异常，并调用 [caller.run\(\)](#)，再通过反射便会调用到 SystemServer.main()方法，该方法主要执行流程：

```
SystemServer.main  
  
SystemServer.run  
  
createSystemContext
```

```
startBootstrapServices();  
startCoreServices();  
startOtherServices();  
Looper.loop();
```

接下来，从其 main 方法说起。

1.1 SystemServer.main

```
public final class SystemServer {  
    ...  
    public static void main(String[] args) {  
        //先初始化 SystemServer 对象，再调用对象的 run()方法， 【见小节 1.2】  
        new SystemServer().run();  
    }  
}
```

1.2 SystemServer.run

```
private void run() {  
    //当系统时间比 1970 年更早，就设置当前系统时间为 1970 年  
    if (System.currentTimeMillis() < EARLIEST_SUPPORTED_TIME) {  
        SystemClock.setCurrentTimeMillis(EARLIEST_SUPPORTED_TIME);  
    }  
  
    //变更虚拟机的库文件，对于 Android 6.0 默认采用的是 libart.so  
    SystemProperties.set("persist.sys.dalvik.vm.lib.2", VMRuntime.getRuntime().  
        vmLibrary());  
  
    if (SamplingProfilerIntegration.isEnabled()) {  
        ...  
    }  
}
```

```
}

//清除 vm 内存增长上限，由于启动过程需要较多的虚拟机内存空间
VMRuntime.getRuntime().clearGrowthLimit();

//设置内存的可能有效使用率为 0.8
VMRuntime.getRuntime().setTargetHeapUtilization(0.8f);

// 针对部分设备依赖于运行时就产生指纹信息，因此需要在开机完成前已经定义
Build.ensureFingerprintProperty();

//访问环境变量前，需要明确地指定用户
Environment.setUserRequired(true);

//确保当前系统进程的 binder 调用，总是运行在前台优先级(foreground priority)
BinderInternal.disableBackgroundScheduling(true);

android.os.Process.setThreadPriority(android.os.Process.THREAD_PRIORITY_FOREGROUND);

android.os.Process.setCanSelfBackground(false);

// 主线程 looper 就在当前线程运行
Looper.prepareMainLooper();

//加载 android_servers.so 库，该库包含的源码在 frameworks/base/services/目录下
System.loadLibrary("android_servers");

//检测上次关机过程是否失败，该方法可能不会返回[见小节 1.2.1]
performPendingShutdown();

//初始化系统上下文 【见小节 1.3】
createSystemContext();
```

```
//创建系统服务管理
mSystemServiceManager = new SystemServiceManager(mSystemContext);

//将 mSystemServiceManager 添加到本地服务的成员 sLocalServiceObjects
LocalServices.addService(SystemServiceManager.class, mSystemServiceManager);

//启动各种系统服务
try {
    startBootstrapServices(); // 启动引导服务【见小节 1.4】
    startCoreServices();      // 启动核心服务【见小节 1.5】
    startOtherServices();     // 启动其他服务【见小节 1.6】
} catch (Throwable ex) {
    Slog.e("System", "***** Failure starting system services", ex);
    throw ex;
}

//用于 debug 版本，将 log 事件不断循环地输出到 dropbox（用于分析）
if (StrictMode.conditionallyEnableDebugLogging()) {
    Slog.i(TAG, "Enabled StrictMode for system server main thread.");
}

//一直循环执行
Looper.loop();

throw new RuntimeException("Main thread loop unexpectedly exited");
}
```

LocalServices 通过用静态 Map 变量 sLocalServiceObjects，来保存以服务类名为 key，以具体服务对象为 value 的 Map 结构。

1.2.1 performPendingShutdown

更多录播视频+架构学习资料免费领取 QQ 3607409029

[-->SystemServer.java]

```
private void performPendingShutdown() {  
    final String shutdownAction = SystemProperties.get(  
        ShutdownThread.SHUTDOWN_ACTION_PROPERTY, "");  
    if (shutdownAction != null && shutdownAction.length() > 0) {  
        boolean reboot = (shutdownAction.charAt(0) == '1');  
  
        final String reason;  
        if (shutdownAction.length() > 1) {  
            reason = shutdownAction.substring(1, shutdownAction.length());  
        } else {  
            reason = null;  
        }  
  
        // 当"sys.shutdown.requested"值不为空,则会重启或者关机  
        ShutdownThread.rebootOrShutdown(null, reboot, reason);  
    }  
}
```

1.3 createSystemContext

[-->SystemServer.java]

```
private void createSystemContext() {  
    //创建 system_server 进程的上下文信息  
    ActivityThread activityThread = ActivityThread.systemMain();  
    mSystemContext = activityThread.getSystemContext();  
  
    //设置主题  
    mSystemContext.setTheme(android.R.style.Theme_DeviceDefault_Light_DarkActionBar);  
}
```



理解 [Application 创建过程](#) 已介绍过 `createSystemContext()` 过程，该过程会创建对象有 `ActivityThread`, `Instrumentation`, `ContextImpl`, `LoadedApk`, `Application`。

1.4 startBootstrapServices

[→SystemServer.java]

```
private void startBootstrapServices() {  
    //阻塞等待与 installd 建立 socket 通道  
    Installer installer = mSystemServiceManager.startService(Installer.class);  
  
    //启动服务 ActivityManagerService  
    mActivityManagerService = mSystemServiceManager.startService(  
        ActivityManagerService.Lifecycle.class).getService();  
    mActivityManagerService.setSystemServiceManager(mSystemServiceManager);  
    mActivityManagerService.setInstaller(installer);  
  
    //启动服务 PowerManagerService  
    mPowerManagerService = mSystemServiceManager.startService(PowerManagerService.class);  
  
    //初始化 power management  
    mActivityManagerService.initPowerManagement();  
  
    //启动服务 LightsService  
    mSystemServiceManager.startService(LightsService.class);  
  
    //启动服务 DisplayManagerService
```

```
mDisplayManagerService = mSystemServiceManager.startService(DisplayManagerService.class);

//Phase100: 在初始化 package manager 之前, 需要默认显示.

mSystemServiceManager.startBootPhase(SystemService.PHASE_WAIT_FOR_DEFAULT_DISPLAY);

//当设备正在加密时, 仅运行核心

String cryptState = SystemProperties.get("vold.decrypt");

if (ENCRYPTING_STATE.equals(cryptState)) {

    mOnlyCore = true;

} else if (ENCRYPTED_STATE.equals(cryptState)) {

    mOnlyCore = true;

}

//启动服务 PackageManagerService

mPackageManagerService = PackageManagerService.main(mSystemContext, installer,

    mFactoryTestMode != FactoryTest.FACTORY_TEST_OFF, mOnlyCore);

mFirstBoot = mPackageManagerService.isFirstBoot();

mPackageManager = mSystemContext.getPackageManager();

//启动服务 UserManagerService, 新建目录/data/user/

ServiceManager.addService(Context.USER_SERVICE, UserManagerService.getInstance());

AttributeCache.init(mSystemContext);

//设置 AMS

mActivityManagerService.setSystemProcess();
```

```
//启动传感器服务  
  
startSensorService();  
  
}
```

该方法所创建的服务：ActivityManagerService, PowerManagerService, LightsService, DisplayManagerService, PackageManagerService, UserManagerService, sensor 服务。

1.5 startCoreServices

```
private void startCoreServices() {  
  
    //启动服务 BatteryService，用于统计电池电量，需要 LightService。  
    mSystemServiceManager.startService(BatteryService.class);  
  
    //启动服务 UsageStatsService，用于统计应用使用情况  
    mSystemServiceManager.startService(UsageStatsService.class);  
    mActivityManagerService.setUsageStatsManager(  
        LocalServices.getService(UsageStatsManagerInternal.class));  
  
    mPackageManagerService.getUsageStatsIfNoPackageUsageInfo();  
  
    //启动服务 WebViewUpdateService  
    mSystemServiceManager.startService(WebViewUpdateService.class);  
  
}
```

启动服务 BatteryService, UsageStatsService, WebViewUpdateService。

1.6 startOtherServices

该方法比较长，有近千行代码，逻辑很简单，主要是启动一系列的服务，这里就不具体列举源码了，在第四节直接对其中的服务进行一个简单分类。


```
private void startOtherServices() {  
    ...  
    SystemConfig.getInstance();  
    mContentResolver = context.getContentResolver(); // resolver  
    ...  
    mActivityManagerService.installSystemProviders(); //provider  
    mSystemServiceManager.startService(AlarmManagerService.class); // alarm  
    // watchdog  
    watchdog.init(context, mActivityManagerService);  
    inputManager = new InputManagerService(context); // input  
    wm = WindowManagerService.main(...); // window  
    inputManager.start(); //启动 input  
    mDisplayManagerService.windowManagerAndInputReady();  
    ...  
    mSystemServiceManager.startService(MOUNT_SERVICE_CLASS); // mount  
    mPackageManagerService.performBootDexOpt(); // dexopt 操作  
    ActivityManagerNative.getDefault().showBootMessage(...); //显示启动界面  
    ...  
    statusBar = new StatusBarManagerService(context, wm); //statusBar  
    //dropbox  
    ServiceManager.addService(Context.DROPBOX_SERVICE,  
        new DropBoxManagerService(context, new File("/data/system/dro  
pbox"))));  
    mSystemServiceManager.startService(JobSchedulerService.class); //JobSch  
eduler  
    lockSettings.systemReady(); //lockSettings  
  
    //phase480 和 phase500  
    mSystemServiceManager.startBootPhase(SystemService.PHASE_LOCK_SETTINGS_R  
EADY);
```

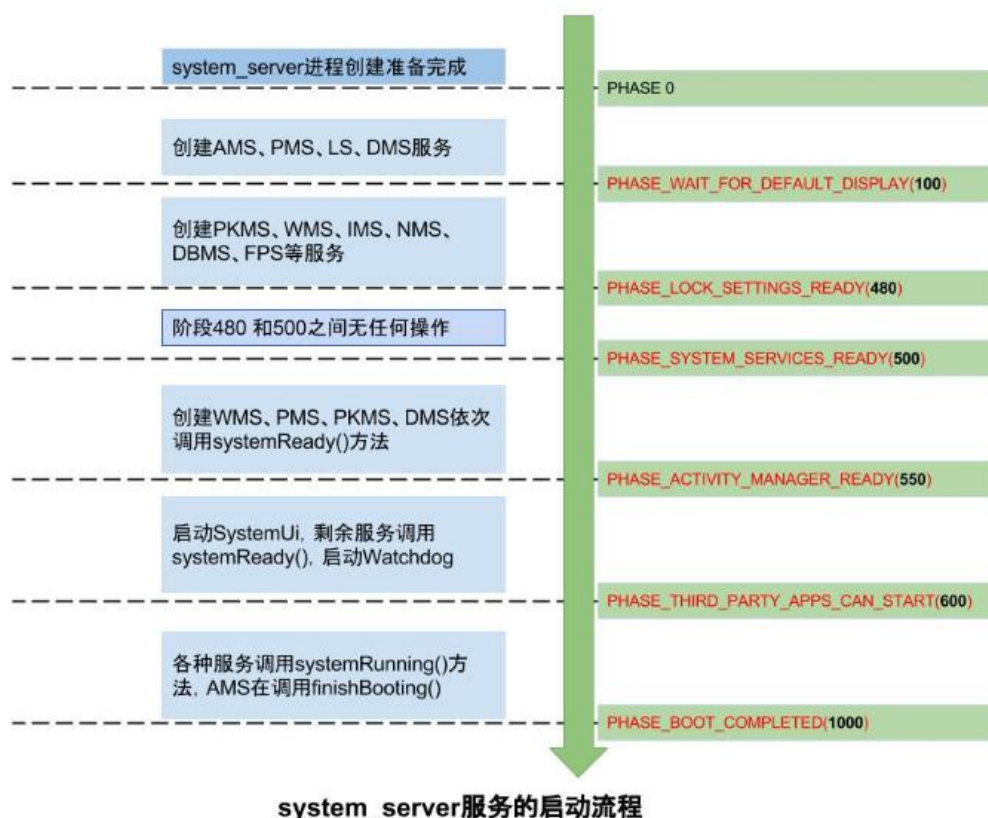
```
mSystemServiceManager.startBootPhase(SystemService.PHASE_SYSTEM_SERVICES_READY);  
  
...  
  
// 准备好 window, power, package, display 服务  
wm.systemReady();  
  
mPowerManagerService.systemReady(...);  
  
mPackageManagerService.systemReady();  
  
mDisplayManagerService.systemReady(...);  
  
...  
  
// 重头戏[见小节 2.1]  
mActivityManagerService.systemReady(new Runnable() {  
    public void run() {  
        ...  
    }  
});  
}
```

SystemServiceManager 启动各种服务中最后的一个环节便是 AMS.systemReady(), 详见 [ActivityManagerService 启动过程](#).

到此, System_server 主线程的启动工作总算完成, 进入 Looper.loop()状态, 等待其他线程通过 handler 发送消息到主线再处理.

二、服务启动阶段

SystemServiceManager 的 startBootPhase()贯穿 system_server 进程的整个启动过程:



其中 **PHASE_BOOT_COMPLETED=1000**，该阶段是发生在 Boot 完成和 home 应用启动完毕。系统服务更倾向于监听该阶段，而不是注册广播 **ACTION_BOOT_COMPLETED**，从而降低系统延迟。

各个启动阶段所在源码的大致位置：

```
public final class SystemServer {  
  
    private void startBootstrapServices() {  
        ...  
        //phase100  
        mSystemServiceManager.startBootPhase(SystemService.PHASE_WAIT_FOR_DEFAULT_DISPLAY);  
        ...  
    }  
}
```

```
private void startCoreServices() {  
    ...  
}  
  
private void startOtherServices() {  
    ...  
    //phase480 && 500  
    mSystemServiceManager.startBootPhase(SystemService.PHASE_LOCK_SETTINGS_READY);  
    mSystemServiceManager.startBootPhase(SystemService.PHASE_SYSTEM_SERVICES_READY);  
    ...  
    mActivityManagerService.systemReady(new Runnable() {  
        public void run() {  
            //phase550  
            mSystemServiceManager.startBootPhase(  
                SystemService.PHASE_ACTIVITY_MANAGER_READY);  
            ...  
            //phase600  
            mSystemServiceManager.startBootPhase(  
                SystemService.PHASE_THIRD_PARTY_APPS_CAN_START);  
        }  
    }  
}
```

接下来再说说简单每个阶段的大概完成的工作：

2.1 Phase0

创建四大引导服务:

- ActivityManagerService
- PowerManagerService
- LightsService
- DisplayManagerService

2.2 Phase100

进入阶段 `PHASE_WAIT_FOR_DEFAULT_DISPLAY=100` 回调服务

onBootPhase(100)

- DisplayManagerService

然后创建大量服务下面列举部分:

- PackageManagerService
- WindowManagerService
- InputManagerService
- NetworkManagerService
- DropBoxManagerService
- FingerprintService
- LauncherAppsService
- ...

2.3 Phase480

进入阶段 `PHASE_LOCK_SETTINGS_READY=480` 回调服务

onBootPhase(480)

- DevicePolicyManagerService

阶段 480 后马上就进入阶段 500.

2.4 Phase500

`PHASE_SYSTEM_SERVICES_READY=500`，进入该阶段服务能安全地调用核心系统服务。

onBootPhase(500)

- AlarmManagerService
- JobSchedulerService
- NotificationManagerService
- BackupManagerService
- UsageStatsService
- DeviceIdleController
- TrustManagerService

•

UiModeManagerService

•

- BluetoothService
- BluetoothManagerService
- EthernetService
- WifiP2pService
- WifiScanningService
- WifiService
- RttService

各大服务执行 `systemReady()`:

- WindowManagerService.systemReady():
- PowerManagerService.systemReady():
- PackageManagerService.systemReady():
- DisplayManagerService.systemReady():

接下来就绪 `AMS.systemReady` 方法。

2.5 Phase550

PHASE_ACTIVITY_MANAGER_READY=550, AMS.mSystemReady=true, 已准备就绪, 进入该阶段服务能广播 Intent;但是 system_server 主线程并没有就绪.

onBootPhase(550)

- MountService
- TelecomLoaderService
- UsbService
- WebViewUpdateService
- DockObserver
- BatteryService

接下来执行: (AMS 启动 native crash 监控, 加载 WebView, 启动 SystemUi 等), 如下

- mActivityManagerService.startObservingNativeCrashes();
- WebViewFactory.prepareWebViewInSystemServer();
-

startSystemUi(context);

-
- networkScoreF.systemReady();
- networkManagementF.systemReady();
- networkStatsF.systemReady();
- networkPolicyF.systemReady();
- connectivityF.systemReady();
- audioServiceF.systemReady();
- Watchdog.getInstance().start();

2.6 Phase600

PHASE_THIRD_PARTY_APPS_CAN_START=600

onBootPhase(600)

- JobSchedulerService
- NotificationManagerService
- BackupManagerService
- AppWidgetService

- GestureLauncherService
- DreamManagerService
- TrustManagerService
- VoiceInteractionManagerService

接下来,各种服务的 `systemRunning` 过程:

WallpaperManagerService、InputMethodManagerService、LocationManagerService、CountryDetectorService、NetworkTimeUpdateService、CommonTimeManagementService、TextServicesManagerService、AssetAtlasService、InputManagerService、TelephonyRegistry、MediaRouterService、MmsServiceBroker 这些服务依次执行其 `systemRunning()` 方法。

2.7 Phase1000

在经过一系列流程,再调用 `AMS.finishBooting()` 时,则进入阶段 `Phase1000`。

到此,系统服务启动阶段完成就绪, `system_server` 进程启动完成则进入 `Looper.loop()` 状态,随时待命,等待消息队列 `MessageQueue` 中的消息到来,则马上进入执行状态。

三、服务类别

`system_server` 进程,从源码角度划分为引导服务、核心服务、其他服务 3 类。以下这些系统服务的注册过程,见 [Android 系统服务的注册方式](#)

1. 引导服务(7 个): ActivityManagerService、PowerManagerService、LightsService、DisplayManagerService、PackageManagerService、UserManagerService、SensorService;
2. 核 心 服 务 (3 个): BatteryService 、 UsageStatsService 、 WebViewUpdateService;
3. 其他服务(70 个+): AlarmManagerService、VibratorService 等。

合计总大约 80 个系统服务:

ActivityManagerService	PackageManagerService	WindowManagerService
PowerManagerService	BatteryService	BatteryStatsService
DreamManagerService	DropBoxManagerService	SamplingProfilerService
UsageStatsService	DiskStatsService	DeviceStorageMonitorService

SchedulingPolicyService	AlarmManagerService	DeviceIdleController
ThermalObserver	JobSchedulerService	AccessibilityManagerService
DisplayManagerService	LightsService	GraphicsStatsService
StatusBarManagerService	NotificationManagerService	WallpaperManagerService

UiModeManagerService	AppWidgetService	LauncherAppsService
TextServicesManagerService	ContentService	LockSettingsService
InputMethodManagerService	InputManagerService	MountService
FingerprintService	TvInputManagerService	DockObserver

NetworkManagementService	NetworkScoreService	NetworkStatsService
NetworkPolicyManagerService	ConnectivityService	BluetoothService
WifiP2pService	WifiService	WifiScanningService
AudioService	MediaRouterService	VoiceInteractionManagerService

MediaProjectionManagerService	MediaSessionService	
DevicePolicyManagerService	PrintManagerService	BackupManagerService
UserManagerService	AccountManagerService	TrustManagerService
SensorService	LocationManagerService	VibratorService

CountryDetectorService	GestureLauncherService	PersistentDataBlockService
EthernetService	WebViewUpdateService	ClipboardService
TelephonyRegistry	TelecomLoaderService	NsdService
UpdateLockService	SerialService	SearchManagerService

CommonTimeManagementService	AssetAtlasService	ConsumerIrService
MidiServiceCameraService	TwilightService	RestrictionsManagerService
MmsServiceBroker	RttService	UsbService

Service 类别众多，其中表中加粗项是指博主挑选的较重要或者较常见的 **Service**，并且在本博客中已经展开或者计划展开讲解的 **Service**，当然如果有精力会讲解更多 **service**，后续再更新。

第四章深入解析 WMS

1. WMS 概述

WMS 是系统的其他服务，无论对于应用开发还是 Framework 开发都是重点的知识，它的职责有很多，主要有以下几点：

窗口管理

WMS 是窗口的管理者，它负责窗口的启动、添加和删除，另外窗口的大小和层级也是由 WMS 进行管理的。窗口管理的核心成员有 DisplayContent、WindowToken 和 WindowState。

窗口动画

窗口间进行切换时，使用窗口动画可以显得更炫一些，窗口动画由 WMS 的动画子系统来负责，动画子系统的管理者为 WindowAnimator。

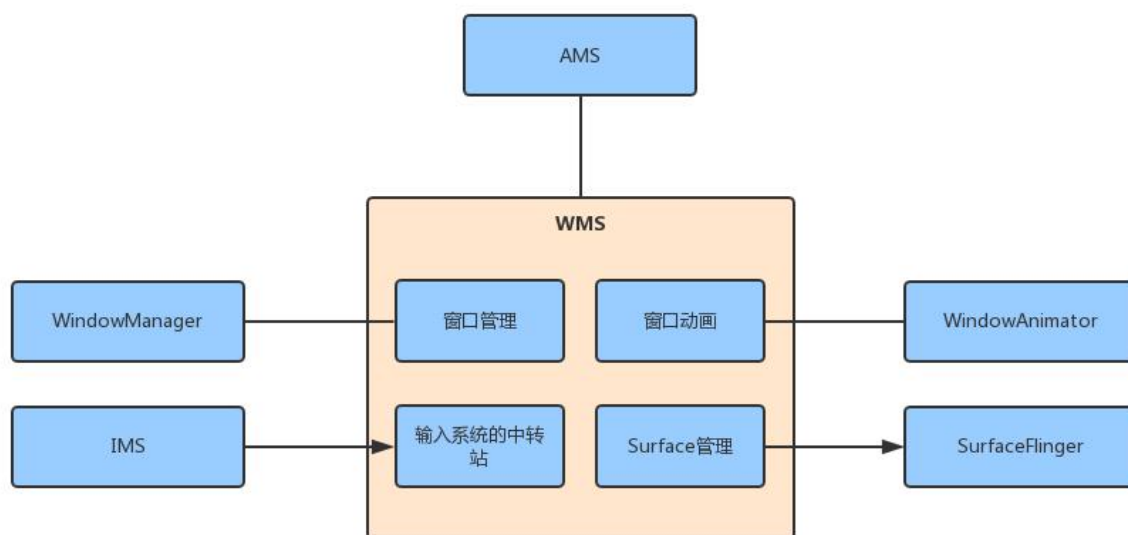
输入系统的中转站

通过对窗口的触摸从而产生触摸事件，InputManagerService (IMS) 会对触摸事件进行处理，它会寻找一个最合适的窗口来处理触摸反馈信息，WMS 是窗口的管理者，因此，WMS“理所应当”的成为了输入系统的中转站。

Surface 管理

窗口并不具备有绘制的功能，因此每个窗口都需要有一块 Surface 来供自己绘制。为每个窗口分配 Surface 是由 WMS 来完成的。

WMS 的职责可以简单总结为下图。



2. WMS 的诞生

WMS 的知识点非常多，在了解这些知识点前，我们十分有必要知道 WMS 是如何产生的。WMS 是在 SystemServer 进程中启动的，不了解 SystemServer 进程的可以查看在

Android 系统启动流程（三）解析 SyetemServer 进程启动过程这篇文章。

先来看 SyetemServer 的 main 方法：

frameworks/base/services/java/com/android/server/SystemServer.java

```
public static void main(String[] args) {  
  
    new SystemServer().run();  
  
}
```

main 方法中只调用了 SystemServer 的 run 方法，如下所示。

frameworks/base/services/java/com/android/server/SystemServer.java

```
private void run() {  
    try {  
        System.loadLibrary("android_servers");//1 ...  
        mSystemServiceManager = new SystemServiceManager(mSystemContext);//2  
        mSystemServiceManager.setRuntimeRestarted(mRuntimeRestart);  
        LocalServices.addService(SystemServiceManager.class,  
mSystemServiceManager);  
        // Prepare the thread pool for init tasks that can be parallelized  
        SystemServerInitThreadPool.get();  
    } finally {  
        traceEnd(); // InitBeforeStartServices    }  
    try {  
        traceBeginAndSlog("StartServices");  
        startBootstrapServices();//3  
        startCoreServices();//4  
        startOtherServices();//5  
        SystemServerInitThreadPool.shutdown();  
    } catch (Throwable ex) {  
        Slog.e("System", "*****");  
        Slog.e("System", "***** Failure starting system services", ex);  
        throw ex;  
    } finally {  
        traceEnd();  
    }  
    ...  
}
```

run 方法代码很多，这里截取了关键的部分，在注释 1 处加载了 libandroid_servers.so。在注释 2 处创建 SystemServiceManager，它会对系统的服务进行创建、启动和生命周期管理。接下来的代码会启动系统的各种服务，在注释 3 中的 startBootstrapServices 方法中用 SystemServiceManager 启动了 ActivityManagerService、

PowerManagerService、PackageManagerService 等服务。在注释 4 处的方法中则启动了 BatteryService、UsageStatsService 和 WebViewUpdateService。注释 5 处的 startOtherServices 方法中则启动了 CameraService、AlarmManagerService、VrManagerService 等服务，这些服务的父类为 SystemService。从注释 3、4、5 的方法名称可以看出，官方把大概 100 多个系统服务分为了三种类型，分别是引导服务、核心服务和其他服务，其中其他服务为一些非紧要和一些不需要立即启动的服务，WMS 就是其他服务的一种。

我们来查看 startOtherServices 方法是如何启动 WMS 的：

frameworks/base/services/java/com/android/server/SystemServer.java

```
private void startOtherServices() {
    ...

    traceBeginAndSlog("InitWatchdog");
    final Watchdog watchdog = Watchdog.getInstance();//1
    watchdog.init(context, mActivityManagerService);//2
    traceEnd();

    traceBeginAndSlog("StartInputManagerService");
    inputManager = new InputManagerService(context);//3
    traceEnd();

    traceBeginAndSlog("StartWindowManagerService");
    ConcurrentUtils.waitForFutureNoInterrupt(mSensorServiceStart,
    START_SENSOR_SERVICE);
    mSensorServiceStart = null;
    wm = WindowManagerService.main(context, inputManager,
        mFactoryTestMode != FactoryTest.FACTORY_TEST_LOW_LEVEL,
        !mFirstBoot, mOnlyCore, new PhoneWindowManager());//4
    ServiceManager.addService(Context.WINDOW_SERVICE, wm);//5
    ServiceManager.addService(Context.INPUT_SERVICE, inputManager);//6
    traceEnd();

    ...
    try {
        wm.displayReady();//7
    } catch (Throwable e) {
        reportWtf("making display ready", e);
    }
    ...
    try {
        wm.systemReady();//8
    } catch (Throwable e) {
        reportWtf("making Window Manager Service ready", e);
    }
    ...
}
```

`startOtherServices` 方法用于启动其他服务，其他服务大概有 70 多个，上面的代码只列出了 WMS 以及和它相关的 IMS 的启动逻辑，剩余的其他服务的启动逻辑也都大同小异。在注释 1、2 处分别得到 Watchdog 实例并对它进行初始化，Watchdog 用来监控系统的一些关键服务的运行状况，后文会再次提到它。在注释 3 处创建了 IMS，并赋值给 IMS 类型的 `inputManager` 对象。注释 4 处执行了 WMS 的 `main` 方法，其内部会创建 WMS，需要注意的是 `main` 方法其中一个传入的参数就是注释 1 处创建的 IMS，WMS 是输入事件的中转站，其内部包含了 IMS 引用并不意外。结合上文，我们可以得知 WMS 的 `main` 方法是运行在 `SystemService` 的 `run` 方法中，换句话说就是运行在“`system_server`”线程”中，后面会再次提到“`system_server`”线程。

注释 5 和注释 6 处分别将 WMS 和 IMS 注册到 `ServiceManager` 中，这样如果某个客户端想要使用 WMS，就需要先去 `ServiceManager` 中查询信息，然后根据信息与 WMS 所在的进程建立通信通路，客户端就可以使用 WMS 了。注释 7 处用来初始化显示信息，注释 8 处则用来通知 WMS，系统的初始化工作已经完成，其内部调用了

`WindowManagerPolicy` 的 `systemReady` 方法。

我们来查看注释 4 处 WMS 的 `main` 方法，如下所示。

frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java

```
public static WindowManagerService main(final Context context, final
InputManagerService im,
    final boolean haveInputMethods, final boolean showBootMsgs, final
boolean onlyCore,
    WindowManagerPolicy policy) {
    DisplayThread.getHandler().runWithScissors(() -> //1
        sInstance = new WindowManagerService(context, im, haveInputMethods,
showBootMsgs,
            onlyCore, policy), 0);
    return sInstance;
}
```

在注释 1 处调用了 `DisplayThread` 的 `getHandler` 方法，用来得到 `DisplayThread` 的 `Handler` 实例。`DisplayThread` 是一个单例的前台线程，这个线程用来处理需要低延时显示的相关操作，并只能由 `WindowManager`、`DisplayManager` 和 `InputManager` 实时执行快速操作。注释 1 处的 `runWithScissors` 方法中使用了 Java8 中的 Lambda 表达式，它等价于如下代码：

```
DisplayThread.getHandler().runWithScissors(new Runnable() {
    @Override
    public void run() {
        sInstance = new WindowManagerService(context, im, haveInputMethods,
showBootMsgs,
            onlyCore, policy); //2
    }
}, 0);
```

在注释 2 处创建了 WMS 的实例，这个过程运行在 Runnable 的 run 方法中，而 Runnable 则传入到了 DisplayThread 对应 Handler 的 runWithScissors 方法中，说明 WMS 的创建是运行在“android.display”线程中。需要注意的是，runWithScissors 方法的第二个参数传入的是 0，后面会提到。来查看 Handler 的 runWithScissors 方法里做了什么：

frameworks/base/core/java/android/os/Handler.java

```
public final boolean runWithScissors(final Runnable r, long timeout) {
    if (r == null) {
        throw new IllegalArgumentException("runnable must not be null");
    }
    if (timeout < 0) {
        throw new IllegalArgumentException("timeout must be non-negative");
    }
    if (Looper.myLooper() == mLooper) { //1          r.run();
        return true;
    }
    BlockingRunnable br = new BlockingRunnable(r);
    return br.postAndWait(this, timeout);
}
```

开头对传入的 Runnable 和 timeout 进行了判断，如果 Runnable 为 null 或者 timeout 小于 0 则抛出异常。注释 1 处根据每个线程只有一个 Looper 的原理来判断当前的线程（“system_server”线程）是否是 Handler 所指向的线程（“android.display”线程），如果是则直接执行 Runnable 的 run 方法，如果不是则调用 BlockingRunnable 的 postAndWait 方法，并将当前线程的 Runnable 作为参数传进去，BlockingRunnable 是 Handler 的内部类，代码如下所示。

frameworks/base/core/java/android/os/Handler.java

```
private static final class BlockingRunnable implements Runnable {
    private final Runnable mTask;
    private boolean mDone;
    public BlockingRunnable(Runnable task) {
        mTask = task;
    }
    @Override
    public void run() {
        try {
            mTask.run(); //1
        } finally {
            synchronized (this) {
                mDone = true;
                notifyAll();
            }
        }
    }
}
```

```
}  
public boolean postAndWait(Handler handler, long timeout) {  
    if (!handler.post(this)) { //2  
        return false;  
    }  
    synchronized (this) {  
        if (timeout > 0) {  
            final long expirationTime = SystemClock.uptimeMillis() +  
timeout;  
  
            while (!mDone) {  
                long delay = expirationTime - SystemClock.uptimeMillis();  
                if (delay <= 0) {  
                    return false; // timeout  
                }  
                try {  
                    wait(delay);  
                } catch (InterruptedException ex) {  
                }  
            }  
        } else {  
            while (!mDone) {  
                try {  
                    wait(); //3  
                } catch (InterruptedException ex) {  
                }  
            }  
        }  
        return true;  
    }  
}
```

注释 2 处将当前的 **BlockingRunnable** 添加到 **Handler** 的任务队列中。前面 **runWithScissors** 方法的第二个参数为 0, 因此 **timeout** 等于 0, 这样如果 **mDone** 为 **false** 的话会一直调用注释 3 处的 **wait** 方法使得当前线程 (“**system_server**”线程) 进入等待状态, 那么等待的是哪个线程呢? 我们往上看, 注释 1 处, 执行了传入的 **Runnable** 的 **run** 方法 (运行在 “**android.display**”线程), 执行完毕后在 **finally** 代码块中将 **mDone** 设置为 **true**, 并调用 **notifyAll** 方法唤醒处于等待状态的线程, 这样就不会继续调用注释 3 处的 **wait** 方法。因此得出结论, “**system_server**”线程等待的就是 “**android.display**”线程, 一直到 “**android.display**”线程执行完毕再执行 “**system_server**”线程, 这是因为 “**android.display**”线程内部执行了 **WMS** 的创建, 显然 **WMS** 的创建优先级更高些。**WMS** 的创建就讲到这, 最后我们来查看 **WMS** 的构造方法:

frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java

```
private WindowManagerService(Context context, InputManagerService inputManager,
    boolean haveInputMethods, boolean showBootMsgs, boolean onlyCore,
    WindowManagerPolicy policy) {
    ...
    mInputManager = inputManager; //1    ...
    mDisplayManager =
        (DisplayManager) context.getSystemService(Context.DISPLAY_SERVICE);
    mDisplays = mDisplayManager.getDisplays(); //2
    for (Display display : mDisplays) {
        createDisplayContentLocked(display); //3    }
    ...
    mActivityManager = ActivityManager.getService(); //4    ...
    mAnimator = new WindowAnimator(this); //5
    mAllowTheaterModeWakeFromLayout = context.getResources().getBoolean(
        com.android.internal.R.bool.config_allowTheaterModeWakeFromWindowLayout);
    LocalServices.addService(WindowManagerInternal.class, new
        LocalService());
    initPolicy(); //6
    // Add ourselves to the Watchdog monitors.
    Watchdog.getInstance().addMonitor(this); //7    ...
}
```

注释 1 处用来保存传进来的 IMS，这样 WMS 就持有了 IMS 的引用。注释 2 处通过 DisplayManager 的 getDisplays 方法得到 Display 数组（每个显示设备都有一个 Display 实例），接着遍历 Display 数组，在注释 3 处的 createDisplayContentLocked 方法会将 Display 封装成 DisplayContent，DisplayContent 用来描述一块屏幕。

注释 4 处得到 AMS 实例，并赋值给 mActivityManager，这样 WMS 就持有了 AMS 的引用。注释 5 处创建了 WindowAnimator，它用于管理所有的窗口动画。注释 6 处初始化了窗口管理策略的接口类 WindowManagerPolicy（WMP），它用来定义一个窗口策略所要遵循的通用规范。注释 7 处将自身也就是 WMS 通过 addMonitor 方法添加到 Watchdog 中，Watchdog 用来监控系统的一些关键服务的运行状况（比如传入的 WMS 的运行状况），这些被监控的服务都会实现 Watchdog.Monitor 接口。Watchdog 每分钟都会对被监控的系统服务进行检查，如果被监控的系统服务出现了死锁，则会杀死 Watchdog 所在的进程，也就是 SystemServer 进程。

查看注释 6 处的 initPolicy 方法，如下所示。

frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java

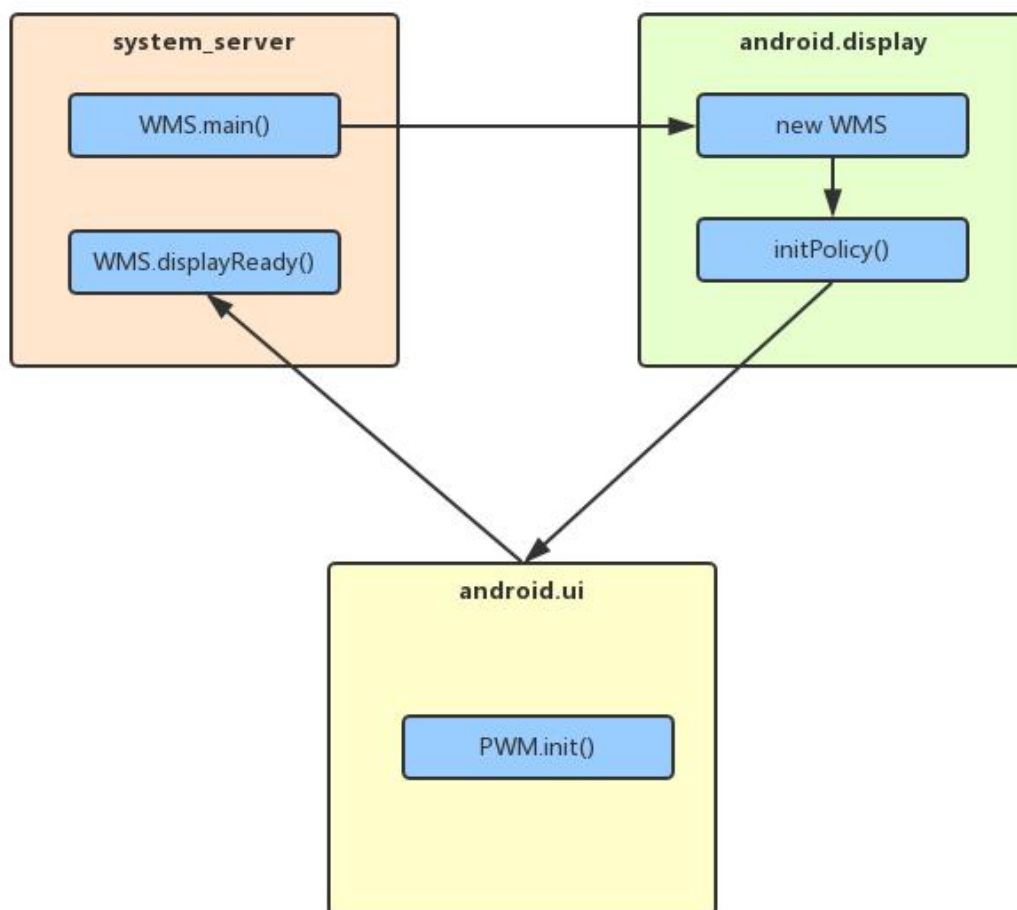
```
private void initPolicy() {
    UiThread.getHandler().runWithScissors(new Runnable() {
        @Override
        public void run() {
```

```
WindowManagerPolicyThread.set(Thread.currentThread(),
Looper.myLooper());

mPolicy.init(mContext, WindowManagerService.this,
WindowManagerService.this); //1
    }, 0);
}
```

initPolicy 方法和此前讲的 WMS 的 main 方法的实现类似，注释 1 处执行了 WMP 的 init 方法，WMP 是一个接口，init 方法的具体实现在 PhoneWindowManager（PWM）中。PWM 的 init 方法运行在“android.ui”线程中，它的优先级要高于 initPolicy 方法所在的“android.display”线程，因此“android.display”线程要等 PWM 的 init 方法执行完毕后，处于等待状态的“android.display”线程才会被唤醒从而继续执行下面的代码。

在本文中共提到了 3 个线程，分别是“system_server”、“android.display”和“android.ui”，为了便于理解，下面给出这三个线程之间的关系。



“system_server”线程中会调用 WMS 的 main 方法，main 方法中会创建 WMS，创建 WMS 的过程运行在“android.display”线程中，它的优先级更高一些，因此要等创建 WMS 完毕后会唤醒处于等待状态的“system_server”线程。

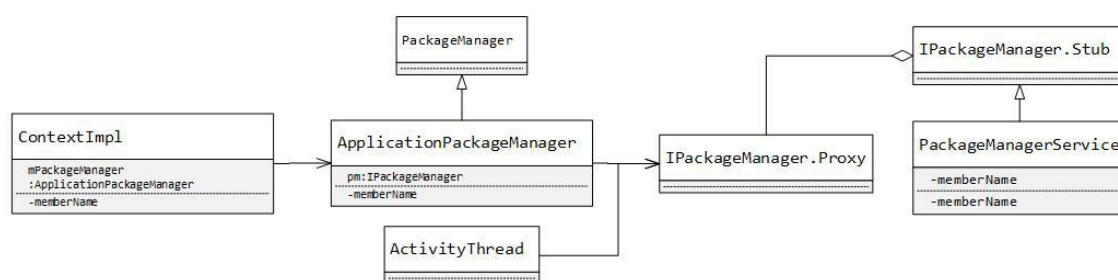
WMS 初始化时会执行 initPolicy 方法，initPolicy 方法会调用 PWM 的 init 方法，这个 init

方法运行在“android.ui”线程，并且优先级更高，因此要先执行完 PWM 的 init 方法后，才会唤醒处于等待状态的“android.display”线程。

PWM 的 init 方法执行完毕后会接着执行运行在“system_server”线程的代码，比如本文前部分提到 WMS 的 systemReady 方法。

第五章 PackageMS 启动

1. PackageMS 相关框架类



2. PackageMS 启动过程

```
# SystemServer.java
private void startBootstrapServices() {
    ...

    mSystemServiceManager.startBootPhase(SystemService.PHASE_WAIT_FOR_DEFAULT_DISPLAY);
    ...

    mPackageManagerService =
PackageManagerService.main(mSystemContext, installer,
        mFactoryTestMode != FactoryTest.FACTORY_TEST_OFF,
mOnlyCore);
    mFirstBoot = mPackageManagerService.isFirstBoot();
    mPackageManager = mSystemContext.getPackageManager();
    ...
}
```

2.2 SystemServer::startOtherServices

```
private void startOtherServices() {
    ...
}
```



```
mSystemServiceManager.startBootPhase(SystemService.PHASE_LOCK_SETTING
S_READY);
```

```
mSystemServiceManager.startBootPhase(SystemService.PHASE_SYSTEM_SERVI
CES_READY);
```

```
...
mPackageManagerService.systemReady();
...
```

```
mSystemServiceManager.startBootPhase(SystemService.PHASE_ACTIVITY_MAN
AGER_READY);
}
```

2.3 PackageManagerService::main

```
public static PackageManagerService main(Context context, Installer
installer,
    boolean factoryTest, boolean onlyCore) {
    ...
    PackageManagerService m = new PackageManagerService(context,
installer,
    ...
    ServiceManager.addService("package", m);
    return m;
}
```

2.4 PackageManagerService 初始化

```
public PackageManagerService(Context context, Installer installer,
    boolean factoryTest, boolean onlyCore) {
    ...
    // PMS_START
    // A.1 创建 Settings
    mSettings = new Settings(mPackages);
    // A.2 system phone log nfc bluetooth shell 添加到 Setting
    mSettings.addSharedUserLPw("android.uid.system",
Process.SYSTEM_UID,
        ApplicationInfo.FLAG_SYSTEM,
ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
    mSettings.addSharedUserLPw("android.uid.phone", RADIO_UID,
        ApplicationInfo.FLAG_SYSTEM,
ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
    mSettings.addSharedUserLPw("android.uid.log", LOG_UID,
        ApplicationInfo.FLAG_SYSTEM,
ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
    mSettings.addSharedUserLPw("android.uid.nfc", NFC_UID,
```

```

        ApplicationInfo.FLAG_SYSTEM,
        ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
        mSettings.addSharedUserLPw("android.uid.bluetooth",
        BLUETOOTH_UID,
        ApplicationInfo.FLAG_SYSTEM,
        ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
        mSettings.addSharedUserLPw("android.uid.shell", SHELL_UID,
        ApplicationInfo.FLAG_SYSTEM,
        ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
        ...
        mPackageDexOptimizer = new PackageDexOptimizer(installer,
        mInstallLock, context,
        "*dexopt*");
        ...
        // A.3 初始化 SystemConfig
        SystemConfig systemConfig = SystemConfig.getInstance();
        mGlobalGids = systemConfig.getGlobalGids();
        mSystemPermissions = systemConfig.getSystemPermissions();
        mAvailableFeatures = systemConfig.getAvailableFeatures();

        synchronized (mInstallLock) {
            synchronized (mPackages) {
                mHandlerThread = new ServiceThread(TAG,
                    Process.THREAD_PRIORITY_BACKGROUND, true /*allowIo*/);
                mHandlerThread.start();
                // A.4 创建 PackageHandler
                mHandler = new PackageHandler(mHandlerThread.getLooper());
                Watchdog.getInstance().addThread(mHandler, WATCHDOG_TIMEOUT);
                ...
                ArrayMap<String, String> libConfig =
                systemConfig.getSharedLibraries();
                for (int i=0; i<libConfig.size(); i++) {
                    mSharedLibraries.put(libConfig.keyAt(i),
                        new SharedLibraryEntry(libConfig.valueAt(i),
                            null));
                }

                // PMS_SYSTEM_SCAN_START
                ...
                final String bootClassPath = System.getenv("BOOTCLASSPATH");
                final String systemServerClassPath =
                System.getenv("SYSTEMSERVERCLASSPATH");
                ...
                // B.1 对满足调价的 Jar Apk 执行 dex 优化

```

```

mInstaller.dexopt(lib, Process.SYSTEM_UID,
dexCodeInstructionSet, dexoptNeeded, DEXOPT_PUBLIC /*dexFlags*/,
getCompilerFilterForReason(REASON_SHARED_APK),
StorageManager.UUID_PRIVATE_INTERNAL, SKIP_SHARED_LIBRARY_CHECK);
...
// B.2 扫描 system/app system/priv-app
// Collect ordinary system packages.
final File systemAppDir = new
File(Environment.getRootDirectory(), "app");
scanDirTracedLI(systemAppDir, mDefParseFlags
| PackageParser.PARSE_IS_SYSTEM
| PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags, 0);
...
// Collect all OEM packages.
final File oemAppDir = new File(Environment.getOemDirectory(),
"app");
scanDirTracedLI(oemAppDir, mDefParseFlags
| PackageParser.PARSE_IS_SYSTEM
| PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags, 0);
...

//look for any incomplete package installations
ArrayList<PackageSetting> deletePkgsList =
mSettings.getListOfIncompleteInstallPackagesLPr();
for (int i = 0; i < deletePkgsList.size(); i++) {
// Actual deletion of code and data will be handled by later
// reconciliation step
final String packageName = deletePkgsList.get(i).name;
logCriticalInfo(Log.WARN, "Cleaning up incompletely
installed app: " + packageName);
synchronized (mPackages) {
mSettings.removePackageLPw(packageName);
}
}

//delete tmp files
deleteTempPackageFiles();

// Remove any shared userIDs that have no associated packages
mSettings.pruneSharedUsersLPw();

// PMS_DATA_SCAN_START

//C.1 处理非系统 App //data/app 、 自定义 app 路径

```

```

        if (!mOnlyCore) {
            scanDirTracedLI(mAppInstallDir, 0, scanFlags |
SCAN_REQUIRE_KNOWN, 0);
            ...

            /**
             * Remove disable package settings for any updated system
             * apps that were removed via an OTA. If they're not a
             * previously-updated app, remove them completely.
             * Otherwise, just revoke their system-level permissions.
             */
            for (String deletedAppName :
possiblyDeletedUpdatedSystemApps) {
                PackageParser.Package deletedPkg =
mPackages.get(deletedAppName);

mSettings.removeDisabledSystemPackageLPw(deletedAppName);

                String msg = "Updated system app + " + deletedAppName
                    + " no longer present; removing system
privileges for "
                    + deletedAppName;

                deletedPkg.applicationInfo.flags
~ApplicationInfo.FLAG_SYSTEM;

                /// M: [Operator] Revoke operator permissions for
the original operator package
                /// under operator folder was gone due to OTA
                deletedPkg.applicationInfo.flagsEx
~ApplicationInfo.FLAG_EX_OPERATOR;

                PackageSetting deletedPs =
mSettings.mPackages.get(deletedAppName);
                deletedPs.pkgFlags
~ApplicationInfo.FLAG_SYSTEM;

                /// M: [Operator] Revoke vendor permissions
                deletedPs.pkgFlagsEx
~ApplicationInfo.FLAG_EX_OPERATOR;
            }
        }

        /**

```

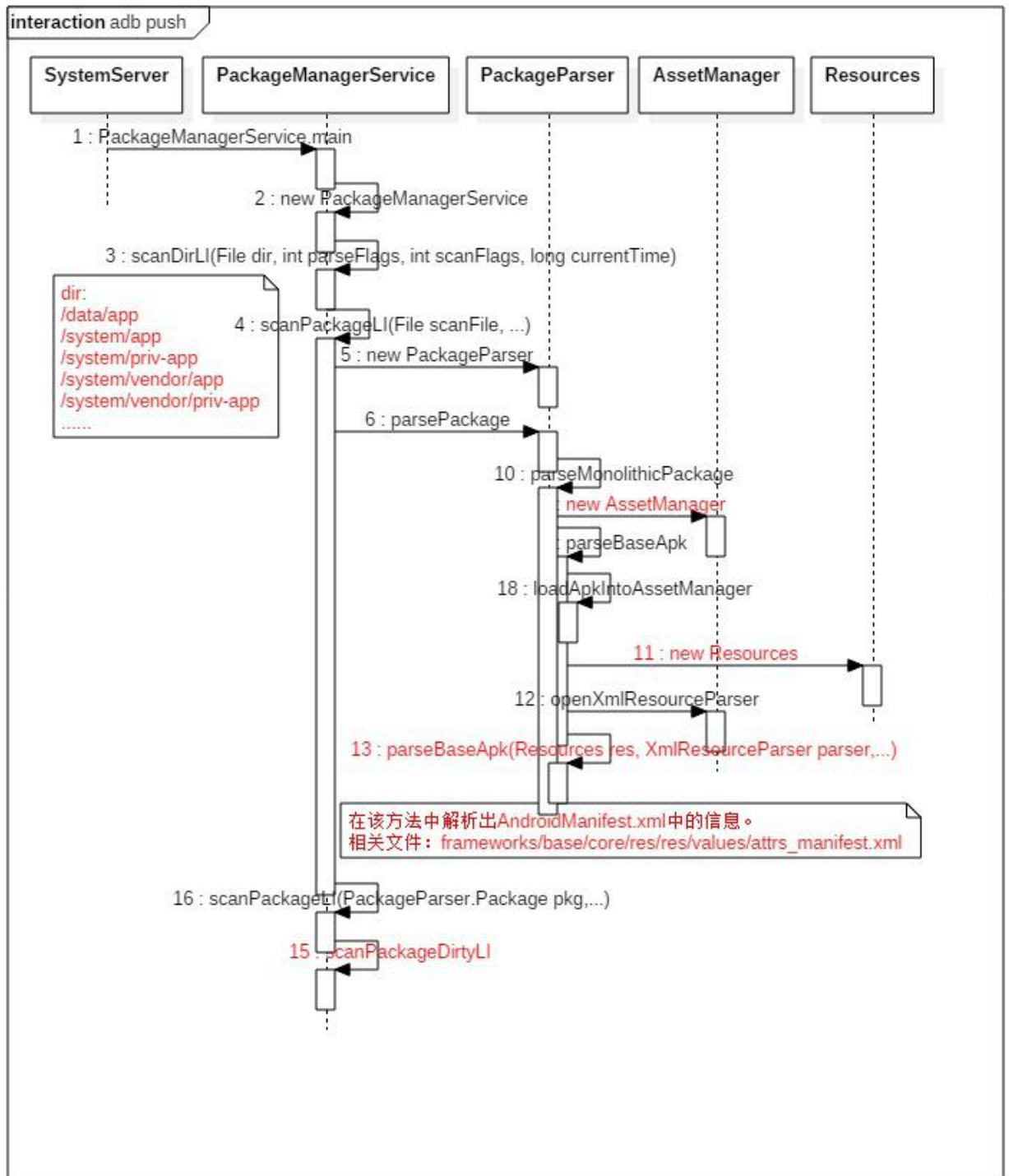
```

    * Make sure all system apps that we expected to appear on
    * the userdata partition actually showed up. If they never
    * appeared, crawl back and revive the system version.
    */
    for (int i = 0; i < mExpectingBetter.size(); i++) {
        final String packageName = mExpectingBetter.keyAt(i);
        if (!mPackages.containsKey(packageName)) {
            final File scanFile = mExpectingBetter.valueAt(i);
            ...
            scanPackageTracedLI(scanFile,          reparseFlags,
scanFlags, 0, null);
        }
    }
}

...
// PMS_SCAN_END
// D.1 回写 package.xml
// can downgrade to reader
mSettings.writeLPr();
...
// PMS_READY
// E1 创建 PackageInstallerService
mInstallerService = new PackageInstallerService(context, this);
...
} // synchronized (mPackages)
} // synchronized (mInstallLock)
}

```

2.5 PackageMS::scanDirLi



腾讯课堂 Android 高级架构系列免费直播公开课
<https://ke.qq.com/course/341933?flowToken=1027385>

更多录播视频+架构学习资料免费领取 QQ 3607409029